Logos 系列产品 GTP 用户指南

(UG020007, Version 1.4)

(2021.08.02)

深圳市紫光同创电子有限公司

版权所有侵权必究

文档版本修订记录

日期	版本	修订记录
2019/07/04	V1.0	初稿
2020/03/16	V1.1	所有 GTP 增加 PGL25G 器件是否支持描述
2020/04/16	V1.2	增加 6.13 小节，GTP_DLL 的介绍
2021/5/11	V1.3	1.所有 GTP 增加 PGL50H/PGL100H 是否支持的描述； 2.新增章节 GTP 分类； 3.第 3 章增加 3.15 节 GTP_MUX2LUT6 使用说明、3.16 节 GTP_MUX2LUT7 使用说明、3.17 节 GTP_MUX2LUT8 使用说明、3.18 节 GTP_LUT5M 使用说明； 4.更新第 4 章第 4.14 节 GTP_FIFO18K 使用说明； 5.去除重复的原第 5 章第 5.5 节 GTP_INV 使用说明（第三章第 3.4 节已经描述了此 GTP）； 6.第五章 5.15 节 GTP_IOBUF_RX_MIPI 使用说明参数列表更新参数； 7.第五章新增 5.17 节 GTP_ISERDES 使用说明、5.18 节 GTP_OSERDES 使用说明； 8.第六章部分章节新增参数描述小节； 9.第六章 第 6.3 节 GTP_CLKBUFGMUX 使用说明更新； 10.第六章第 6.6 小节 GTP_IOCLKDIV 使用说明去除端口列表中多余端口； 11.第六章第 6.11 小节 GTP_PLL_E1 使用说明参数列表更新参数； 12.新增第七章配置相关 GTP 使用说明，原第八章 GTP_EFUSECODE、GTP_IPAL_E1、GTP_SCANCHAIN_E1、GTP_FLASHIF、GTP_UDID 使用说明修改到现第七章第 7.1 节~第 7.5 节； 13.第七章第 7.2 节 GTP_IPAL_E1 使用说明更新； 14.第七章第 7.5 节 GTP_UDID 使用说明增加波形图； 15.第八章新增第 8.4 节 GTP_IDDR_E2 使用说明、8.5 节 GTP_ODDR_E2 使用说明； 16.第九章新增 9.5 节 GTP_HSST_E2 使用说明、9.6 节 GTP_ADC_E1 使用说明、9.7 节 GTP_PCIEGEN2 使用说明。
2021/08/02	V1.4	修改 GTP_IODELAY 的 STEP 值为 0~15

目录

文档版本修订记录 1
目录 2
表目录 20
图目录 29
GTP 分类 34
1. 触发器 GTP 使用说明 37
1.1. 总体介绍 37
1.2. GTP_DFF 使用说明 37
1.2.1. 支持器件 37
1.2.2. 功能介绍 37
1.2.3. 端口描述 37
1.2.4. 例化模板 37
1.3. GTP_DFF_C 使用说明 38
1.3.1. 支持器件 38
1.3.2. 功能介绍 38
1.3.3. 端口描述 38
1.3.4. 例化模板 39
1.4. GTP_DFF_E 使用说明 39
1.4.1. 支持器件 39
1.4.2. 功能介绍 39
1.4.3. 端口描述 39
1.4.4. 例化模板 40
1.5. GTP_DFF_CE 使用说明 40
1.5.1. 支持器件 40
1.5.2. 功能介绍 40
1.5.3. 端口描述 41
1.5.4. 例化模板 41
1.6. GTP_DFF_P 使用说明 41
1.6.1. 支持器件 41
1.6.2. 功能介绍 41
1.6.3. 端口描述 42
1.6.4. 例化模板 42
1.7. GTP_DFF_PE 使用说明 42
1.7.1. 支持器件 42

1.7.2. 功能介绍 43
1.7.3. 端口描述 43
1.7.4. 例化模板 43
1.8. GTP_DFF_R 使用说明 44
1.8.1. 支持器件 44
1.8.2. 功能介绍 44
1.8.3. 端口描述 44
1.8.4. 例化模板 44
1.9. GTP_DFF_RE 使用说明 45
1.9.1. 支持器件 45
1.9.2. 功能介绍 45
1.9.3. 端口描述 45
1.9.4. 例化模板 46
1.10. GTP_DFF_S 使用说明 46
1.10.1. 支持器件 46
1.10.2. 功能介绍 46
1.10.3. 端口描述 46
1.10.4. 例化模板 47
1.11. GTP_DFF_SE 使用说明 47
1.11.1. 支持器件 47
1.11.2. 功能介绍 47
1.11.3. 端口描述 47
1.11.4. 例化模板 48
2. 锁存器 GTP 使用说明 49
2.1. 总体介绍 49
2.2. GTP_DLATCH 使用说明 49
2.2.1. 支持器件 49
2.2.2. 功能介绍 49
2.2.3. 端口描述 49
2.2.4. 例化模板 49
2.3. GTP_DLATCH_E 使用说明 50
2.3.1. 支持器件 50
2.3.2. 功能介绍 50
2.3.3. 端口描述 50

2.3.4. 例化模板.....51
2.4. GTP_DLATCH_C 使用说明.....51
2.4.1. 支持器件.....51
2.4.2. 功能介绍.....51
2.4.3. 端口描述.....51
2.4.4. 例化模板.....52
2.5. GTP_DLATCH_CE 使用说明.....52
2.5.1. 支持器件.....52
2.5.2. 功能介绍.....52
2.5.3. 端口描述.....52
2.5.4. 例化模板.....53
2.6. GTP_DLATCH_P 使用说明.....53
2.6.1. 支持器件.....53
2.6.2. 功能介绍.....53
2.6.3. 端口描述.....54
2.6.4. 例化模板.....54
2.7. GTP_DLATCH_PE 使用说明.....54
2.7.1. 支持器件.....54
2.7.2. 功能介绍.....55
2.7.3. 端口描述.....55
2.7.4. 例化模板.....55
3. LUT 相关 GTP 使用说明.....56
3.1. GTP_ONE 使用说明.....56
3.1.1. 支持器件.....56
3.1.2. 功能介绍.....56
3.1.3. 端口描述.....56
3.1.4. 例化模板.....56
3.2. GTP_ZERO 使用说明.....56
3.2.1. 支持器件.....56
3.2.2. 功能介绍.....57
3.2.3. 端口描述.....57
3.2.4. 例化模板.....57
3.3. GTP_BUF 使用说明.....57
3.3.1. 支持器件.....57

3.3.2. 功能介绍 57
3.3.3. 端口描述 58
3.3.4. 例化模板 58
3.3.5. 详细功能说明 58
3.4. GTP_INV 使用说明 58
3.4.1. 支持器件 58
3.4.2. 功能介绍 58
3.4.3. 端口描述 59
3.4.4. 例化模板 59
3.5. GTP_LUT1 使用说明 59
3.5.1. 支持器件 59
3.5.2. 功能介绍 59
3.5.3. 端口描述 60
3.5.4. 参数描述 60
3.5.5. 例化模板 60
3.6. GTP_LUT2 使用说明 61
3.6.1. 支持器件 61
3.6.2. 功能介绍 61
3.6.3. 端口描述 61
3.6.4. 参数描述 62
3.6.5. 例化模板 62
3.7. GTP_LUT3 使用说明 62
3.7.1. 支持器件 62
3.7.2. 功能介绍 62
3.7.3. 端口描述 63
3.7.4. 参数描述 63
3.7.5. 例化模板 63
3.8. GTP_LUT4 使用说明 64
3.8.1. 支持器件 64
3.8.2. 功能介绍 64
3.8.3. 端口描述 64
3.8.4. 参数描述 65
3.8.5. 例化模板 65

3.9. GTP_LUT5 使用说明 66
3.9.1. 支持器件 66
3.9.2. 功能介绍 66
3.9.3. 端口描述 66
3.9.4. 参数描述 68
3.9.5. 例化模板 68
3.10. GTP_LUT5CARRY 使用说明 68
3.10.1. 支持器件 68
3.10.2. 功能介绍 68
3.10.3. 端口描述 69
3.10.4. 参数描述 69
3.10.5. 例化模板 70
3.11. GTP_LUT6 使用说明 70
3.11.1. 支持器件 70
3.11.2. 功能介绍 70
3.11.3. 端口描述 71
3.11.4. 参数描述 73
3.11.5. 例化模板 74
3.12. GTP_LUT7 使用说明 74
3.12.1. 支持器件 74
3.12.2. 功能介绍 74
3.12.3. 端口描述 74
3.12.4. 参数描述 79
3.12.5. 例化模板 79
3.13. GTP_LUT8 使用说明 80
3.13.1. 支持器件 80
3.13.2. 功能介绍 80
3.13.3. 端口描述 80
3.13.4. 参数描述 89
3.13.5. 例化模板 90
3.14. GTP_LUTMUX4 使用说明 90
3.14.1. 支持器件 90
3.14.2. 功能介绍 90
3.14.3. 端口描述 91

3.14.4. 例化模板 91
3.15. GTP_MUX2LUT6 使用说明 92
3.15.1. 支持器件 92
3.15.2. 功能介绍 92
3.15.3. 端口描述 92
3.15.4. 例化模板 92
3.16. GTP_MUX2LUT7 使用说明 93
3.16.1. 支持器件 93
3.16.2. 功能介绍 93
3.16.3. 端口描述 93
3.16.4. 例化模板 93
3.17. GTP_MUX2LUT8 使用说明 94
3.17.1. 支持器件 94
3.17.2. 功能介绍 94
3.17.3. 端口描述 94
3.17.4. 例化模板 94
3.18. GTP_LUT5M 使用说明 95
3.18.1. 支持器件 95
3.18.2. 功能介绍 95
3.18.3. 端口描述 95
3.18.4. 参数描述 97
3.18.5. 例化模板 97
4. ROM&RAM 相关 GTP 使用说明 98
4.1. 总体介绍 98
4.2. GTP_ROM32X1 使用说明 98
4.2.1. 支持器件 98
4.2.2. 功能介绍 98
4.2.3. 端口描述 98
4.2.4. 参数描述 99
4.2.5. 详细功能说明 99
4.2.6. 例化模板 99
4.3. GTP_ROM64X1 使用说明 100
4.3.1. 支持器件 100
4.3.2. 功能介绍 100

4.3.3. 端口描述 100
4.3.4. 参数描述 101
4.3.5. 详细功能说明 101
4.3.6. 例化模板 101
4.4. GTP_ROM128X1 使用说明 102
4.4.1. 支持器件 102
4.4.2. 功能介绍 102
4.4.3. 端口描述 102
4.4.4. 参数描述 103
4.4.5. 详细功能说明 103
4.4.6. 例化模板 104
4.5. GTP_ROM256X1 使用说明 104
4.5.1. 支持器件 104
4.5.2. 功能介绍 105
4.5.3. 端口描述 105
4.5.4. 参数描述 105
4.5.5. 详细功能说明 106
4.5.6. 例化模板 106
4.6. GTP_RAM16X4SP 使用说明 107
4.6.1. 支持器件 107
4.6.2. 功能介绍 107
4.6.3. 端口描述 107
4.6.4. 参数描述 108
4.6.5. 详细功能说明 108
4.6.6. 例化模板 108
4.7. GTP_RAM16X4DP 使用说明 109
4.7.1. 支持器件 109
4.7.2. 功能介绍 109
4.7.3. 端口描述 109
4.7.4. 参数描述 110
4.7.5. 详细功能说明 110
4.7.6. 例化模板 110
4.8. GTP_RAM16X1SP 使用说明 111

4.8.1. 支持器件 111
4.8.2. 功能介绍 111
4.8.3. 端口描述 111
4.8.4. 参数描述 112
4.8.5. 详细功能说明 112
4.8.6. 例化模板 112
4.9. GTP_RAM16X1DP 使用说明 113
4.9.1. 支持器件 113
4.9.2. 功能介绍 113
4.9.3. 端口描述 113
4.9.4. 参数描述 114
4.9.5. 详细功能说明 114
4.9.6. 例化模板 114
4.10. GTP_RAM32X1SP 使用说明 115
4.10.1. 支持器件 115
4.10.2. 功能介绍 115
4.10.3. 端口描述 115
4.10.4. 参数描述 116
4.10.5. 详细功能说明 116
4.10.6. 例化模板 116
4.11. GTP_RAM32X1DP 使用说明 117
4.11.1. 支持器件 117
4.11.2. 功能介绍 117
4.11.3. 端口描述 117
4.11.4. 参数描述 118
4.11.5. 详细功能说明 118
4.11.6. 例化模板 118
4.12. GTP_DRM9K 使用说明 119
4.12.1. 支持器件 119
4.12.2. 功能介绍 119
4.12.3. 端口描述 120
4.12.4. 参数描述 121
4.12.5. 详细功能说明 123

4.12.6. 例化模板 126
4.13. GTP_DRM18K 使用说明 128
4.13.1. 支持器件 128
4.13.2. 功能介绍 128
4.13.3. 端口描述 129
4.13.4. 参数描述 130
4.13.5. 详细功能说明 132
4.13.6. 例化模板 135
4.13.7. GTP 使用方法 138
4.14. GTP_FIFO18K 使用说明 138
4.14.1. 支持器件 138
4.14.2. 功能介绍 138
4.14.3. 端口描述 139
4.14.4. 参数描述 140
4.14.5. 详细功能说明 141
4.14.6. 级联拓展 144
4.14.7. 例化模板 146
5. IO 相关 GTP 使用说明 148
5.1. GTP_INBUF 使用说明 148
5.1.1. 支持器件 148
5.1.2. 功能介绍 148
5.1.3. 端口描述 148
5.1.4. 参数描述 148
5.1.5. 例化模板 149
5.2. GTP_INBUFDS 使用说明 149
5.2.1. 支持器件 149
5.2.2. 功能介绍 149
5.2.3. 端口描述 149
5.2.4. 参数描述 150
5.2.5. 例化模板 150
5.3. GTP_INBUFG 使用说明 150
5.3.1. 支持器件 150
5.3.2. 功能介绍 150
5.3.3. 端口描述 151

5.3.4. 参数描述 151
5.3.5. 例化模板 151
5.4. GTP_INBUFGDS 使用说明 152
5.4.1. 支持器件 152
5.4.2. 功能介绍 152
5.4.3. 端口描述 152
5.4.4. 参数描述 152
5.4.5. 例化模板 153
5.5. GTP_IOBUF 使用说明 153
5.5.1. 功能介绍 153
5.5.2. 端口描述 154
5.5.3. 参数描述 154
5.5.4. 例化模板 154
5.6. GTP_IOBUFCO 使用说明 155
5.6.1. 支持器件 155
5.6.2. 功能介绍 155
5.6.3. 端口描述 155
5.6.4. 参数描述 156
5.6.5. 例化模板 156
5.7. GTP_IOBUFDS 使用说明 157
5.7.1. 支持器件 157
5.7.2. 功能介绍 157
5.7.3. 端口描述 157
5.7.4. 参数描述 157
5.7.5. 例化模板 158
5.8. GTP_OUTBUF 使用说明 158
5.8.1. 支持器件 158
5.8.2. 功能介绍 158
5.8.3. 端口描述 159
5.8.4. 参数描述 159
5.8.5. 例化模板 159
5.9. GTP_OUTBUFCO 使用说明 160
5.9.1. 支持器件 160
5.9.2. 功能介绍 160

5.9.3. 端口描述 160
5.9.4. 参数描述 160
5.9.5. 例化模板 161
5.10. GTP_OUTBUFDS 使用说明 161
5.10.1. 支持器件 161
5.10.2. 功能介绍 161
5.10.3. 端口描述 161
5.10.4. 参数描述 162
5.10.5. 实例模板 162
5.11. GTP_OUTBUFT 使用说明 162
5.11.1. 支持器件 162
5.11.2. 功能介绍 162
5.11.3. 端口描述 163
5.11.4. 参数描述 163
5.11.5. 实例模板 163
5.12. GTP_OUTBUFTCO 使用说明 164
5.12.1. 支持器件 164
5.12.2. 功能介绍 164
5.12.3. 端口描述 164
5.12.4. 参数描述 164
5.12.5. 实例模板 165
5.13. GTP_OUTBUFTDS 使用说明 165
5.13.1. 支持器件 165
5.13.2. 功能介绍 165
5.13.3. 端口描述 166
5.13.4. 参数描述 166
5.13.5. 实例模板 166
5.14. GTP_IODELAY 使用说明 167
5.14.1. 支持器件 167
5.14.2. 功能介绍 167
5.14.3. 端口描述 167
5.14.4. 参数描述 167
5.14.5. 实例模板 168

5.15. GTP_IOBUF_RX_MIPI 使用说明 168
5.15.1. 支持器件 168
5.15.2. 功能介绍 168
5.15.3. 端口描述 169
5.15.4. 参数描述 170
5.15.5. 实例模板 170
5.16. GTP_IOBUF_TX_MIPI 使用说明 171
5.16.1. 支持器件 171
5.16.2. 功能介绍 171
5.16.3. 端口描述 172
5.16.4. 参数描述 172
5.16.5. 实例模板 173
5.17. GTP_ISERDES 使用说明 173
5.17.1. 支持器件 173
5.17.2. 功能介绍 173
5.17.3. 端口描述 174
5.17.4. 参数描述 174
5.17.5. 例化模板 175
5.18. GTP_OSERDES 使用说明 175
5.18.1. 支持器件 175
5.18.2. 功能介绍 175
5.18.3. 端口描述 176
5.18.4. 参数描述 176
5.18.5. 例化模板 177
6. 时钟相关 GTP 使用说明 178
6.1. GTP_CLKBUFG 178
6.1.1. 支持器件 178
6.1.2. 功能介绍 178
6.1.3. 端口描述 178
6.1.4. 例化模块 178
6.1.5. 详细功能介绍 178
6.2. GTP_CLKBUFGCE 179
6.2.1. 支持器件 179
6.2.2. 功能介绍 179

6.2.3. 端口描述 179
6.2.4. 参数描述 179
6.2.5. 例化模块 179
6.2.6. 详细功能介绍 180
6.3. GTP_CLKBUFGMUX 180
6.3.1. 支持器件 180
6.3.2. 功能介绍 180
6.3.3. 端口描述 180
6.3.4. 参数描述 181
6.3.5. 例化模块 181
6.3.6. 详细功能介绍 182
6.4. GTP_CLKBUFR 183
6.4.1. 支持器件 183
6.4.2. 功能介绍 183
6.4.3. 端口描述 183
6.4.4. 例化模块 183
6.4.5. 详细功能介绍 183
6.5. GTP_IOCLKBUF 184
6.5.1. 支持器件 184
6.5.2. 功能介绍 184
6.5.3. 端口描述 184
6.5.4. 参数描述 184
6.5.5. 例化模块 184
6.5.6. 详细功能介绍 184
6.6. GTP_IOCLKDIV 185
6.6.1. 支持器件 185
6.6.2. 功能介绍 185
6.6.3. 端口描述 185
6.6.4. 参数描述 185
6.6.5. 例化模块 186
6.6.6. 详细功能介绍 186
6.7. GTP_IOCLKDELAY 186
6.7.1. 支持器件 186

6.7.2. 功能介绍 186
6.7.3. 端口描述 186
6.7.4. 参数描述 187
6.7.5. 例化模块 187
6.7.6. 详细功能介绍 188
6.8. GTP_OSC_E1 188
6.8.1. 支持器件 188
6.8.2. 功能介绍 188
6.8.3. 端口描述 188
6.8.4. 参数描述 188
6.8.5. 例化模块 188
6.8.6. 详细功能介绍 189
6.9. GTP_OSC_E3 189
6.9.1. 支持器件 189
6.9.2. 功能介绍 189
6.9.3. 端口描述 189
6.9.4. 参数描述 190
6.9.5. 例化模块 190
6.9.6. 详细功能介绍 190
6.10. GTP_CRYSTAL 191
6.10.1. 支持器件 191
6.10.2. 功能介绍 191
6.10.3. 端口描述 191
6.10.4. 参数描述 191
6.10.5. 例化模块 191
6.10.6. 详细功能介绍 192
6.11. GTP_PLL_E1 192
6.11.1. 支持器件 192
6.11.2. 功能介绍 192
6.11.3. 端口描述 192
6.11.4. 参数描述 194
6.11.5. 例化模块 196
6.11.6. 详细功能说明 200

6.12. GTP_PLL_E3 200
6.12.1. 支持器件 200
6.12.2. 功能介绍 200
6.12.3. 端口描述 200
6.12.4. 参数描述 202
6.12.5. 例化模块 205
6.12.6. 详细功能说明 208
6.13. GTP_DLL 208
6.13.1. 支持器件 208
6.13.2. 功能介绍 208
6.13.3. 端口描述 209
6.13.4. 参数描述 209
6.13.5. 例化模块 209
6.13.6. 详细功能介绍 210
7. 配置相关 GTP 使用说明 212
7.1. GTP_EFUSECODE 使用说明 212
7.1.1. 支持器件 212
7.1.2. 功能介绍 212
7.1.3. 端口描述 212
7.1.4. 参数描述 212
7.1.5. 例化模板 212
7.1.6. 详细功能说明 212
7.2. GTP_IPAL_E1 使用说明 213
7.2.1. 支持器件 213
7.2.2. 功能介绍 213
7.2.3. 端口描述 213
7.2.4. 参数描述 214
7.2.5. 例化模板 214
7.2.6. 详细功能说明 215
7.3. GTP_SCANCHAIN_E1 使用说明 215
7.3.1. 支持器件 215
7.3.2. 功能介绍 215
7.3.3. 端口描述 215
7.3.4. 参数描述 216

7.3.5. 例化模板 216
7.3.6. 详细功能说明 217
7.4. GTP_FLASHIF 使用说明 217
7.4.1. 支持器件 217
7.4.2. 功能介绍 217
7.4.3. 端口描述 217
7.4.4. 例化模板 217
7.5. GTP_UDID 使用说明 218
7.5.1. 支持器件 218
7.5.2. 功能介绍 218
7.5.3. 端口描述 219
7.5.4. 参数描述 219
7.5.5. 例化模板 219
7.5.6. 详细功能说明 219
8. DDR GTP 使用说明 220
8.1. GTP_DDC_E1 使用说明 220
8.1.1. 支持器件 220
8.1.2. 功能介绍 220
8.1.3. 端口描述 220
8.1.4. 参数描述 222
8.1.5. 例化模板 223
8.1.6. 详细功能说明 224
8.2. GTP_DDRC 使用说明 225
8.2.1. 支持器件 225
8.2.2. 功能介绍 225
8.2.3. 端口描述 225
8.2.4. 例化模板 232
8.2.5. 详细功能说明 236
8.2.6. GTP 使用方法 236
8.3. GTP_DDRPHY 使用说明 236
8.3.1. 支持器件 236
8.3.2. 功能介绍 236
8.3.3. 端口描述 236
8.3.4. 参数描述 244

8.3.5. 详细功能说明 250
8.4. GTP_IDDR_E2 使用说明 250
8.4.1. 支持器件 250
8.4.2. 功能介绍 251
8.4.3. 端口描述 251
8.4.4. 参数描述 251
8.4.5. 例化模板 251
8.5. GTP_ODDR_E2 使用说明 252
8.5.1. 支持器件 252
8.5.2. 功能介绍 252
8.5.3. 端口描述 252
8.5.4. 参数描述 253
8.5.5. 例化模板 253
9. 其它 GTP 使用说明 254
9.1. GTP_GRS 使用说明 254
9.1.1. 支持器件 254
9.1.2. 功能介绍 254
9.1.3. 端口描述 254
9.1.4. 例化模板 254
9.2. GTP_START_E1 使用说明 254
9.2.1. 支持器件 254
9.2.2. 功能介绍 254
9.2.3. 端口描述 255
9.2.4. 例化模板 255
9.2.5. 详细功能说明 255
9.3. GTP_RES_CAL 使用说明 255
9.3.1. 支持器件 255
9.3.2. 功能介绍 255
9.3.3. 端口描述 256
9.3.4. 参数描述 256
9.3.5. 例化模板 256
9.3.6. 详细功能说明 256
9.3.7. GTP 使用方法 257
9.4. GTP_APM_E1 使用说明 257

9.4.1. 支持器件 257
9.4.2. 功能介绍 257
9.4.3. 端口描述 258
9.4.4. 参数描述 259
9.4.5. 详细功能说明 262
9.4.6. 例化模板 265
9.5. GTP_HSST_E2 使用说明 266
9.5.1. 支持器件 266
9.5.2. 功能介绍 266
9.5.3. 端口描述 267
9.5.4. 参数描述 271
9.5.5. 例化模板 357
9.6. GTP_ADC_E1 使用说明 401
9.6.1. 支持器件 401
9.6.2. 功能介绍 401
9.6.3. 端口描述 401
9.6.4. 参数描述 402
9.6.5. 例化模板 406
9.7. GTP_PCIEGEN2 使用说明 407
9.7.1. 支持器件 407
9.7.2. 功能介绍 407
9.7.3. 端口描述 408
9.7.4. 参数描述 428
9.7.5. 例化模板 430
免责声明 436

表目录

表 1 Logos 系列 GTP 分类表.....34
表 2 GTP_DFF 器件支持情况.....37
表 3 GTP_DFF 端口描述.....37
表 4 GTP_DFF_C 器件支持情况.....38
表 5 GTP_DFF_C 端口描述.....38
表 6 GTP_DFF_E 器件支持情况.....39
表 7 GTP_DFF_E 端口描述.....39
表 8 GTP_DFF_CE 器件支持情况.....40
表 9 GTP_DFF_CE 端口描述.....41
表 10 GTP_DFF_P 器件支持情况.....41
表 11 GTP_DFF_P 端口描述.....42
表 12 GTP_DFF_PE 器件支持情况.....42
表 13 GTP_DFF_PE 端口描述.....43
表 14 GTP_DFF_R 器件支持情况.....44
表 15 GTP_DFF_R 端口描述.....44
表 16 GTP_DFF_RE 器件支持情况.....45
表 17 GTP_DFF_RE 端口描述.....45
表 18 GTP_DFF_S 器件支持情况.....46
表 19 GTP_DFF_S 端口描述.....46
表 20 GTP_DFF_SE 器件支持情况.....47
表 21 GTP_DFF_SE 端口描述.....47
表 22 GTP_DLATCH 器件支持情况.....49
表 23 GTP_DLATCH 端口列表.....49
表 24 GTP_DLATCH_E 器件支持情况.....50
表 25 GTP_DLATCH_E 端口列表.....50
表 26 GTP_DLATCH_C 器件支持情况.....51
表 27 GTP_DLATCH_C 端口列表.....51
表 28 GTP_DLATCH 器件支持情况.....52
表 29 GTP_DLATCH_CE 端口列表.....52

表 30 GTP_DLATCH_P 器件支持情况.....53
表 31 GTP_DLATCH_P 端口列表.....54
表 32GTP_DLATCH_PE 器件支持情况.....54
表 33 GTP_DLATCH_PE 端口列表.....55
表 34 GTP_ONE 器件支持情况.....56
表 35 GTP_ONE 端口列表.....56
表 36 GTP_ZERO 器件支持情况.....56
表 37 GTP_ZERO 端口列表.....57
表 38 GTP_BUF 器件支持情况.....57
表 39 GTP_BUF 端口列表.....58
表 40 GTP_INV 器件支持情况.....58
表 41 GTP_INV 端口列表.....59
表 42 GTP_LUT1 器件支持情况.....59
表 43 GTP_LUT1 端口列表.....60
表 44 GTP_LUT1 真值表.....60
表 45 GTP_LUT1 参数列表.....60
表 46 GTP_LUT2 器件支持情况.....61
表 47 GTP_LUT2 端口列表.....61
表 48 GTP_LUT2 真值表.....61
表 49 GTP_LUT2 参数列表.....62
表 50 GTP_LUT3 器件支持情况.....62
表 51 GTP_LUT3 端口列表.....63
表 52 GTP_LUT3 真值表.....63
表 53 GTP_LUT3 参数列表.....63
表 54 GTP_LUT4 器件支持情况.....64
表 55 GTP_LUT4 端口列表.....64
表 56 GTP_LUT4 真值表.....64
表 57 GTP_LUT4 参数列表.....65
表 58 GTP_LUT5 器件支持情况.....66
表 59 GTP_LUT5 端口列表.....66
表 60 GTP_LUT5 真值表.....66

表 61 GTP_LUT5 参数 68
表 62 GTP_LUT5CARRY 器件支持情况 68
表 63 GTP_LUT5CARRY 端口 69
表 64 GTP_LUT5CARRY 参数 69
表 65 GTP_LUT6 器件支持情况 70
表 66 GTP_LUT6 端口 71
表 67 GTP_LUT6 真值表 71
表 68 GTP_LUT6 参数 73
表 69 GTP_LUT7 器件支持情况 74
表 70 GTP_LUT7 端口 74
表 71 GTP_LUT7 真值表 75
表 72 GTP_LUT7 参数 79
表 73 GTP_LUT8 器件支持情况 80
表 74 GTP_LUT8 端口 80
表 75 GTP_LUT8 真值表 81
表 76 GTP_LUT8 参数 89
表 77 GTP_LUTMUX4 器件支持情况 90
表 78 GTP_LUTMUX4 端口 91
表 79 GTP_LUTMUX4 真值表 91
表 80 GTP_MUX2LUT6 器件支持情况 92
表 81 GTP_MUX2LUT6 端口 92
表 82 GTP_MUX2LUT7 器件支持情况 93
表 83 GTP_MUX2LUT7 端口 93
表 84 GTP_MUX2LUT8 器件支持情况 94
表 85 GTP_MUX2LUT8 端口 94
表 86 GTP_LUT5M 器件支持情况 95
表 87 GTP_LUT5CARRY 端口 95
表 88 GTP_LUT5M 真值表 96
表 89 GTP_LUT5M 参数 97
表 90 GTP_ROM32X1 器件支持情况 98
表 91 GTP_ROM32X1 端口列表 98

表 92 GTP_ROM32X1 参数列表.....99
表 93 GTP_ROM64X1 器件支持情况.....100
表 94 GTP_ROM64X1 端口列表.....100
表 95 GTP_ROM64X1 参数列表.....101
表 96 GTP_ROM128X1 器件支持情况.....102
表 97 GTP_ROM128X1 端口列表.....102
表 98 GTP_ROM128X1 参数列表.....103
表 99 GTP_ROM256X1 器件支持情况.....104
表 100 GTP_ROM256X1 端口列表.....105
表 101 GTP_ROM256X1 参数列表.....105
表 102 GTP_RAM16X4SP 器件支持情况.....107
表 103 GTP_RAM16X4SP 端口列表.....107
表 104 GTP_RAM16X4SP 参数列表.....108
表 105 GTP_RAM16X4DP 器件支持情况.....109
表 106 GTP_RAM16X4DP 端口列表.....109
表 107 GTP_RAM16X4DP 参数列表.....110
表 108 GTP_RAM16X1SP 器件支持情况.....111
表 109 GTP_RAM16X1SP 端口列表.....111
表 110 GTP_RAM16X4SP 参数列表.....112
表 111 GTP_RAM16X1DP 器件支持情况.....113
表 112 GTP_RAM16X1DP 端口列表.....113
表 113 GTP_RAM16X1DP 参数列表.....114
表 114 GTP_RAM32X1SP 器件支持情况.....115
表 115 GTP_RAM32X1SP 端口列表.....115
表 116 GTP_RAM32X1SP 参数列表.....116
表 117 GTP_RAM32X1DP 器件支持情况.....117
表 118 GTP_RAM32X1DP 端口列表.....117
表 119 GTP_RAM32X1DP 参数列表.....118
表 120 GTP_DRM9K 器件支持情况.....119
表 121 GTP_DRM9K 端口列表.....120
表 122 GTP_DRM9K 参数列表.....121

表 123 GTP_DRM18K 器件支持情况.....128
表 124 GTP_DRM18K 端口列表.....129
表 125 GTP_DRM18K 参数列表.....130
表 126 GTP_FIFO18K 器件支持情况.....138
表 127 GTP_FIFO18K 端口列表.....139
表 128 GTP_FIFO18K 参数列表.....140
表 129 GTP_INBUF 器件支持情况.....148
表 130 GTP_INBUF 端口列表描述.....148
表 131 GTP_INBUF 参数列表.....148
表 132 GTP_INBUFDS 器件支持情况.....149
表 133 GTP_INBUFDS 端口列表描述.....149
表 134 GTP_INBUFDS 参数列表.....150
表 135 GTP_INBUFG 器件支持情况.....150
表 136 GTP_INBUFG 端口列表描述.....151
表 137 GTP_INBUFG 参数列表描述.....151
表 138 GTP_INBUFGDS 器件支持情况.....152
表 139 GTP_INBUFGDS 端口列表描述.....152
表 140 GTP_INBUFGDS 参数列表描述.....152
表 141 GTP_IOBUF 器件支持情况.....153
表 142 GTP_IOBUF 端口列表描述.....154
表 143 GTP_IOBUF 参数列表描述.....154
表 144 GTP_IOBUFCO 器件支持情况.....155
表 145 GTP_IOBUFCO 端口列表描述.....155
表 146 GTP_IOBUFCO 参数列表描述.....156
表 147 GTP_IOBUFDS 器件支持情况.....157
表 148 GTP_IOBUFDS 端口列表描述.....157
表 149 GTP_IOBUFDS 参数列表描述.....157
表 150 GTP_OUTBUF 器件支持情况.....158
表 151 GTP_OUTBUF 端口列表描述.....159
表 152 GTP_OUTBUF 参数列表描述.....159
表 153 GTP_OUTBUFCO 器件支持情况.....160

表 154 GTP_OUTBUFCO 端口列表信息 160
表 155 GTP_OUTBUFCO 参数列表信息 160
表 156 GTP_OUTBUFDS 器件支持情况 161
表 157 GTP_OUTBUFDS 端口列表描述 161
表 158 GTP_OUTBUFDS 参数列表描述 162
表 159 GTP_OUTBUFT 器件支持情况 162
表 160 GTP_OUTBUFT 端口列表描述 163
表 161 GTP_OUTBUFTCO 器件支持情况 164
表 162 GTP_OUTBUFTCO 端口列表描述 164
表 163 GTP_OUTBUFTDS 器件支持情况 165
表 164 GTP_OUTBUFTDS 端口列表描述 166
表 165 GTP_IODELAY 器件支持情况 167
表 166 GTP_IODELAY 端口列表描述 167
表 167 GTP_IOBUF_RX_MIPI 器件支持情况 168
表 168 GTP_IOBUF_RX_MIPI 端口列表描述 169
表 169 GTP_IOBUF_TX_MIPI 器件支持情况 171
表 170 GTP_IOBUF_TX_MIPI 端口列表描述 172
表 171 GTP_ISERDES 器件支持情况 173
表 172 GTP_ISERDES 端口列表 174
表 173 GTP_ISERDES 参数列表 174
表 174 GTP_OSERDES 器件支持情况 175
表 175 GTP_OSERDES 端口列表 176
表 176 GTP_OSERDES 参数列表 176
表 177 GTP_CLKBUFG 器件支持情况 178
表 178 GTP_CLKBUFG 的端口说明 178
表 179 GTP_CLKBUFGCE 器件支持情况 179
表 180 GTP_CLKBUFGCE 的端口说明 179
表 181 GTP_CLKBUFGCE 的参数说明 179
表 182 GTP_CLKBUFGMUX 器件支持情况 180
表 183 GTP_CLKBUFGMUX 的端口说明 180
表 184 GTP_CLKBUFGMUX 的参数说明 181

表 185 GTP_CLKBUFR 器件支持情况 183
表 186 GTP_CLKBUFR 的端口说明 183
表 187 GTP_IOCLKBUF 器件支持情况 184
表 188 GTP_IOCLKBUF 端口说明 184
表 189 GTP_IOCLKBUF 的参数说明 184
表 190 GTP_IOCLKDIV 器件支持情况 185
表 191 GTP_IOCLKDIV 的端口说明 185
表 192 GTP_IOCLKDIV 的参数说明 185
表 193 GTP_IOCLKDELAY 器件支持情况 186
表 194 GTP_IOCLKDELAY 端口说明 186
表 195 GTP_IOCLKDELAY 端口说明 187
表 196 GTP_OSC_E1 器件支持情况 188
表 197 GTP_OSC_E1 端口说明 188
表 198 GTP_OSC_E1 参数说明 188
表 199 GTP_OSC_E1 分频系数与输出时钟关系 189
表 200 GTP_OSC_E3 器件支持情况 189
表 201 GTP_OSC_E3 端口说明 189
表 202 GTP_OSC_E3 参数说明 190
表 203 GTP_OSC_E3 分频系数与输出时钟关系 190
表 204 GTP_CRYSTAL 器件支持情况 191
表 205 GTP_CRYSTAL 的端口说明 191
表 206 GTP_CRYSTAL 的参数说明 191
表 207 GTP_PLL_E1 器件支持情况 192
表 208 GTP_PLL_E1 的端口说明 192
表 209 GTP_PLL_E1 的参数说明 194
表 210 GTP_PLL_E3 器件支持情况 200
表 211 GTP_PLL_E3 端口描述 200
表 212 GTP_PLL_E3 参数描述 202
表 213 GTP_DLL 器件支持情况 208
表 214 GTP_DLL 端口说明 209
表 215 GTP_DLL 参数说明 209

表 216 不同配置下的相移列表 210
表 217 不同频率不同相移设置下的数字码列表 210
表 218 GTP_EFUSECODE 器件支持情况 212
表 219 GTP_EFUSECODE 端口列表 212
表 220 GTP_EFUSECODE 参数列表 212
表 221 GTP_IPAL_E1 器件支持情况 213
表 222 GTP_IPAL_E1 端口列表 213
表 223 GTP_IPAL_E1 参数列表 214
表 224 GTP_SCANCHAIN_E1 器件支持情况 215
表 225 GTP_SCANCHAIN_E1 端口列表 215
表 226 GTP_SCANCHAIN_E1 参数列表 216
表 227 GTP_FLASHIF 器件支持情况 217
表 228 GTP_FLASHIF 端口列表 217
表 229 GTP_UDID 器件支持情况 218
表 230 GTP_UDID 端口列表 219
表 231 GTP_UDID 参数列表 219
表 232 GTP_DDC_E1 器件支持情况 220
表 233 GTP_DDC_E1 端口列表 220
表 234 GTP_DDC_E1 参数列表 222
表 235 GTP_DDRC 器件支持情况 225
表 236 GTP_DDRC 端口列表 225
表 237 GTP_DDRPHY 器件支持情况 236
表 238 GTP_DDRPHY 端口列表 236
表 239 GTP_DDRPHY 参数列表 244
表 240 GTP_IDDR_E2 器件支持情况 250
表 241 GTP_IDDR_E2 端口列表 251
表 242 GTP_IDDR_E2 参数列表 251
表 243 GTP_ODDR_E2 器件支持情况 252
表 244 GTP_ODDR_E2 端口列表 252
表 245 GTP_ODDR_E2 参数列表 253
表 246 GTP_GRS 器件支持情况 254

表 247 GTP_GRS 端口列表 254
表 248 GTP_START_E1 器件支持情况 254
表 249 GTP_START_E1 端口列表 255
表 250 GTP_RES_CAL 器件支持情况 255
表 251 GTP_RES_CAL 端口列表 256
表 252 GTP_RES_CAL 参数列表 256
表 253 GTP_APM_E1 器件支持情况 257
表 254 GTP_APM_E1 端口列表 258
表 255 GTP_APM_E1 参数列表 259
表 256 GTP_APM_E1MODEX 列表 261
表 257 ZMUX 列表 262
表 258 GTP_HSST_E2 器件支持情况 266
表 259 GTP_HSST_E2 端口列表 267
表 260 GTP_HSST_E2 参数列表 271
表 261 GTP_ADC_E1 器件支持情况 401
表 262 GTP_ADC_E1 端口列表 401
表 263 GTP_ADC_E1 参数列表 402
表 264 GTP_PCIEGEN2 器件支持情况 407
表 265 GTP_PCIEGEN2 端口列表 408
表 266 GTP_PCIEGEN2 参数列表 428

图目录

图 1 GTP_DFF 37
图 2 GTP_DFF_C 38
图 3 GTP_DFF_E 39
图 4 GTP_DFF_CE 40
图 5 GTP_DFF_P 42
图 6 GTP_DFF_PE 43
图 7 GTP_DFF_R 44
图 8 GTP_DFF_RE 45
图 9 GTP_DFF_S 46
图 10 GTP_DFF_SE 47
图 11 GTP_DLATCH 示意图 49
图 12 GTP_DLATCH_E 示意图 50
图 13 GTP_DLATCH_C 示意图 51
图 14 GTP_DLATCH_CE 示意图 52
图 15 GTP_DLATCH_P 示意图 54
图 16 GTP_DLATCH_PE 示意图 55
图 17 GTP_ONE 结构图 56
图 18 GTP_ZERO 结构图 57
图 19 GTP_BUF 结构图 58
图 20 GTP_INV 结构图 59
图 21 GTP_LUT1 结构图 60
图 22 GTP_LUT2 结构图 61
图 23 GTP_LUT3 结构图 62
图 24 GTP_LUT4 结构图 64
图 25 GTP_LUT5 结构图 66
图 26 GTP_LUT5CARRY 结构图 69
图 27 GTP_LUT6 结构图 71
图 28 GTP_LUT7 结构图 74
图 29 GTP_LUT8 结构图 80

图 30 GTP_LUTMUX4 结构图.....90
图 31 GTP_MUX2LUT6 结构图.....92
图 32 GTP_MUX2LUT7 结构图.....93
图 33 GTP_MUX2LUT8 结构图.....94
图 34 GTP_LUT5M 结构图.....95
图 35 GTP_ROM32X1 结构框图.....98
图 36 GTP_ROM32X1 波形示意图.....99
图 37 GTP_ROM64X1 结构框图.....100
图 38 GTP_ROM64X1 波形示意图.....101
图 39 GTP_ROM128X1 结构框图.....102
图 40 GTP_ROM128X1 波形示意图.....104
图 41 GTP_ROM256X1 结构框图.....105
图 42 GTP_ROM256X1 波形示意图.....106
图 43 GTP_RAM16X4SP 结构框图.....107
图 44 GTP_RAM16X4SP 波形示意图.....108
图 45 GTP_RAM16X4DP 结构框图.....109
图 46 GTP_RAM16X4DP 波形示意图.....110
图 47 GTP_RAM16X1SP 结构框图.....111
图 48 GTP_RAM16X1SP 波形示意图.....112
图 49 GTP_RAM16X1DP 结构框图.....113
图 50 GTP_RAM16X1DP 波形示意图.....114
图 51 GTP_RAM32X1SP 结构框图.....115
图 52 GTP_RAM32X1SP 波形示意图.....116
图 53 GTP_RAM32X1DP 结构框图.....117
图 54 GTP_RAM32X1DP 波形示意图.....118
图 55 GTP_DRM9K 结构框图.....120
图 56 GTP_DRM9K NW 模式波形示意图.....123
图 57 GTP_DRM9K TW 模式波形示意图.....124
图 58 GTP_DRM9K RBW 模式波形示意图.....125
图 59 GTP_DRM9K SP 接口示意图.....125
图 60 GTP_DRM9K 寄存器模式波形示意图.....126

图 61 GTP_DRM18K 结构框图.....129
图 62 GTP_DRM18K NW 模式波形示意图.....133
图 63 GTP_DRM18K TW 模式波形示意图.....133
图 64 GTP_DRM18K RBW 模式波形示意图.....134
图 65 GTP_DRM18K SP 接口示意图.....135
图 66 GTP_DRM18K 寄存器模式波形示意图.....135
图 67GTP_FIFO18K 结构框图.....139
图 68 GTP_FIFO18K 写入空 FIFO 模式波形示意图.....141
图 69 GTP_FIFO18K 写入将满 FIFO 模式波形示意图.....142
图 70 GTP_FIFO18K 满 FIFO 读出模式波形示意图.....142
图 71 GTP_FIFO18K 将空 FIFO 读出模式波形示意图.....143
图 72 GTP_FIFO18K FIFO 写丢包模式波形示意图.....143
图 73 GTP_FIFO18K FIFO 读重发模式波形示意图.....144
图 74 GTP_FIFO18K FIFO 深度拓展示意图.....145
图 75 GTP_FIFO18K FIFO 宽度拓展示意图.....146
图 76 GTP_INBUF.....148
图 77 GTP_INBUFDS.....149
图 78 GTP_INBUFG.....151
图 79 GTP_INBUFGDS.....152
图 80 GTP_IOBUF.....153
图 81 GTP_IOBUFCO.....155
图 82 GTP_IOBUFDS.....157
图 83 GTP_OUTBUF.....159
图 84 GTP_OUTBUFCO.....160
图 85 GTP_OUTBUFDS.....161
图 86 GTP_OUTBUFT.....163
图 87 GTP_OUTBUFTCO.....164
图 88 GTP_OUTBUFTDS.....166
图 89 GTP_IODELAY.....167
图 90 GTP_IOBUF_RX_MIPI.....169
图 91 GTP_IOBUF_TX_MIPI.....171

图 92 GTP_ISERDES 结构框图.....174
图 93 GTP_OSERDES 结构框图.....175
图 94 GTP_CLKBUFGCE 波形图（DEFAULT_VALUE=1）.....180
图 95 GTP_CLKBUFGCE 波形图（DEFAULT_VALUE=0）.....180
图 96 GTP_CLKBUFGMUX 的参数 TRIGGER_MODE="NORMAL"的时序图.....182
图 97 GTP_CLKBUFGMUX 的参数 TRIGGER_MODE="NEGEDGE"的时序图.....182
图 98 GTP_CLKBUFR 的波形图.....183
图 99 GATEN="FALSE"的 GTP_IOCLKBUF 时序图.....185
图 100 GATEN="TRUE "的 GTP_IOCLKBUF 时序图.....185
图 101 GTP_CRYSTAL 的波形图.....192
图 102 GTP_DLL 的波形图.....211
图 103 GTP_IPAL_E1 结构框图.....213
图 104 GTP_IPAL_E1 内部从并行接口读写接口时序.....215
图 105 读 UID 时序.....218
图 106 扩展 UID 位宽.....218
图 107 GTP_DDC_E1 结构框图.....220
图 108 GTP_IDDR_E2 结构框图.....251
图 109 GTP_ODDR_E2 结构框图.....252
图 110 GTP_RES_CAL 校准模式时序.....257
图 111 GTP_APM_E1 结构框图.....258
图 112 GTP_APM_E1 乘法模式应用示意图.....262
图 113 带预加的乘法模式应用示意图.....262
图 114 乘法模式的典型时序图.....263
图 115 GTP_APM_E1 乘法模式应用示意图.....263
图 116 乘加模式的典型时序图.....263
图 117 GTP_APM_E1 乘法模式应用示意图.....264
图 118 带预加的乘法模式应用示意图.....264
图 119 乘加模式的典型时序图.....264
图 120 Systolic FIR 功能示意图.....265
图 121 GTP_HSST_E2 结构框图.....267
图 122 GTP_ADC_E1 结构框图.....401

图 123 GTP_ADC_E1 结构框图.....407

GTP 分类

本章节对本文描述的 GTP 原语按功能分为了触发器、锁存器、LUT 相关、ROM 和 RAM 相关、IO 相关、时钟相关、配置相关、DDR 相关、其他九个类别，具体分类见下表。

表 1 Logos 系列 GTP 分类表

GTP名称	GTP说明
触发器	
GTP_DFF	触发器
GTP_DFF_C	异步清零触发器
GTP_DFF_E	带使能端的触发器
GTP_DFF_CE	带使能端的异步清零触发器
GTP_DFF_P	异步置位触发器
GTP_DFF_PE	带使能端的异步置位触发器
GTP_DFF_R	同步清零触发器
GTP_DFF_RE	带使能端的同步清零触发器
GTP_DFF_S	同步置位触发器
GTP_DFF_SE	带使能端的同步置位触发器
锁存器	
GTP_DLATCH	锁存器
GTP_DLATCH_E	带使能端的锁存器
GTP_DLATCH_C	异步清零锁存器
GTP_DLATCH_CE	带使能端的异步清零锁存器
GTP_DLATCH_P	异步置位锁存器
GTP_DLATCH_PE	带使能端的异步置位锁存器
LUT相关	
GTP_ONE	GTP_ONE 输出高电平
GTP_ZERO	GTP_ZERO 输出低电平
GTP_BUF	通用 1bit BUF
GTP_INV	反向器
GTP_LUT1	1 输入 LUT
GTP_LUT2	2 输入 LUT
GTP_LUT3	3 输入 LUT
GTP_LUT4	4 输入 LUT
GTP_LUT5	5 输入 LUT
GTP_LUT5CARRY	带进位链的 5 输入 LUT
GTP_LUT6	6 输入 LUT
GTP_LUT7	7 输入 LUT
GTP_LUT8	8 输入 LUT
GTP_LUTMUX4	4:1 多路选择器
GTP_MUX2LUT6	GTP_LUT6 专用两输入选择器
GTP_MUX2LUT7	GTP_LUT7 专用两输入选择器
GTP_MUX2LUT8	GTP_LUT8 专用两输入选择器
GTP_LUT5M	5 输入查找表，不同的 INIT 参数值对应 LUT5M 不同的功能

GTP名称	GTP说明
ROM和RAM相关	
GTP_ROM32X1	32bits 地址 1bit 数据 ROM
GTP_ROM64X1	64bits 地址 1bit 数据 ROM
GTP_ROM128X1	128bits 地址 1bit 数据 ROM
GTP_ROM256X1	256bits 地址 1bit 数据 ROM
GTP_RAM16X4SP	16bits 地址 4bit 数据单端口 RAM
GTP_RAM16X4DP	16bits 地址 4bit 数据简单双端口 RAM
GTP_RAM16X1SP	16bits 地址 1bit 数据单端口 RAM
GTP_RAM16X1DP	16bits 地址 1bit 数据简单双端口 RAM
GTP_RAM32X1SP	32bits 地址 1bit 数据单端口 RAM
GTP_RAM32X1DP	32bits 地址 1bit 数据简单双端口 RAM
GTP_DRM9K	9K bits 专用 RAM 模块（DRM）
GTP_DRM18K	18K bits 专用 RAM 模块（DRM）
GTP_FIFO18K	18K bits FIFO
IO相关	
GTP_INBUF	单端输入 BUFFER
GTP_INBUFDS	差分输入 BUFFER
GTP_INBUFG	单端输入时钟 BUFFER
GTP_INBUFGDS	差分输入时钟 BUFFER
GTP_IOBUF	双向输入输出 BUFFER
GTP_IOBUFCO	双向伪差分输入输出 BUFFER
GTP_IOBUFDS	双向真差分输入输出 BUFFER
GTP_OUTBUF	单端输出 BUFFER
GTP_OUTBUFCO	伪差分输出 BUFFER
GTP_OUTBUFDS	真差分输出 BUFFER
GTP_OUTBUFT	三态输出 BUFFER
GTP_OUTBUFTCO	三态伪差分输出 BUFFER
GTP_OUTBUFTDS	三态真差分输出 BUFFER
GTP_IODELAY	数据延迟单元，支持动态或者静态延迟控制
GTP_IOBUF_RX_MIPI	支持 MIPI DPHY 高速输入，及低功耗模式（LP）下单端输入、输出
GTP_IOBUF_TX_MIPI	支持 MIPI DPHY 三态输出，包含低功耗模式（LP）下输入/输出及高速(HS)模式下差分输出
GTP_ISERDES	输入方向的数据处理模块
GTP_OSERDES	数据输出转换模块
时钟相关	
GTP_CLKBUFG	全局时钟 BUFFER
GTP_CLKBUFGCE	带使能端的全局时钟 BUFFER
GTP_CLKBUFGMUX	在两个全局时钟输入源之间的动态切换 GTP
GTP_CLKBUFR	区域时钟 BUFFER
GTP_IOCLKBUF	相当于 gate 功能，该 GTP 可用来关闭或者开启 IO 时钟
GTP_IOCLKDIV	时钟分频 GTP
GTP_IOCLKDELAY	输入输出时钟延迟单元，支持动态或者静态延迟控制
GTP_OSC_E1	为用户提供可配置的输出时钟
GTP_OSC_E3	为用户提供可配置的输出时钟

GTP名称	GTP说明
GTP_CRYSTAL	GTP_CRYSTAL 的输出时钟可以作为全局时钟驱动逻辑单元，也可以作为 IO 时钟
GTP_PLL_E1	PLL 是 FPGA 提供时钟资源的核心子系统，主要实现分频、倍频、相位调整功能
GTP_PLL_E3	PLL 是 FPGA 提供时钟资源的核心子系统，主要实现分频、倍频、相位调整功能
GTP_DLL	动态锁定输入参考时钟的频率，输出等效 delay step 数目
配置相关	
GTP_EFUSECODE	该 GTP 用来读取 EFUSECODE，向用户并行输出 32bit 存储在 efuse 中的数据
GTP_IPAL_E1	用来实现对回读的数据进行 CRC 检测或者 SEU 检测
GTP_SCANCHAIN_E1	通过 JTAG 接口，读取用户数据寄存器的值
GTP_FLASHIF	该模块为用户和内部或者外部 Flash 的时钟接口
GTP_UDID	该 GTP 用来读取 UDID CODE 值
DDR相关	
GTP_DDC_E1	为 DDR memory 存储器产生写操作时钟
GTP_DDRC	主要实现 DDR 系统中内存控制器的功能
GTP_DDRPHY	与 DDRC 配合使用，完成对 DDR 的 calibration 和 read-write 操作
GTP_IDDR_E2	GTP_IDDR_E2 支持同沿 pipeline 的 DDR1TO2 速率转换
GTP_ODDR_E2	GTP_ODDR_E2 支持同沿的 DDR2TO1 速率转换
其他	
GTP_GRS	该 GTP 用来控制全局复位信号
GTP_START_E1	该 GTP 用来描述唤醒操作的全局信号释放过程。
GTP_RES_CAL	描述用户模式时，进行 IO 的阻抗校准的输入、输出特性
GTP_APM_E1	逻辑运算单元
GTP_HSST_E2	高速串行收发器模块
GTP_ADC_E1	用于实现 ADC 功能
GTP_PCIEGEN2	高速串行接口模块，兼容 PCIE2.1 协议

1. 触发器 GTP 使用说明

1.1. 总体介绍

GTP_DFF 系列 GTP 主要实现触发器相关功能的实现。

1.2. GTP_DFF 使用说明

1.2.1. 支持器件

表 2 GTP_DFF 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

1.2.2. 功能介绍

GTP_DFF 是一个带数据输入 D 和数据输出 Q 的 D 触发器，时钟上升沿时触发，把输入信号传递给输出信号。上电输出为低。

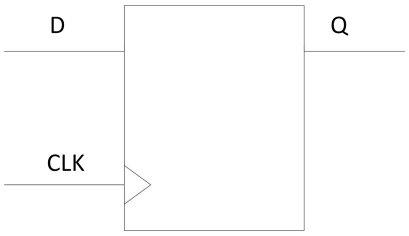


图 1 GTP_DFF

1.2.3. 端口描述

表 3 GTP_DFF 端口描述

端口名称	方向	功能描述
CLK	输入	输入时钟
D	输入	触发器输入信号
Q	输出	触发器输出信号

1.2.4. 例化模板

```
GTP_DFF(
.CLK (CLK),
```

```
.D (D),
.Q (Q)
);
```

1.3. GTP_DFF_C 使用说明

1.3.1. 支持器件

表 4 GTP_DFF_C 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

1.3.2. 功能介绍

GTP_DFF_C 是一个带数据输入 D 和数据输出 Q 的 D 触发器，时钟上升沿时触发，把输入信号传递给输出信号。上电输出为低。

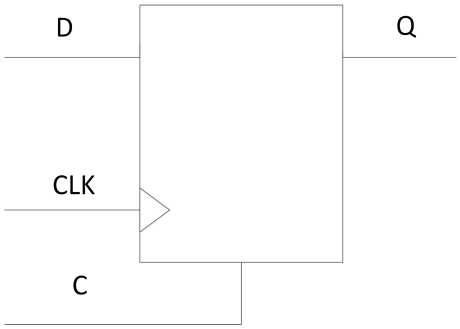


图 2 GTP_DFF_C

1.3.3. 端口描述

表 5 GTP_DFF_C 端口描述

端口名称	方向	功能描述
CLK	输入	输入时钟
D	输入	触发器输入信号
Q	输出	触发器输出信号
C	输入	异步清零信号

1.3.4. 例化模板

```
GTP_DFF_C
.CLK (CLK),
.D (D),
.C (C),
.Q (Q)
);
```

1.4. GTP_DFF_E 使用说明

1.4.1. 支持器件

表 6 GTP_DFF_E 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

1.4.2. 功能介绍

GTP_DFF E 是一个带数据输入 D 和数据输出 Q 的 D 触发器，时钟上升沿时触发且 CE 为高时，把输入信号传递给输出信号。

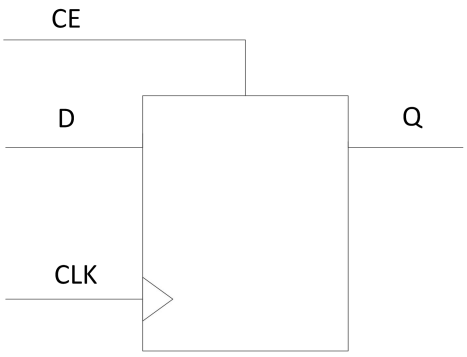


图 3 GTP_DFF_E

1.4.3. 端口描述

表 7 GTP_DFF_E 端口描述

端口名称	方向	功能描述

CLK	输入	输入时钟
D	输入	触发器输入信号
Q	输出	触发器输出信号
CE	输入	使能信号，高有效

1.4.4. 例化模板

```
GTP_DFF_E(
.CLK (CLK),
.CE (CE),
.D (D),
.Q (Q)
);
```

1.5. GTP_DFF_CE 使用说明

1.5.1. 支持器件

表 8 GTP_DFF_CE 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

1.5.2. 功能介绍

GTP_DFF_CE 是一个带数据输入 D 和数据输出 Q 的 D 触发器，时钟上升沿时触发且 CE 为高时，把输入信号传递给输出信号。异步清零。

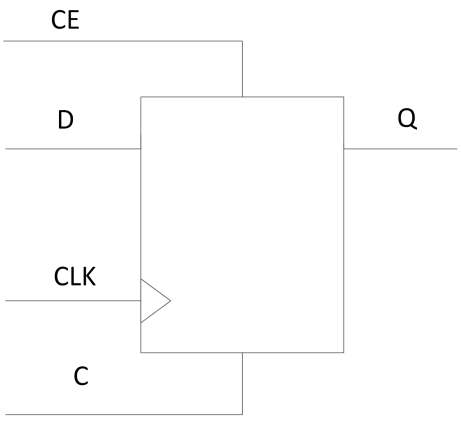


图 4 GTP_DFF_CE

1.5.3. 端口描述

表 9 GTP_DFF_CE 端口描述

端口名称	方向	功能描述
CLK	输入	输入时钟
D	输入	触发器输入信号
Q	输出	触发器输出信号
C	输入	异步清零信号
CE	输入	使能信号，高有效

1.5.4. 例化模板

```
GTP_DFF_CE(
.CLK (CLK),
.C (C),
.CE (CE),
.D (D),
.Q (Q)
);
```

1.6. GTP_DFF_P 使用说明

1.6.1. 支持器件

表 10 GTP_DFF_P 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

1.6.2. 功能介绍

GTP_DFF_P 是一个带数据输入 D 和数据输出 Q 的 D 触发器，时钟上升沿时触发，把输入信号传递给输出信号。异步置位。

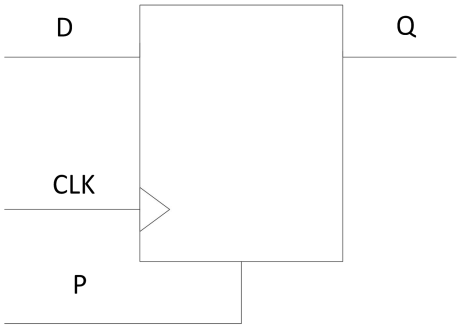


图 5 GTP_DFF_P

1.6.3. 端口描述

表 11 GTP_DFF_P 端口描述

端口名称	方向	功能描述
CLK	输入	输入时钟
D	输入	触发器输入信号
Q	输出	触发器输出信号
P	输入	异步置位信号

1.6.4. 例化模板

```
GTP_DFF_P(
.CLK (CLK),
.P(P),
.D (D),
.Q (Q)
);
```

1.7. GTP_DFF_PE 使用说明

1.7.1. 支持器件

表 12 GTP_DFF_PE 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

1.7.2. 功能介绍

GTP_DFF_PE 是一个带数据输入 D 和数据输出 Q 的 D 触发器，CE 为高时在时钟上升沿触发，把输入信号传递给输出信号。异步置位。

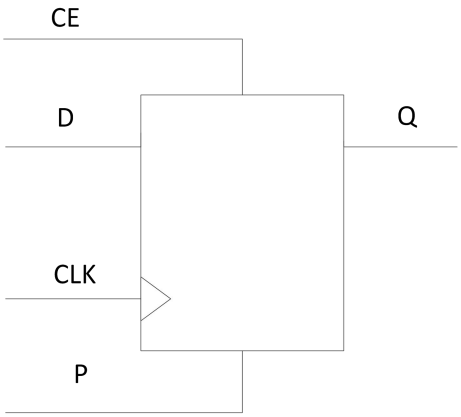


图 6 GTP_DFF_PE

1.7.3. 端口描述

表 13 GTP_DFF_PE 端口描述

端口名称	方向	功能描述
CLK	输入	输入时钟
D	输入	触发器输入信号
Q	输出	触发器输出信号
P	输入	异步置位信号
CE	输入	使能信号，高有效

1.7.4. 例化模板

```
GTP_DFF_PE(
.CLK (CLK),
.P (P),
.CE (CE),
.D (D),
.Q (Q)
);
```

1.8. GTP_DFF_R 使用说明

1.8.1. 支持器件

表 14 GTP_DFF_R 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

1.8.2. 功能介绍

GTP_DFF_R 是一个带数据输入 D 和数据输出 Q 的 D 触发器，时钟上升沿时触发，把输入信号传递给输出信号。同步清零。

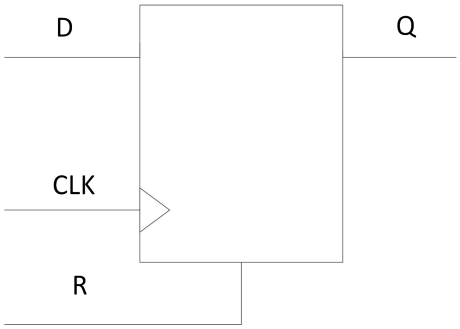


图 7 GTP_DFF_R

1.8.3. 端口描述

表 15 GTP_DFF_R 端口描述

端口名称	方向	功能描述
CLK	输入	输入时钟
D	输入	触发器输入信号
Q	输出	触发器输出信号
R	输入	同步清零信号

1.8.4. 例化模板

```
GTP_DFF_R(
.CLK (CLK),
.R (R),
.D (D),
.Q (Q)
```

);

1.9. GTP_DFF_RE 使用说明

1.9.1. 支持器件

表 16 GTP_DFF_RE 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

1.9.2. 功能介绍

GTP_DFF_RE 是一个带数据输入 D 和数据输出 Q 的 D 触发器，CE 为高时在时钟上升沿触发，把输入信号传递给输出信号。同步清零。

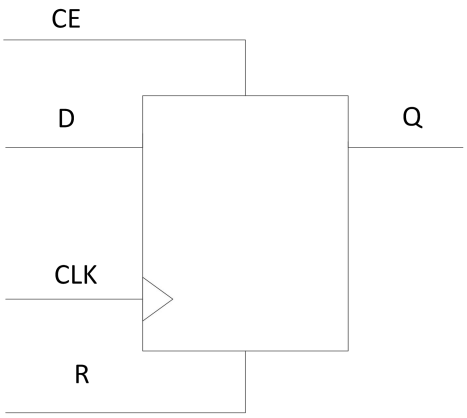


图 8 GTP_DFF_RE

1.9.3. 端口描述

表 17 GTP_DFF_RE 端口描述

端口名称	方向	功能描述
CLK	输入	输入时钟
D	输入	触发器输入信号
Q	输出	触发器输出信号
R	输入	同步清零信号
CE	输入	使能信号，高有效

1.9.4. 例化模板

```
GTP_DFF_RE(
.CLK (CLK),
.R (R),
.CE (CE),
.D (D),
.Q (Q)
);
```

1.10. GTP_DFF_S 使用说明

1.10.1. 支持器件

表 18 GTP_DFF_S 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

1.10.2. 功能介绍

GTP_DFF_S 是一个带数据输入 D 和数据输出 Q 的 D 触发器，时钟上升沿时触发，把输入信号传递给输出信号。同步置位。

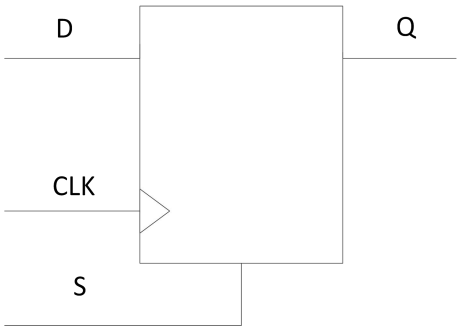


图 9 GTP_DFF_S

1.10.3. 端口描述

表 19 GTP_DFF_S 端口描述

端口名称	方向	功能描述
CLK	输入	输入时钟
D	输入	触发器输入信号

Q	输出	触发器输出信号
S	输入	同步置位信号

1.10.4. 例化模板

```
GTP_DFF_S(
.CLK (CLK),
.S (S),
.D (D),
.Q (Q)
);
```

1.11. GTP_DFF_SE 使用说明

1.11.1. 支持器件

表 20 GTP_DFF_SE 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

1.11.2. 功能介绍

GTP_DFF_SE 是一个带数据输入 D 和数据输出 Q 的 D 触发器，时钟上升沿时触发且 CE 为高时，把输入信号传递给输出信号。同步置位。

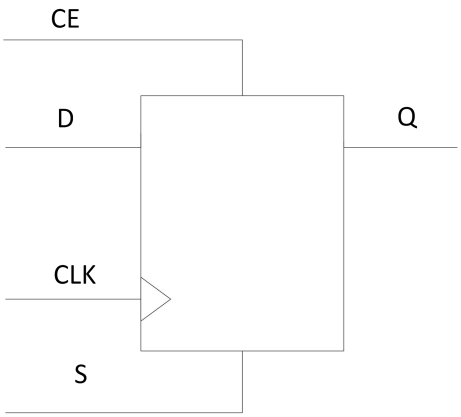


图 10 GTP_DFF_SE

1.11.3. 端口描述

表 21 GTP_DFF_SE 端口描述

端口名称	方向	功能描述
CLK	输入	输入时钟
D	输入	触发器输入信号
Q	输出	触发器输出信号
S	输入	同步置位信号
CE	输入	使能信号，高有效

1.11.4. 例化模板

```
GTP_DFF_SE(
.CLK (CLK),
.S (S),
.CE (CE),
.D (D),
.Q (Q)
);
```

2. 锁存器 GTP 使用说明

2.1. 总体介绍

GTP_DLATCH 系列 GTP 主要实现触发器相关功能的实现。

2.2. GTP_DLATCH 使用说明

2.2.1. 支持器件

表 22 GTP_DLATCH 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

2.2.2. 功能介绍

GTP_DLATCH 是一个带数据输入 D 和数据输出 Q 的锁存器，G 为高电平的时候，把输入信号传递给输出信号。

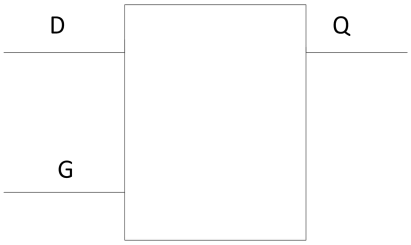


图 11 GTP_DLATCH 示意图

2.2.3. 端口描述

表 23 GTP_DLATCH 端口列表

端口名称	方向	功能描述
G	输入	电平输入
D	输入	锁存器输入信号
Q	输出	锁存器输出信号

2.2.4. 例化模板

```
GTP_DLATCH#(
.INIT(1'b0),
.GRS_EN("TURE")//"TURE"; "FALSE"
)
```

```
GTP_DLATCH (
.Q (q),
.D (d),
.G (g)
);
```

2.3. GTP_DLATCH_E 使用说明

2.3.1. 支持器件

表 24 GTP_DLATCH_E 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

2.3.2. 功能介绍

GTP_DLATCH_E 是一个带数据输入 D 和数据输出 Q 的锁存器，G 为高电平且 GE 为高时，把输入信号传递给输出信号。

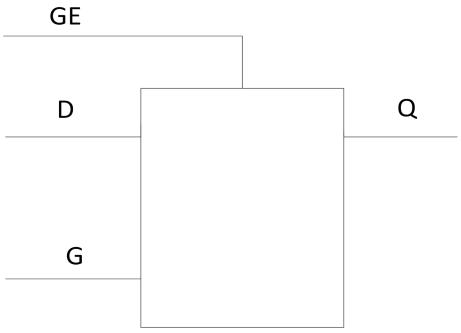


图 12 GTP_DLATCH_E 示意图

2.3.3. 端口描述

表 25 GTP_DLATCH_E 端口列表

端口名称	方向	功能描述
G	输入	输入电平
D	输入	锁存器输入信号
Q	输出	锁存器输出信号
GE	输入	使能信号，高有效

2.3.4. 例化模板

```
GTP_DLATCH_E#(
.INIT (1'b0),
.GRS_EN("TURE")//"TURE"; "FALSE"
)
GTP_DLATCH_E_inst (
.Q (q),
.D (d),
.G (g),
.GE (en)
);
```

2.4. GTP_DLATCH_C 使用说明

2.4.1. 支持器件

表 26 GTP_DLATCH_C 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

2.4.2. 功能介绍

GTP_DLATCH_C 是一个带数据输入 D 和数据输出 Q 的锁存器，G 为高电平时，把输入信号传递给输出信号。异步清零。

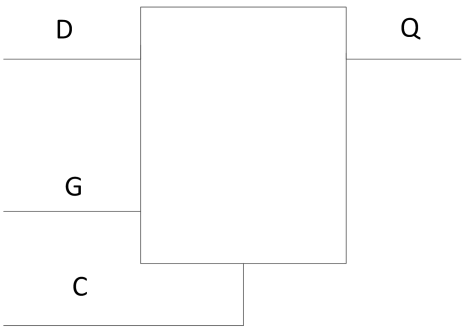


图 13 GTP_DLATCH_C 示意图

2.4.3. 端口描述

表 27 GTP_DLATCH_C 端口列表

端口名称	方向	功能描述
G	输入	输入电平
D	输入	锁存器输入信号

Q	输出	锁存器输出信号
C	输入	异步清零信号

2.4.4. 例化模板

```
GTP_DLATCH_C#(
.INIT(1'b0),
.GRS_EN("TURE")//"TURE"; "FALSE"
)
GTP_DLATCH_C_inst (
.Q (q),
.D (d),
.G (g),
.C (c)
);
```

2.5. GTP_DLATCH_CE 使用说明

2.5.1. 支持器件

表 28 GTP_DLATCH 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

2.5.2. 功能介绍

GTP_DLATCH_CE 是一个带数据输入 D 和数据输出 Q 的锁存器，G 为高电平且 GE 为高时，把输入信号传递给输出信号。异步清零。

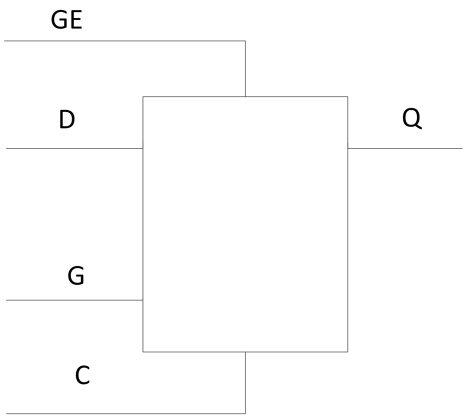


图 14 GTP_DLATCH_CE 示意图

2.5.3. 端口描述

表 29 GTP_DLATCH_CE 端口列表

端口名称	方向	功能描述
G	输入	输入电平
D	输入	锁存器输入信号
Q	输出	锁存器输出信号
C	输入	异步清零信号
GE	输入	使能信号，高有效

2.5.4. 例化模板

```
GTP_DLATCH_CE#(
.INIT(1'b0),
.GRS_EN("TURE")//"TURE"; "FALSE"
)
GTP_DLATCH_CE_inst (
.Q (q),
.D (d),
.G (g),
.C (c),
.GE (ge)
);
```

2.6. GTP_DLATCH_P 使用说明

2.6.1. 支持器件

表 30 GTP_DLATCH_P 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

2.6.2. 功能介绍

GTP_DLATCH_P 是一个带数据输入 D 和数据输出 Q 的锁存器，G 为高电平时，把输入信号传递给输出信号。异步置位。

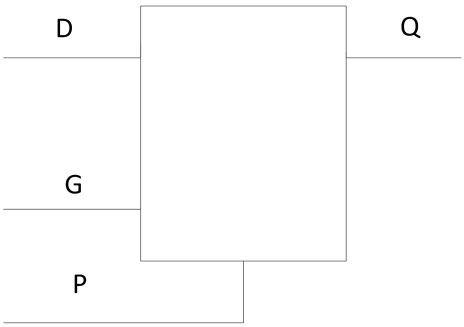


图 15 GTP_DLATCH_P 示意图

2.6.3. 端口描述

表 31 GTP_DLATCH_P 端口列表

端口名称	方向	功能描述
G	输入	输入电平
D	输入	锁存器输入信号
Q	输出	锁存器输出信号
P	输入	异步置位信号

2.6.4. 例化模板

```
GTP_DLATCH_P#(
.INIT(1'b0),
.GRS_EN("TURE")//"TURE"; "FALSE"
)
GTP_DLATCH_P_inst (
.Q (q),
.D (d),
.G (g),
.P (p)
);
```

2.7. GTP_DLATCH_PE 使用说明

2.7.1. 支持器件

表 32GTP_DLATCH_PE 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

2.7.2. 功能介绍

GTP_DLATCH_PE 是一个带数据输入 D 和数据输出 Q 的锁存器，G 为高电平且 GE 为高时，把输入信号传递给输出信号。异步置位。

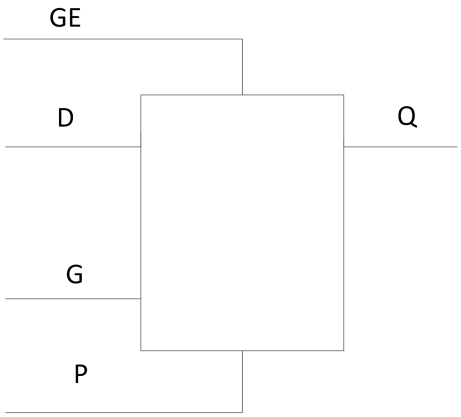


图 16 GTP_DLATCH_PE 示意图

2.7.3. 端口描述

表 33 GTP_DLATCH_PE 端口列表

端口名称	方向	功能描述
G	输入	输入电平
D	输入	锁存器输入信号
Q	输出	锁存器输出信号
P	输入	异步置位信号
GE	输入	使能信号，高有效

2.7.4. 例化模板

```
GTP_DLATCH_PE#(
.INIT(1'b0),
.GRS_EN("TURE")//"TURE"; "FALSE"
)
GTP_DLATCH_PE_inst (
.Q (q),
.D (d),
.G (g),
.P (p),
.GE (en)
```

3. LUT 相关 GTP 使用说明

3.1. GTP_ONE 使用说明

3.1.1. 支持器件

表 34 GTP_ONE 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.1.2. 功能介绍

GTP_ONE 输出高电平。如下图所示：

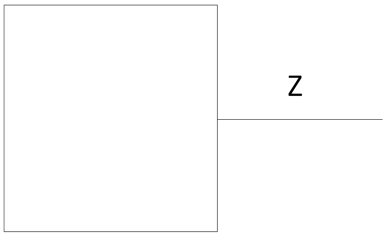


图 17 GTP_ONE 结构图

3.1.3. 端口描述

表 35 GTP_ONE 端口列表

端口名称	方向	功能描述
Z	输出	高电平输出

3.1.4. 例化模板

```
GTP_ONE GTP_ONE_inst (
.Z    (Z)
);
```

3.2. GTP_ZERO 使用说明

3.2.1. 支持器件

表 36 GTP_ZERO 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H

是否支持该GTP	支持	支持	支持	支持	支持

3.2.2. 功能介绍

GTP_ZERO 输出低电平。结构图如下所示：

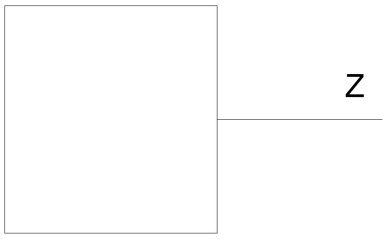


图 18 GTP_ZERO 结构图

3.2.3. 端口描述

表 37 GTP_ZERO 端口列表

端口名称	方向	功能描述
Z	输出	低电平输出

3.2.4. 例化模板

```
GTP_ZERO GTP_ZERO_inst (
.Z    (z)
);
```

3.3. GTP_BUF 使用说明

3.3.1. 支持器件

表 38 GTP_BUF 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.3.2. 功能介绍

GTP_BUF 是一个通用的 1bit 的 BUFFER，结构图如下所示：

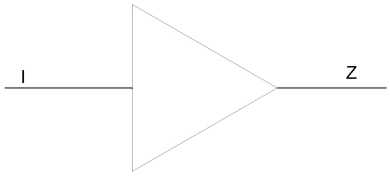


图 19 GTP_BUF 结构图

3.3.3. 端口描述

表 39 GTP_BUF 端口列表

端口名称	方向	功能描述
Z	输出	输出信号
I	输入	输入信号

3.3.4. 例化模板

```
GTP_BUF GTP_BUF_inst (
.I    (I),
.Z    (Z)
);
```

3.3.5. 详细功能说明

这个 GTP 并不是必须的，MAP 后软件会优化掉 GTP_BUF。

3.4. GTP_INV 使用说明

3.4.1. 支持器件

表 40 GTP_INV 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.4.2. 功能介绍

GTP_INV 是一个反向器。结构图如下所示：

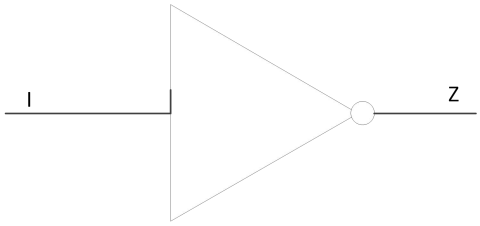


图 20 GTP_INV 结构图

3.4.3. 端口描述

表 41 GTP_INV 端口列表

端口名称	方向	功能描述
Z	输出	反向输出信号
I	输入	输入信号

3.4.4. 例化模板

```
GTP_INV GTP_INV_inst (
.I    (i),
.Z    (z)
);
```

3.5. GTP_LUT1 使用说明

3.5.1. 支持器件

表 42 GTP_LUT1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.5.2. 功能介绍

GTP_LUT1 是 1bit 输入 1bit 输出的查找表。其结构框图如下所示：

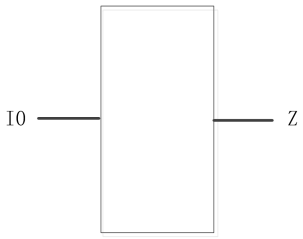


图 21 GTP_LUT1 结构图

3.5.3. 端口描述

表 43 GTP_LUT1 端口列表

端口名称	方向	功能描述
Z	输出	输出信号
I0	输入	输入信号

真值表：

表 44 GTP_LUT1 真值表

输入	输出
I0	Z
0	INIT[0]
1	INIT[1]

3.5.4. 参数描述

表 45 GTP_LUT1 参数列表

参数名称	描述	默认值	有效值
INIT	输出配置参数	2'h0	0~3

3.5.5. 例化模板

```
GTP_LUT1
#(
.INIT       (2'h0)
)GTP_LUT1_inst (
.Z          (z),
.I0         (i0)
);
```

3.6. GTP_LUT2 使用说明

3.6.1. 支持器件

表 46 GTP_LUT2 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.6.2. 功能介绍

GTP_LUT2 是一个两输入查找表，不同的 INIT 参数值对应 LUT2 不同的功能，使用时必须指定对应的 INIT 参数。其结构框图如下所示：

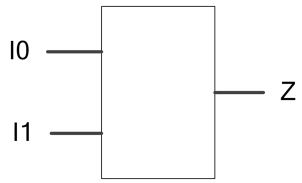


图 22 GTP_LUT2 结构图

3.6.3. 端口描述

表 47 GTP_LUT2 端口列表

端口名称	方向	功能描述
Z	输出	输出信号
I0	输入	输入信号
I1	输入	输入信号

真值表：

表 48 GTP_LUT2 真值表

输入		输出
I1	I0	Z
0	0	INIT[0]
0	1	INIT[1]
1	0	INIT[2]
1	1	INIT[3]

3.6.4. 参数描述

表 49 GTP_LUT2 参数列表

参数名称	描述	默认值	有效值
INIT	输出配置参数	4'h0	0~15

3.6.5. 例化模板

```
GTP_LUT2
#(
.INIT       (4'h0)
)GTP_LUT2_inst (
.Z          (z),
.I0         (i0),
.I1         (i1)
);
```

3.7. GTP_LUT3 使用说明

3.7.1. 支持器件

表 50 GTP_LUT3 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.7.2. 功能介绍

GTP_LUT3 是一个三输入查找表，不同的 INIT 参数值对应 LUT3 不同的功能，使用时必须指定对应的 INIT 参数。其结构框图如下所示：

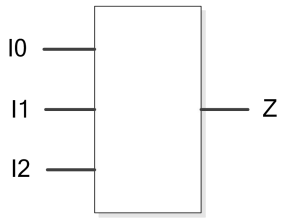


图 23 GTP_LUT3 结构图

3.7.3. 端口描述

表 51 GTP_LUT3 端口列表

端口名称	方向	功能描述
Z	输出	输出信号
I0	输入	输入信号
I1	输入	输入信号
I2	输入	输入信号

真值表：

表 52 GTP_LUT3 真值表

输入			输出
I2	I1	I0	Z
0	0	0	INIT[0]
0	0	1	INIT[1]
0	1	0	INIT[2]
0	1	1	INIT[3]
1	0	0	INIT[4]
1	0	1	INIT[5]
1	1	0	INIT[6]
1	1	1	INIT[7]

3.7.4. 参数描述

表 53 GTP_LUT3 参数列表

参数名称	描述	默认值	有效值
INIT	输出配置参数	8'h00	0~255

3.7.5. 例化模板

```
GTP_LUT3
#(
.INIT           (8'h00)
)GTP_LUT3_inst (
.Z              (z),
```

```
.I0         (i0),
.I1         (i1),
.I2         (i2)
);
```

3.8. GTP_LUT4 使用说明

3.8.1. 支持器件

表 54 GTP_LUT4 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.8.2. 功能介绍

GTP_LUT4 是一个 4 输入查找表，不同的 INIT 参数值对应 LUT4 不同的功能，使用时必须指定对应的 INIT 参数。其结构框图如下所示：

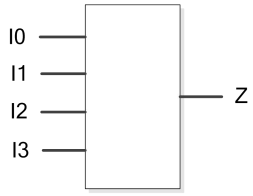


图 24 GTP_LUT4 结构图

3.8.3. 端口描述

表 55 GTP_LUT4 端口列表

端口名称	方向	功能描述
Z	输出	输出信号
I0	输入	输入信号
I1	输入	输入信号
I2	输入	输入信号
I3	输入	输入信号

真值表：

表 56 GTP_LUT4 真值表

输入				输出
I3	I2	I1	I0	Z
0	0	0	0	INIT[0]

0	0	0	1	INIT[1]
0	0	1	0	INIT[2]
0	0	1	1	INIT[3]
0	1	0	0	INIT[4]
0	1	0	1	INIT[5]
0	1	1	0	INIT[6]
0	1	1	1	INIT[7]
1	0	0	0	INIT[8]
1	0	0	1	INIT[9]
1	0	1	0	INIT[10]
1	0	1	1	INIT[11]
1	1	0	0	INIT[12]
1	1	0	1	INIT[13]
1	1	1	0	INIT[14]
1	1	1	1	INIT[15]

3.8.4. 参数描述

表 57 GTP_LUT4 参数列表

参数名称	描述	默认值	有效值
INIT	输出配置参数	16'h0000	0~2^16-1

3.8.5. 例化模板

```
GTP_LUT4
#(
.INIT       (16'h0000)
)GTP_LUT4_inst (
.Z          (z),
.I0         (i0),
.I1         (i1),
.I2         (i2),
.I3         (i3)
);
```

3.9. GTP_LUT5 使用说明

3.9.1. 支持器件

表 58 GTP_LUT5 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.9.2. 功能介绍

GTP_LUT5 是一个 5 输入查找表，不同的 INIT 参数值对应 LUT5 不同的功能，使用时必须指定对应的 INIT 参数。其结构框图如下所示：

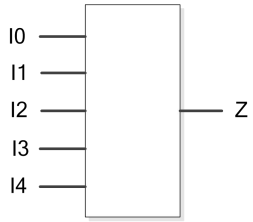


图 25 GTP_LUT5 结构图

3.9.3. 端口描述

表 59 GTP_LUT5 端口列表

端口名称	方向	功能描述
Z	输出	输出信号
I0	输入	输入信号
I1	输入	输入信号
I2	输入	输入信号
I3	输入	输入信号
I4	输入	输入信号

真值表：

表 60 GTP_LUT5 真值表

输入					输出
I4	I3	I2	I1	I0	Z
0	0	0	0	0	INIT[0]
0	0	0	0	1	INIT[1]
0	0	0	1	0	INIT[2]

0	0	0	1	1	INIT[3]
0	0	1	0	0	INIT[4]
0	0	1	0	1	INIT[5]
0	0	1	1	0	INIT[6]
0	0	1	1	1	INIT[7]
0	1	0	0	0	INIT[8]
0	1	0	0	1	INIT[9]
0	1	0	1	0	INIT[10]
0	1	0	1	1	INIT[11]
0	1	1	0	0	INIT[12]
0	1	1	0	1	INIT[13]
0	1	1	1	0	INIT[14]
0	1	1	1	1	INIT[15]
1	0	0	0	0	INIT[16]
1	0	0	0	1	INIT[17]
1	0	0	1	0	INIT[18]
1	0	0	1	1	INIT[19]
1	0	1	0	0	INIT[20]
1	0	1	0	1	INIT[21]
1	0	1	1	0	INIT[22]
1	0	1	1	1	INIT[23]
1	1	0	0	0	INIT[24]
1	1	0	0	1	INIT[25]
1	1	0	1	0	INIT[26]
1	1	0	1	1	INIT[27]
1	1	1	0	0	INIT[28]
1	1	1	0	1	INIT[29]
1	1	1	1	0	INIT[30]
1	1	1	1	1	INIT[31]

3.9.4. 参数描述

表 61 GTP_LUT5 参数

参数名	描述	默认值	设置值
INIT	输出配置参数	32'h0000_0000	0 ~ 2^32-1

3.9.5. 例化模板

```
GTP_LUT5
#(
.INIT      (32'h0000_0000)
)GTP_LUT5_inst (
.Z         (z),
.I0        (i0),
.I1        (i1),
.I2        (i2),
.I3        (i3),
.I4        (i4)
);
```

3.10. GTP_LUT5CARRY 使用说明

3.10.1. 支持器件

表 62 GTP_LUT5CARRY 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.10.2. 功能介绍

GTP_LUT5CARRY 是一个快速进位逻辑，典型应用有 counter 记数，减法运算，地址逻辑等。其结构框图如下所示：

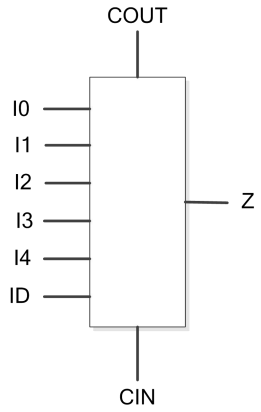


图 26 GTP_LUT5CARRY 结构图

3.10.3. 端口描述

表 63 GTP_LUT5CARRY 端口

端口名称	方向	功能描述
I0	输入	输入信号
ID	输入	输入信号，ID_TO_LUT = "TRUE"时有效
I1	输入	输入信号
I2	输入	输入信号
I3	输入	输入信号
I4	输入	输入信号，I4_TO_CARRY = "TRUE"时有效
CIN	输入	级联输入，CIN_TO_LUT = "TRUE"时有效，必须与 COUT 连接
COUT	输出	级联输出
Z	输出	输出信号

3.10.4. 参数描述

表 64 GTP_LUT5CARRY 参数

参数名	描述	默认值	设置值
INIT	输出配置参数	32'h0000_0000	32'h0 ~ 32'hffff_ffff
ID_TO_LUT	从 I0 和 ID 选择一个作为 LUT 输入	"FALSE"	TRUE：选择 ID 有效； FLASE：选择 I0 有效。
CIN_TO_LUT	从 CIN 和 I0 选择一个作为 LUT 的输入	"TRUE"	TRUE：选择 CIN 有效； FLASE：选择 I0 有效。
I4_TO_CARRY	从 I4 和 LUT4 的输出选	"TRUE"	TRUE：选择 I4 有效；

	择一个到 CARRY CHAIN		FLASE：选择 LUT4 输出。
I4_TO_LUT	从 I4 和 1’b1 选择一个作为 LUT 输入	"FALSE"	TRUE：选择 I4 有效； FLASE：选择 1’b1 有效。

3.10.5. 例化模板

```
GTP_LUT5CARRY
#(
.INIT               (32'h0000_0000),
.ID_TO_LUT          ("FALSE"),
.CIN_TO_LUT         ("TRUE"),
.I4_TO_CARRY        ("TRUE"),
.I4_TO_LUT          ("TRUE")
)GTP_LUT5CARRY_inst (
.COUT           (cout),
.Z              (z),
.CIN            (cin),
.I0             (i0),
.ID             (id),
.I1             (i1),
.I2             (i2),
.I3             (i3),
.I4             (i4)
);
```

3.11. GTP_LUT6 使用说明

3.11.1. 支持器件

表 65 GTP_LUT6 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.11.2. 功能介绍

GTP_LUT6 是一个 6 输入查找表，不同的 INIT 参数值对应 LUT6 不同的功能，使用时必须指定对应的 INIT 参数。其结构框图如下所示：

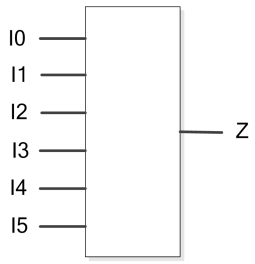


图 27 GTP_LUT6 结构图

3.11.3. 端口描述

表 66 GTP_LUT6 端口

端口名称	方向	功能描述
Z	输出	输出信号
I0	输入	输入信号
I1	输入	输入信号
I2	输入	输入信号
I3	输入	输入信号
I4	输入	输入信号
I5	输入	输入信号

真值表：

表 67 GTP_LUT6 真值表

输入						输出
I5	I4	I3	I2	I1	I0	Z
0	0	0	0	0	0	INIT[0]
0	0	0	0	0	1	INIT[1]
0	0	0	0	1	0	INIT[2]
0	0	0	0	1	1	INIT[3]
0	0	0	1	0	0	INIT[4]
0	0	0	1	0	1	INIT[5]
0	0	0	1	1	0	INIT[6]
0	0	0	1	1	1	INIT[7]
0	0	1	0	0	0	INIT[8]
0	0	1	0	0	1	INIT[9]

0	0	1	0	1	0	INIT[10]
0	0	1	0	1	1	INIT[11]
0	0	1	1	0	0	INIT[12]
0	0	1	1	0	1	INIT[13]
0	0	1	1	1	0	INIT[14]
0	0	1	1	1	1	INIT[15]
0	1	0	0	0	0	INIT[16]
0	1	0	0	0	1	INIT[17]
0	1	0	0	1	0	INIT[18]
0	1	0	0	1	1	INIT[19]
0	1	0	1	0	0	INIT[20]
0	1	0	1	0	1	INIT[21]
0	1	0	1	1	0	INIT[22]
0	1	0	1	1	1	INIT[23]
0	1	1	0	0	0	INIT[24]
0	1	1	0	0	1	INIT[25]
0	1	1	0	1	0	INIT[26]
0	1	1	0	1	1	INIT[27]
0	1	1	1	0	0	INIT[28]
0	1	1	1	0	1	INIT[29]
0	1	1	1	1	0	INIT[30]
0	1	1	1	1	1	INIT[31]
1	0	0	0	0	0	INIT[32]
1	0	0	0	0	1	INIT[33]
1	0	0	0	1	0	INIT[34]
1	0	0	0	1	1	INIT[35]
1	0	0	1	0	0	INIT[36]
1	0	0	1	0	1	INIT[37]
1	0	0	1	1	0	INIT[38]
1	0	0	1	1	1	INIT[39]

1	0	1	0	0	0	INIT[40]
1	0	1	0	0	1	INIT[41]
1	0	1	0	1	0	INIT[42]
1	0	1	0	1	1	INIT[43]
1	0	1	1	0	0	INIT[44]
1	0	1	1	0	1	INIT[45]
1	0	1	1	1	0	INIT[46]
1	0	1	1	1	1	INIT[47]
1	1	0	0	0	0	INIT[48]
1	1	0	0	0	1	INIT[49]
1	1	0	0	1	0	INIT[50]
1	1	0	0	1	1	INIT[51]
1	1	0	1	0	0	INIT[52]
1	1	0	1	0	1	INIT[53]
1	1	0	1	1	0	INIT[54]
1	1	0	1	1	1	INIT[55]
1	1	1	0	0	0	INIT[56]
1	1	1	0	0	1	INIT[57]
1	1	1	0	1	0	INIT[58]
1	1	1	0	1	1	INIT[59]
1	1	1	1	0	0	INIT[60]
1	1	1	1	0	1	INIT[61]
1	1	1	1	1	0	INIT[62]
1	1	1	1	1	1	INIT[63]

3.11.4. 参数描述

表 68 GTP_LUT6 参数

参数名	描述	默认值	设置值
INIT	输出配置参数	64’h0000_0000_0000_0000	0 ~ 2^64-1

3.11.5. 例化模板

```
GTP_LUT6
#(
.INIT       (64'h0000_0000_0000_0000)
)GTP_LUT6_inst (
.Z          (z),
.I0         (i0),
.I1         (i1),
.I2         (i2),
.I3         (i3),
.I4         (i4),
.I5         (i5)
);
```

3.12. GTP_LUT7 使用说明

3.12.1. 支持器件

表 69 GTP_LUT7 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.12.2. 功能介绍

GTP_LUT7 是一个 7 输入查找表，不同的 INIT 参数值对应 LUT7 不同的功能，使用时必须指定对应的 INIT 参数。其结构框图如下所示：

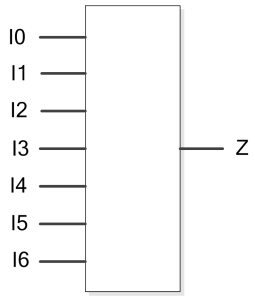


图 28 GTP_LUT7 结构图

3.12.3. 端口描述

表 70 GTP_LUT7 端口

端口名称	方向	功能描述

Z	输出	输出信号
I0	输入	输入信号
I1	输入	输入信号
I2	输入	输入信号
I3	输入	输入信号
I4	输入	输入信号
I5	输入	输入信号
I6	输入	输入信号

真值表：

表 71 GTP_LUT7 真值表

输入							输出
I6	I5	I4	I3	I2	I1	I0	Z
0	0	0	0	0	0	0	INIT[0]
0	0	0	0	0	0	1	INIT[1]
0	0	0	0	0	1	0	INIT[2]
0	0	0	0	0	1	1	INIT[3]
0	0	0	0	1	0	0	INIT[4]
0	0	0	0	1	0	1	INIT[5]
0	0	0	0	1	1	0	INIT[6]
0	0	0	0	1	1	1	INIT[7]
0	0	0	1	0	0	0	INIT[8]
0	0	0	1	0	0	1	INIT[9]
0	0	0	1	0	1	0	INIT[10]
0	0	0	1	0	1	1	INIT[11]
0	0	0	1	1	0	0	INIT[12]
0	0	0	1	1	0	1	INIT[13]
0	0	0	1	1	1	0	INIT[14]
0	0	0	1	1	1	1	INIT[15]
0	0	1	0	0	0	0	INIT[16]
0	0	1	0	0	0	1	INIT[17]

0	0	1	0	0	1	0	INIT[18]
0	0	1	0	0	1	1	INIT[19]
0	0	1	0	1	0	0	INIT[20]
0	0	1	0	1	0	1	INIT[21]
0	0	1	0	1	1	0	INIT[22]
0	0	1	0	1	1	1	INIT[23]
0	0	1	1	0	0	0	INIT[24]
0	0	1	1	0	0	1	INIT[25]
0	0	1	1	0	1	0	INIT[26]
0	0	1	1	0	1	1	INIT[27]
0	0	1	1	1	0	0	INIT[28]
0	0	1	1	1	0	1	INIT[29]
0	0	1	1	1	1	0	INIT[30]
0	0	1	1	1	1	1	INIT[31]
0	1	0	0	0	0	0	INIT[32]
0	1	0	0	0	0	1	INIT[33]
0	1	0	0	0	1	0	INIT[34]
0	1	0	0	0	1	1	INIT[35]
0	1	0	0	1	0	0	INIT[36]
0	1	0	0	1	0	1	INIT[37]
0	1	0	0	1	1	0	INIT[38]
0	1	0	0	1	1	1	INIT[39]
0	1	0	1	0	0	0	INIT[40]
0	1	0	1	0	0	1	INIT[41]
0	1	0	1	0	1	0	INIT[42]
0	1	0	1	0	1	1	INIT[43]
0	1	0	1	1	0	0	INIT[44]
0	1	0	1	1	0	1	INIT[45]
0	1	0	1	1	1	0	INIT[46]
0	1	0	1	1	1	1	INIT[47]

0	1	1	0	0	0	0	INIT[48]
0	1	1	0	0	0	1	INIT[49]
0	1	1	0	0	1	0	INIT[50]
0	1	1	0	0	1	1	INIT[51]
0	1	1	0	1	0	0	INIT[52]
0	1	1	0	1	0	1	INIT[53]
0	1	1	0	1	1	0	INIT[54]
0	1	1	0	1	1	1	INIT[55]
0	1	1	1	0	0	0	INIT[56]
0	1	1	1	0	0	1	INIT[57]
0	1	1	1	0	1	0	INIT[58]
0	1	1	1	0	1	1	INIT[59]
0	1	1	1	1	0	0	INIT[60]
0	1	1	1	1	0	1	INIT[61]
0	1	1	1	1	1	0	INIT[62]
0	1	1	1	1	1	1	INIT[63]
1	0	0	0	0	0	0	INIT[64]
1	0	0	0	0	0	1	INIT[65]
1	0	0	0	0	1	0	INIT[66]
1	0	0	0	0	1	1	INIT[67]
1	0	0	0	1	0	0	INIT[68]
1	0	0	0	1	0	1	INIT[69]
1	0	0	0	1	1	0	INIT[70]
1	0	0	0	1	1	1	INIT[71]
1	0	0	1	0	0	0	INIT[72]
1	0	0	1	0	0	1	INIT[73]
1	0	0	1	0	1	0	INIT[74]
1	0	0	1	0	1	1	INIT[75]
1	0	0	1	1	0	0	INIT[76]
1	0	0	1	1	0	1	INIT[77]

1	0	0	1	1	1	0	INIT[78]
1	0	0	1	1	1	1	INIT[79]
1	0	1	0	0	0	0	INIT[80]
1	0	1	0	0	0	1	INIT[81]
1	0	1	0	0	1	0	INIT[82]
1	0	1	0	0	1	1	INIT[83]
1	0	1	0	1	0	0	INIT[84]
1	0	1	0	1	0	1	INIT[85]
1	0	1	0	1	1	0	INIT[86]
1	0	1	0	1	1	1	INIT[87]
1	0	1	1	0	0	0	INIT[88]
1	0	1	1	0	0	1	INIT[89]
1	0	1	1	0	1	0	INIT[90]
1	0	1	1	0	1	1	INIT[91]
1	0	1	1	1	0	0	INIT[92]
1	0	1	1	1	0	1	INIT[93]
1	0	1	1	1	1	0	INIT[94]
1	0	1	1	1	1	1	INIT[95]
1	1	0	0	0	0	0	INIT[96]
1	1	0	0	0	0	1	INIT[97]
1	1	0	0	0	1	0	INIT[98]
1	1	0	0	0	1	1	INIT[99]
1	1	0	0	1	0	0	INIT[100]
1	1	0	0	1	0	1	INIT[101]
1	1	0	0	1	1	0	INIT[102]
1	1	0	0	1	1	1	INIT[103]
1	1	0	1	0	0	0	INIT[104]
1	1	0	1	0	0	1	INIT[105]
1	1	0	1	0	1	0	INIT[106]
1	1	0	1	0	1	1	INIT[107]

1	1	0	1	1	0	0	INIT[108]
1	1	0	1	1	0	1	INIT[109]
1	1	0	1	1	1	0	INIT[110]
1	1	0	1	1	1	1	INIT[111]
1	1	1	0	0	0	0	INIT[112]
1	1	1	0	0	0	1	INIT[113]
1	1	1	0	0	1	0	INIT[114]
1	1	1	0	0	1	1	INIT[115]
1	1	1	0	1	0	0	INIT[116]
1	1	1	0	1	0	1	INIT[117]
1	1	1	0	1	1	0	INIT[118]
1	1	1	0	1	1	1	INIT[119]
1	1	1	1	0	0	0	INIT[120]
1	1	1	1	0	0	1	INIT[121]
1	1	1	1	0	1	0	INIT[122]
1	1	1	1	0	1	1	INIT[123]
1	1	1	1	1	0	0	INIT[124]
1	1	1	1	1	0	1	INIT[125]
1	1	1	1	1	1	0	INIT[126]
1	1	1	1	1	1	1	INIT[127]

3.12.4. 参数描述

表 72 GTP_LUT7 参数

参数名	描述	默认值	设置值
INIT	输出配置参数	128'h00000000_00000000_00000000_00000000	0 ~ 2^128-1

3.12.5. 例化模板

```
GTP_LUT7
#(
.INIT       (128'h00000000_00000000_00000000_00000000)
)GTP_LUT7_inst (
.Z          (z),
.I0         (i0),
```

```
    .I1         (i1),
    .I2         (i2),
    .I3         (i3),
    .I4         (i4),
    .I5         (i5),
    .I6         (i6)
    );
```

3.13. GTP_LUT8 使用说明

3.13.1. 支持器件

表 73 GTP_LUT8 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.13.2. 功能介绍

GTP_LUT8 是一个 8 输入查找表，不同的 INIT 参数值对应 LUT8 不同的功能，使用时必须指定对应的 INIT 参数。其结构框图如下所示：

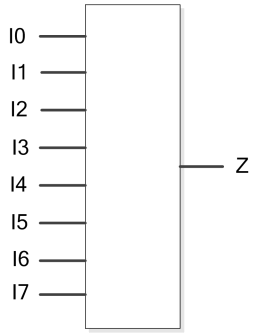


图 29 GTP_LUT8 结构图

3.13.3. 端口描述

表 74 GTP_LUT8 端口

端口名称	方向	功能描述
Z	输出	输出信号
I0	输入	输入信号
I1	输入	输入信号
I2	输入	输入信号
I3	输入	输入信号

I4	输入	输入信号
I5	输入	输入信号
I6	输入	输入信号
I7	输入	输入信号

真值表：

表 75 GTP_LUT8 真值表

输入								输出
I7	I6	I5	I4	I3	I2	I1	I0	Z
0	0	0	0	0	0	0	0	INIT[0]
0	0	0	0	0	0	0	1	INIT[1]
0	0	0	0	0	0	1	0	INIT[2]
0	0	0	0	0	0	1	1	INIT[3]
0	0	0	0	0	1	0	0	INIT[4]
0	0	0	0	0	1	0	1	INIT[5]
0	0	0	0	0	1	1	0	INIT[6]
0	0	0	0	0	1	1	1	INIT[7]
0	0	0	0	1	0	0	0	INIT[8]
0	0	0	0	1	0	0	1	INIT[9]
0	0	0	0	1	0	1	0	INIT[10]
0	0	0	0	1	0	1	1	INIT[11]
0	0	0	0	1	1	0	0	INIT[12]
0	0	0	0	1	1	0	1	INIT[13]
0	0	0	0	1	1	1	0	INIT[14]
0	0	0	0	1	1	1	1	INIT[15]
0	0	0	1	0	0	0	0	INIT[16]
0	0	0	1	0	0	0	1	INIT[17]
0	0	0	1	0	0	1	0	INIT[18]
0	0	0	1	0	0	1	1	INIT[19]
0	0	0	1	0	1	0	0	INIT[20]
0	0	0	1	0	1	0	1	INIT[21]

0	0	0	1	0	1	1	0	INIT[22]
0	0	0	1	0	1	1	1	INIT[23]
0	0	0	1	1	0	0	0	INIT[24]
0	0	0	1	1	0	0	1	INIT[25]
0	0	0	1	1	0	1	0	INIT[26]
0	0	0	1	1	0	1	1	INIT[27]
0	0	0	1	1	1	0	0	INIT[28]
0	0	0	1	1	1	0	1	INIT[29]
0	0	0	1	1	1	1	0	INIT[30]
0	0	0	1	1	1	1	1	INIT[31]
0	0	1	0	0	0	0	0	INIT[32]
0	0	1	0	0	0	0	1	INIT[33]
0	0	1	0	0	0	1	0	INIT[34]
0	0	1	0	0	0	1	1	INIT[35]
0	0	1	0	0	1	0	0	INIT[36]
0	0	1	0	0	1	0	1	INIT[37]
0	0	1	0	0	1	1	0	INIT[38]
0	0	1	0	0	1	1	1	INIT[39]
0	0	1	0	1	0	0	0	INIT[40]
0	0	1	0	1	0	0	1	INIT[41]
0	0	1	0	1	0	1	0	INIT[42]
0	0	1	0	1	0	1	1	INIT[43]
0	0	1	0	1	1	0	0	INIT[44]
0	0	1	0	1	1	0	1	INIT[45]
0	0	1	0	1	1	1	0	INIT[46]
0	0	1	0	1	1	1	1	INIT[47]
0	0	1	1	0	0	0	0	INIT[48]
0	0	1	1	0	0	0	1	INIT[49]
0	0	1	1	0	0	1	0	INIT[50]
0	0	1	1	0	0	1	1	INIT[51]

0	0	1	1	0	1	0	0	INIT[52]
0	0	1	1	0	1	0	1	INIT[53]
0	0	1	1	0	1	1	0	INIT[54]
0	0	1	1	0	1	1	1	INIT[55]
0	0	1	1	1	0	0	0	INIT[56]
0	0	1	1	1	0	0	1	INIT[57]
0	0	1	1	1	0	1	0	INIT[58]
0	0	1	1	1	0	1	1	INIT[59]
0	0	1	1	1	1	0	0	INIT[60]
0	0	1	1	1	1	0	1	INIT[61]
0	0	1	1	1	1	1	0	INIT[62]
0	0	1	1	1	1	1	1	INIT[63]
0	1	0	0	0	0	0	0	INIT[64]
0	1	0	0	0	0	0	1	INIT[65]
0	1	0	0	0	0	1	0	INIT[66]
0	1	0	0	0	0	1	1	INIT[67]
0	1	0	0	0	1	0	0	INIT[68]
0	1	0	0	0	1	0	1	INIT[69]
0	1	0	0	0	1	1	0	INIT[70]
0	1	0	0	0	1	1	1	INIT[71]
0	1	0	0	1	0	0	0	INIT[72]
0	1	0	0	1	0	0	1	INIT[73]
0	1	0	0	1	0	1	0	INIT[74]
0	1	0	0	1	0	1	1	INIT[75]
0	1	0	0	1	1	0	0	INIT[76]
0	1	0	0	1	1	0	1	INIT[77]
0	1	0	0	1	1	1	0	INIT[78]
0	1	0	0	1	1	1	1	INIT[79]
0	1	0	1	0	0	0	0	INIT[80]
0	1	0	1	0	0	0	1	INIT[81]

0	1	0	1	0	0	1	0	INIT[82]
0	1	0	1	0	0	1	1	INIT[83]
0	1	0	1	0	1	0	0	INIT[84]
0	1	0	1	0	1	0	1	INIT[85]
0	1	0	1	0	1	1	0	INIT[86]
0	1	0	1	0	1	1	1	INIT[87]
0	1	0	1	1	0	0	0	INIT[88]
0	1	0	1	1	0	0	1	INIT[89]
0	1	0	1	1	0	1	0	INIT[90]
0	1	0	1	1	0	1	1	INIT[91]
0	1	0	1	1	1	0	0	INIT[92]
0	1	0	1	1	1	0	1	INIT[93]
0	1	0	1	1	1	1	0	INIT[94]
0	1	0	1	1	1	1	1	INIT[95]
0	1	1	0	0	0	0	0	INIT[96]
0	1	1	0	0	0	0	1	INIT[97]
0	1	1	0	0	0	1	0	INIT[98]
0	1	1	0	0	0	1	1	INIT[99]
0	1	1	0	0	1	0	0	INIT[100]
0	1	1	0	0	1	0	1	INIT[101]
0	1	1	0	0	1	1	0	INIT[102]
0	1	1	0	0	1	1	1	INIT[103]
0	1	1	0	1	0	0	0	INIT[104]
0	1	1	0	1	0	0	1	INIT[105]
0	1	1	0	1	0	1	0	INIT[106]
0	1	1	0	1	0	1	1	INIT[107]
0	1	1	0	1	1	0	0	INIT[108]
0	1	1	0	1	1	0	1	INIT[109]
0	1	1	0	1	1	1	0	INIT[110]
0	1	1	0	1	1	1	1	INIT[111]

0	1	1	1	0	0	0	0	INIT[112]
0	1	1	1	0	0	0	1	INIT[113]
0	1	1	1	0	0	1	0	INIT[114]
0	1	1	1	0	0	1	1	INIT[115]
0	1	1	1	0	1	0	0	INIT[116]
0	1	1	1	0	1	0	1	INIT[117]
0	1	1	1	0	1	1	0	INIT[118]
0	1	1	1	0	1	1	1	INIT[119]
0	1	1	1	1	0	0	0	INIT[120]
0	1	1	1	1	0	0	1	INIT[121]
0	1	1	1	1	0	1	0	INIT[122]
0	1	1	1	1	0	1	1	INIT[123]
0	1	1	1	1	1	0	0	INIT[124]
0	1	1	1	1	1	0	1	INIT[125]
0	1	1	1	1	1	1	0	INIT[126]
0	1	1	1	1	1	1	1	INIT[127]
1	0	0	0	0	0	0	0	INIT[128]
1	0	0	0	0	0	0	1	INIT[129]
1	0	0	0	0	0	1	0	INIT[130]
1	0	0	0	0	0	1	1	INIT[131]
1	0	0	0	0	1	0	0	INIT[132]
1	0	0	0	0	1	0	1	INIT[133]
1	0	0	0	0	1	1	0	INIT[134]
1	0	0	0	0	1	1	1	INIT[135]
1	0	0	0	1	0	0	0	INIT[136]
1	0	0	0	1	0	0	1	INIT[137]
1	0	0	0	1	0	1	0	INIT[138]
1	0	0	0	1	0	1	1	INIT[139]
1	0	0	0	1	1	0	0	INIT[140]
1	0	0	0	1	1	0	1	INIT[141]

1	0	0	0	1	1	1	0	INIT[142]
1	0	0	0	1	1	1	1	INIT[143]
1	0	0	1	0	0	0	0	INIT[144]
1	0	0	1	0	0	0	1	INIT[145]
1	0	0	1	0	0	1	0	INIT[146]
1	0	0	1	0	0	1	1	INIT[147]
1	0	0	1	0	1	0	0	INIT[148]
1	0	0	1	0	1	0	1	INIT[149]
1	0	0	1	0	1	1	0	INIT[150]
1	0	0	1	0	1	1	1	INIT[151]
1	0	0	1	1	0	0	0	INIT[152]
1	0	0	1	1	0	0	1	INIT[153]
1	0	0	1	1	0	1	0	INIT[154]
1	0	0	1	1	0	1	1	INIT[155]
1	0	0	1	1	1	0	0	INIT[156]
1	0	0	1	1	1	0	1	INIT[157]
1	0	0	1	1	1	1	0	INIT[158]
1	0	0	1	1	1	1	1	INIT[159]
1	0	1	0	0	0	0	0	INIT[160]
1	0	1	0	0	0	0	1	INIT[161]
1	0	1	0	0	0	1	0	INIT[162]
1	0	1	0	0	0	1	1	INIT[163]
1	0	1	0	0	1	0	0	INIT[164]
1	0	1	0	0	1	0	1	INIT[165]
1	0	1	0	0	1	1	0	INIT[166]
1	0	1	0	0	1	1	1	INIT[167]
1	0	1	0	1	0	0	0	INIT[168]
1	0	1	0	1	0	0	1	INIT[169]
1	0	1	0	1	0	1	0	INIT[170]
1	0	1	0	1	0	1	1	INIT[171]

1	0	1	0	1	1	0	0	INIT[172]
1	0	1	0	1	1	0	1	INIT[173]
1	0	1	0	1	1	1	0	INIT[174]
1	0	1	0	1	1	1	1	INIT[175]
1	0	1	1	0	0	0	0	INIT[176]
1	0	1	1	0	0	0	1	INIT[177]
1	0	1	1	0	0	1	0	INIT[178]
1	0	1	1	0	0	1	1	INIT[179]
1	0	1	1	0	1	0	0	INIT[180]
1	0	1	1	0	1	0	1	INIT[181]
1	0	1	1	0	1	1	0	INIT[182]
1	0	1	1	0	1	1	1	INIT[183]
1	0	1	1	1	0	0	0	INIT[184]
1	0	1	1	1	0	0	1	INIT[185]
1	0	1	1	1	0	1	0	INIT[186]
1	0	1	1	1	0	1	1	INIT[187]
1	0	1	1	1	1	0	0	INIT[188]
1	0	1	1	1	1	0	1	INIT[189]
1	0	1	1	1	1	1	0	INIT[190]
1	0	1	1	1	1	1	1	INIT[191]
1	1	0	0	0	0	0	0	INIT[192]
1	1	0	0	0	0	0	1	INIT[193]
1	1	0	0	0	0	1	0	INIT[194]
1	1	0	0	0	0	1	1	INIT[195]
1	1	0	0	0	1	0	0	INIT[196]
1	1	0	0	0	1	0	1	INIT[197]
1	1	0	0	0	1	1	0	INIT[198]
1	1	0	0	0	1	1	1	INIT[199]
1	1	0	0	1	0	0	0	INIT[200]
1	1	0	0	1	0	0	1	INIT[201]

1	1	0	0	1	0	1	0	INIT[202]
1	1	0	0	1	0	1	1	INIT[203]
1	1	0	0	1	1	0	0	INIT[204]
1	1	0	0	1	1	0	1	INIT[205]
1	1	0	0	1	1	1	0	INIT[206]
1	1	0	0	1	1	1	1	INIT[207]
1	1	0	1	0	0	0	0	INIT[208]
1	1	0	1	0	0	0	1	INIT[209]
1	1	0	1	0	0	1	0	INIT[210]
1	1	0	1	0	0	1	1	INIT[211]
1	1	0	1	0	1	0	0	INIT[212]
1	1	0	1	0	1	0	1	INIT[213]
1	1	0	1	0	1	1	0	INIT[214]
1	1	0	1	0	1	1	1	INIT[215]
1	1	0	1	1	0	0	0	INIT[216]
1	1	0	1	1	0	0	1	INIT[217]
1	1	0	1	1	0	1	0	INIT[218]
1	1	0	1	1	0	1	1	INIT[219]
1	1	0	1	1	1	0	0	INIT[220]
1	1	0	1	1	1	0	1	INIT[221]
1	1	0	1	1	1	1	0	INIT[222]
1	1	0	1	1	1	1	1	INIT[223]
1	1	1	0	0	0	0	0	INIT[224]
1	1	1	0	0	0	0	1	INIT[225]
1	1	1	0	0	0	1	0	INIT[226]
1	1	1	0	0	0	1	1	INIT[227]
1	1	1	0	0	1	0	0	INIT[228]
1	1	1	0	0	1	0	1	INIT[229]
1	1	1	0	0	1	1	0	INIT[230]
1	1	1	0	0	1	1	1	INIT[231]

1	1	1	0	1	0	0	0	INIT[232]
1	1	1	0	1	0	0	1	INIT[233]
1	1	1	0	1	0	1	0	INIT[234]
1	1	1	0	1	0	1	1	INIT[235]
1	1	1	0	1	1	0	0	INIT[236]
1	1	1	0	1	1	0	1	INIT[237]
1	1	1	0	1	1	1	0	INIT[238]
1	1	1	0	1	1	1	1	INIT[239]
1	1	1	1	0	0	0	0	INIT[240]
1	1	1	1	0	0	0	1	INIT[241]
1	1	1	1	0	0	1	0	INIT[242]
1	1	1	1	0	0	1	1	INIT[243]
1	1	1	1	0	1	0	0	INIT[244]
1	1	1	1	0	1	0	1	INIT[245]
1	1	1	1	0	1	1	0	INIT[246]
1	1	1	1	0	1	1	1	INIT[247]
1	1	1	1	1	0	0	0	INIT[248]
1	1	1	1	1	0	0	1	INIT[249]
1	1	1	1	1	0	1	0	INIT[250]
1	1	1	1	1	0	1	1	INIT[251]
1	1	1	1	1	1	0	0	INIT[252]
1	1	1	1	1	1	0	1	INIT[253]
1	1	1	1	1	1	1	0	INIT[254]
1	1	1	1	1	1	1	1	INIT[255]

3.13.4. 参数描述

表 76 GTP_LUT8 参数

参数名	描述	默认值	设置值
INIT	输出配置参数	256'h0000000000000000_0000000000000000_0000000000000000_0000000000000000	0 ~ 2^256-1

3.13.5. 例化模板

```
GTP_LUT8
#(
.INIT      (256'h0000000000000000_0000000000000000_0000000000000000_0000000000000000)
)GTP_LUT8_inst (
.Z         (z),
.I0        (i0),
.I1        (i1),
.I2        (i2),
.I3        (i3),
.I4        (i4),
.I5        (i5),
.I6        (i6),
.I7        (i7)
);
```

3.14. GTP_LUTMUX4 使用说明

3.14.1. 支持器件

表 77 GTP_LUTMUX4 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.14.2. 功能介绍

GTP_LUTMUX4 是一个 4:1 多路选择器，其中 I0、I1、I2、I3 为数据输入，S1、S0 为选择信号，当{S1，S0}为 2’b00 时输出 I0，当{S1，S0}为 2’b01 时输出 I1，当{S1，S0}为 2’b10 时输出 I2，当{S1，S0}为 2’b11 时输出 I3，其结构框图如下所示：

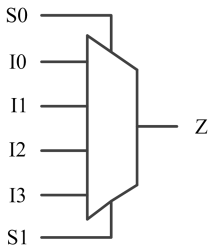


图 30 GTP_LUTMUX4 结构图

3.14.3. 端口描述

表 78 GTP_LUTMUX4 端口

端口名称	方向	功能描述
S0	输入	选择信号 0
S1	输入	选择信号 1
I0	输入	MUX 输入信号
I1	输入	MUX 输入信号
I2	输入	MUX 输入信号
I3	输入	MUX 输入信号
Z	输出	MUX 输出信号

真值表：

表 79 GTP_LUTMUX4 真值表

输入						输出
S1	S0	I3	I2	I1	I0	Z
0	0	?	?	?	0	0
0	0	?	?	?	1	1
0	1	?	?	0	?	0
0	1	?	?	1	?	1
1	0	?	0	?	?	0
1	0	?	1	?	?	1
1	1	0	?	?	?	0
1	1	1	?	?	?	1

3.14.4. 例化模板

```
GTP_LUTMUX4 GTP_LUTMUX4_inst
(
.S0    (s0),
.S1    (s1),
.I0    (i0),
.I1    (i1),
.I2    (i2),
```

```
.I3    (i3),
. Z    (z)
);
```

3.15. GTP_MUX2LUT6 使用说明

3.15.1. 支持器件

表 80 GTP_MUX2LUT6 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.15.2. 功能介绍

GTP_MUX2LUT6 是 GTP_LUT6 专用两输入选择器，其结构框图如下所示。其中 I0、I1 为数据输入，S 为选择信号，当 S 为 1’b0 时输出 I0，S 为 1’b1 时输出 I1。其结构框图如下所示。GTP_MUX2LUT6 的功能是选择两个 LUT5 的输出，形成一个 LUT6，因此必须与 LUT5 同时使用。

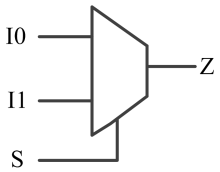


图 31 GTP_MUX2LUT6 结构图

3.15.3. 端口描述

表 81 GTP_MUX2LUT6 端口

端口名称	方向	功能描述
I0	输入	输入信号
I1	输入	输入信号
S	输入	选择信号
Z	输出	输出信号

3.15.4. 例化模板

```
GTP_MUX2LUT6 GTP_MUX2LUT6_inst (
    .Z(), // OUTPUT
    .I0(),// INPUT
```

```
    .I1(),// INPUT
    .S()    // INPUT
);
```

3.16. GTP_MUX2LUT7 使用说明

3.16.1. 支持器件

表 82 GTP_MUX2LUT7 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.16.2. 功能介绍

GTP_MUX2LUT7 是 GTP_LUT7 专用两输入选择器，其结构框图如下所示。其中 I0、I1 为数据输入，S 为选择信号，当 S 为 1’b0 时输出 I0，S 为 1’b1 时输出 I1。其结构框图如下所示。GTP_MUX2LUT7 的功能是选择两个 LUT6 的输出，形成一个 LUT7，因此必须与 LUT6 同时使用。

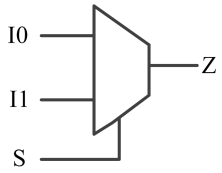


图 32 GTP_MUX2LUT7 结构图

3.16.3. 端口描述

表 83 GTP_MUX2LUT7 端口

端口名称	方向	功能描述
I0	输入	输入信号
I1	输入	输入信号
S	输入	选择信号
Z	输出	输出信号

3.16.4. 例化模板

```
GTP_MUX2LUT7 GTP_MUX2LUT7_inst (
    .Z(), // OUTPUT
    .I0(),// INPUT
    .I1(),// INPUT
```

```
    .S()    // INPUT
);
```

3.17. GTP_MUX2LUT8 使用说明

3.17.1. 支持器件

表 84 GTP_MUX2LUT8 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.17.2. 功能介绍

GTP_MUX2LUT8 是 GTP_LUT8 专用两输入选择器，其结构框图如下所示。其中 I0、I1 为数据输入，S 为选择信号，当 S 为 1’b0 时输出 I0，S 为 1’b1 时输出 I1。其结构框图如下所示。GTP_MUX2LUT8 的功能是选择两个 LUT7 的输出，形成一个 LUT8，因此必须与 LUT7 同时使用。

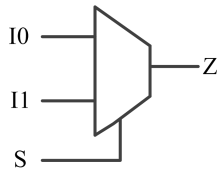


图 33 GTP_MUX2LUT8 结构图

3.17.3. 端口描述

表 85 GTP_MUX2LUT8 端口

端口名称	方向	功能描述
I0	输入	输入信号
I1	输入	输入信号
S	输入	选择信号
Z	输出	输出信号

3.17.4. 例化模板

```
GTP_MUX2LUT8 GTP_MUX2LUT8_inst (
    .Z(), // OUTPUT
    .I0(),// INPUT
    .I1(),// INPUT
    .S()    // INPUT
```

);

3.18. GTP_LUT5M 使用说明

3.18.1. 支持器件

表 86 GTP_LUT5M 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

3.18.2. 功能介绍

GTP_LUT5M 是一个 5 输入查找表，不同的 INIT 参数值对应 LUT5M 不同的功能，使用时必须指定对应的 INIT 参数。其结构框图如下所示：

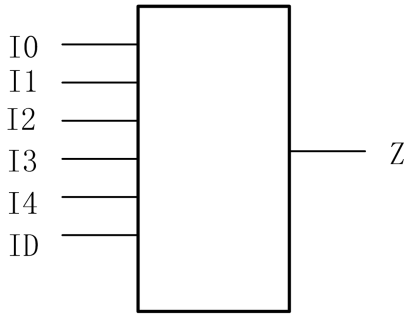


图 34 GTP_LUT5M 结构图

3.18.3. 端口描述

表 87 GTP_LUT5CARRY 端口

端口名称	方向	功能描述
Z	输出	输出信号
I0	输入	输入信号
I1	输入	输入信号
I2	输入	输入信号
I3	输入	输入信号
I4	输入	输入信号
ID	输入	输入信号

真值表：

表 88 GTP_LUT5M 真值表

输入						输出
I4	I3	I2	I1	I0	ID	Z
0	0	0	0	0/1	0	INIT[0]
0	0	0	0	0/1	1	INIT[1]
0	0	0	1	0/1	0	INIT[2]
0	0	0	1	0/1	1	INIT[3]
0	0	1	0	0/1	0	INIT[4]
0	0	1	0	0/1	1	INIT[5]
0	0	1	1	0/1	0	INIT[6]
0	0	1	1	0/1	1	INIT[7]
0	1	0	0	0/1	0	INIT[8]
0	1	0	0	0/1	1	INIT[9]
0	1	0	1	0/1	0	INIT[10]
0	1	0	1	0/1	1	INIT[11]
0	1	1	0	0/1	0	INIT[12]
0	1	1	0	0/1	1	INIT[13]
0	1	1	1	0/1	0	INIT[14]
0	1	1	1	0/1	1	INIT[15]
1	0	0	0	0	0/1	INIT[16]
1	0	0	0	1	0/1	INIT[17]
1	0	0	1	0	0/1	INIT[18]
1	0	0	1	1	0/1	INIT[19]
1	0	1	0	0	0/1	INIT[20]
1	0	1	0	1	0/1	INIT[21]
1	0	1	1	0	0/1	INIT[22]
1	0	1	1	1	0/1	INIT[23]
1	1	0	0	0	0/1	INIT[24]

1	1	0	0	1	0/1	INIT[25]
1	1	0	1	0	0/1	INIT[26]
1	1	0	1	1	0/1	INIT[27]
1	1	1	0	0	0/1	INIT[28]
1	1	1	0	1	0/1	INIT[29]
1	1	1	1	0	0/1	INIT[30]
1	1	1	1	1	0/1	INIT[31]

3.18.4. 参数描述

表 89 GTP_LUT5M 参数

参数名	描述	默认值	设置值
INIT	输出配置参数	32'h0000_0000	32'h0 ~ 32'hffff_ffff

3.18.5. 例化模板

```
GTP_LUT5M #(
        .INIT('b00000000000000000000000000000000)
) GTP_LUT5M_inst (
        .Z(), // OUTPUT
        .I0(),// INPUT
        .I1(),// INPUT
        .I2(),// INPUT
        .I3(),// INPUT
        .I4(),// INPUT
        .ID() // INPUT
    );
```

4. ROM&RAM 相关 GTP 使用说明

4.1. 总体介绍

ROM&RAM 相关 GTP 主要实现了各类寄存器。GTP_RAM 系列实现单/双输入端口简单寄存器。GTP_DRM 系列实现双口/简单双口/单端口的寄存器，并且实现三种不同的写模式，NW,TW 和 RBW 模式。GTP_FIFO 实现提示空满状态，且具有丢包重发功能的寄存器。

4.2. GTP_ROM32X1 使用说明

4.2.1. 支持器件

表 90 GTP_ROM32X1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

4.2.2. 功能介绍

GTP_ROM32X1 是一个地址深度为 32，数据位宽为 1bit 的存储 ROM。结构框图如下图所示。

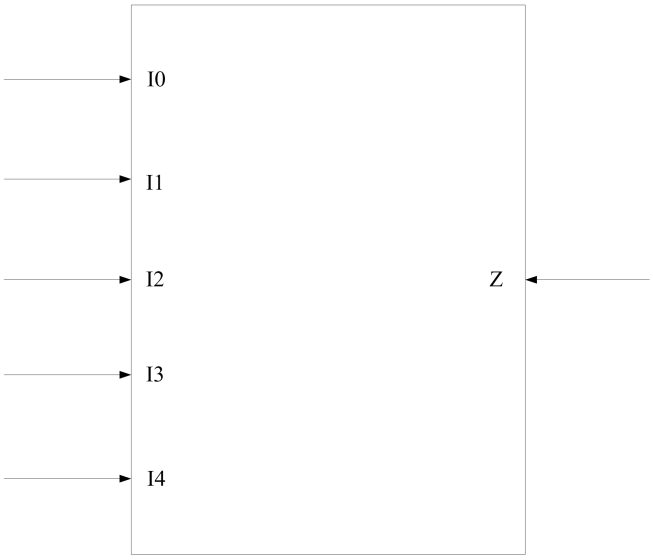


图 35 GTP_ROM32X1 结构框图

4.2.3. 端口描述

表 91 GTP_ROM32X1 端口列表

端口名称	方向	功能描述
I0	输入	ROM读地址addr[0]

I1	输入	ROM读地址addr[1]
I2	输入	ROM读地址addr[2]
I3	输入	ROM读地址addr[3]
I4	输入	ROM读地址addr[4]
Z	输出	读数据

4.2.4. 参数描述

表 92 GTP_ROM32X1 参数列表

参数名称	参数类型	设置值	默认值	功能描述
INIT	<binary>	32'h00000000~32'hffffffff	32'h00000000	ROM初始化配置参数

4.2.5. 详细功能说明

该 GTP 实现存储 ROM 功能。输入 I4~I0 构成读取数据地址，读取 ROM 初始配置参数的指定位的数值。

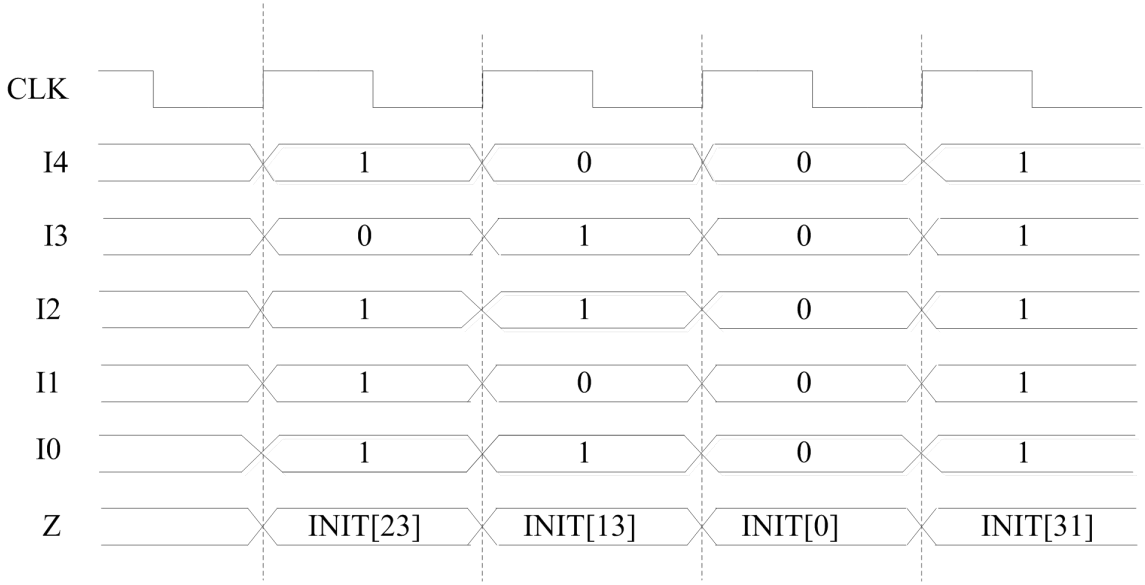


图 36 GTP_ROM32X1 波形示意图

4.2.6. 例化模板

```
GTP_ROM32X1    #(
    .INIT          (32'h00000000)
)
GTP_ROM32X1_inst(
    .I0                (    I0        ),
    .I1                (    II        ),
```

```
        .I2             (   I2          ),
        .I3             (   I3          ),
        .I4             (   I4          ),
        .Z              (   Z           )
    );
```

4.3. GTP_ROM64X1 使用说明

4.3.1. 支持器件

表 93 GTP_ROM64X1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

4.3.2. 功能介绍

GTP_ROM64X1 是一个地址深度为 64，数据位宽为 1bit 的存储 ROM。结构框图如下图所示。

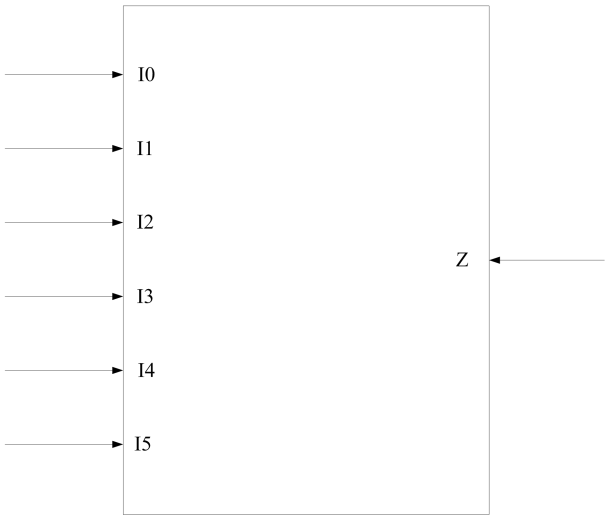


图 37 GTP_ROM64X1 结构框图

4.3.3. 端口描述

表 94 GTP_ROM64X1 端口列表

端口名称	方向	功能描述
I0	输入	ROM读地址addr[0]
I1	输入	ROM读地址addr[1]

I2	输入	ROM读地址addr[2]
I3	输入	ROM读地址addr[3]
I4	输入	ROM读地址addr[4]
I5	输入	ROM读地址addr[5]
Z	输出	读数据

4.3.4. 参数描述

表 95 GTP_ROM64X1 参数列表

参数名称	参数类型	设置值	默认值	功能描述
INIT	<binary>	64'h00000000_00000000~ 64'hffffffff_ffffffff	64'h00000000_00000000	ROM初始化配置参数

4.3.5. 详细功能说明

该 GTP 实现存储 ROM 功能。输入 I4~I0 构成读取数据地址，读取 ROM 初始配置参数的指定位的数值。

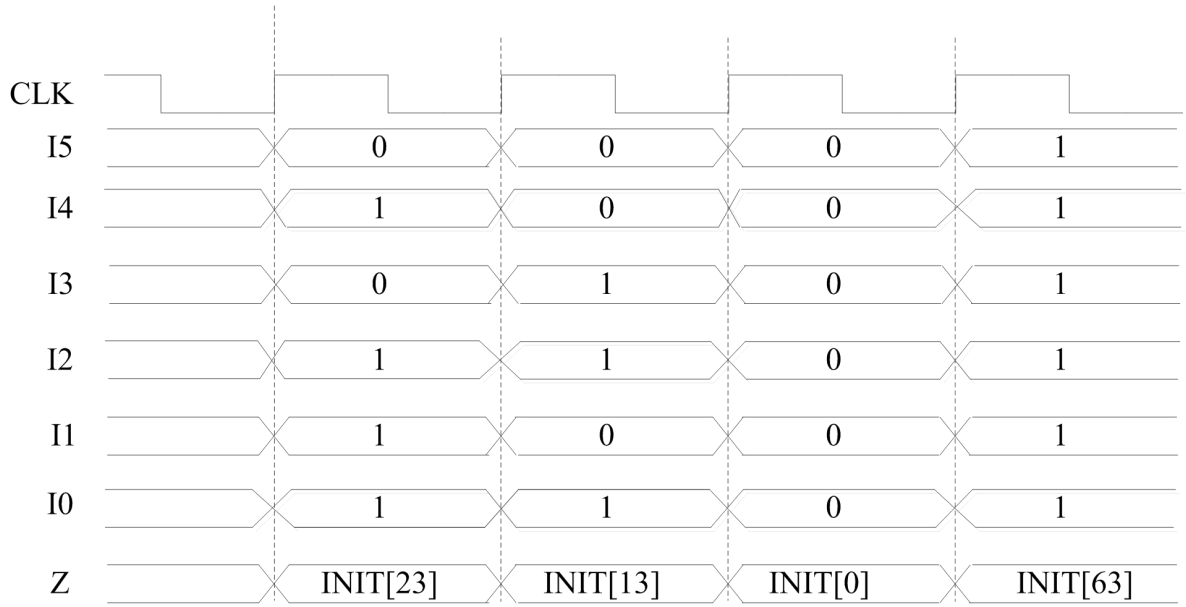


图 38 GTP_ROM64X1 波形示意图

4.3.6. 例化模板

```
GTP_ROM64X1#(
    .INIT           (64'h00000000_00000000)
```

```
)
GTP_ROM64X1_inst(
    .I0             (   I0      ),
    .I1             (   II      ),
    .I2             (   I2      ),
    .I3             (   I3      ),
    .I4             (   I4      ),
    .I5             (   I5      ),
    .Z              (   Z       )
);
```

4.4. GTP_ROM128X1 使用说明

4.4.1. 支持器件

表 96 GTP_ROM128X1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

4.4.2. 功能介绍

GTP_ROM128X1 是一个地址深度为 128，数据位宽为 1bit 的存储 ROM。结构框图如下图所示。

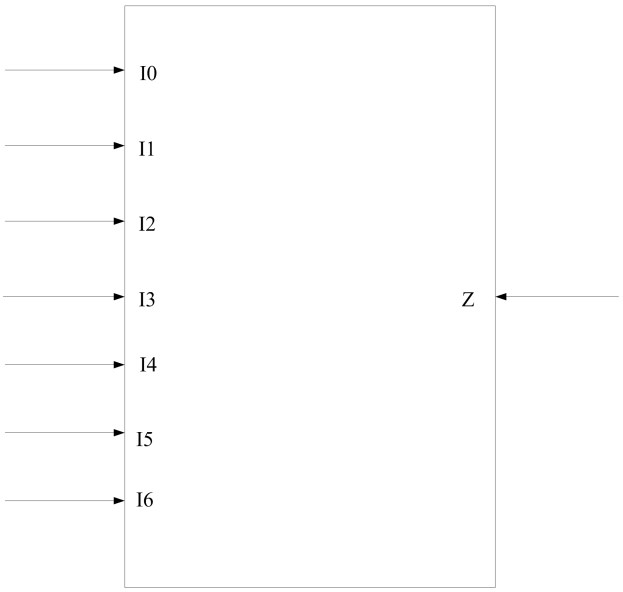


图 39 GTP_ROM128X1 结构框图

4.4.3. 端口描述

表 97 GTP_ROM128X1 端口列表

端口名称	方向	功能描述
I0	输入	ROM读地址addr[0]
I1	输入	ROM读地址addr[1]
I2	输入	ROM读地址addr[2]
I3	输入	ROM读地址addr[3]
I4	输入	ROM读地址addr[4]
I5	输入	ROM读地址addr[5]
I6	输入	ROM读地址addr[6]
Z	输出	读数据

4.4.4. 参数描述

表 98 GTP_ROM128X1 参数列表

参数名称	参数类型	设置值	默认值	功能描述
INIT	<binary>	128'h00000000_00000000_00000000_00000000~128'hffffffff_ffffffff_ffffffff_ffffffff	128'h00000000_00000000_00000000_00000000	ROM初始化配置参数

4.4.5. 详细功能说明

该 GTP 实现存储 ROM 功能。输入 I4~I0 构成读取数据地址，读取 ROM 初始配置参数的指定位的数值。

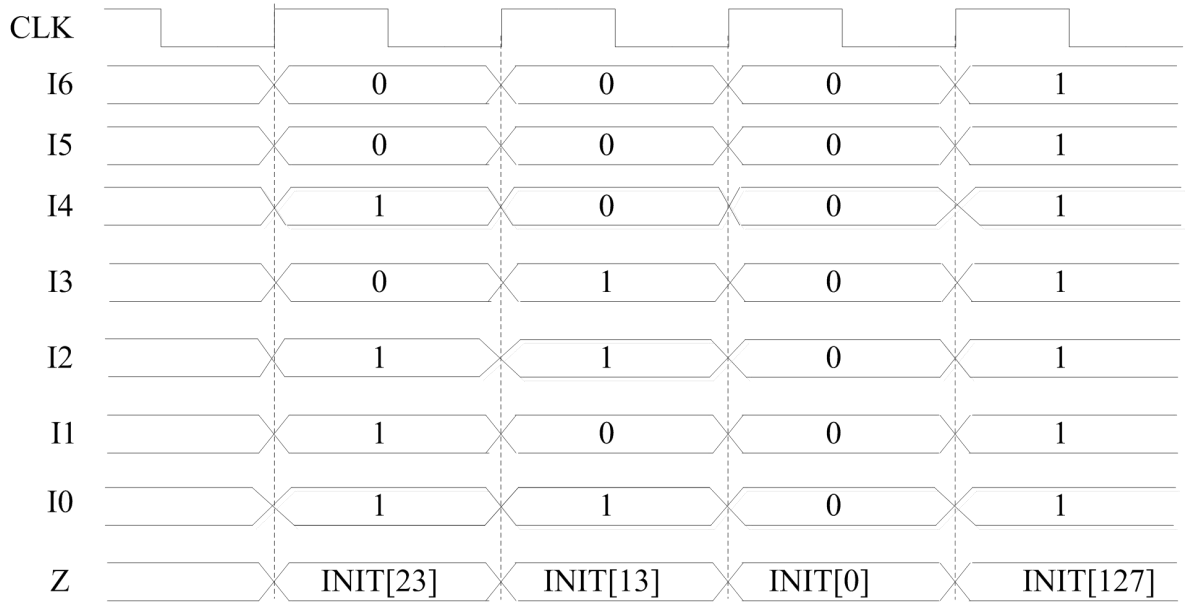


图 40 GTP_ROM128X1 波形示意图

4.4.6. 例化模板

```
GTP_ROM128X1    #(
    .INIT           (128'h00000000_00000000_00000000_00000000)
)
GTP_ROM128X1_inst(
    .I0             (   I0      ),
    .I1             (   II      ),
    .I2             (   I2      ),
    .I3             (   I3      ),
    .I4             (   I4      ),
    .I5             (   I5      ),
    .I6             (   I6      ),
    .Z              (   Z       )
);
```

4.5. GTP_ROM256X1 使用说明

4.5.1. 支持器件

表 99 GTP_ROM256X1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

4.5.2. 功能介绍

GTP_ROM256X1 是一个地址深度为 256，数据位宽为 1bit 的存储 ROM。结构框图如下图所示。

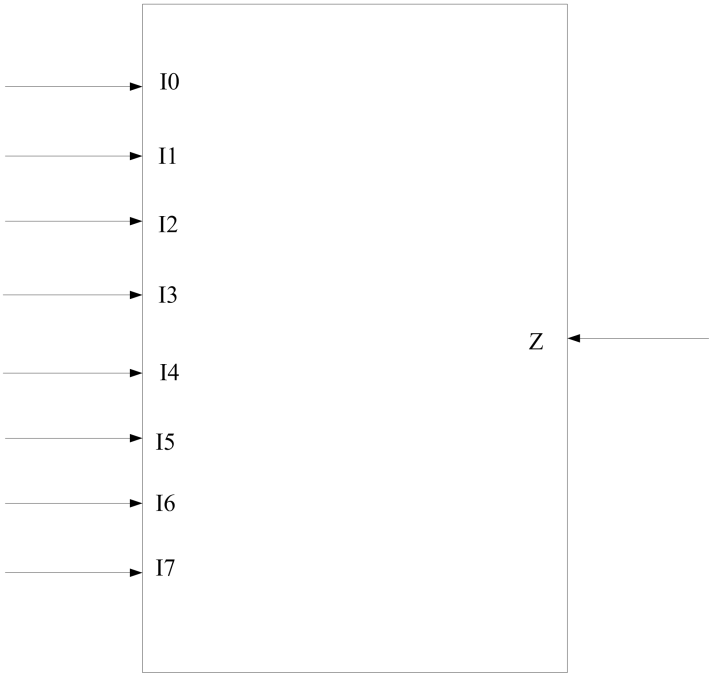


图 41 GTP_ROM256X1 结构框图

4.5.3. 端口描述

表 100 GTP_ROM256X1 端口列表

端口名称	方向	功能描述
I0	输入	ROM读地址addr[0]
I1	输入	ROM读地址addr[1]
I2	输入	ROM读地址addr[2]
I3	输入	ROM读地址addr[3]
I4	输入	ROM读地址addr[4]
I5	输入	ROM读地址addr[5]
I6	输入	ROM读地址addr[6]
I7	输入	ROM读地址addr[7]
Z	输出	读数据

4.5.4. 参数描述

表 101 GTP_ROM256X1 参数列表

参数名称	参数类型	设置值	默认值	功能描述
INIT	<binary>	256'h00000000_00000000_00000000_00000000_00000000_00000000_00000000_00000000~256'hffffffff_ffffffff_ffffffff_ffffffff_ffffffff_ffffffff_ffffffff_ffffffff	256'h00000000_00000000_00000000_00000000_00000000_00000000_00000000_00000000	ROM初始化配置参数

4.5.5. 详细功能说明

该 GTP 实现存储 ROM 功能。输入 I4~I0 构成读取数据地址，读取 ROM 初始配置参数的指定位的数值。

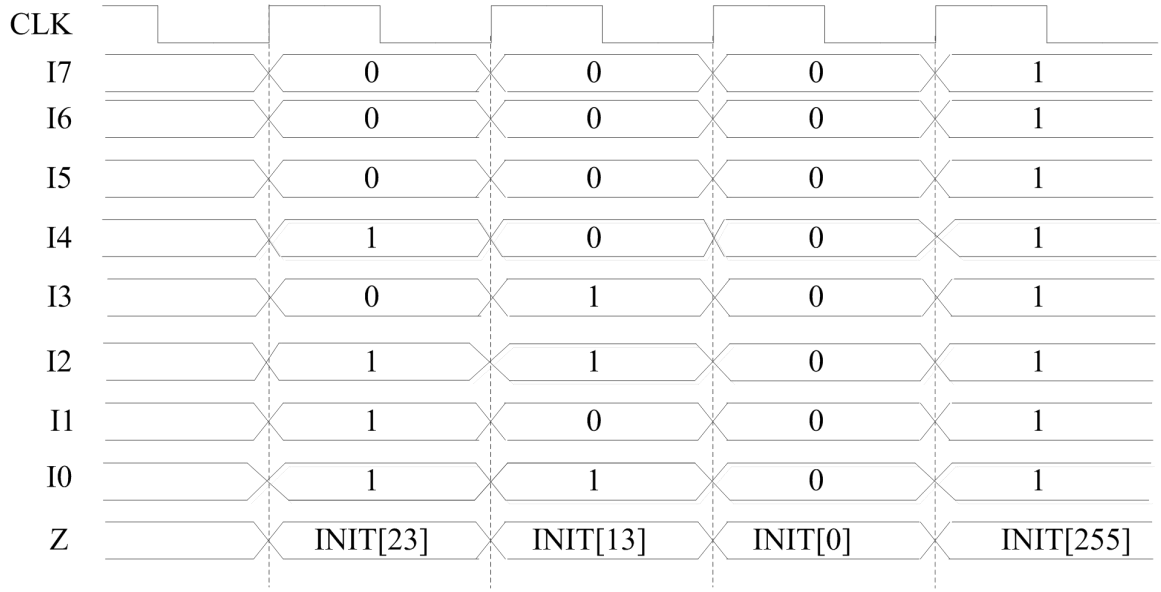


图 42 GTP_ROM256X1 波形示意图

4.5.6. 例化模板

```
GTP_ROM256X1    #(
    .INIT           (256'h00000000_00000000_00000000_00000000_00000000_00000000_00000000_00000000)
)
GTP_ROM256X1_inst(
    .I0             (  I0       ),
    .I1             (  I1       ),
    .I2             (  I2       ),
    .I3             (  I3       ),
    .I4             (  I4       ),
    .I5             (  I5       ),
    .I6             (  I6       ),
    .I7             (  I7       ),
    .Z          (  Z        )
```

);

4.6. GTP_RAM16X4SP 使用说明

4.6.1. 支持器件

表 102 GTP_RAM16X4SP 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	不支持	不支持	不支持	不支持

4.6.2. 功能介绍

GTP_RAM16X4SP 是一个数据位宽为 4，地址深度为 16 的单端口随机存储模块,读写地址相同。结构框图如下图所示。

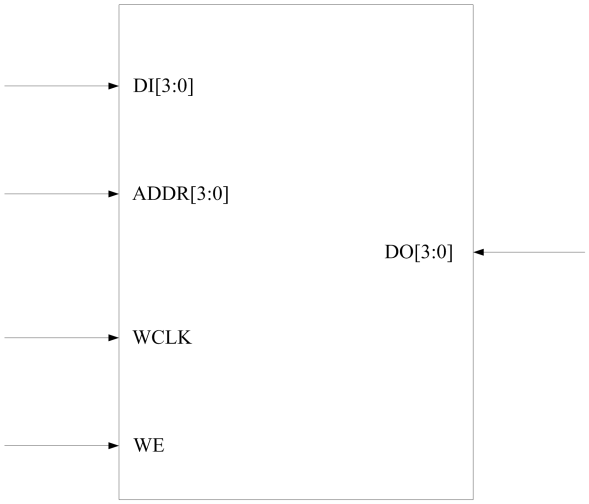


图 43 GTP_RAM16X4SP 结构框图

4.6.3. 端口描述

表 103 GTP_RAM16X4SP 端口列表

端口名称	方向	功能描述
DI	输入	写数据
ADDR	输入	读写地址
WCLK	输入	写时钟
WE	输入	写使能
DO	输出	读数据

4.6.4. 参数描述

表 104 GTP_RAM16X4SP 参数列表

参数名称	参数类型	设置值	默认值	功能描述
INIT_0	<binary>	16'h0000~16'hffff	16'h0000	Memory初始化配置参数
INIT_1	<binary>	16'h0000~16'hffff	16'h0000	Memory初始化配置参数
INIT_2	<binary>	16'h0000~16'hffff	16'h0000	Memory初始化配置参数
INIT_3	<binary>	16'h0000~16'hffff	16'h0000	Memory初始化配置参数

4.6.5. 详细功能说明

该 GTP 实现单口 RAM 功能，仅在 WE 为高电平时，进行写操作。当 WE 为高电平时，在 WCLK 上升沿时将输入数据 DI 写入当前地址 ADDR 中，并且将输入数据从 DO 端口读出。当 WE 为低电平时，在 WCLK 上升沿时将当前地址 ADDR 中的数据从 DO 端口读出，不进行写入操作。其波形图如下图所示。

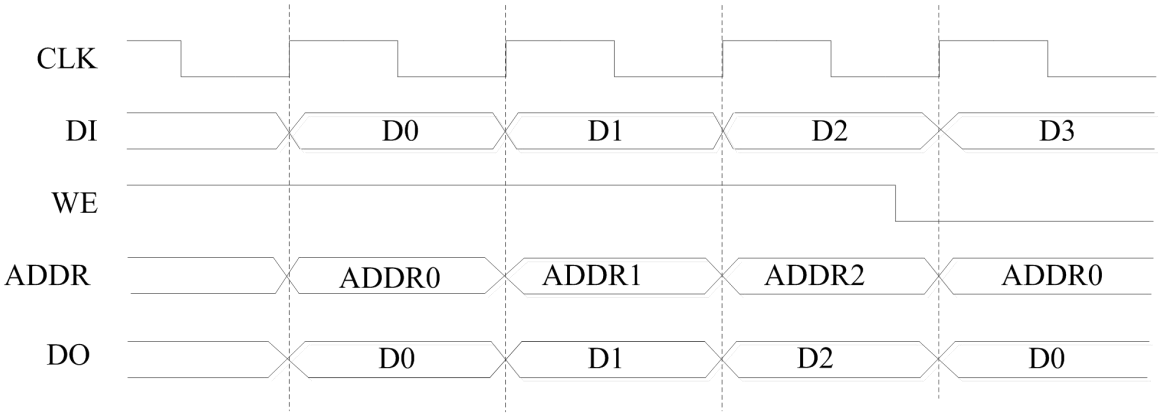


图 44 GTP_RAM16X4SP 波形示意图

4.6.6. 例化模板

```
GTP_RAM16X4SP  #(
    .INIT_0          (16'h0000),
    .INIT_1          (16'h0000),
    .INIT_2          (16'h0000),
    .INIT_3          (16'h0000)
)
GTP_RAM16X4SP_inst(
    .DO              (  DO     ),
    .DI              (  DI     ),
    .ADDR            (  ADDR   ),
    .WCLK            (  WCLK   ),
```

```
        .WE                ( WE )
    );
```

4.7. GTP_RAM16X4DP 使用说明

4.7.1. 支持器件

表 105 GTP_RAM16X4DP 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	不支持	不支持	不支持	不支持

4.7.2. 功能介绍

GTP_RAM16X4DP 是一个数据位宽为 4，地址深度为 16 的双端口随机存储模块,读写地址分别由两个独立的端口输入。结构框图如下图所示。

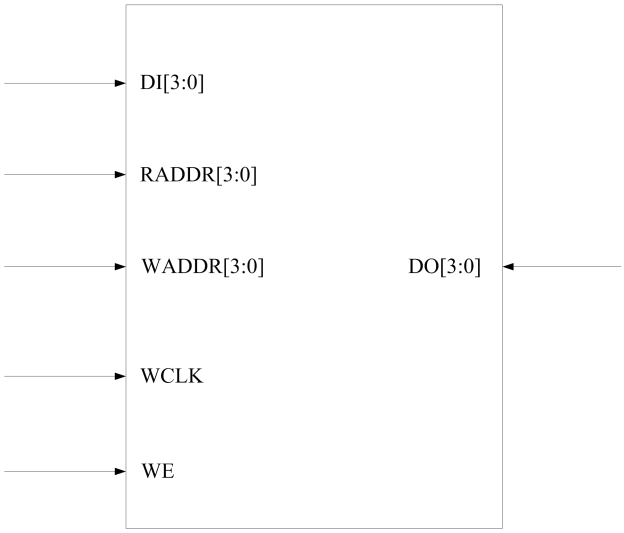


图 45 GTP_RAM16X4DP 结构框图

4.7.3. 端口描述

表 106 GTP_RAM16X4DP 端口列表

端口名称	方向	功能描述
DI	输入	写数据
RADDR	输入	读地址
WADDR	输入	写地址
WCLK	输入	写时钟

WE	输入	写使能
DO	输出	读数据

4.7.4. 参数描述

表 107 GTP_RAM16X4DP 参数列表

参数名称	参数类型	设置值	默认值	功能描述
INIT_0	<binary>	16'h0000~16'hffff	16'h0000	Memory初始化配置参数
INIT_1	<binary>	16'h0000~16'hffff	16'h0000	Memory初始化配置参数
INIT_2	<binary>	16'h0000~16'hffff	16'h0000	Memory初始化配置参数
INIT_3	<binary>	16'h0000~16'hffff	16'h0000	Memory初始化配置参数

4.7.5. 详细功能说明

该 GTP 实现双口 RAM 功能，读写地址互相独立。仅当 WE 为高电平时，将输入数据写入 RAM。当 WE 为高电平时，在 WCLK 上升沿时将输入数据 DI 写入写地址 WADDR 中，将读地址 RADDR 中的数据从 DO 端口读出。当 WE 为低电平时，在 WCLK 上升沿时将读地址 RADDR 中的数据从 DO 端口读出，不进行写入操作。

读写为异步进行，在同一时刻读写地址不能相同，防止读写冲突。其波形图如下图所示。

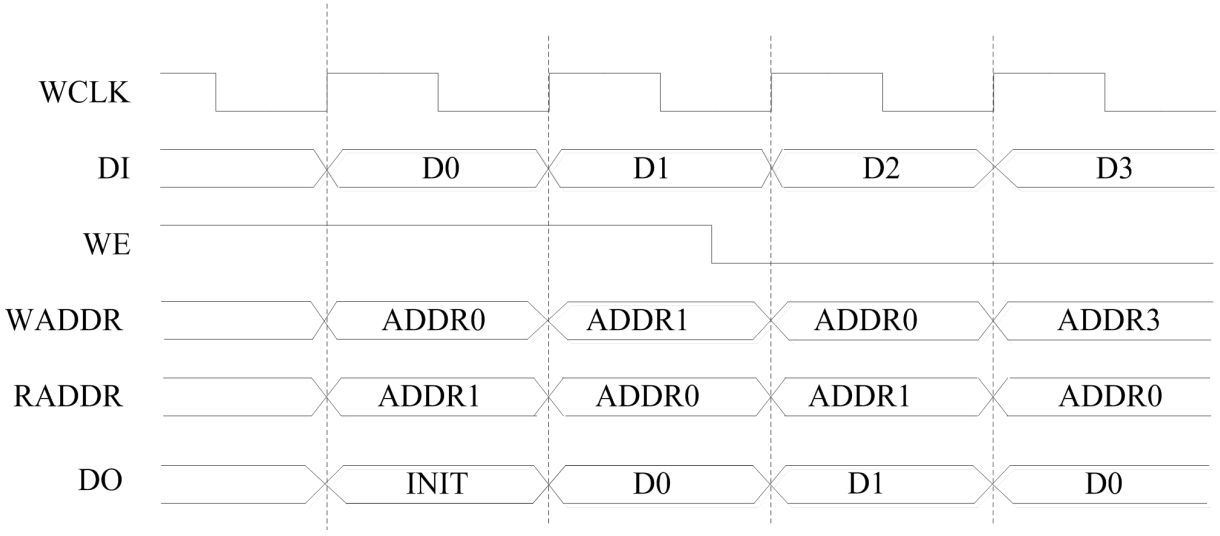


图 46 GTP_RAM16X4DP 波形示意图

4.7.6. 例化模板

```
GTP_RAM16X4DP    #(
        .INIT_0             (16'h0000),
```

```
        .INIT_1         (16'h0000),
        .INIT_2         (16'h0000),
        .INIT_3         (16'h0000)
    )
    GTP_RAM16X4DP_inst(
        .DO             (  DO     ),
        .DI             (  DI     ),
        .RADDR          (  RADDR  ),
        .WADDR          (  WADDR  ),
        .WCLK           (  WCLK   ),
        .WE             (  WE     )
    );
```

4.8. GTP_RAM16X1SP 使用说明

4.8.1. 支持器件

表 108 GTP_RAM16X1SP 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

4.8.2. 功能介绍

GTP_RAM16X1SP 是一个数据位宽为 1，地址深度为 16 的单端口随机存储模块,读写地址相同。结构框图如下图所示。

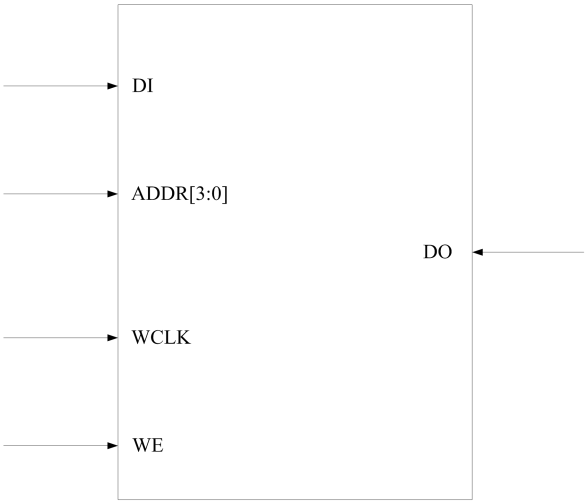


图 47 GTP_RAM16X1SP 结构框图

4.8.3. 端口描述

表 109 GTP_RAM16X1SP 端口列表

端口名称	方向	功能描述
DI	输入	写数据
ADDR	输入	读写地址
WCLK	输入	写时钟
WE	输入	写使能
DO	输出	读数据

4.8.4. 参数描述

表 110 GTP_RAM16X4SP 参数列表

参数名称	参数类型	设置值	默认值	功能描述
INIT	<binary>	16'h0000~16'hffff	16'h0000	Memory初始化配置参数

4.8.5. 详细功能说明

该 GTP 实现单口 RAM 功能，仅在 WE 为高电平时，进行写操作。当 WE 为高电平时，在 WCLK 上升沿时将输入数据 DI 写入当前地址 ADDR 中，并且将输入数据从 DO 端口读出。当 WE 为低电平时，在 WCLK 上升沿时将当前地址 ADDR 中的数据从 DO 端口读出，不进行写入操作，其波形图如下图所示。

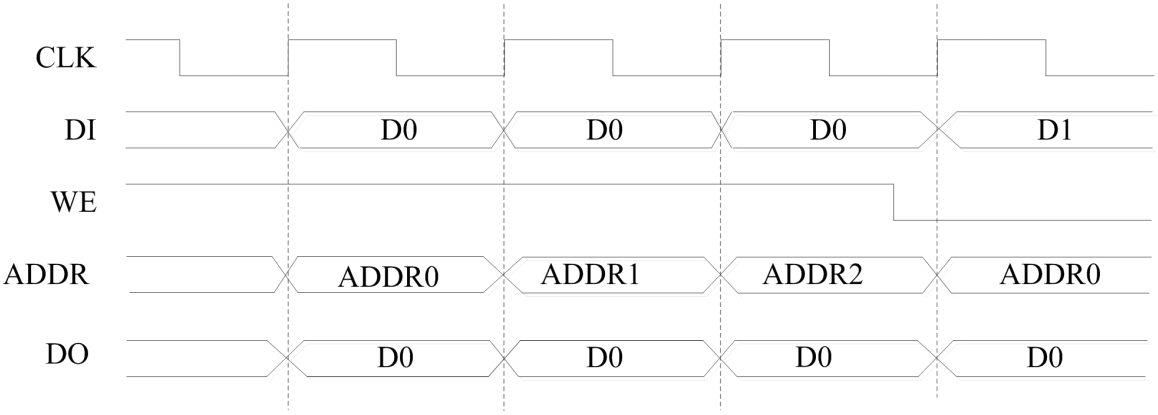


图 48 GTP_RAM16X1SP 波形示意图

4.8.6. 例化模板

```
GTP_RAM16X1SP#(
    .INIT           (16'h0000)
)
GTP_RAM16X1SP_inst(
    .DO             (  DO     ),
    .DI             (  DI     ),
```

```
        .ADDR       (   ADDR    ),
        .WCLK       (   WCLK    ),
        .WE         (   WE      )
    );
```

4.9. GTP_RAM16X1DP 使用说明

4.9.1. 支持器件

表 111 GTP_RAM16X1DP 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

4.9.2. 功能介绍

GTP_RAM16X1DP 是一个数据位宽为 1，地址深度为 16 的双端口随机存储模块,读写地址分别由两个独立的端口输入。结构框图如下图所示。

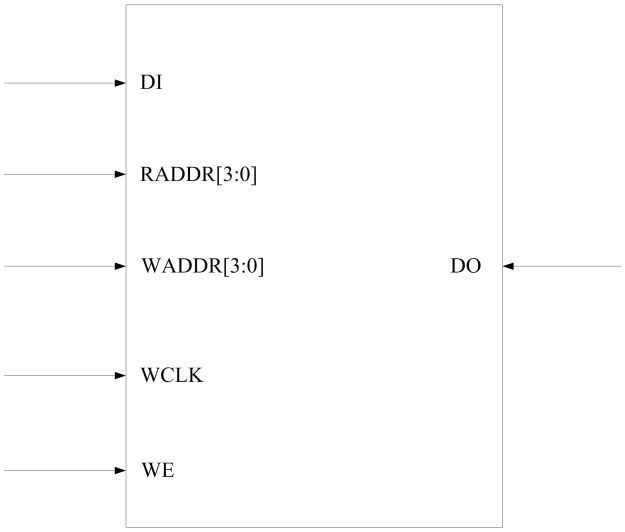


图 49 GTP_RAM16X1DP 结构框图

4.9.3. 端口描述

表 112 GTP_RAM16X1DP 端口列表

端口名称	方向	功能描述
DI	输入	写数据
RADDR	输入	读地址
WADDR	输入	写地址

WCLK	输入	写时钟
WE	输入	写使能
DO	输出	读数据

4.9.4. 参数描述

表 113 GTP_RAM16X1DP 参数列表

参数名称	参数类型	设置值	默认值	功能描述
INIT	<binary>	16’h0000~16’hffff	16’h0000	Memory初始化配置参数

4.9.5. 详细功能说明

该 GTP 实现双口 RAM 功能，读写地址互相独立。仅当 WE 为高电平时，将输入数据写入 RAM。当 WE 为高电平时，在 WCLK 上升沿时将输入数据 DI 写入写地址 WADDR 中，将读地址 RADDR 中的数据从 DO 端口读出。当 WE 为低电平时，在 WCLK 上升沿时将读地址 RADDR 中的数据从 DO 端口读出，不进行写入操作。

读写为异步进行，在同一时刻读写地址不能相同，防止读写冲突。其波形图如下图所示。

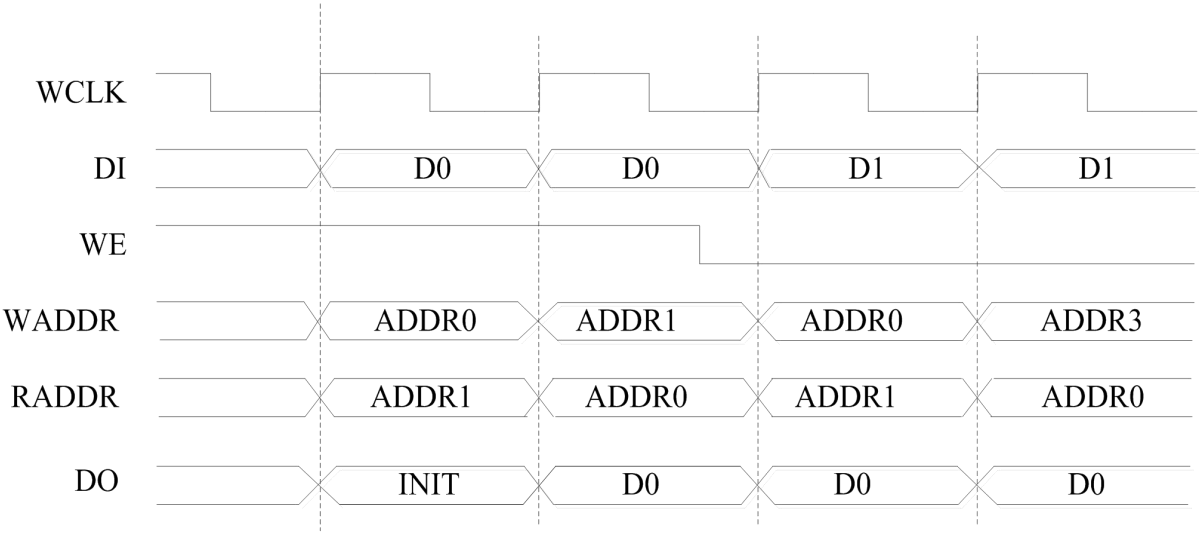


图 50 GTP_RAM16X1DP 波形示意图

4.9.6. 例化模板

```
GTP_RAM16X1DP #(
    .INIT           (16'h0000)
)
GTP_RAM16X1DP_inst(
    .DO             (   DO      ),
    .DI             (   DI      ),
```

```
        .RADDR          (   RADDR ),
        .WADDR          (   WADDR ),
        .WCLK           (   WCLK  ),
        .WE             (   WE    )
    );
```

4.10. GTP_RAM32X1SP 使用说明

4.10.1. 支持器件

表 114 GTP_RAM32X1SP 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	不支持	支持	支持	支持	支持

4.10.2. 功能介绍

GTP_RAM32X1SP 是一个数据位宽为 1，地址深度为 32 的双端口随机存储模块,读写地址分别由两个独立的端口输入。结构框图如下图所示。

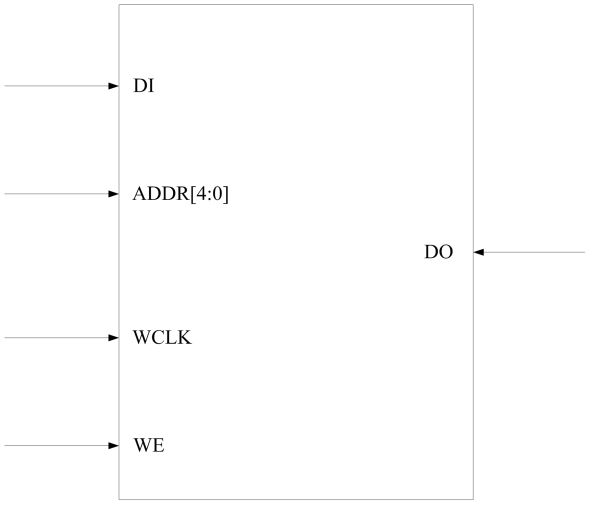


图 51 GTP_RAM32X1SP 结构框图

4.10.3. 端口描述

表 115 GTP_RAM32X1SP 端口列表

端口名称	方向	功能描述
DI	输入	写数据
ADDR	输入	写地址
WCLK	输入	写时钟

WE	输入	写使能
DO	输出	读数据

4.10.4. 参数描述

表 116 GTP_RAM32X1SP 参数列表

参数名称	参数类型	设置值	默认值	功能描述
INIT	<binary>	32'h00000000~32'hffffffff	32'h00000000	Memory初始化配置参数

4.10.5. 详细功能说明

该 GTP 实现单口 RAM 功能，仅在 WE 为高电平时，进行写操作。当 WE 为高电平时，在 WCLK 上升沿时将输入数据 DI 写入当前地址 ADDR 中，并且将输入数据从 DO 端口读出。当 WE 为低电平时，在 WCLK 上升沿时将当前地址 ADDR 中的数据从 DO 端口读出，不进行写入操作，其波形图如下图所示。

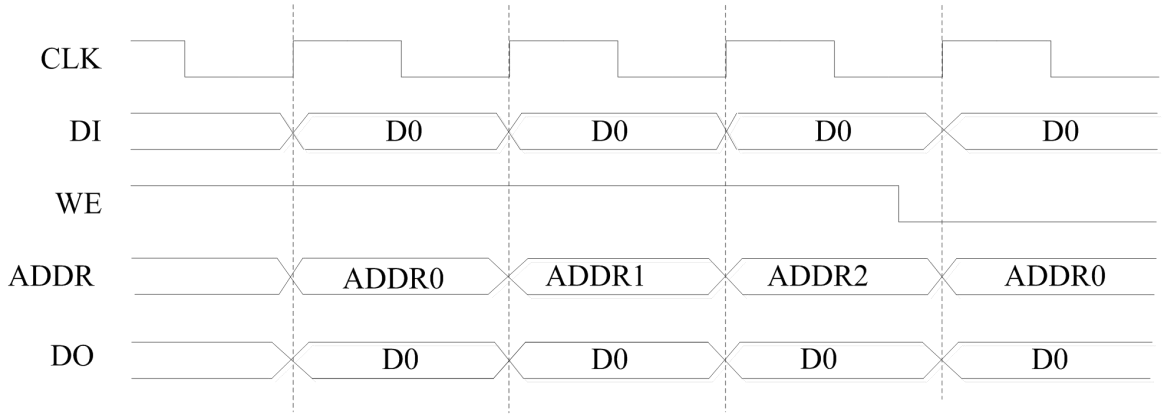


图 52 GTP_RAM32X1SP 波形示意图

4.10.6. 例化模板

```
GTP_RAM32X1SP#(
    .INIT           (32'h00000000)
)
GTP_RAM32X1SP_inst(
    .DO             (  DO     ),
    .DI             (  DI     ),
    .ADDR           (  ADDR   ),
    .WCLK           (  WCLK   ),
    .WE             (  WE     )
```

);

4.11. GTP_RAM32X1DP 使用说明

4.11.1. 支持器件

表 117 GTP_RAM32X1DP 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	不支持	支持	支持	支持	支持

4.11.2. 功能介绍

GTP_RAM32X1DP 是一个数据位宽为 1，地址深度为 32 的双端口随机存储模块,读写地址分别由两个独立的端口输入。结构框图如下图所示。

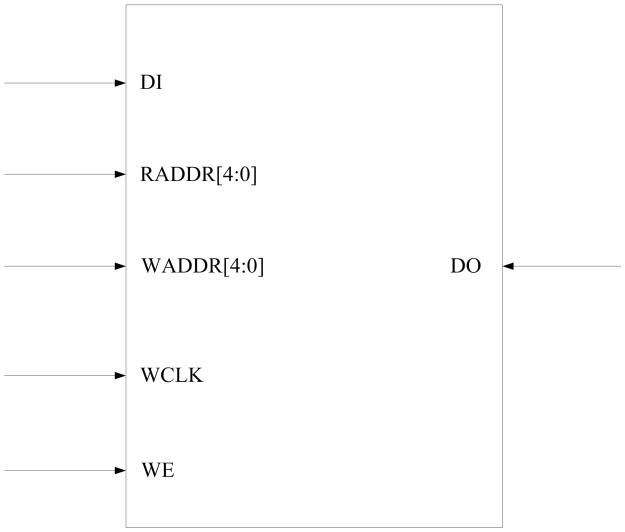


图 53 GTP_RAM32X1DP 结构框图

4.11.3. 端口描述

表 118 GTP_RAM32X1DP 端口列表

端口名称	方向	功能描述
DI	输入	写数据
RADDR	输入	读地址
WADDR	输入	写地址

WCLK	输入	写时钟
WE	输入	写使能
DO	输出	读数据

4.11.4. 参数描述

表 119 GTP_RAM32X1DP 参数列表

参数名称	参数类型	设置值	默认值	功能描述
INIT	<binary>	32'h00000000~32'hffffffff	32'h00000000	Memory初始化配置参数

4.11.5. 详细功能说明

该 GTP 实现双口 RAM 功能，读写地址互相独立。仅当 WE 为高电平时，将输入数据写入 RAM。当 WE 为高电平时，在 WCLK 上升沿时将输入数据 DI 写入写地址 WADDR 中，将读地址 RADDR 中的数据从 DO 端口读出。当 WE 为低电平时，在 WCLK 上升沿时将读地址 RADDR 中的数据从 DO 端口读出，不进行写入操作。

读写为异步进行，在同一时刻读写地址不能相同，防止读写冲突。其波形图如下图所示。

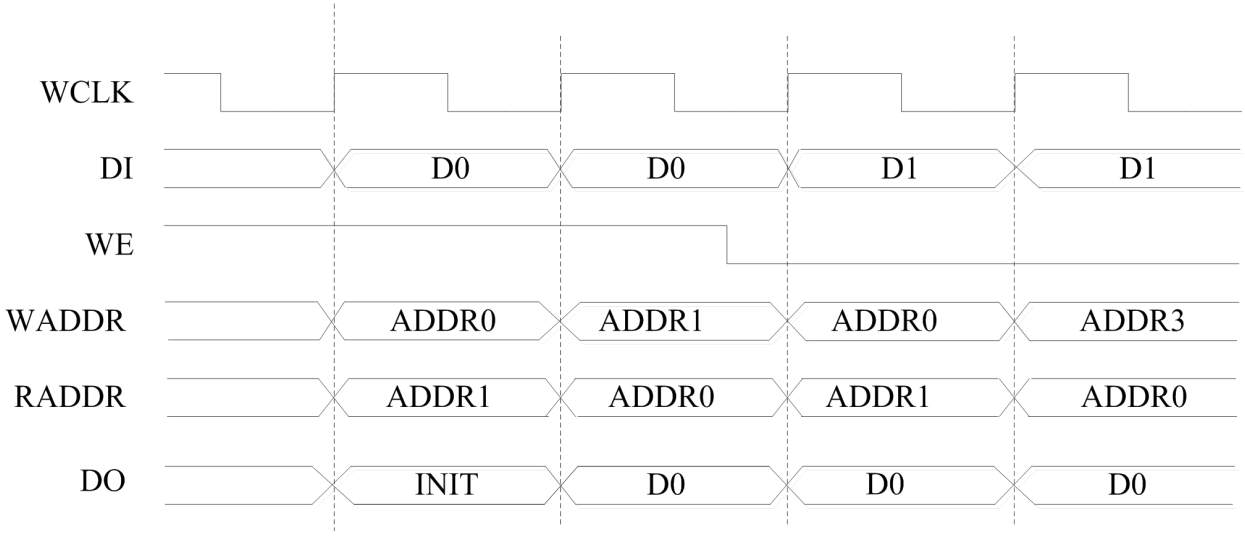


图 54 GTP_RAM32X1DP 波形示意图

4.11.6. 例化模板

```
GTP_RAM32X1DP  #(
.INIT                (32'h00000000)
```

```
    )
    GTP_RAM32X1DP_inst(
        .DO             (   DO      ),
        .DI             (   DI      ),
        .RADDR          (   RADDR   ),
        .WADDR          (   WADDR   ),
        .WCLK           (   WCLK    ),
        .WE             (   WE      )
    );
```

4.12. GTP_DRM9K 使用说明

4.12.1. 支持器件

表 120 GTP_DRM9K 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

4.12.2. 功能介绍

GTP_DRM9K 有 9K bits 存储单元，支持多种工作模式，包括双口（DP，True Dual Port）RAM，简单双口（SDP，Simple Dual Port）RAM，单口（SP，Single Port）RAM 或 ROM 模式。DRM 支持可配置的数据位宽，并在 DP RAM 和 SDP RAM 模式下支持双端口混合数据位宽。

DP RAM 模块有两个独立的端口 A、B，均可以独立进行读写操作，即可同时进行读操作，写操作，或一端口进行读操作一端口进行写操作。两端口均支持不同的时钟，且位宽可以独立设置。不可同时通过两个端口对同一地址进行读写操作，否则将引起冲突.

SDP RAM 中，A 端口固定为写入端口，B 端口固定为读出端口，两个端口支持不同时钟，且均有独立的位宽设置。不可同时通过两个端口对同一地址进行读写操作，否则将引起冲突。

SP RAM 模式下，可以对 DRM 的两个端口分别进行独立操作。不可同时通过两个端口对同一地址进行读写操作，否则将引起冲突。

DRM 可以配置为 ROM，通过配置接口可以初始化 ROM 内容。由于没有内建逻辑去控制 ROM 模式，需要 PDS 软件禁止写操作。

DRM 的端口写操作支持 Normal Write mode（NW），Transparent Write mode（TW），Read before Write mode（RBW）三种模式。

所有端口均默认为高电平有效。

GTP_DRM9K 的结构框图见下图。

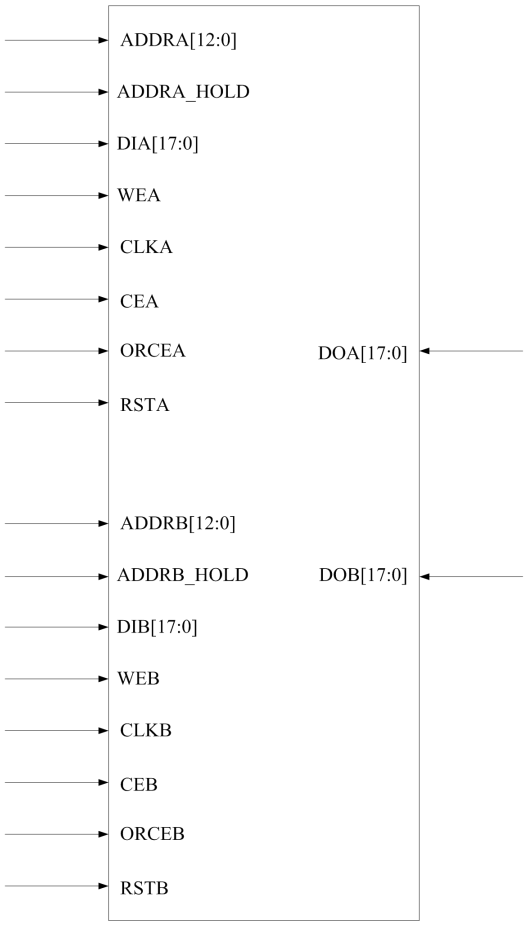


图 55 GTP_DRM9K 结构框图

4.12.3. 端口描述

表 121 GTP_DRM9K 端口列表

端口名称	方向	功能描述
ADDRA	输入	A端口输入地址
ADDRA_HOLD	输入	A端口地址输入选择
DIA	输入	A端口数据输入
WEA	输入	A端口写使能
CLKA	输入	A端口时钟
CEA	输入	A端口时钟使能
ORCEA	输入	A端口输出寄存器使能
RSTA	输入	A端口数据寄存器复位

端口名称	方向	功能描述
DOA	输出	A端口数据输出
ADDRB	输入	B端口输入地址
ADDRB_HOLD	输入	B端口地址输入选择
DIB	输入	B端口数据输入
WEB	输入	B端口写使能
CLKB	输入	B端口时钟
CEB	输入	B端口时钟使能
ORCEB	输入	B端口输出寄存器使能
RSTB	输入	B端口数据寄存器复位
DOB	输出	B端口数据输出

4.12.4. 参数描述

表 122 GTP_DRM9K 参数列表

参数名	描述	设置值
DATA_WIDTH_A	A端口数据最大位宽	1、2、4、8、16、32、9、18、36
DATA_WIDTH_B	B端口数据最大位宽	1、2、4、8、16、32、9、18、36
WRITE_MODE_A	A端口写模式	"NORMAL_WRITE" "TRANSPARENT_WRITE" "READ_BEFORE_WRITE"
WRITE_MODE_B	B端口写模式	"NORMAL_WRITE" "TRANSPARENT_WRITE" "READ_BEFORE_WRITE"
DOA_REG	A端口输出寄存	0：不使能输出寄存 1：使能输出寄存
DOB_REG	B端口输出寄存	0：不使能输出寄存 1：使能输出寄存

RST_TYPE	复位模式选择	"SYNC"：同步复位 "ASYNC"：异步复位 "ASYNC_SYNC_RELEASE"：异步复位同步释放
RAM_MODE	RAM模式选择	"TRUE_DUAL_PORT"：双口RAM "SIMPLE_DUAL_PORT"：简单双口RAM "SINGLE_PORT"：单口RAM ROM：ROM
GRS_EN	全局复位使能信号（芯片内部）	"FALSE":不使能全局复位； "TRUE":使能全局复位。
DOA_REG_CLKINV	A端口输出寄存器时钟翻转	0：时钟不翻转 1：时钟翻转
DOB_REG_CLKINV	B端口输出寄存器时钟翻转	0：时钟不翻转 1：时钟翻转
INIT_00 INIT_01 INIT_02 … INIT_1F	RAM初始化配置参数	0 ~2^288-1
INIT_FILE	初始化文件	"NONE"：不指定初始化文件，则默认初始化数据全为零； "XXX"：XXX表示具体初始化文件路径
BLOCK_X	RAM18K级联时数据级联坐标	视器件DRM个数而定
BLOCK_Y	RAM18K级联时地址级联坐标	视器件DRM个数而定
RAM_DATA_WIDTH	RAM级联后的数据最大位宽	视器件DRM个数而定
RAM_ADDR_WIDTH	RAM级联后的地址最大位宽	视器件DRM个数而定

INIT_FORMAT	初始化文件格式	"BIN"：二进制 "HEX"：十六进制
SIM_DEVICE	支持器件	"PGL22G"：PGL22G "LOGOS"：PGL系列后续器件

注：不支持以下条件同时满足情况：

SIM_DEVICE = “PGL22G”

RAM_MODE = “TRUE_DUAL_PORT”

WRITE_MODE_A = “READ_BEFORE_WRITE”

WRITE_MODE_B = “READ_BEFORE_WRITE

4.12.5. 详细功能说明

4.12.5.1 写模式---NW 模式

NW 模式是 Normal Write 模式，为默认模式。当写使能信号 WEA 为高电平时，不进行读操作，将输入数据 DIA 写入当前地址。当写使能信号 WEA 为低电平时，将当前地址 ADDRA 所保存的数据读出，并在下一个时钟周期从输出端口 DOA 输出。其波形图如下图所示。

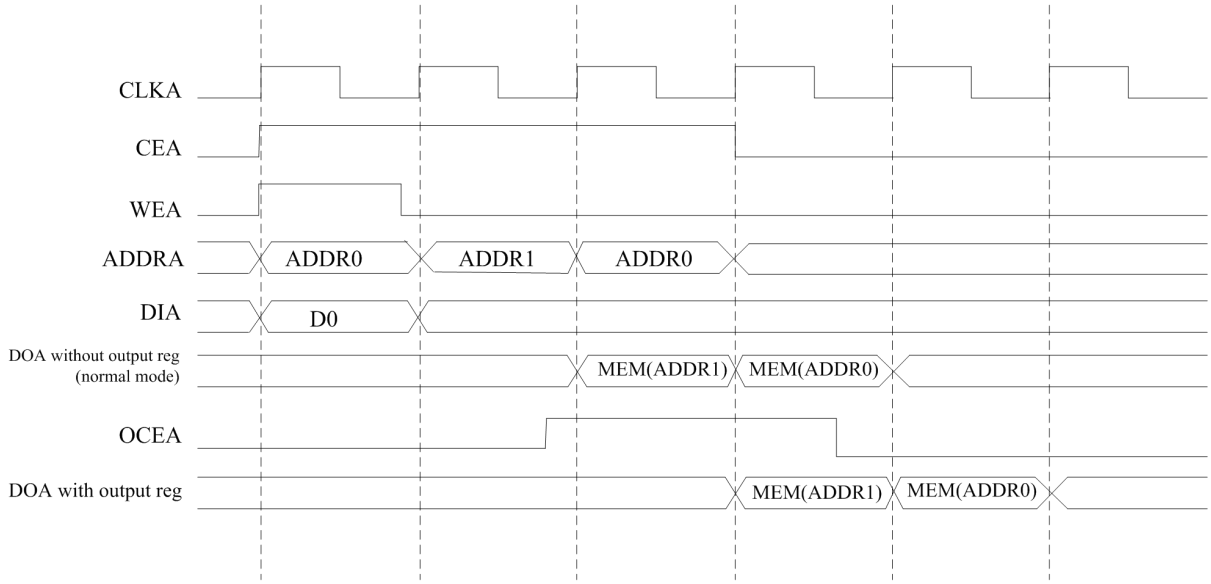


图 56 GTP_DRM9K NW 模式波形示意图

注：1.DRM 的端口写操作模式仅适用于具有读写功能的端口，即 DP RAM 的 A/B 端口与 SP RAM 的端口。

2.SDP 模式不支持读写模式设置，在使用时禁止设置成 TW 或 RBW 模式，只能设置为默认的 NW 模式，否则 DRM 将进入非正常的写操作状态。

4.12.5.2 写模式---TW 模式

TW 模式中，在写使能信号 WEA 为高电平时，输入数据 DIA 写入地址 ADDRA 的同时，将在下一个时钟周期时直接从输出端口 DOA 读出。使能信号 WEA 为低电平时，将当前地址 ADDRA 所保存的数据读出，并在下一个时钟周期从输出端口 DOA 输出。其波形图如下图所示。

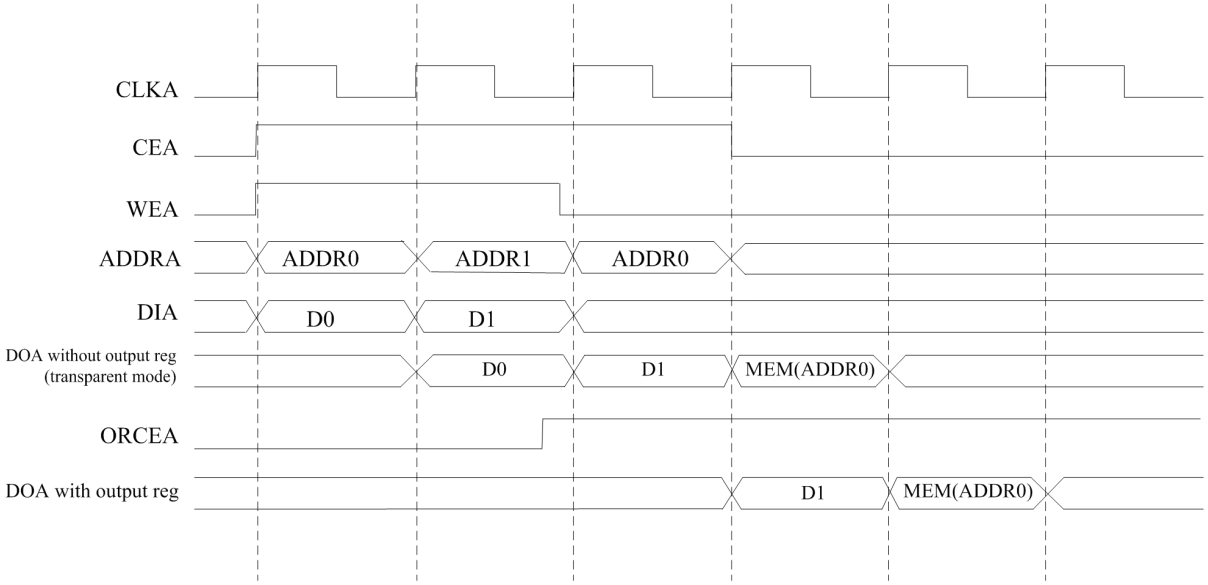


图 57 GTP_DRM9K TW 模式波形示意图

4.12.5.3 写模式---RBW 模式

在 RBW 模式中，当写使能信号 WEA 为高电平时，将首先读出地址 ADDRA 所引的原数据,在写操作的下一个时钟周期输出到输出端口，再将输入数据 DIA 写入地址 ADDRA 中，覆盖原数据。使能信号 WEA 为低电平时，将当前地址 ADDRA 所保存的数据读出，并在下一个时钟周期从输出端口 DOA 输出。其波形图如下图所示。

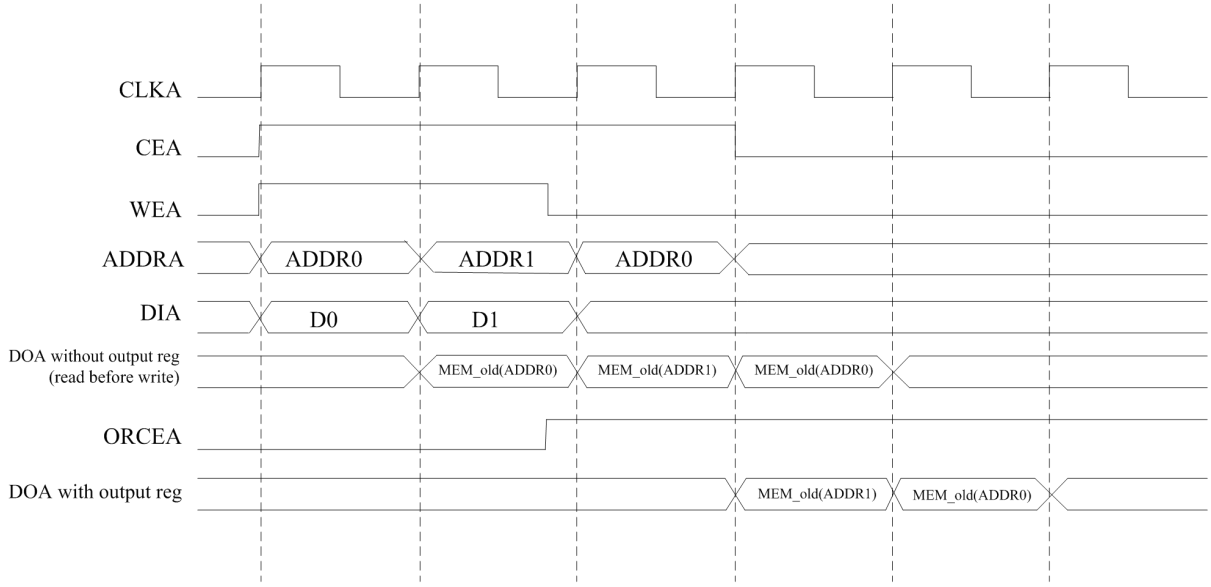


图 58 GTP_DRM9K RBW 模式波形示意图

4.12.5.4 SP RAM 的 TW 和 RBW 模式的实现

DRM 无法通过直接配置实现 32/36 bit SP 模式下的 TW（Transparent Write）和 RBW（Read before Write）模式（在 GTP 配置为 32/36 bit SP 模式时禁止设置读写模式为 TW 或 RBW 模式，否则 DRM 将进入非正常的写操作状态），需要额外的配置，具体实现方式如下：

a. Port A 和 Port B 同时配置为 16/18 bit DP 模式（分别针对 32/36 bit SP）；

b. Port A 和 Port B 配置为相同的写模式；其他配置位也需要相同。

c. Port A 和 Port B 的 CE，WE，ADDR，ORCE，RST，CLK 等控制信号分别并联起来；

d. DIA、DIB 并联作为数据输入；DOA、DOB 并联作为数据输出；

e. Port A 和 Port B 地址输入端口需同时接地址输入；ADDRA[4]和 ADDRB[4]需分别接 0 或者 1（相反）

如下图所示。

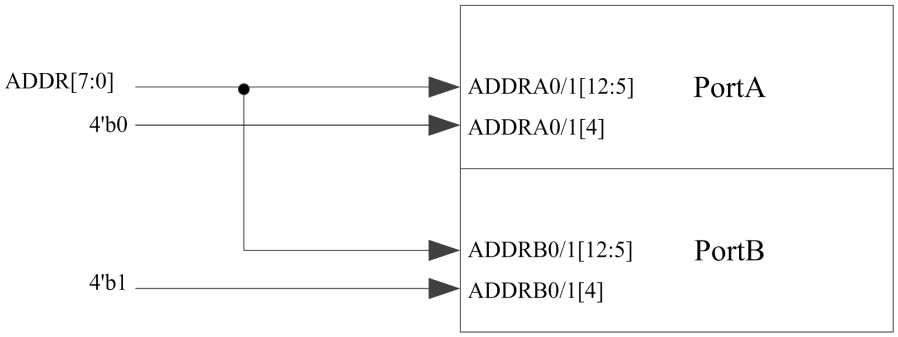


图 59 GTP_DRM9K SP 接口示意图

4.12.5.5 输出寄存器模式

针对数据输出端口，DRM 特别提供了可选的输出寄存器，以取得更好的时序性能。如下图所示，输出寄存器可由独立的使能信号 OCE 控制，当 OCE 为常量 1 时，输出寄存器的使用会把读操作的延迟从一个时钟周期增加到两个时钟周期（Case 1）；在具有流控功能的流水线设计应用中，用户也可以用逻辑灵活控制 OCE（Case 2）。

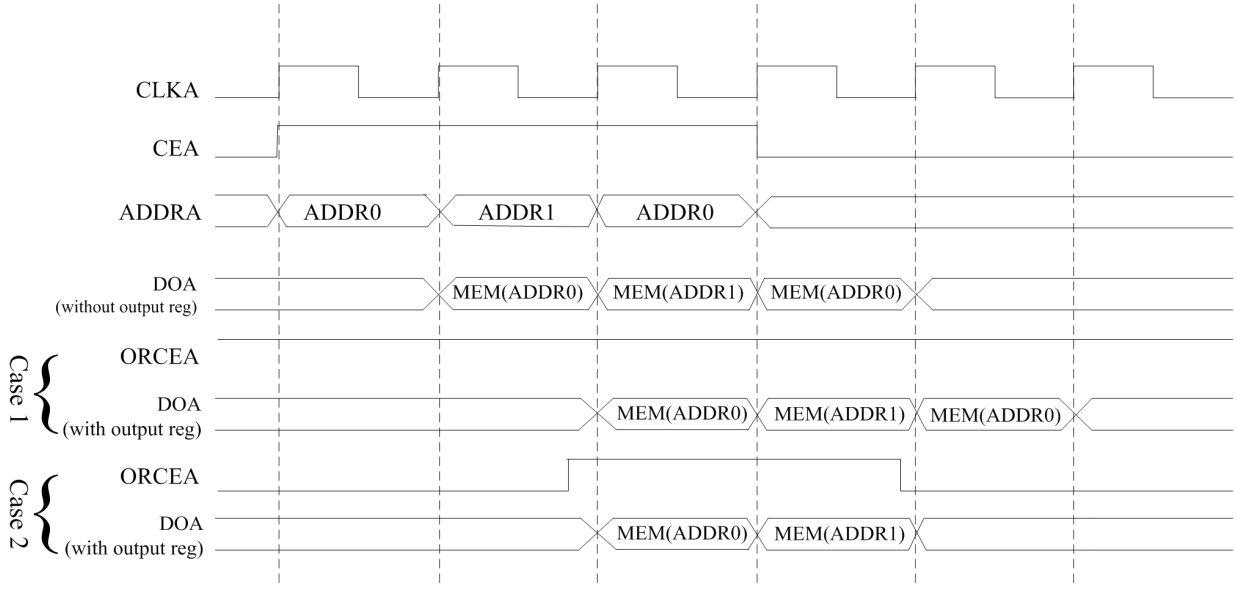


图 60 GTP_DRM9K 寄存器模式波形示意图

4.12.6. 例化模板

```
GTP_DRM9K
#(
.DATA_WIDTH_A       ( 18), // 1 2 4 8 16 32 9 18 36
.DATA_WIDTH_B       ( 18), // 1 2 4 8 16 32 9 18 36
.WRITE_MODE_A       ( "NORMAL_WRITE"), // TRANSPARENT_WRITE READ_BEFORE_WRITE
.WRITE_MODE_B       ( "NORMAL_WRITE"), // TRANSPARENT_WRITE READ_BEFORE_WRITE
.DOA_REG            ( 0),
.DOB_REG            ( 0),
.RST_TYPE           ("SYNC"), // ASYNC,ASYNC_SYNC_RELEASE
.RAM_MODE           ("TRUE_DUAL_PORT"),    // SIMPLE_DUAL_PORT SINGLE_PORT ROM
.GRS_EN                 ("TRUE"),                   //"TRUE"; "FALSE"
.DOA_REG_CLKINV   (1'b0),     //clka polarity invert for output register
.DOB_REG_CLKINV   (1'b0),     //clkb polarity invert for output register
.INIT_00 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_01 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_02 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_03 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_04 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_05 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_06 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_07 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_08 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_09 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_0A (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_0B (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_0C (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
```

```
.INIT_0D (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_0E (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_0F (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_10 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_11 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_12 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_13 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_14 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_15 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_16 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_17 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_18 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_19 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_1A (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_1B (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_1C (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_1D (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_1E (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_1F (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_FILE ("NONE"),
 //add for initialization memory
.BLOCK_X          (0 )          , //indicate X location of block memory when cascaded with DRM18Ks
.BLOCK_Y          (0 )          , //indicate Y location of block memory when cascaded with DRM18Ks
.RAM_DATA_WIDTH  ( 9   ), //the total DATA WIDTH of cascaded DRMS
.RAM_ADDR_WIDTH ( 11 ), //the total ADDR WIDTH of cascaded DRMS
.INIT_FORMAT      ("BIN")     //initial file data type binary or hexadecimal
)
GTP_DRM9K_inst (
    .DOA            (DOA          ),
    .ADDRA          (ADDRA        ),
    .ADDRA_HOLD     (ADDRA_HOLD),
    .DIA            (DIA                ),
    .WEA            (WEA          ),
    .CLKA           (CLKA         ),
    .CEA            (CEA          ),
    .ORCEA          (ORCEA              ),
    .RSTA           (RSTA         ),
    .DOB            (DOB          ),
    .ADDRB          (ADDRB        ),
    .ADDRB_HOLD     (ADDRB_HOLD ),
    .DIB            (DIB                ),
    .WEB            (WEB                ),
    .CLKB           (CLKB               ),
    .CEB            (CEB          ),
    .ORCEB          (ORCEB        ),
```

```
        .RSTB                (RSTB          )
    );
```

4.13. GTP_DRM18K 使用说明

4.13.1. 支持器件

表 123 GTP_DRM18K 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

4.13.2. 功能介绍

GTP_DRM18K 有 18K bits 存储单元，支持多种工作模式，包括双口（DP，True Dual Port）RAM，简单双口（SDP，Simple Dual Port）RAM，单口（SP，Single Port） RAM 或 ROM 模式。DRM 支持可配置的数据位宽，并在 DP RAM 和 SDP RAM 模式下支持双端口混合数据位宽。

DP RAM 模块有两个独立的端口 A、B，均可以独立进行读写操作，即可同时进行读操作，写操作，或一端口进行读操作一端口进行写操作。两端口均支持不同的时钟，且位宽可以独立设置。不可同时通过两个端口对同一地址进行读写操作，否则将引起冲突。

SDP RAM 中，A 端口固定为写入端口，B 端口固定为读出端口，两个端口支持不同时钟，且均有独立的位宽设置。不可同时通过两个端口对同一地址进行读写操作，否则将引起冲突。

SP RAM 模式下，可以对 DRM 的两个端口分别进行独立操作。不可同时通过两个端口对同一地址进行读写操作，否则将引起冲突。

DRM 可以配置为 ROM，通过配置接口可以初始化 ROM 内容。

DRM 的端口写操作支持 Normal Write mode（NW），Transparent Write mode（TW），Read before Write mode（RBW）三种模式。

所有端口均默认为高电平有效。

结构框图见下图。

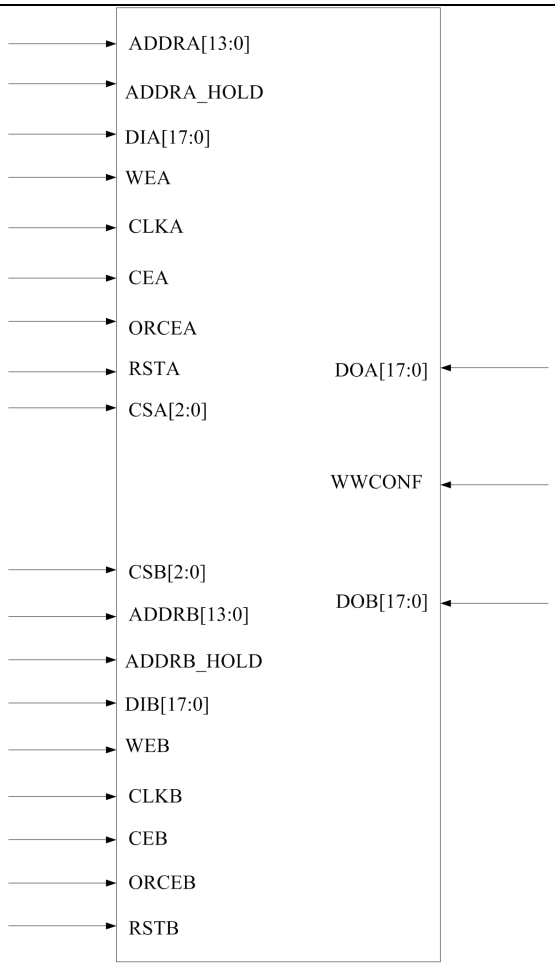


图 61 GTP_DRM18K 结构框图

4.13.3. 端口描述

表 124 GTP_DRM18K 端口列表

端口名称	方向	功能描述
ADDRA	输入	A端口输入地址
ADDRA_HOLD	输入	A端口地址输入选择
DIA	输入	A端口数据输入
CSA	输入	A端口地址扩展
WEA	输入	A端口写使能
CLKA	输入	A端口时钟
CEA	输入	A端口时钟使能
ORCEA	输入	A端口输出寄存器使能
RSTA	输入	A端口数据寄存器复位

端口名称	方向	功能描述
DOA	输出	A端口数据输出
ADDRB	输入	B端口输入地址
ADDRB_HOLD	输入	B端口地址输入选择
DIB	输入	B端口数据输入
CSB	输入	B端口地址扩展
WEB	输入	B端口写使能
CLKB	输入	B端口时钟
CEB	输入	B端口时钟使能
ORCEB	输入	B端口输出寄存器使能
RSTB	输入	B端口数据寄存器复位
DOB	输出	B端口数据输出
WWCONF	输出	写冲突

4.13.4. 参数描述

表 125 GTP_DRM18K 参数列表

参数名	描述	设置值
CSA_MASK[2:0]	A端口地址扩展控制信号	0～7
CSB_MASK[2:0]	B端口地址扩展控制信号	0～7
DATA_WIDTH_A	A端口数据最大位宽	1、2、4、8、16、32、9、18、36
DATA_WIDTH_B	B端口数据最大位宽	1、2、4、8、16、32、9、18、36
WRITE_MODE_A	A端口写模式	"NORMAL_WRITE" "TRANSPARENT_WRITE" "READ_BEFORE_WRITE"
WRITE_MODE_B	B端口写模式	"NORMAL_WRITE" "TRANSPARENT_WRITE" "READ_BEFORE_WRITE"
DOA_REG	A端口输出寄存	0：不使能输出寄存

		1：使能输出寄存
DOB_REG	B端口输出寄存	0：不使能输出寄存 1：使能输出寄存
RST_TYPE	复位模式选择	"SYNC"：同步复位 "ASYNC"：异步复位 "ASYNC_SYNC_RELEASE"：异步复位同步释放
RAM_MODE	RAM模式选择	"TRUE_DUAL_PORT"：双口RAM "SIMPLE_DUAL_PORT"：简单双口RAM "SINGLE_PORT"：单口RAM "ROM"：ROM
WRITE_COLLISION_ARBITER	写冲突仲裁	"NULL"：不进行仲裁
GRS_EN	全局复位使能信号（芯片内部）	"FALSE":不使能全局复位； "TRUE":使能全局复位。
DOA_REG_CLKINV	A端口输出寄存器时钟翻转	0：时钟不翻转 1：时钟翻转
DOB_REG_CLKINV	B端口输出寄存器时钟翻转	0：时钟不翻转 1：时钟翻转
INIT_00 INIT_01 INIT_02 … INIT_3F	RAM初始化配置参数	0 ~2^288-1
INIT_FILE	初始化文件	"NONE"：不指定初始化文件，则默认初始化数据全为零； "XXX"：XXX表示具体初始化文件路径
BLOCK_X	RAM18K级联时数据	视器件DRM个数而定

	级联坐标	
BLOCK_Y	RAM18K级联时地址级联坐标	视器件DRM个数而定
RAM_DATA_WIDTH	RAM级联后的数据最大位宽	视器件DRM个数而定
RAM_ADDR_WIDTH	RAM级联后的地址最大位宽	视器件DRM个数而定
INIT_FORMAT	初始化文件格式	"BIN"：二进制 "HEX"：十六进制
SIM_DEVICE	仿真模型器件识别	"TITAN"：For TITAN器件 "TITAN"：For PGL22G器件 "LOGOS"：For LOGOS系列后续器件

不支持以下条件同时满足情况：

SIM_DEVICE = “PGL22G”

RAM_MODE = “TRUE_DUAL_PORT”

WRITE_MODE_A = “READ_BEFORE_WRITE”

WRITE_MODE_B = “READ_BEFORE_WRITE”

4.13.5. 详细功能说明

4.13.6.1 写模式---NW 模式

NW 模式是 Normal Write 模式，为默认模式。当写使能信号 WEA 为高电平时，不进行读操作，将输入数据 DIA 写入当前地址。当写使能信号 WEA 为低电平时，将当前地址 ADDRA 所保存的数据读出，并在下一个时钟周期从输出端口 DOA 输出。其波形图如下图所示。

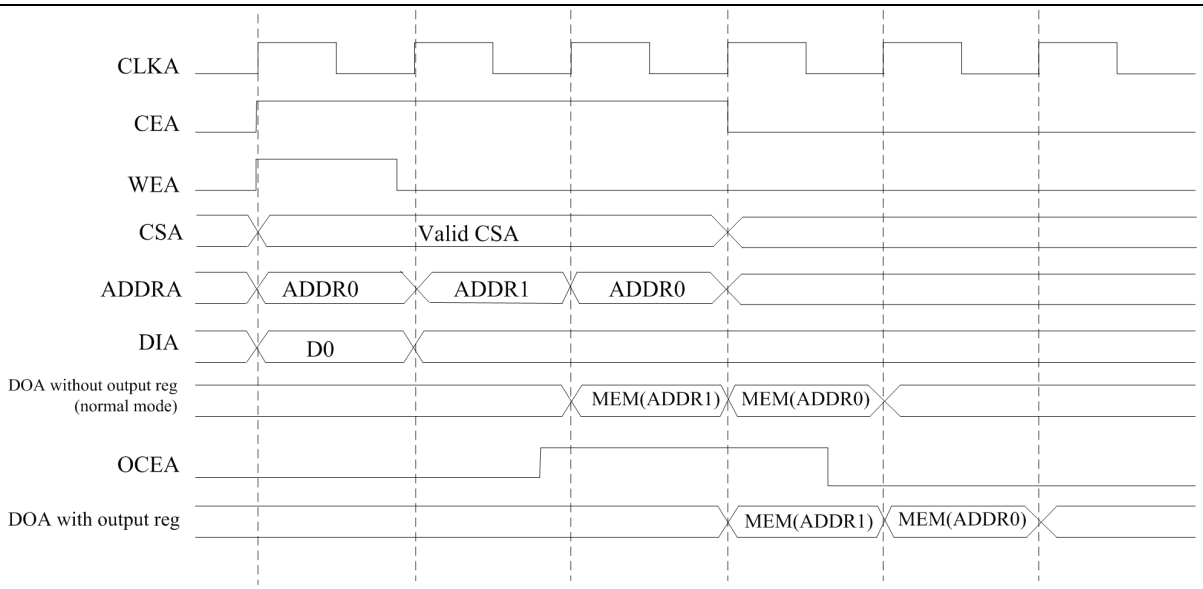


图 62 GTP_DRM18K NW 模式波形示意图

注：1.DRM 的端口写操作模式仅适用于具有读写功能的端口，即 DP RAM 的 A/B 端口与 SP RAM 的端口。
2.SDP 模式不支持读写模式设置，在使用时禁止设置成 TW 或 RBW 模式，只能设置为默认的 NW 模式，否则 DRM 将进入非正常的写操作状态。

4.13.6.2 写模式---TW 模式

TW 模式中，在写使能信号 WEA 为高电平时，输入数据 DIA 写入地址 ADDRA 的同时，将在下一个时钟周期时直接从输出端口 DOA 读出。使能信号 WEA 为低电平时，将当前地址 ADDRA 所保存的数据读出，并在下一个时钟周期从输出端口 DOA 输出。其波形图如下图所示。

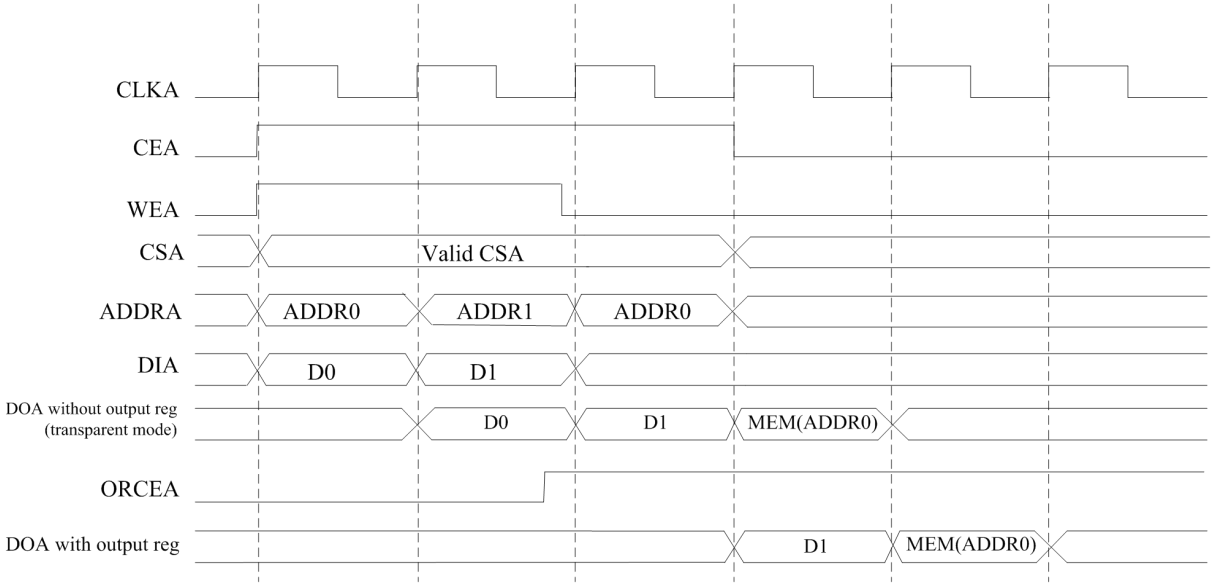


图 63 GTP_DRM18K TW 模式波形示意图

4.13.6.3 写模式---RBW 模式

在 RBW 模式中，当写使能信号 WEA 为高电平时，将首先读出地址 ADDRA 所引的原数据并在

写操作的下一个时钟周期输出到输出端口，再将输入数据 DIA 写入地址 ADDRA 中，覆盖原数据。使能信号 WEA 为低电平时，将当前地址 ADDRA 所保存的数据读出，并在下一个时钟周期从输出端口 DOA 输出。其波形图如下图所示。

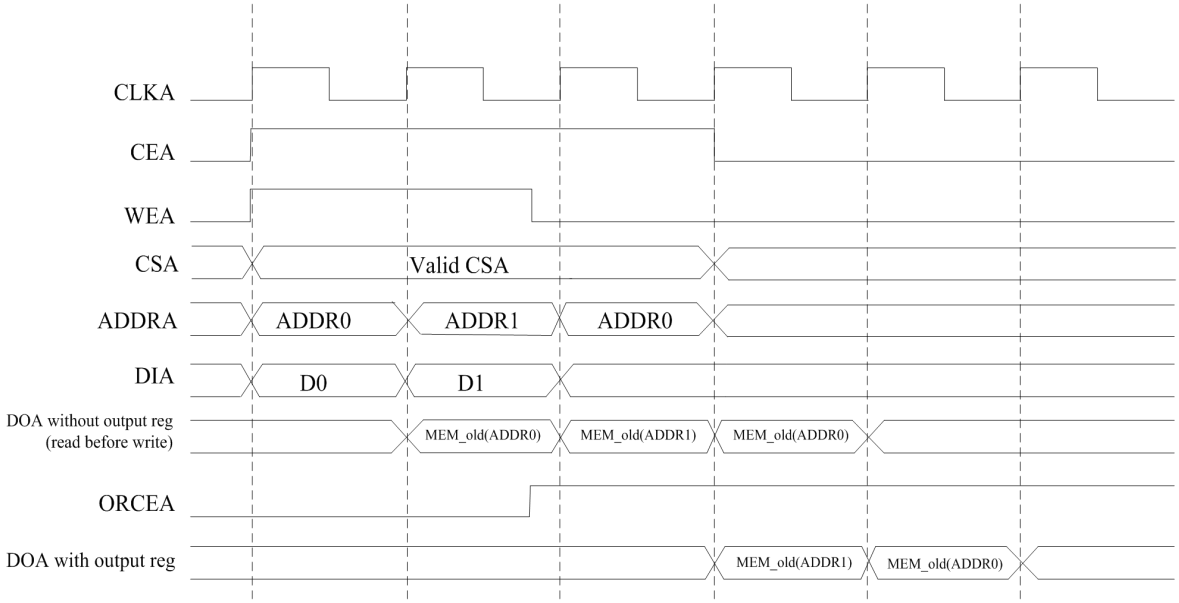


图 64 GTP_DRM18K RBW 模式波形示意图

4.13.6.4 SP RAM 的 TB 和 RBW 模式的实现

DRM 无法通过直接配置实现 32/36 bit SP 模式下的 TW（Transparent Write）和 RBW（Read before Write）模式（在 GTP 配置为 32/36 bit SP 模式时禁止设置读写模式为 TW 或 RBW 模式，否则 DRM 将进入非正常的写操作状态），需要额外的配置，具体实现方式如下：

a. Port A 和 Port B 同时配置为 16/18 bit DP 模式（分别针对 32/36 bit SP）；

b. Port A 和 Port B 配置为相同的写模式；其他配置位也需要相同。

c. Port A 和 Port B 的 CE, CS, WE, ADDR, ORCE, RST, CLK 等控制信号分别并联起来；

d. DIA, DIB 并联作为数据输入；DOA, DOB 并联作为数据输出；

e. Port A 和 Port B 地址输入端口需同时接地址输入；ADDRA[4]和 ADDRB[4]需分别接 0 或者 1（相反）

如下图所示。

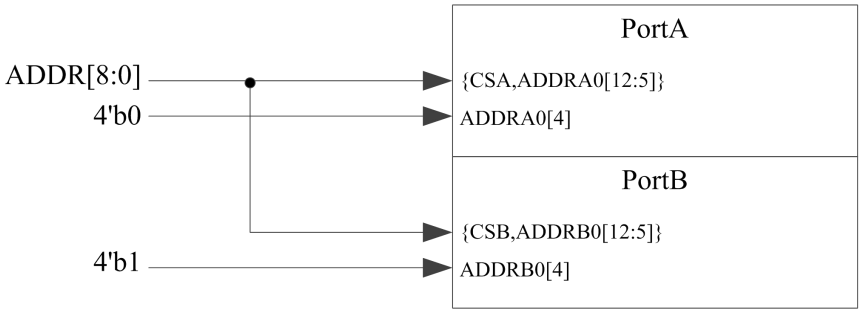


图 65 GTP_DRM18K SP 接口示意图

4.13.6.5 输出寄存器模式

针对数据输出端口，DRM 特别提供了可选的输出寄存器，以取得更好的时序性能。如下图所示，输出寄存器可由独立的使能信号 OCE 控制，当 OCE 为常量 1 时，输出寄存器的使用会把读操作的延迟从一个时钟周期增加到两个时钟周期（Case 1）；在具有流控功能的流水线设计应用中，用户也可以用逻辑灵活控制 OCE（Case 2）。

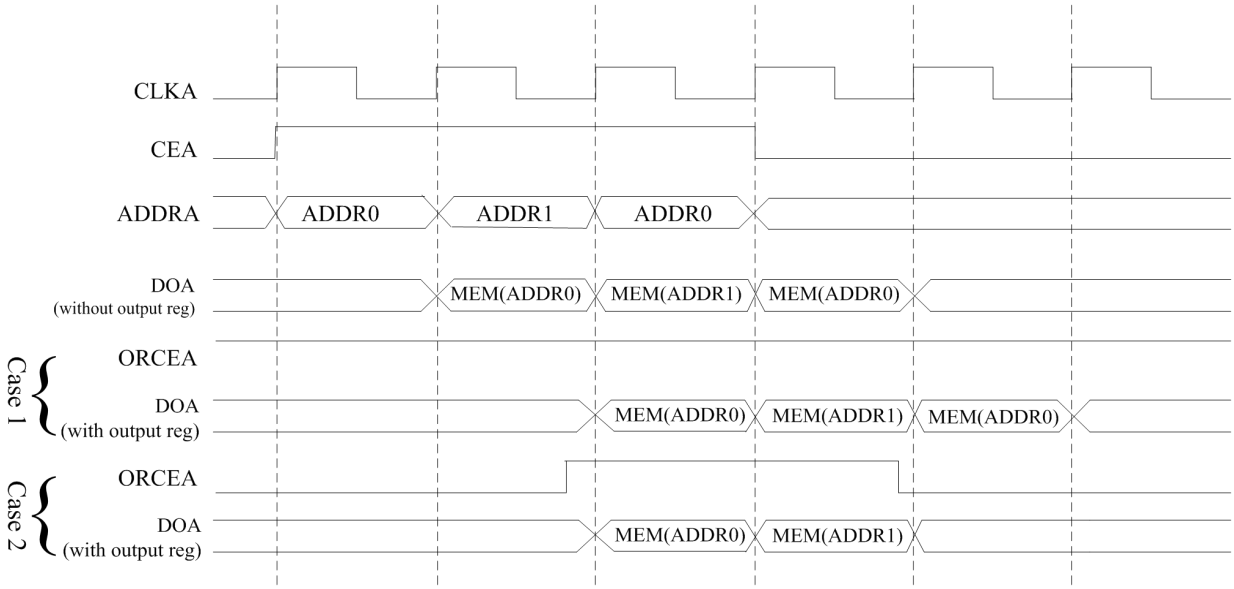


图 66 GTP_DRM18K 寄存器模式波形示意图

4.13.6. 例化模板

```
GTP_DRM18K
#(
    .CSA_MASK           (3'b000),
    .CSB_MASK           (3'b000),
    .DATA_WIDTH_A       ( 18), // 1 2 4 8 16 32 9 18 36
    .DATA_WIDTH_B       ( 18), // 1 2 4 8 16 32 9 18 36
```

```
.WRITE_MODE_A   ( "NORMAL_WRITE"), // TRANSPARENT_WRITE READ_BEFORE_WRITE
.WRITE_MODE_B   ( "NORMAL_WRITE"), // TRANSPARENT_WRITE READ_BEFORE_WRITE
.DOA_REG        ( 0),
.DOB_REG        ( 0),
.RST_TYPE           ("SYNC"), // ASYNC,ASYNC_SYNC_RELEASE
.RAM_MODE           ("TRUE_DUAL_PORT"),    // SIMPLE_DUAL_PORT SINGLE_PORT ROM
.WRITE_COLLISION_ARBITER ("NULL"), // PORTA PORTB
.GRS_EN             ("TRUE"),                 //"TRUE"; "FALSE"
.DOA_REG_CLKINV     (1'b0),     //clka polarity invert for output register
.DOB_REG_CLKINV     (1'b0),     //clkb polarity invert for output register
.INIT_00 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_01 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_02 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_03 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_04 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_05 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_06 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_07 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_08 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_09 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_0A (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_0B (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_0C (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_0D (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_0E (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_0F (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_10 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_11 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_12 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_13 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_14 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_15 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_16 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_17 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_18 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_19 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_1A (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_1B (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_1C (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_1D (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_1E (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_1F (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_20 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_21 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
.INIT_22 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
```

```
    .INIT_23 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_24 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_25 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_26 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_27 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_28 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_29 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_2A (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_2B (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_2C (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_2D (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_2E (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_2F (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_30 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_31 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_32 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_33 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_34 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_35 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_36 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_37 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_38 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_39 (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_3A (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_3B (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_3C (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_3D (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_3E (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_3F (288'h000000000000000000000000000000000000000000000000000000000000000000000000),
    .INIT_FILE ("NONE"),
//add for initialization memory
    .BLOCK_X          (0 )      ,  //indicate X location of block memory when cascaded with DRM18Ks
    .BLOCK_Y          (0 )      ,  //indicate Y location of block memory when cascaded with DRM18Ks
    .RAM_DATA_WIDTH   ( 9  ),  //the total DATA WIDTH of cascaded DRMS
    .RAM_ADDR_WIDTH   ( 11 ),   //the total ADDR WIDTH of cascaded DRMS
    .INIT_FORMAT      ("BIN")      //initial file data type binary or hexadecimal
)
GTP_DRM18K_inst (
      .DOA            (DOA          ),
      .ADDRA          (ADDRA        ),
      .ADDRA_HOLD     (ADDRA_HOLD),
      .DIA                (DIA              ),
      .CSA                (CSA          ),
      .WEA            (WEA          ),
      .CLKA           (CLKA         ),
```

```
        .CEA            (CEA        ),
        .ORCEA          (ORCEA          ),
        .RSTA           (RSTA       ),
        .DOB            (DOB        ),
        .ADDRB          (ADDRB      ),
        .ADDRB_HOLD     (ADDRB_HOLD ),
        .DIB            (DIB            ),
        .CSB                (CSB        ),
        .WEB            (WEB            ),
        .CLKB           (CLKB           ),
        .CEB                (CEB        ),
        .ORCEB          (ORCEB      ),
        .RSTB           (RSTB       ),
        .WWCONF         (WWCONF     )
    );
```

4.13.7. GTP 使用方法

LOGOS器件 mapping时：无WWCONF端口。

4.14. GTP_FIFO18K 使用说明

4.14.1. 支持器件

表 126 GTP_FIFO18K 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

4.14.2. 功能介绍

DRM_FIFO 支持 18K bits 存储单元，可以配成 16K*1，8K*2，4K*4，2K*9(8)，1K*18(16)，512*36(32)数据位宽模式。DRM 配置为 FIFO 模式时，DRM 配置成 Simple dual port 模式，且不支持混合位宽，不支持 Byte enable 的写操作。

DRM_FIFO 可将 PG DRM 配置成同步 FIFO 或异步 FIFO 模式。在 DRM SDP RAM 模式下，一个端口专用于 FIFO 数据写入，另一个端口专用于 FIFO 数据读取，读写端口可以采用不同的时钟。DRM_FIFO 在同步和异步模式下都支持可选的丢包重发模式。机构框图见下图。

DRM_FIFO 支持寄存器模式，具体使用方式和 GTP_DRM 的寄存器模式相同。

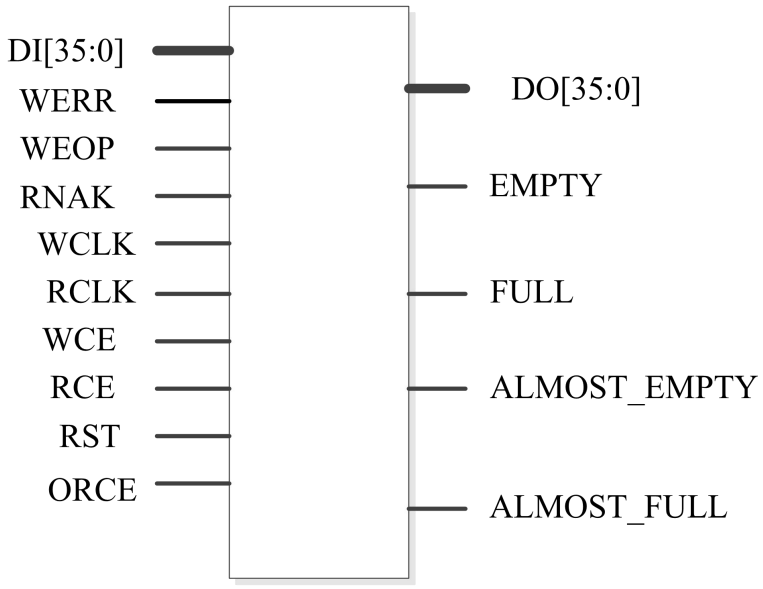


图 67GTP_FIFO18K 结构框图

4.14.3. 端口描述

表 127 GTP_FIFO18K 端口列表

端口名称	方向	功能描述
DI	输入	数据写入
WERR	输入	写端口错误数据包指示
WEOP	输入	包尾信号指示
RNAK	输入	读端口重发指示
WCLK	输入	写时钟信号
RCLK	输入	读时钟信号
WCE	输入	写使能信号
RCE	输入	读使能信号
RST	输入	复位信号
ORCE	输入	输出寄存器使能信号
DO	输出	数据读出
EMPTY	输出	读端口读空标志
FULL	输出	写端口写满标志
ALMOST_EMPTY	输出	读端口将空标志
ALMOST_FULL	输出	写端口将满标志

4.14.4. 参数描述

下表为该 GTP 参数描述，默认值均为 0。

表 128 GTP_FIFO18K 参数列表

参数命名	功能描述	设置值
GRS_EN	全局复位使能信号（芯片内部）	"TRUE":使能全局复位 "FALSE":不使能全局复位
ALMOST_FULL_OFFSET	当FIFO将满且写指针与读指针位置之差值等于ALMOST_FULL_OFFSET，产生almost_full标志。	16382：16K地址 8190：8K地址 4094：4K地址 2046：2K地址 1022：1K地址 510：512地址
ALMOST_EMPTY_OFFSET	当FIFO将空且读指针与写指针位置之差值等于ALMOST_EMPTY_OFFSET,产生almost_empty标志。	16382：16K地址 8190：8K地址 4094：4K地址 2046：2K地址 1022：1K地址 510：512地址
SYNC_FIFO	异步/同步FIFO选择	"TRUE":使用同步FIFO "FALSE":使用异步FIFO
DATA_WIDTH	FIFO数据位宽	1,2,4,8,9,16,18,32,36
REWRITE_EN	丢包使能	"FALSE"：无丢包功能； "TRUE"：使能丢包功能
RESEND_EN	重发使能	"FALSE"：无重发功能； "TRUE"：使能重发功能
USE_EMPTY	使能读空标志	1:使能读空标志； 0:不使能读空标志
USE_FULL	使能写满标志	1:使能写满标志； 0:不使能写满标志

DO_REG	使能输出寄存器使能	1:使能； 0:不使能

4.14.5. 详细功能说明

FIFO 模式支持同步\异步模式。同步模式下，读写时钟为同一个时钟。异步模式下，读写时钟为两个互相独立的时钟。

4.14.5.1 写入空 FIFO 时序

EMPTY 信号指示 FIFO 为空，当 WR_EN 有效并且有效数据写入后，同步 FIFO 时，EMPTY 信号在 1 个 RCLK 时钟周期后置“0”；写入异步 FIFO 时，EMPTY 信号在 2 个 RCLK 时钟周期后置零。当 WR_EN 继续有效，ALMOST_EMPTY 信号根据 ALMOST_EMPTY_OFFSET 的配置延迟置零。其波形图如下图所示。

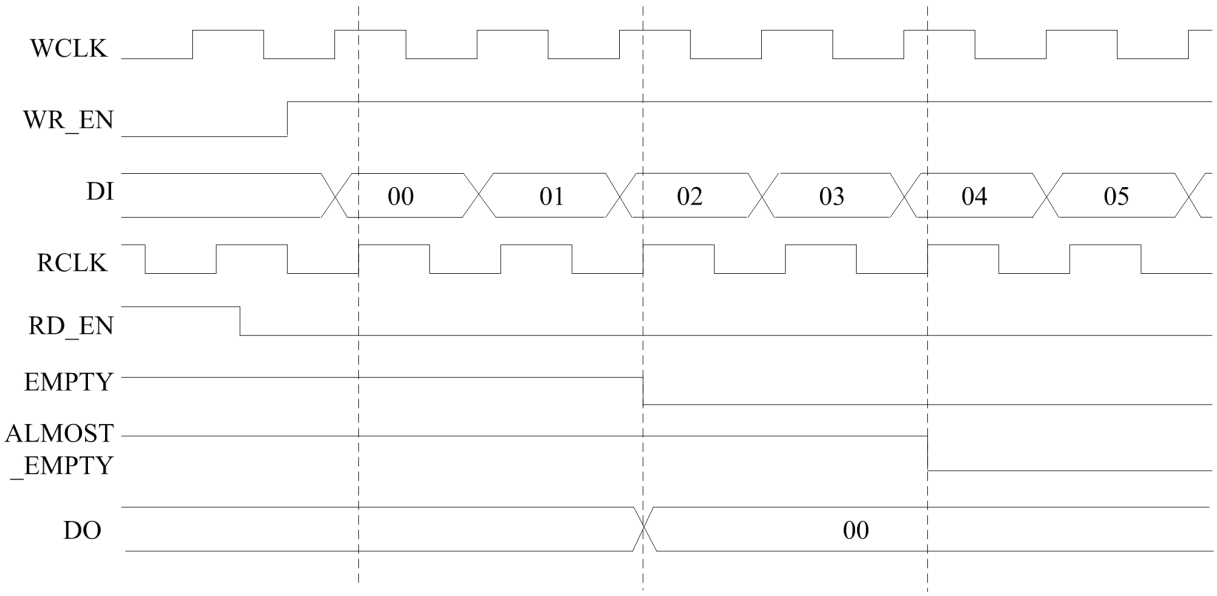


图 68 GTP_FIFO18K 写入空 FIFO 模式波形示意图

4.14.5.2 写入将满 FIFO 时序

FIFO 将满时，ALMOST_FULL 将根据 ALMOST_FULL_OFFSET 的配置提前置 1。当 FIFO full 时，写地址不再增加。其波形图如下图所示。

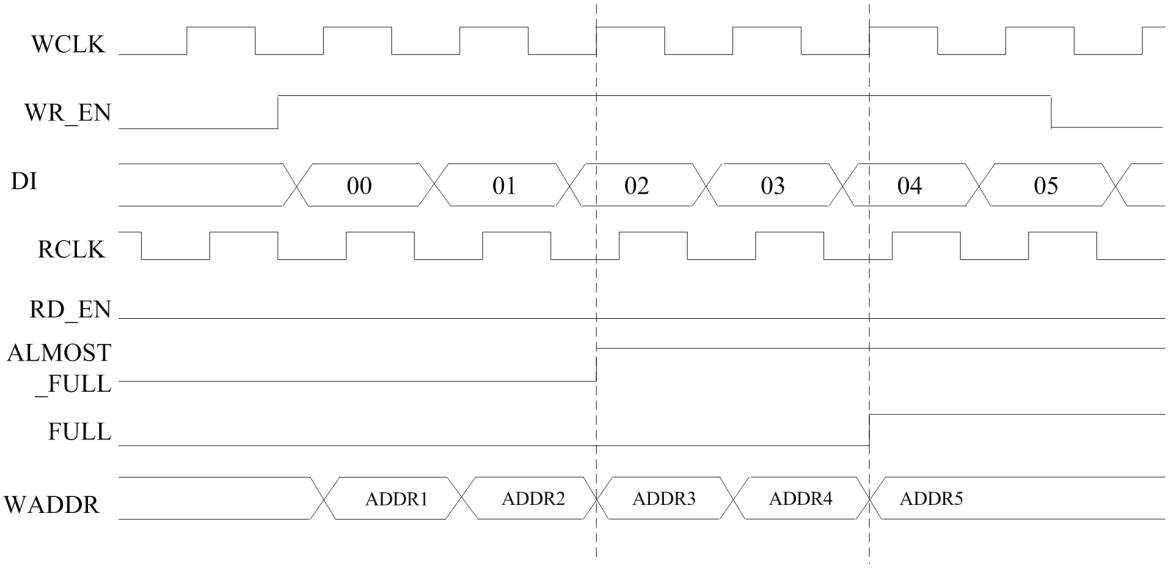


图 69 GTP_FIFO18K 写入将满 FIFO 模式波形示意图

4.14.5.3 满 FIFO 读出时序

FULL 信号指示 FIFO 为满，当 RD_EN 有效读出数据后，FULL 信号延迟 1（同步 FIFO）到 2（异步 FIFO）个 WCLK 时钟周期置零。当 RD_EN 继续有效，ALMOST_FULL 信号根据 ALMOST_FULL_OFFSET 的配置延迟置零。从满 FIFO 读出时序图如下图所示

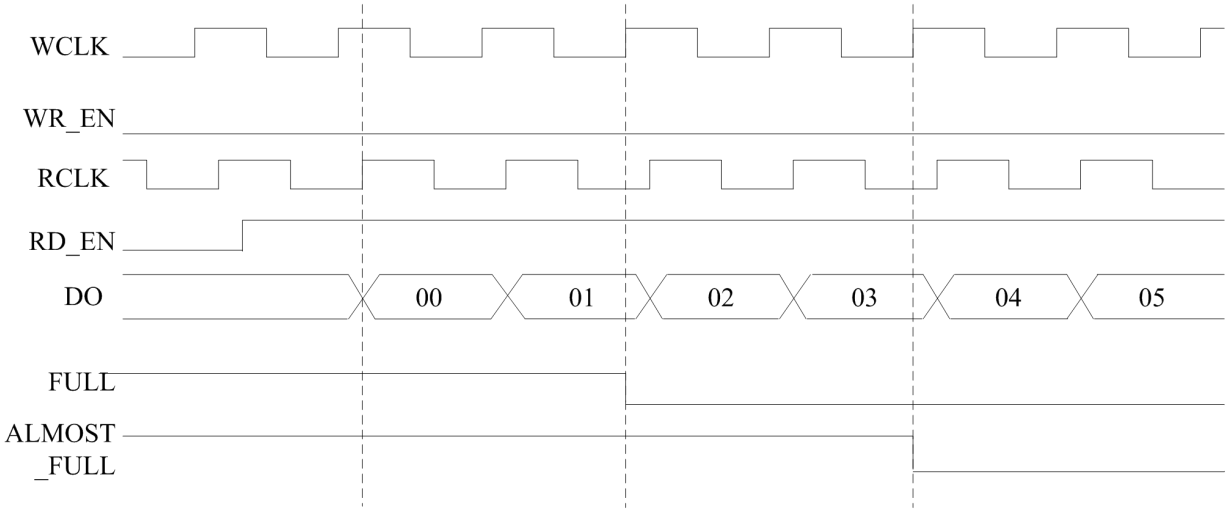


图 70 GTP_FIFO18K 满 FIFO 读出模式波形示意图

4.14.5.4 将空 FIFO 读出时序

FIFO 将空时，ALMOST_EMPTY 将根据 ALMOST_EMPTY_OFFSET 的配置提前置 1。当 FIFO EMPTY 时，读地址不再增加。其波形图如下图所示。

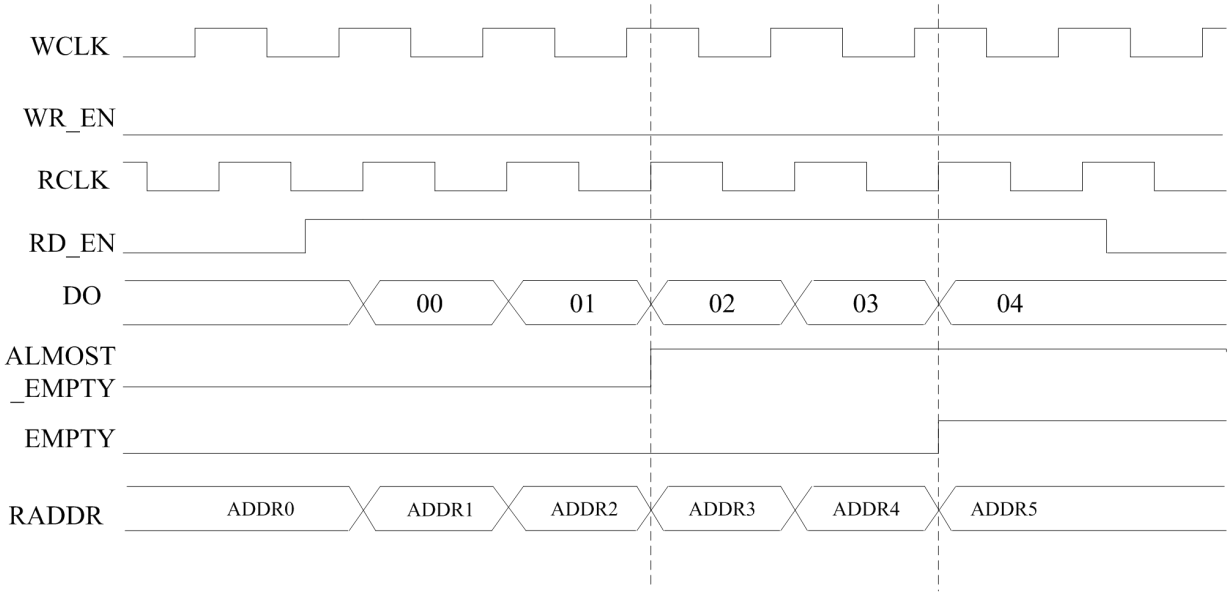


图 71 GTP_FIFO18K 将空 FIFO 读出模式波形示意图

4.14.5.5 写丢包时序

FIFO 写入端检测到 wr_err 数据包错误信号时，写入端的写地址在下一个 WCLK 周期退回到数据包包头所在的写地址。丢包功能可以单独使能，wr_eop 为输入信号，置 1 时使能丢包功能。丢包重发模式下，wr_eop 信号需要占用一位数据位，该信号需要连接到数据输入 DI[8]（2K*9、1K*18）或 DI[26]（512*36）。其波形图如下图所示。其波形图如下图所示。

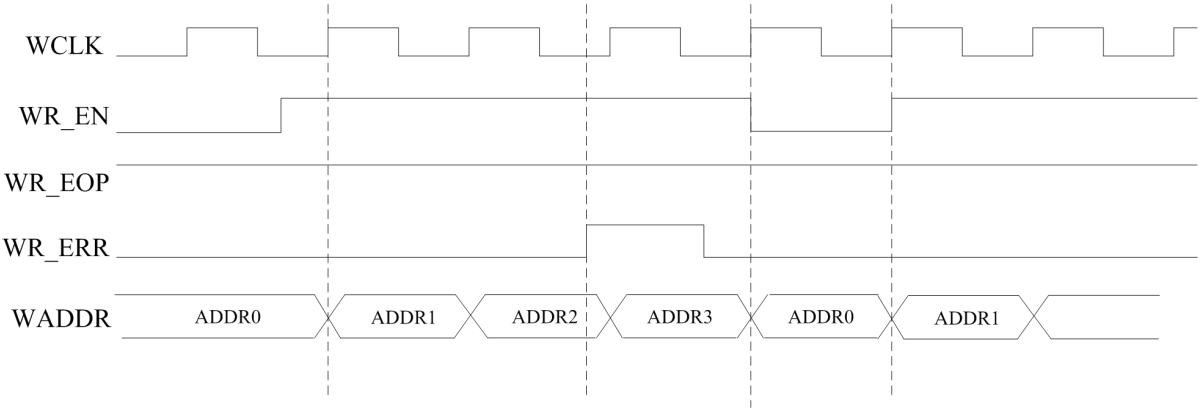


图 72 GTP_FIFO18K FIFO 写丢包模式波形示意图

4.14.5.6 读重发时序

FIFO 写入端检测到 wr_err 数据包错误信号时，写入端的写地址退回到数据包包头所在的写地址。读指针在 RD_NAK 信号置“1”后退回到当前数据包包头所在地址。丢包功能和重发功能需要同时使能。

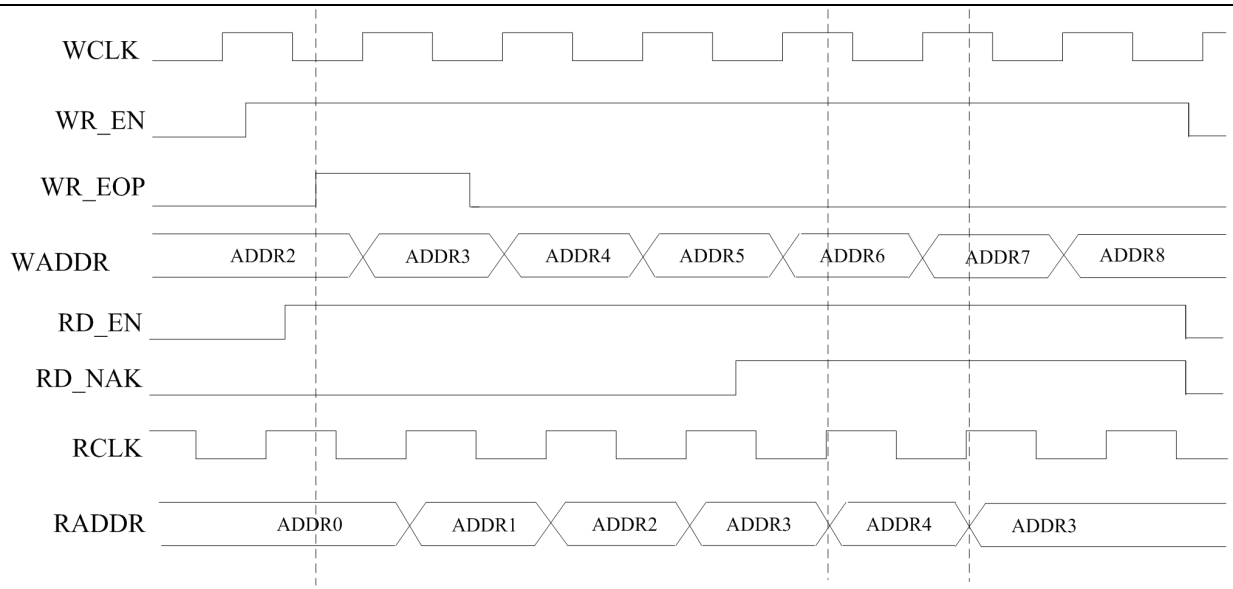
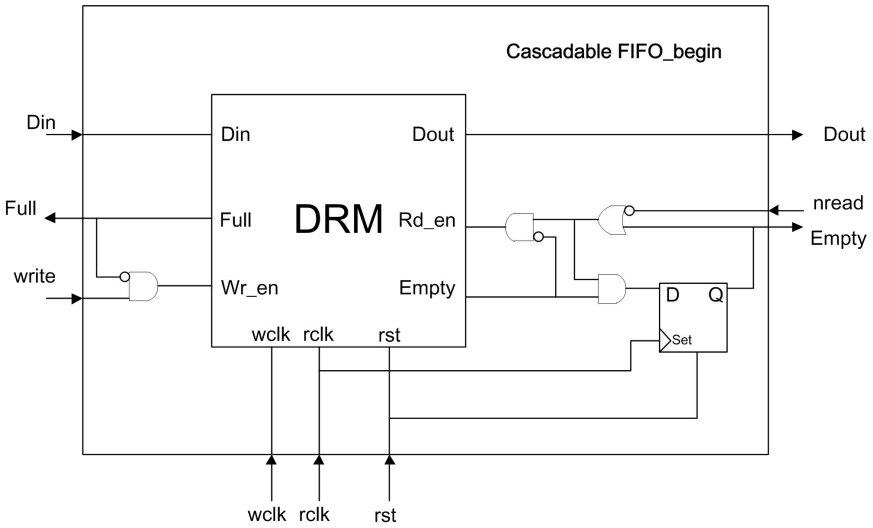


图 73 GTP_FIFO18K FIFO 读重发模式波形示意图

4.14.6. 级联拓展

4.14.6.1 深度拓展

深度扩展可以通过 N 个 FIFO 级联实现：如果 WCLK 频率高于 RCLK 频率，INTCLK = WCLK；如果 WCLK 频率小于等于 RCLK 频率，INTCLK = RCLK。ALMOST_EMPTY 信号由最后一个 FIFO 产生，ALMOST_FULL 信号由第一个 FIFO 产生。如下图所示。



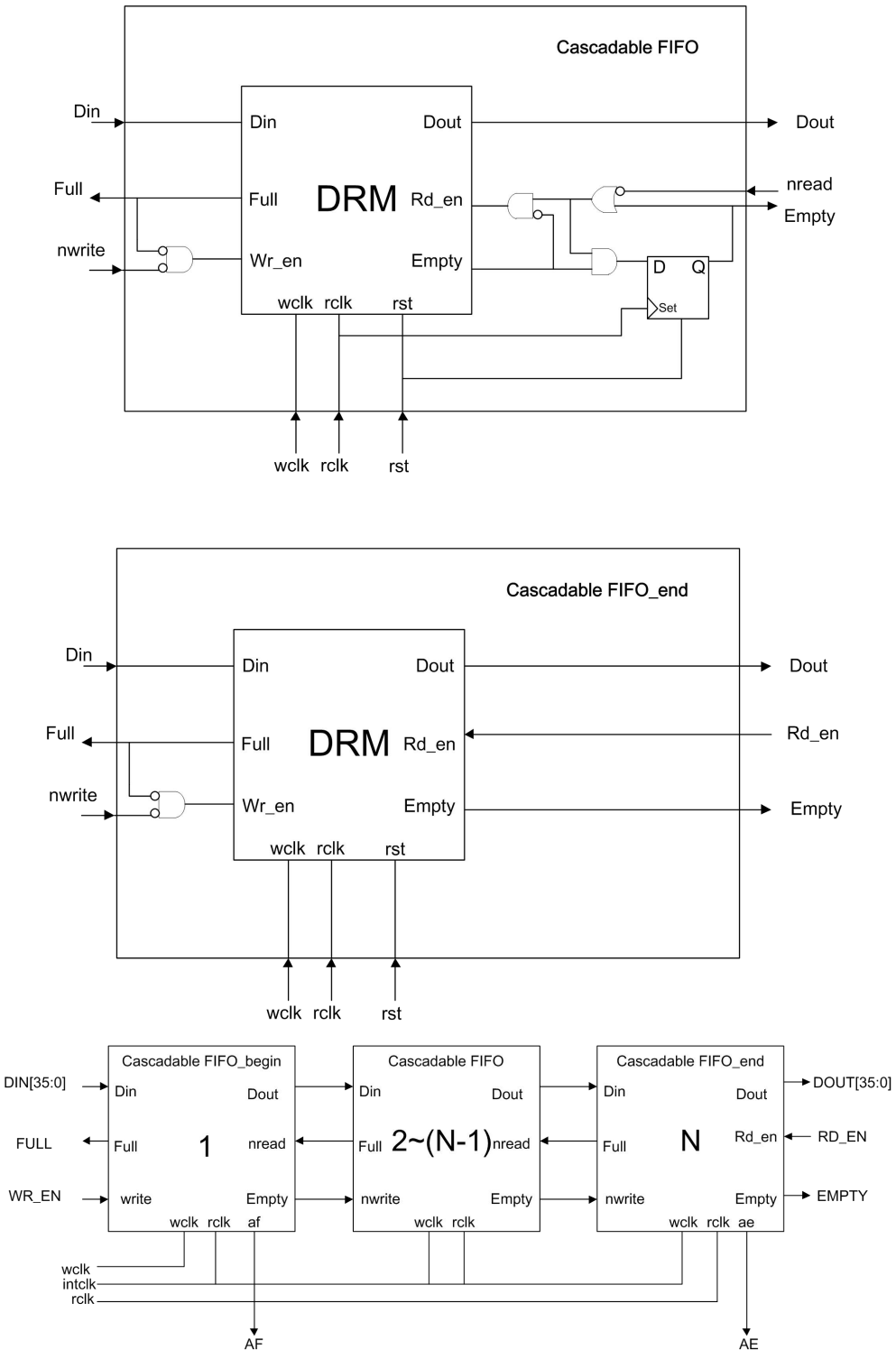


图 74 GTP_FIFO18K FIFO 深度拓展示意图

以 N=3 为例，如下图所示。

drm1 实例化成 cascadable FIFO_begin。其中，级联后数据输入端口是 din[35:0]； AF 和 Full 作为级联后标志位；WR_EN 输入到 write。

drm2 实例化成 cascadable FIFO。

drm3 实例化成 cascadable FIFO_end。其中 AE 和 Empty 作为级联后的标志位；RD_EN 输入 rd_en。

Dout 为输出。

4.14.6.2 宽度拓展

宽度拓展可以通过并联 FIFO 实现，如下图所示。

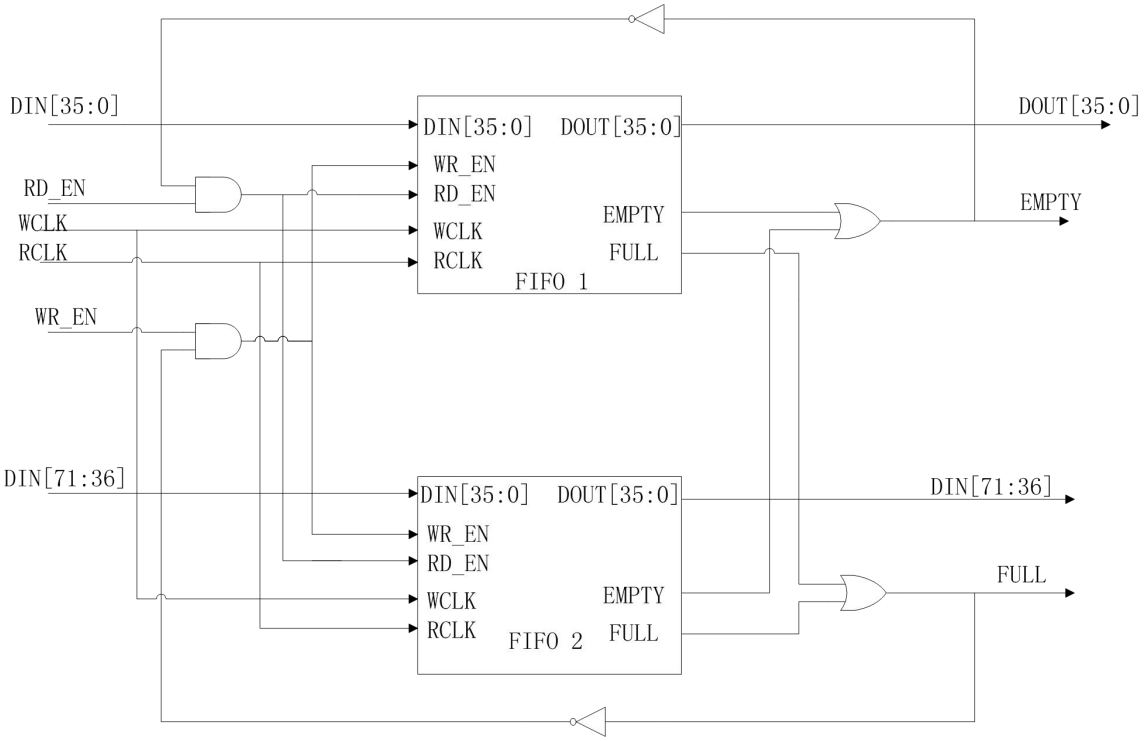


图 75 GTP_FIFO18K FIFO 宽度拓展示意图

4.14.7. 例化模板

```
GTP_FIFO18K #(
        .GRS_EN("TRUE"),
        .DATA_WIDTH(18),
        .DO_REG(0),
        .ALMOST_FULL_OFFSET('b00000000000000),
        .ALMOST_EMPTY_OFFSET('b00000000000000),
        .USE_EMPTY(0),
        .USE_FULL(0),
        .REWRITE_EN("FALSE"),
        .RESEND_EN("FALSE"),
        .SYNC_FIFO("FALSE")
) GTP_FIFO18K_inst (
        .DO(),                  // OUTPUT[35:0]
        .DI(),                  // INPUT[35:0]
        .ALMOST_EMPTY(),// OUTPUT
        .ALMOST_FULL(), // OUTPUT
```

```
    .EMPTY(),          // OUTPUT
    .FULL(),           // OUTPUT
    .ORCE(),           // INPUT
    .RCE(),            // INPUT
    .RCLK(),           // INPUT
    .RNAK(),           // INPUT
    .RST(),            // INPUT
    .WCE(),            // INPUT
    .WCLK(),           // INPUT
    .WEOP(),           // INPUT
    .WERR()            // INPUT
);
```

5. IO 相关 GTP 使用说明

5.1. GTP_INBUF 使用说明

5.1.1. 支持器件

表 129 GTP_INBUF 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.1.2. 功能介绍

GTP_INBUF 是一个输入 BUFFER。如下图：

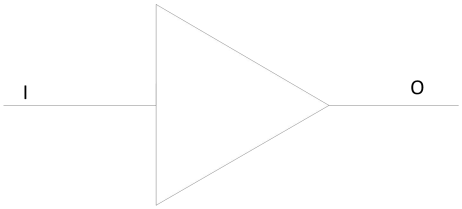


图 76 GTP_INBUF

5.1.3. 端口描述

表 130 GTP_INBUF 端口列表描述

端口名	输入/输出	描述
I	输入	输入信号
O	输出	输出信号

5.1.4. 参数描述

表 131 GTP_INBUF 参数列表

参数名	描述	默认值	设置值
IOSTANDARD	IO 标准	"DEFAULT"	IO 支持的标准包括"LVTTL33", "PCI33",等
TERM_DDR	HSTL、SSTL 标准输入时，内置 terminal resistor 开启或关闭	"ON"	"ON"

5.1.5. 例化模板

```
GTP_INBUF#(
.IOSTANDARD ("DEFAULT"),
. TERM_DDR    ("ON")
)
GTP_INBUF_inst (
.I    (i),
.O    (o)
);
```

5.2. GTP_INBUFDS 使用说明

5.2.1. 支持器件

表 132 GTP_INBUFDS 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.2.2. 功能介绍

GTP_INBUFDS 是一个差分输入 BUFFER。如下图:

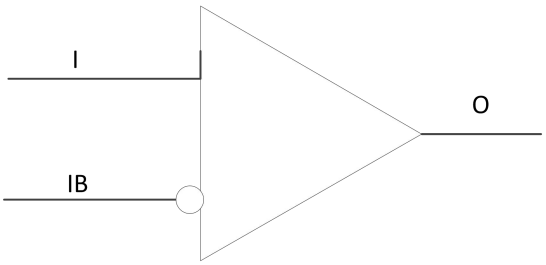


图 77 GTP_INBUFDS

5.2.3. 端口描述

表 133 GTP_INBUFDS 端口列表描述

端口名	输入/输出	描述
I	输入	差分 P 输入信号
IB	输入	差分 N 输入信号
O	输出	输出信号

5.2.4. 参数描述

表 134 GTP_INBUFDS 参数列表

参数名	描述	默认值	设置值
IOSTANDARD	IO 标准	"DEFAULT"	IO 支持的标准包括"LVDS", "MINI-LVDS", "LVPECL",等
TERM_DIFF	差分输入时，内置 terminal resistor 开启或关闭	"ON"	"ON"

5.2.5. 例化模板

```
GTP_INBUFDS#(
.IOSTANDARD ("DEFAULT"),
. TERM_DIFF("ON")
)
 GTP_INBUFDS_inst (
   .I          (i),
   .IB         (ib ),
   .O      (o)
);
```

5.3. GTP_INBUFG 使用说明

5.3.1. 支持器件

表 135 GTP_INBUFG 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.3.2. 功能介绍

GTP_INBUFG 是一个时钟输入 BUFFER。如下图:

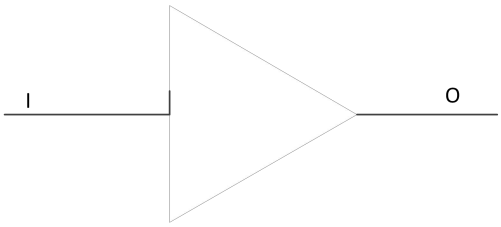


图 78 GTP_INBUFG

5.3.3. 端口描述

表 136 GTP_INBUFG 端口列表描述

端口名	输入/输出	描述
I	输入	输入信号
O	输出	输出信号

5.3.4. 参数描述

表 137 GTP_INBUFG 参数列表描述

参数名	描述	默认值	设置值
IOSTANDARD	IO 标准	"DEFAULT"	IO 支持的标准包括"LVTTL33", "LVCMOS25",等
TERM_DDR	HSTL、SSTL 标准输入时，内置 terminal resistor 开启或关闭	"ON"	"ON"

5.3.5. 例化模板

```
GTP_INBUFG#(
.IOSTANDARD ("DEFAULT"),
. TERM_DDR ("ON")
)
GTP_INBUFG_inst (
.I    (i),
.O    (o)
);
```

5.4. GTP_INBUFGDS 使用说明

5.4.1. 支持器件

表 138 GTP_INBUFGDS 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.4.2. 功能介绍

GTP_INBUFGDS 是一个差分时钟输入 BUFFER。如下图:

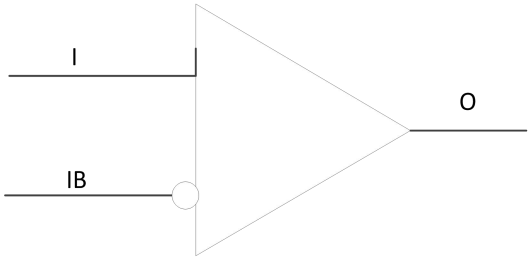


图 79 GTP_INBUFGDS

5.4.3. 端口描述

表 139 GTP_INBUFGDS 端口列表描述

端口名	输入/输出	描述
I	输入	差分 P 输入信号
IB	输入	差分 N 输入信号
O	输出	输出信号

5.4.4. 参数描述

表 140 GTP_INBUFGDS 参数列表描述

参数名	描述	默认值	设置值
IOSTANDARD	IO 标准	"DEFAULT"	IO 支持的标准包括"LVTTL33", "LVCMOS25",等
TERM_DIFF	差分输入时，内置 terminal resistor 开启或关闭	"ON"	"ON"

5.4.5. 例化模板

```
GTP_INBUFGDS#(
    .IOSTANDARD ("DEFAULT"),
    . TERM_DIFF ("ON")
)
 GTP_INBUFGDS_inst (
.I    (i),
.IB   (ib ),
.O    (o)
);
```

5.5. GTP_IOBUF 使用说明

表 141 GTP_IOBUF 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.5.1. 功能介绍

GTP_IOBUF 是一个双向单信号 BUFFER，连接外部双向引脚和内部逻辑电路。如下图:

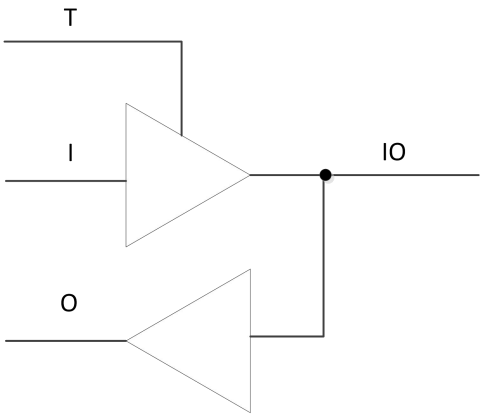


图 80 GTP_IOBUF

5.5.2. 端口描述

表 142 GTP_IOBUF 端口列表描述

端口名	输入/输出	描述
I	输入	输入信号
T	输入	三态输入使能，低有效
O	输出	输出信号
IO	输入输出	输入输出信号（inout）

5.5.3. 参数描述

表 143 GTP_IOBUF 参数列表描述

参数名	描述	默认值	设置值
IOSTANDARD	IO 标准	"DEFAULT"	IO 支持的标准包括"LVTTL33", "LVCMOS25",等
SLEW_RATE	Slew rate 属性	"SLOW"	"FAST", "SLOW"
DRIVE_STRENGTH	输出驱动	"8"	2, 4, 6, 8, 12, 16, 24
TERM_DDR	内置 terminal resistor 开启或关闭	"ON"	

5.5.4. 例化模板

```
 GTP_IOBUF#(
.IOSTANDARD ("DEFAULT"),
.SLEW_RATE ("SLOW"),
.DRIVE_STRENGTH ("8"),
. TERM_DDR("ON")
 )
  GTP_IOBUF_inst (
.I    (i),
.T    (t),
.IO   (io ),
```

.O (o)

);

5.6. GTP_IOBUFCO 使用说明

5.6.1. 支持器件

表 144 GTP_IOBUFCO 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.6.2. 功能介绍

GTP_IOBUFCO 是一个双向伪差分缓冲器，其只支持伪差分电平（与 GTP_IOBUFDS 相对应，GTP_IOBUFDS 只支持真差分电平）。其结构框图如下所示：

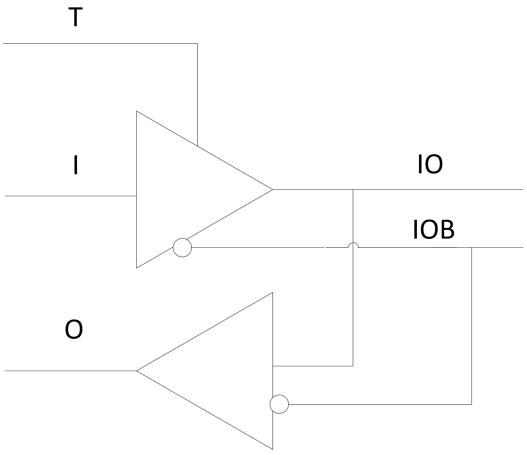


图 81 GTP_IOBUFCO

5.6.3. 端口描述

表 145 GTP_IOBUFCO 端口列表描述

端口名	输入/输出	描述
I	输入	输入信号
T	输入	三态使能信号
IO	输入/输出	差分 P 端输入/输出
IOB	输入/输出	差分 Q 端输入/输出

O	输出	输出信号

5.6.4. 参数描述

表 146 GTP_IOBUFCO 参数列表描述

参数名	描述	默认值	设置值
IOSTANDARD	IO 标准	"DEFAULT"	"SSTL18D_I","SSTL18D_II","SSTL15D_I","SSTL15D_II", "HSTL15D_I","SSTL25D_I","SSTL25D_II","SSTL15D_I_CAL", "SSTL15D_II_CAL", "HSTL15D_I_CAL", "LVPECL", "RSDS", "PPDS", "BLVDS", "DEFAULT"
TERM_DDR	HSTL、SSTL 标准输入时，内置 terminal resistor 开启或关闭	"ON"	"ON", "OFF"

5.6.5. 例化模板

```
GTP_IOBUFCO#(
   .IOSTANDARD ("DEFAULT"),
   . TERM_DDR    ("ON")
)
 GTP_IOBUFCO_inst(
.I     (i),
.T     (t),
.IO    (io),
.IOB(iob),
.O     (o)
);
```

5.7. GTP_IOBUFDS 使用说明

5.7.1. 支持器件

表 147 GTP_IOBUFDS 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.7.2. 功能介绍

GTP_IOBUFDS 是一个双向单信号差分输入 BUFFER。如下图:

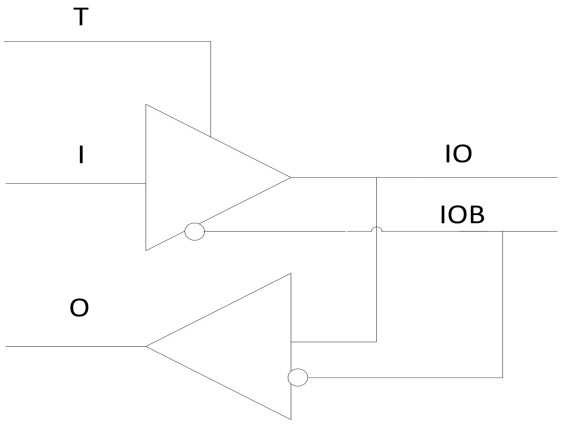


图 82 GTP_IOBUFDS

5.7.3. 端口描述

表 148 GTP_IOBUFDS 端口列表描述

端口名	输入/输出	描述
I	输入	输入信号
T	输入	三态输入使能,低有效
O	输出	输出信号
IO	输入输出	差分 P 输入输出（inout）
IOB	输入输出	差分 N 输入输出

5.7.4. 参数描述

表 149 GTP_IOBUFDS 参数列表描述

参数名	描述	默认值	设置值

IOSTANDARD	IO 标准	"DEFAULT"	"LVDS","MINI-LVDS","SUB-LVDS","TMDS"
TERM_DIFF	LVDS 等标准输入时，内置差分 terminal resistor 开启或关闭	"ON"	"ON", "OFF"

5.7.5. 例化模板

```
GTP_IOBUFDS#(
    .IOSTANDARD ("DEFAULT"),
   . TERM_DIFF    ("ON")
)
 GTP_IOBUFDS_inst (
.I          (i),
.T          (t),
.IO         (io),
.IOB(iob),
.O          (o)
);
```

5.8. GTP_OUTBUF 使用说明

5.8.1. 支持器件

表 150 GTP_OUTBUF 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.8.2. 功能介绍

GTP_OUTBUF 是一个输出 BUFFER。如下图:

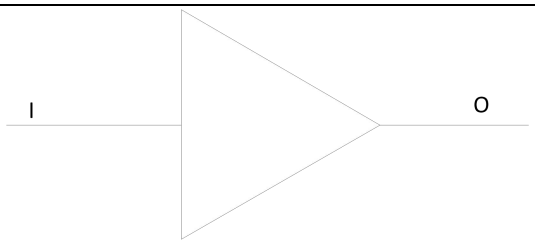


图 83 GTP_OUTBUF

5.8.3. 端口描述

表 151 GTP_OUTBUF 端口列表描述

端口名	输入/输出	描述
I	输入	输入信号
O	输出	输出信号

5.8.4. 参数描述

表 152 GTP_OUTBUF 参数列表描述

参数名	描述	默认值	设置值
IOSTANDARD	IO 标准	"DEFAULT"	IO 支持的标准包括"LVTTL33"，"PCI33"，"PCIX33"等
SLEW_RATE	Slew rate 属性	"SLOW"	"FAST","SLOW"
DRIVE_STRENGTH	输出驱动	"8"	2,4,6,8,12,16,24

5.8.5. 例化模板

```
GTP_OUTBUF#(
  .IOSTANDARD ("DEFAULT"),
  .SLEW_RATE ("SLOW"),
  .DRIVE_STRENGTH (8)
)
  GTP_OUTBUF_inst (
.I    (i),
.O    (o)
```

);

5.9. GTP_OUTBUFCO 使用说明

5.9.1. 支持器件

表 153 GTP_OUTBUFCO 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.9.2. 功能介绍

GTP_OUTBUFCO 是一个伪差分输出 buffer。如下图:

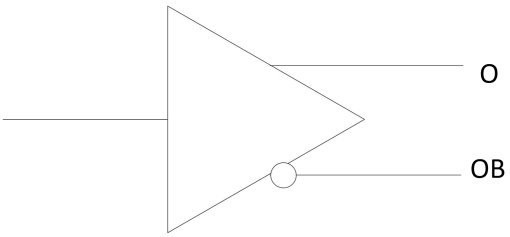


图 84 GTP_OUTBUFCO

5.9.3. 端口描述

表 154 GTP_OUTBUFCO 端口列表信息

端口名	输入/输出	描述
I	输入	输入信号
OB	输出	差分 N 输出信号
O	输出	差分 P 输出信号

5.9.4. 参数描述

表 155 GTP_OUTBUFCO 参数列表信息

参数名	描述	默认值	设置值
IOSTANDARD	IO 标准	"DEFAULT"	IO 支持的标准包括"SSTL18D_I"，"SSTL18D_II"，"SSTL15D_I"等

5.9.5. 例化模板

```
GTP_OUTBUFCO#(
    .IOSTANDARD ("DEFAULT")
)
 GTP_OUTBUFCO_inst (
.I     (i),
.OB(ob ),
.O    (o)
);
```

5.10. GTP_OUTBUFDS 使用说明

5.10.1. 支持器件

表 156 GTP_OUTBUFDS 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.10.2. 功能介绍

GTP_OUTBUFDS 是一个真差分输出 buffer 如下图:

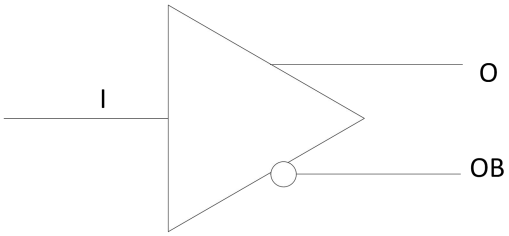


图 85 GTP_OUTBUFDS

5.10.3. 端口描述

表 157 GTP_OUTBUFDS 端口列表描述

端口名	输入/输出	描述

I	输入	输入信号
OB	输出	差分 N 输出信号
O	输出	差分 P 输出信号

5.10.4. 参数描述

表 158 GTP_OUTBUFDS 参数列表描述

参数名	描述	默认值	设置值
IOSTANDARD	IO 标准	"DEFAULT"	IO 支持的标准包括"LVDS", "MINI-LVDS", "SUB-LVDS","TMDS", "DEFAULT",等

5.10.5. 实例模板

```
GTP_OUTBUFDS#(
    .IOSTANDARD ("DEFAULT ")
)
 GTP_OUTBUFDS_inst (
.I    (i),
.OB(ob ),
.O   (o)
);
```

5.11. GTP_OUTBUFT 使用说明

5.11.1. 支持器件

表 159 GTP_OUTBUFT 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.11.2. 功能介绍

GTP_OUTBUFT 是一个三态输出 BUFFER。如下图:

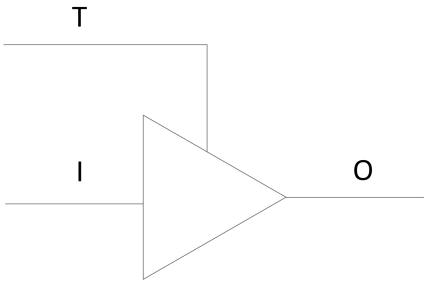


图 86 GTP_OUTBUFT

5.11.3. 端口描述

表 160 GTP_OUTBUFT 端口列表描述

端口名	输入/输出	描述
I	输入	输入信号
O	输出	输出信号
T	输入	三态使能输入，低有效

5.11.4. 参数描述

参数名	描述	默认值	设置值
IOSTANDARD	IO 标准	"DEFAULT"	IO 支持的标准包括"LVTTL33", "PCI33", "PCIX33", "LVCMOS33",等
SLEW_RATE	Slew rate 属性	"SLOW "	"FAST", "SLOW"
DRIVE_STRENGTH	输出驱动	"8"	2, 4, 6, 8, 12, 16, 24

5.11.5. 实例模板

```
GTP_OUTBUFT#(
  .IOSTANDARD ("DEFAULT"),
  .SLEW_RATE ("SLOW "),
  .DRIVE_STRENGTH (8)
)
  GTP_OUTBUFT_inst (
.I     (i),
.O     (o),
```

.T (t)

);

5.12. GTP_OUTBUFTCO 使用说明

5.12.1. 支持器件

表 161 GTP_OUTBUFTCO 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.12.2. 功能介绍

GTP_OUTBUFTCO 是一个三态输出伪差分 BUFFER。如下图:

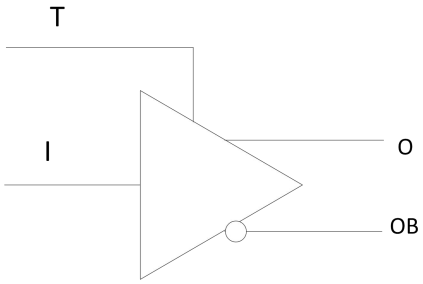


图 87 GTP_OUTBUFTCO

5.12.3. 端口描述

表 162 GTP_OUTBUFTCO 端口列表描述

端口名	输入/输出	描述
I	输入	输入信号
OB	输出	差分 N 输出信号
O	输出	差分 P 输出信号
T	输入	三态使能输入，低有效

5.12.4. 参数描述

参数名	描述	默认值	设置值
IOSTANDARD	IO 标准	"DEFAULT"	IO 支持的标准包括"SSTL18D_I",

			"SSTL18D_II", "SSTL15D_I",等

5.12.5. 实例模板

```
GTP_OUTBUFTCO#(
    .IOSTANDARD ("DEFAULT")
)
 GTP_OUTBUFTCO_inst (
.I    (i),
.T    (t),
.OB(ob ),
.O    (o)
);
```

5.13. GTP_OUTBUFTDS 使用说明

5.13.1. 支持器件

表 163 GTP_OUTBUFTDS 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.13.2. 功能介绍

GTP_OUTBUFTDS 是一个三态输出真差分 BUFFER。如下图:

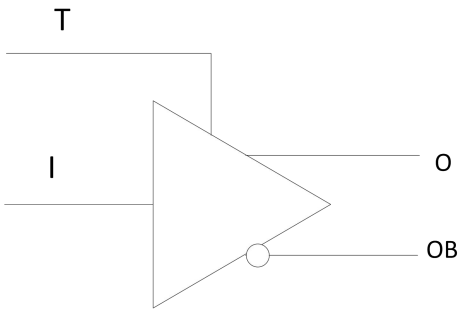


图 88 GTP_OUTBUFTDS

5.13.3. 端口描述

表 164 GTP_OUTBUFTDS 端口列表描述

端口名	输入/输出	描述
I	输入	输入信号
OB	输出	差分 N 输出信号
O	输出	差分 P 输出信号
T	输入	三态使能输入，低有效

5.13.4. 参数描述

参数名	描述	默认值	设置值
IOSTANDARD	IO 标准	"DEFAULT"	IO 支持的标准包括"LVDS", "MINI-LVDS", "SUB-LVDS","TMDS", "DEFAULT",等

5.13.5. 实例模板

```
GTP_OUTBUFTDS#(
    .IOSTANDARD ("DEFAULT")
)
 GTP_OUTBUFTDS_inst (
.I    (i),
.T    (t),
.OB  (ob ),
.O    (o)
```

);

5.14. GTP_IODELAY 使用说明

5.14.1. 支持器件

表 165 GTP_IODELAY 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.14.2. 功能介绍

每个 IO PAD 包含一个 IO DELAY 单元，可用于输入或输出延迟，可以提供 16 steps 的延迟每个 step 大约 25ps；它可以提供静态配置或动态调节的 DELAY 模式。典型应用 IO DELAY 常用于调节采样窗口或调整输出时序。结构框图如下所示：

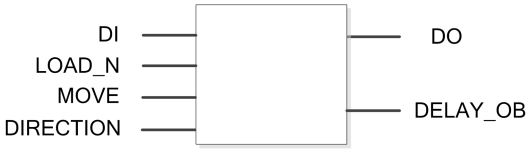


图 89 GTP_IODELAY

5.14.3. 端口描述

表 166 GTP_IODELAY 端口列表描述

端口名	输入/输出	描述
DI	输入	输入数据
LOAD_N	输入	低有效，将 delay step 复位至 DELAY_STEP 值。
MOVE	输入	下降沿触发动态微调，根据 DIRECTION 不同增加一个 step 或减小一个 step。
DIRECTION	输入	需动态增加 delay step 时置“0”，否则置“1”。
DO	输出	输出数据
DELAY_OB	输出	溢出指示信号，当 DIRECTION 置 0 时，内部 delay step 为 15 时变高；当 DIRECTION 置 1 时，delay step 为 0 时变高。

5.14.4. 参数描述

参数名	描述	默认值	设置值

DELAY_STEP	Step 数设置，0~15 对应延迟 1~16 个 step	0	0 ~ 15
DELAY_DEPTH	Step 数位宽设置	7	7，4。默认为 7

5.14.5. 实例模板

```
GTP_IODELAY
#(
.DELAY_STEP     (7'd0),         //set delay parameter
.DELAY_DEPTH   (7)
)I_GTP_IODELAY(
.DI             (din),
.DO             (dout),
.LOAD_N         (load_n),
.MOVE           (move),
.DIRECTION      (direction),
.DELAY_OB       (delay_ob)
);
```

5.15. GTP_IOBUF_RX_MIPI 使用说明

5.15.1. 支持器件

表 167 GTP_IOBUF_RX_MIPI 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.15.2. 功能介绍

GTP_IOBUF_RX_MIPI 支持 MIPI DPHY 高速输入，及低功耗模式（LP）下单端输入、输出。

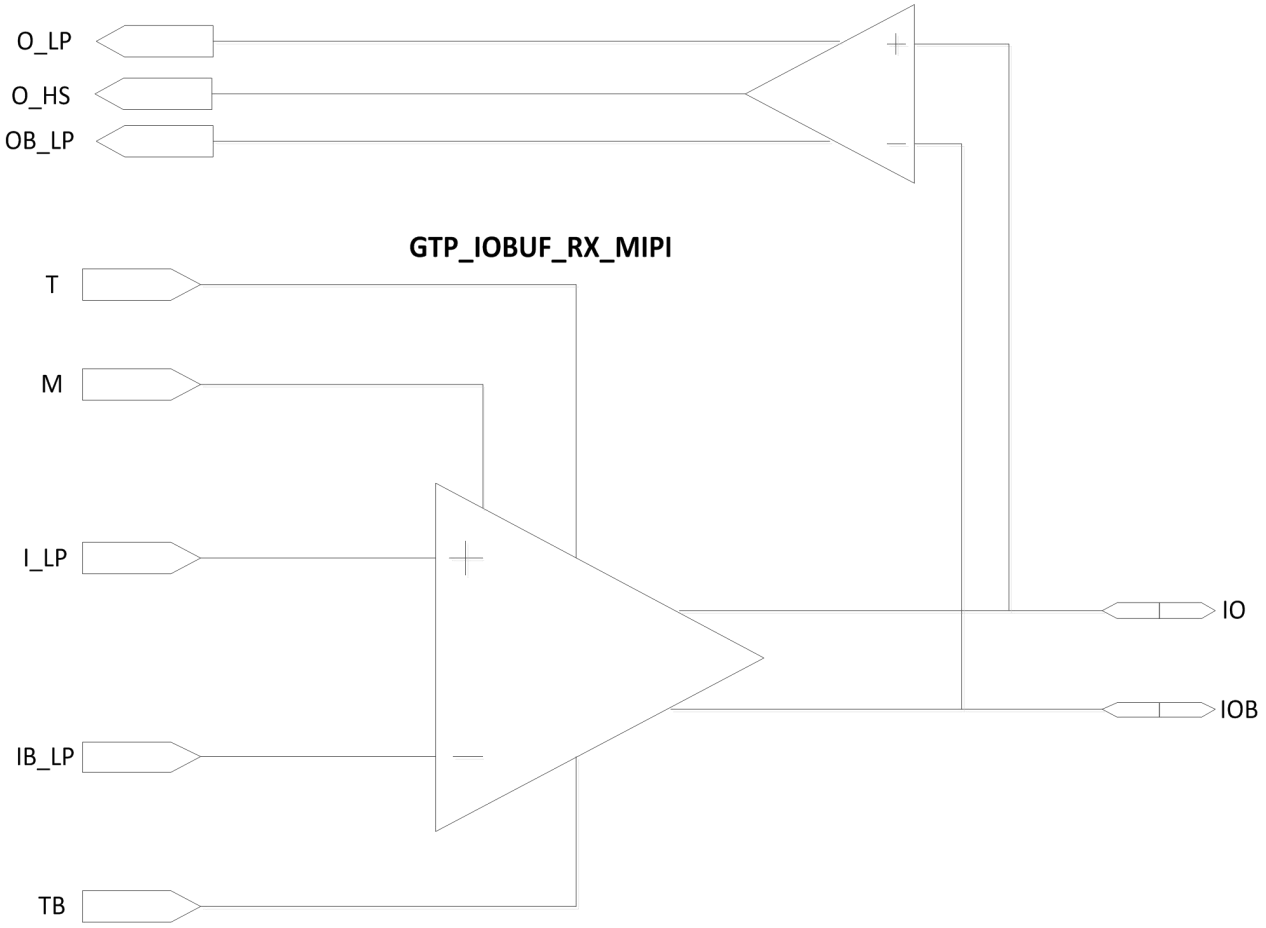


图 90 GTP_IOBUF_RX_MIPI

5.15.3. 端口描述

表 168 GTP_IOBUF_RX_MIPI 端口列表描述

端口名	输入/输出	描述
I_LP	输入	LP模式下单端输出buffer的输入信号，来自fabric
IB_LP	输入	LP 模式下单端输出buffer的输入信号，来自fabric
M	输入	模式选择信号. 1: HS 模式，差分输入；0：LP模式，来自fabric
T	输入	单端输出使能信号
TB	输入	单端输出使能信号
O_HS	输出	差分输出(HS)到fabric
O_LP	输出	单端（LP）输出
OB_LP	输出	单端（LP）输出

IO	双向	同相差分输入（HS）和输入/输出（LP）
IOB	双向	反向差分输入（HS）和输入/输出（LP）

5.15.4. 参数描述

参数名称	参数类型	有效值	默认值	功能描述
IOSTANDARD	string	MIPI	DEFAULT	输入IO标准
DRIVE_STRENGTH	string	“2”,“6”	“6”	驱动电流强度
SLEW_RATE	string	“SLOW” , “FAST”	“SLOW”	摆率
TERM_DIFF	string	“ON”, “OFF”	“OFF”	差分输入时，内置terminal resistor 开启或关闭
HPIO	string	“TRUE”, “FALSE”	“FALSE”	映射到HPIO BANK为“TRUE”; 映射到HRIO BANK为“FALSE”;

注：Logos 系列不支持 HPIO，故参数 HPIO 必须设置为“FALSE”。

5.15.5. 实例模板

```
GTP_IOBUF_RX_MIPI #(
    .HPIO("FALSE"),
    .IOSTANDARD("DEFAULT"),
    .SLEW_RATE("SLOW"),
    .DRIVE_STRENGTH("6"),
    .TERM_DIFF("ON")
) GTP_IOBUF_RX_MIPI_inst (
    .IO(),      // INOUT
    .IOB(),     // INOUT
    .OB_LP(),// OUTPUT
    .O_HS(), // OUTPUT
    .O_LP(), // OUTPUT
    .IB_LP(),// INPUT
    .I_LP(), // INPUT
```

```
    .M(),       // INPUT
    .T(),       // INPUT
    .TB()       // INPUT
);
```

5.16. GTP_IOBUF_TX_MIPI 使用说明

5.16.1. 支持器件

表 169 GTP_IOBUF_TX_MIPI 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.16.2. 功能介绍

GTP_IOBUF_TX_MIPI 支持 MIPI DPHY 三态输出，包含低功耗模式（LP）下输入/输出及高速(HS)模式下差分输出，且两个模式根据实际应用可以自由切换。

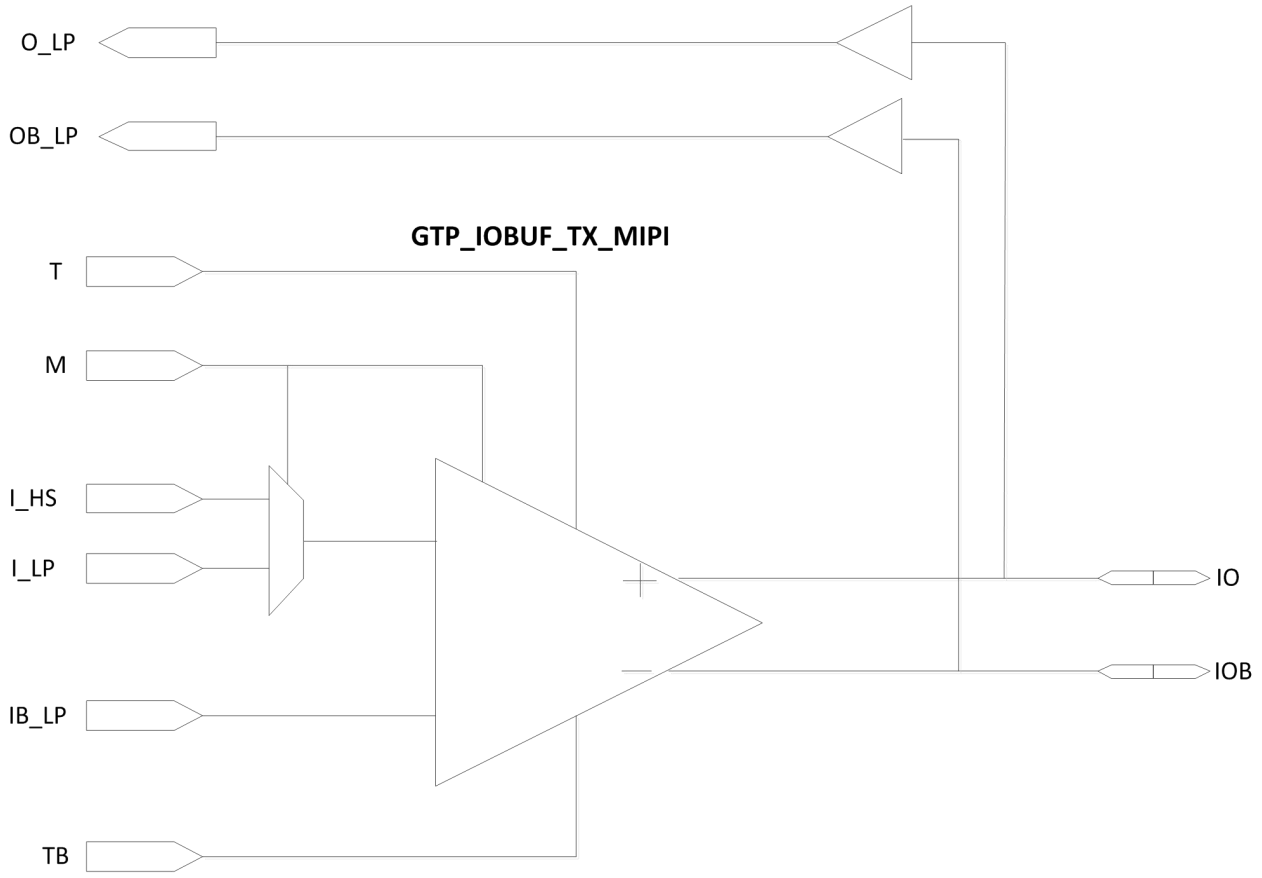


图 91 GTP_IOBUF_TX_MIPI

5.16.3. 端口描述

表 170 GTP_IOBUF_TX_MIPI 端口列表描述

端口名	输入/输出	描述
I_HS	输入	HS 差分输出buffer的输入信号，来自IOL
I_LP	输入	LP 模式下单端输出buffer的输入信号，来自IOL
IB_LP	输入	LP 模式下单端输出buffer的输入信号，来自IOL
M	输入	模式选择信号. 1: HS 模式，差分输出；0：LP模式，单端输入输出。来自IOL
T	输入	差分及单端输出使能信号
TB	输入	单端输出使能信号
O_LP	输出	单端（LP）输入到IOL
OB_LP	输出	单端（LP）输入到IOL
IO	双向	同相差分输出
IOB	双向	反向差分输出

5.16.4. 参数描述

参数名称	参数类型	有效值	默认值	功能描述
IOSTANDARD	string	MIPI	DEFAULT	输入IO标准 VDDIO=1.2V
DRIVE_STRENGTH	string	“2”, “6”	“6”	驱动电流强度
SLEW_RATE	string	“SLOW” , “FAST”	“SLOW”	摆率
TERM_DIFF	string	“ON”, “OFF”	“ON”	差分输入时，内置 terminal resistor 开启或关闭

5.16.5. 实例模板

```
GTP_IOBUF_TX_MIPI #
(
.IOSTANDARD ( "DEFAULT" ),//"MIPI","DEFAULT"
.DRIVE_STRENGTH ( "2" ),
.SLEW_RATE ( "FAST" ),
.TERM_DIFF ( "OFF" )
)
GTP_IOBUF_TX_MIPI_INST
(
.O_LP     (O_LP ),
.OB_LP    (OB_LP ),
.IO       ( IO ),
.IOB      ( IOB ),
.I_HS     ( I_HS ),
.I_LP     ( I_LP ),
.IB_LP    ( IB_LP),
.T        ( T ),
.TB       ( TB ),
.M        ( M )
);
```

5.17. GTP_ISERDES 使用说明

5.17.1. 支持器件

表 171 GTP_ISERDES 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.17.2. 功能介绍

IOL 灵活支持各种应用接口，除了常用的直接输入/输出和输入/输出寄存器之外，IOL 具有数据输入输出速率转换功能。GTP_ISERDES 是数据输入解串器，在上升沿和下降沿进行双

沿数据采样，并支持 1:2、1:4、1:7、1:8 速率转换。其结构框图如下所示：

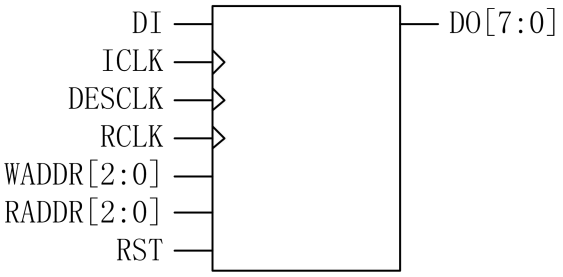


图 92 GTP_ISERDES 结构框图

5.17.3. 端口描述

表 172 GTP_ISERDES 端口列表

端口名称	宽度	方向	功能描述
DI	1	输入	数据输入
WADDR	3	输入	与DDC相连FIFO写控制指针
RADDR	3	输入	与DDC相连FIFO读控制指针
ICLK	1	输入	外部输入时钟
DESCLK	1	输入	解串时钟
RCLK	1	输入	输出系统时钟
RST	1	输入	复位信号，高有效
DO	8	输出	数据输出

5.17.4. 参数描述

表 173 GTP_ISERDES 参数列表

参数名称	参数类型	有效值	默认值	功能描述
ISERDES_MODE	String	“IDDR” “IMDDR” “IDES4” “IMDES4” “IDES7” “IDES8” “IMDES8”	“IDDR”	解串器模式： “IDDR”: General DDR 1:2解串模式 “IMDDR”: Memory DDR 1:2解串模式 “IDES4”: General DDR 1:4解串模式 “IMDES4”: Memory DDR 1:4解串模式 “IDES7”: General DDR 1:7解串模式 “IDES8”: General DDR 1:8解串模式 “IMDES8”: Memory DDR 1:8解串模式
GRS_EN	String	“TRUE”	“TRUE”	全局复位使能信号

		“FLASE”		“TRUE”：使能；”FALSE”：不使能
LRS_EN	String	“TRUE” “FLASE”	“TRUE”	局部复位使能信号 “TRUE”：使能；”FALSE”：不使能

5.17.5. 例化模板

```
GTP_ISERDES #(
        .ISERDES_MODE("IDDR"),
        .GRS_EN("TRUE"),
        .LRS_EN("TRUE")
) GTP_ISERDES_inst (
        .DO(),       // OUTPUT[7:0]
        .RADDR(), // INPUT[2:0]
        .WADDR(), // INPUT[2:0]
        .DESCLK(),// INPUT
        .DI(),       // INPUT
        .ICLK(),    // INPUT
        .RCLK(),    // INPUT
        .RST()       // INPUT
    );
```

5.18. GTP_OSERDES 使用说明

5.18.1. 支持器件

表 174 GTP_OSERDES 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

5.18.2. 功能介绍

IOL 灵活支持各种应用接口，除了常用的直接输入/输出和输入/输出寄存器之外，IOL 具有数据输入输出速率转换功能。GTP_OSERDES 是 DDR 数据输出转换模块支持 2:1、4:1、7:1、8:1 速率转换功能。其结构框图如下所示：

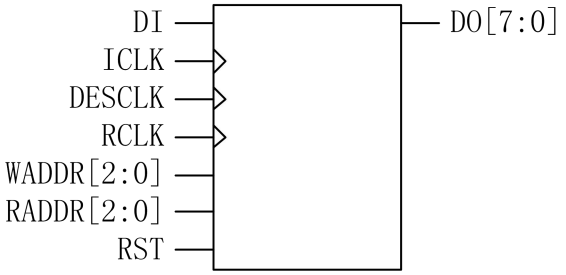


图 93 GTP_OSERDES 结构框图

5.18.3. 端口描述

表 175 GTP_OSERDES 端口列表

端口名称	宽度	方向	功能描述
DI	输入	8	数据输入
TI	输入	4	选通数据输入
RCLK	输入	1	输入系统时钟
SERCLK	输入	1	串行时钟
OCLK	输入	1	输出时钟
RST	输入	1	复位信号，高有效
DO	输出	1	数据输出
TQ	输出	1	输出选通

5.18.4. 参数描述

表 176 GTP_OSERDES 参数列表

参数名称	参数类型	有效值	默认值	功能描述
OSERDES_MODE	String	“ODDR” “OMDDR” “OSER4” “OMSER4” “OSER7” “OSER8” “OMSER8”	"ODDR"	输出转换器模式： “ODDR”: General DDR 1:2输出转换模式 “OMDDR”: Memory DDR 1:2输出转换模式 “OSER4”: General DDR 1:4输出转换模式 “OMSER4”: Memory DDR 1:4输出转换模式 “OSER7”: General DDR 1:7输出转换模式 “OSER8”: General DDR 1:8输出转换模式 “OMSER8”: Memory DDR 1:8输出转换模式
WL_EXTEND	String	“TRUE” “FALSE”	“FALSE”	Write Leveling 扩展
GRS_EN	String	“TRUE” “FLASE”	“TRUE”	全局复位使能信号 “TRUE”: 使能；”FALSE”: 不使能
LRS_EN	String	“TRUE” “FLASE”	“TRUE”	局部复位使能信号 “TRUE”: 使能；”FALSE”: 不使能
TSDDR_INIT	Bit	1’b0,1’b1	1’b0	TO初始态控制， 1’b0:TO初始态为0, 1’b1：TO初始态为1

5.18.5. 例化模板

```
GTP_OSERDES #(
        .OSERDES_MODE("ODDR"),
        .WL_EXTEND("FALSE"),
        .GRS_EN("TRUE"),
        .LRS_EN("TRUE"),
        .TSDDR_INIT(1'b0)
) GTP_OSERDES_inst (
        .DI(),       // INPUT[7:0]
        .TI(),       // INPUT[3:0]
        .DO(),       // OUTPUT
        .TQ(),       // OUTPUT
        .OCLK(),    // INPUT
        .RCLK(),    // INPUT
        .RST(),     // INPUT
        .SERCLK() // INPUT
    );
```

6. 时钟相关 GTP 使用说明

6.1. GTP_CLKBUFG

6.1.1. 支持器件

表 177 GTP_CLKBUFG 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

6.1.2. 功能介绍

GTP_CLKBUFG 提供了简单的时钟 BUFFER 功能，时钟上全局可以通过例化 GTP_CLKBUFG 来实现。

6.1.3. 端口描述

表 178 GTP_CLKBUFG 的端口说明

端口信号	输入/输出	描述
CLKIN	输入	输入时钟
CLKOUT	输出	输出时钟

6.1.4. 例化模块

```
GTP_CLKBUFG U_CLKBUFG (
    .CLKOUT          (CLKOUT),
    .CLKIN           (CLKIN )
);
```

6.1.5. 详细功能介绍

GTP_CLKBUFG 提供了简单的时钟 BUFFER 功能，时钟上全局可以通过例化 GTP_CLKBUFG 来实现。

6.2. GTP_CLKBUFGCE

6.2.1. 支持器件

表 179 GTP_CLKBUFGCE 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

6.2.2. 功能介绍

GTP_CLKBUFGCE 属于带有使能控制端口的时钟 BUFFER，时钟上全局也可以通过例化 GTP_CLKBUFGCE 来实现。

6.2.3. 端口描述

表 180 GTP_CLKBUFGCE 的端口说明

端口信号	输入/输出	说明
CLK	输入	输入时钟
CE	输入	时钟使能，高有效使能
CLKOUT	输出	输出时钟

6.2.4. 参数描述

表 181 GTP_CLKBUFGCE 的参数说明

参数	说明
DEFAULT_VALUE	0/1；CE为0时控制输出时钟CLKOUT的状态

6.2.5. 例化模块

```
GTP_CLKBUFGCE
#(.DEFAULT_VALUE        (1'b0           )     //1'b0; 1'b1
) I_GTP_CLKBUFGCE (
    .CLKIN              (CLK            ),
    .CE                 ( CE      ),
    .CLKOUT             (CLKOUT     )
);
```

6.2.6. 详细功能介绍

当使能 CE="1"或"0"时对应的波形图如下所示：

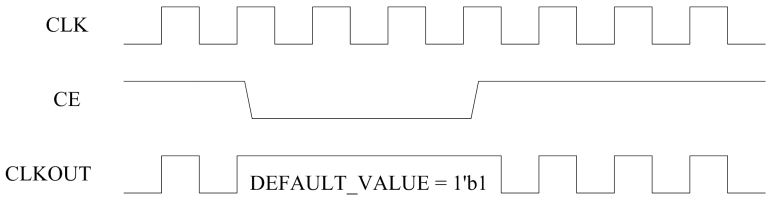


图 94 GTP_CLKBUFGCE 波形图（DEFAULT_VALUE=1）

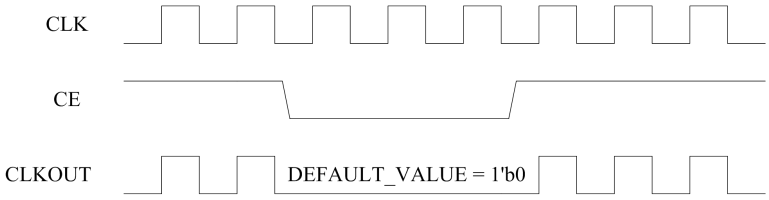


图 95 GTP_CLKBUFGCE 波形图（DEFAULT_VALUE=0）

6.3. GTP_CLKBUFGMUX

6.3.1. 支持器件

表 182 GTP_CLKBUFGMUX 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

6.3.2. 功能介绍

GTP_CLKBUFGMUX 可实现在两个全局时钟输入源源之间的动态切换，用户可以根据需要选择强制切换（对应 TRIGGER_MODE="NORMAL"），或者由时钟下降沿触发的去毛刺切换（对应 TRIGGER_MODE=" NEGEDGE "）。值得注意的是强制切换可能会引入毛刺，切换后用户需对相关逻辑复位，以实现预期功能。

6.3.3. 端口描述

表 183 GTP_CLKBUFGMUX 的端口说明

端口信号	输入/输出	说明
CLKIN0	输入	输入时钟CLKIN0
CLKIN1	输入	输入时钟CLKIN1

SEL	输入	时钟选择信号，0选择CLKIN0；1选择CLKIN1；
CLKOUT	输出	输出时钟

6.3.4. 参数描述

表 184 GTP_CLKBUFGMUX 的参数说明

参数名称	设置值	默认值	功能描述
TRIGGER_MODE	"NORMAL" "NEGEDGE" "POSEDGE"	"NORMAL"	（1）"NORMAL"：此模式下，自由切换输入时钟（CLKIN0和CLKIN1），不提供GLITCHLESS功能 （2）"NEGEDGE"：此模式下，自由切换输入时钟（CLKIN0和CLKIN1），提供GLITCHLESS功能，由时钟下降沿触发。 （3）"POSEDGE"：此模式下，自由切换输入时钟（CLKIN0和CLKIN1），提供GLITCHLESS功能，由时钟上升沿触发。
SIM_DEVICE	"TITAN" "LOGOS" "COMPACT" "LOGOS2" "TITAN2"	"TITAN"	仿真模型器件识别 "TITAN":在上升沿或者下降沿触发时，在两个时钟域下分别打1拍就切换 "LOGOS""COMPACT""LOGOS2""TITAN2":在上升沿或者下降沿触发时，在两个时钟域下分别打2拍才切换

注：1.LOGOS 系列只支持"NORMAL"，"NEGEDGE"两种模式，其它器件支持情况请参考对应的时钟资源用户指南。

2.TRIGGER_MODE 为值为"NEGEDGE"或"POSEDGE"时，需要分别在两个时钟域下采集时钟上升沿或下降沿才能切换切换时钟，因此必须 CLKIN0/CLKIN1 两端口都有时钟上升沿或下降沿时该 GTP 才能够正常切换时钟。

6.3.5. 例化模块

```
GTP_CLKBUFGMUX
#(
.TRIGGER_MODE ("NORMAL"),
.SIM_DEVICE("TITAN")

)I_CLKBUFGMUX          (
.CLKIN0      (clkin0   ),
.CLKIN1      (clkin1   ),
.SEL         (sel      ),
.CLKOUT      (clkout )
);
```

6.3.6. 详细功能介绍

例化 GTP_CLKBUFGMUX 可实现在两个全局时钟输入源之间的动态切换，用户可以根据需要选择强制切换（对应 TRIGGER_MODE="NORMAL"），或者由时钟下降沿触发的去毛刺切换（对应 TRIGGER_MODE=" NEGEDGE "）。值得注意的是强制切换可能会引入毛刺，切换后用户需对相关逻辑复位，以实现预期功能。当参数 TRIGGER_MODE 值为"NORMAL"，SEL 信号跳变后，CLKOUT 立刻被切换，对应的时序图如下：

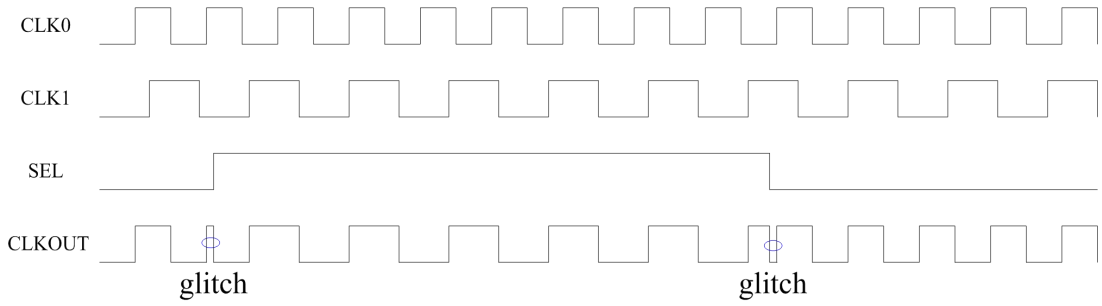


图 96 GTP_CLKBUFGMUX 的参数 TRIGGER_MODE="NORMAL"的时序图

当参数 TRIGGER_MODE 值为"NEGEDGE"，参数 SIM_DEVICE 值为"LOGOS"时，SEL 从 0 变化到 1，需要先检测 2 次当前时钟 CLKIN0 的下降沿 2 次（在检测当前时钟上升沿的过程中，CLKOUT 输出维持当前时钟），再检测被切换时钟 CLKIN1 的下降沿 2 次（在检测被切换时钟下降沿的过程中，CLKOUT 输出低电平），检测完成后 CLKOUT 由 CLKIN0 切换到 CLKIN1；反之，当 SEL 从 1 变化到 0 时，先检测 CLKIN1 的下降沿 2 次（在检测当前时钟上升沿的过程中，CLKOUT 输出维持当前时钟），再检测 CLKIN0 的下降沿 2 次（检测被切换时钟 CLKIN0 下降沿的过程中，CLKOUT 输出低电平），检测完成后 CLKOUT 由 CLKIN1 切换到 CLKIN0。对应的时序图如下：

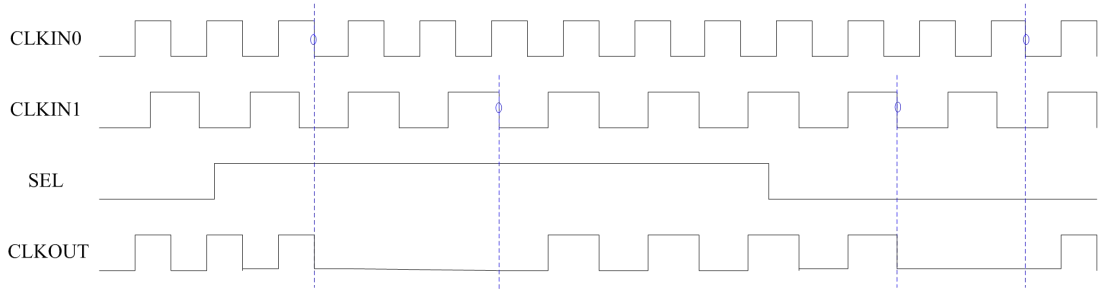


图 97 GTP_CLKBUFGMUX 的参数 TRIGGER_MODE="NEGEDGE"的时序图

6.4. GTP_CLKBUFR

6.4.1. 支持器件

表 185 GTP_CLKBUFR 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

6.4.2. 功能介绍

GTP_CLKBUFR 是一个区域时钟的 BUFFER，区域时钟驱动某一个区域的单元逻辑。

6.4.3. 端口描述

表 186 GTP_CLKBUFR 的端口说明

端口信号	输入/输出	描述
CLKIN	输入	输入时钟
CLKOUT	输出	输出时钟

6.4.4. 例化模块

```
GTP_CLKBUFR    U_BUFR(
    .CLKOUT          (CLKOUT),
    .CLKIN           (CLKIN    )
);
```

6.4.5. 详细功能介绍

GTP_CLKBUFR 是一个区域时钟的 BUFFER，区域时钟驱动某一个区域的单元逻辑。

下图所示的是 GTP_CLKBUFR 的输入输出波形图

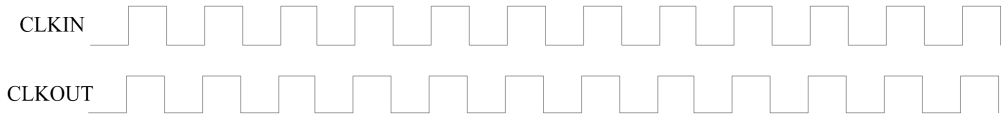


图 98 GTP_CLKBUFR 的波形图

6.5. GTP_IOCLKBUF

6.5.1. 支持器件

表 187 GTP_IOCLKBUF 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

6.5.2. 功能介绍

GTP_IOCLKBUF 相当于 gate 功能，例化 GTP_IOCLKBUF 来关闭或者开启 IO 时钟。

6.5.3. 端口描述

表 188 GTP_IOCLKBUF 端口说明

端口信号	输入/输出	描述
CLKIN	输入	输入时钟
DI	输入	使能信号
CLKOUT	输出	输出时钟

6.5.4. 参数描述

表 189 GTP_IOCLKBUF 的参数说明

参数名	描述
GATE_EN	"TRUE" 时使能信号DI有效；"FALSE" 时使能信号DI无效；

6.5.5. 例化模块

```
GTP_IOCLKBUF
#(
    .GATE_EN            ("FALSE"    )   //FALSE; TRUE
) u_IOCLKBUF    (
.CLKOUT             (CLKOUT     ),
.CLKIN              (CLIN       ),
.DI                 (DI         )
);
```

6.5.6. 详细功能介绍

GTP_IOCLKBUF 相当于 gate 功能，例化 GTP_IOCLKBUF 来关闭或者开启 IO 时钟。当参数 GATE_EN 为 "TRUE" 或 "FALSE" 时，其对应的波形图如下两图。

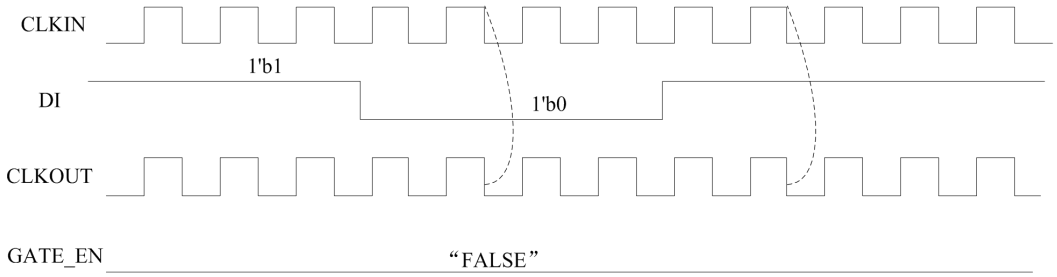


图 99 GATEN="FALSE"的 GTP_IOCLKBUF 时序图

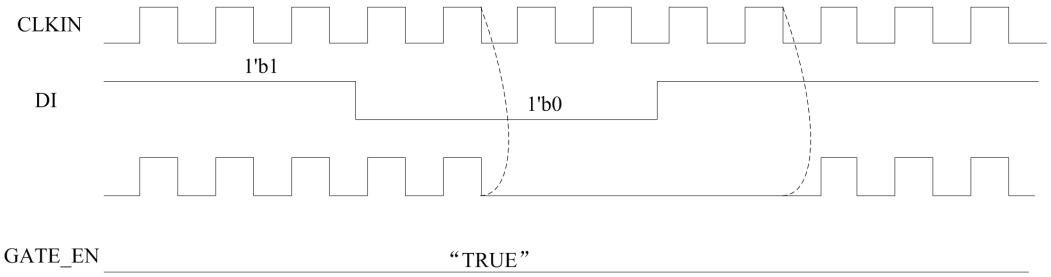


图 100 GATEN="TRUE "的 GTP_IOCLKBUF 时序图

6.6. GTP_IOCLKDIV

6.6.1. 支持器件

表 190 GTP_IOCLKDIV 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

6.6.2. 功能介绍

GTP_IOCLKDIV 对来自于 GTP_IOCLKBUF 的时钟进行 2、3.5、4、5 分频。

6.6.3. 端口描述

表 191 GTP_IOCLKDIV 的端口说明

端口信号	输入/输出	描述
CLKIN	输入	输入时钟
RST_N	输入	低有效复位
CLKDIVOUT	输出	分频输出时钟

6.6.4. 参数描述

表 192 GTP_IOCLKDIV 的参数说明

参数名	描述
DIV_FACTOR	当为"DIV_DIS"时，CLKOUT=1’b1,CLKDIVOUT=1’b1，为分频比，"2"; "3.5"; "4";时，CLKOUT=CLKIN, CLKDIVOUT输出分频时钟。

GRS_EN	全局使能信号，“TRUE”，全局使能信号有效；“FALSE”，全局使能信号无效。

6.6.5. 例化模块

```
GTP_IOCLKDIV #(
.DIV_FACTOR ("2"            ),      //"2"; "3.5"; "4"; "5";
.GRS_EN       ("FALSE"    ) //"TRUE"; "FALSE"
)I_IOCLKDIV(
.CLKIN              (CLKIN       ),
.RST_N              (RST_N       ),
.CLKDIVOUT      (CLKDIVOUT)
);
```

6.6.6. 详细功能介绍

GTP_IOCLKDIV 对来自于 GTP_IOCLKBUF 的时钟进行 2、3.5、4、5 分频。分频之后的时钟可作为全局时钟的输入源。

6.7. GTP_IOCLKDELAY

6.7.1. 支持器件

表 193 GTP_IOCLKDELAY 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

6.7.2. 功能介绍

为满足一些应用中调整输入时钟相位的需求，时钟输入管脚提供了可选的 IOCLKDELAY 功能。用户可以静态配置 delay step（调节延迟链的数量，每个 step 的 delay 约为 25ps，最小的延迟链数量为 0，最大的延迟链数量为 127（8’b0~8’b0111_1111）），也可以利用片上 DLL 获得动态的 delay step，更可以在动/静态配置的基础上通过用户控制逻辑，动态微调 delay step。

6.7.3. 端口描述

表 194 GTP_IOCLKDELAY 端口说明

端口信号	输入/输出	说明
CLKIN	输入	来自管脚的时钟输入
DELAY_STEP[7:0]	输入	来自DLL的delay step
DIRECTION	输入	置0为动态增加delay step，置1为动态减少delay step。

端口信号	输入/输出	说明
LOAD	输入	高有效，将delay step复位至DELAY_STEP或静态delay值。
MOVE	输入	下降沿触发动态微调，根据DIRECTION不同增加一个step或减小一个step。
DELAY_OB	输出	Delay step动态微调溢出标志，当DIRECTION置0时，内部delay step为255时变高；当DIRECTION置1时，内部delay step为0时变高。
CLKOUT	输出	Delay后的时钟输出

6.7.4. 参数描述

表 195 GTP_IOCLKDELAY 端口说明

参数名称	参数类型	有效值	默认值	功能描述
DELAY_STEP_VALUE	binary	8’d0~8’d255	8’d0	静态delay step，当参数DELAY_STEP_SEL为“PARAMETER”时有效
DELAY_STEP_SEL	string	”PARAMETER”，“PORT”	“PARAMETER”	“PORT”选择来自DLL的DELAY_STEP为基准delay step；“PARAMETER”选择CLKDLY_STEP为基准delay step
SIM_DEVICE	string	“TITAN”,”LOGOS”,”COMPACT”	“TITAN”	设置“TITAN”，CLOCK STEP最大值255；设置“LOGOS”或“COMPACT”，CLOCK STEP最大值127。

6.7.5. 例化模块

```
GTP_IOCLKDELAY #(
    .DELAY_STEP_VALUE('b00000000),
    .DELAY_STEP_SEL("PARAMETER"),
    .SIM_DEVICE("TITAN")
) <InstanceName> (
    .DELAY_STEP(),// INPUT[7:0]
    .CLKOUT(),      // OUTPUT
    .DELAY_OB(),    // OUTPUT
    .CLKIN(),       // INPUT
    .DIRECTION(), // INPUT
    .LOAD(),        // INPUT
    .MOVE()         // INPUT
);
```

6.7.6. 详细功能介绍

参考《UG020004_Logos 系列 FPGA 时钟资源（Clock）用户指南》

6.8. GTP_OSC_E1

6.8.1. 支持器件

表 196 GTP_OSC_E1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	不支持	不支持	不支持	不支持

6.8.2. 功能介绍

为用户提供可配置的输出时钟。

6.8.3. 端口描述

表 197 GTP_OSC_E1 端口说明

端口名	类型	描述
CLKOUT	输出	输出时钟，频率可以随着参数值得设置而改变
RST_N	输出	复位，低电平有效
EN	输入	使能，高电平有效

6.8.4. 参数描述

表 198 GTP_OSC_E1 参数说明

参数名	默认值	描述
CLK_DIV	2	时钟分频比设置，有效值为 0~127

6.8.5. 例化模块

```
GTP_OSC_E1
#(
. CLK_DIV          (2                )
) U_OSC_E1 (
        .CLKOUT    (CLKOUT      ),
        .EN        (EN          ),
```

```
    .RST_N      (RST_N      )
    );
```

6.8.6. 详细功能介绍

为用户提供可配置的输出时钟，输出的时钟可以作为全局时钟的输入源。输出的时钟与配置的分频系数对应关系如下：

表 199 GTP_OSC_E1 分频系数与输出时钟关系

CLK_DIV	CLKOUT(MHz)
0	200/128
1	200/1
2	200/2
3	200/3
…	…
125	200/125
126	200/126
127	200/127

6.9. GTP_OSC_E3

6.9.1. 支持器件

表 200 GTP_OSC_E3 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	不支持	支持	支持	支持	支持

6.9.2. 功能介绍

为用户提供可配置的输出时钟。

6.9.3. 端口描述

表 201 GTP_OSC_E3 端口说明

端口名	类型	描述
CLKOUT	输出	输出时钟，频率可以随着参数值得设置而改变

RST_N	输出	复位，低电平有效
EN_N	输入	使能，高电平有效

6.9.4. 参数描述

表 202 GTP_OSC_E3 参数说明

参数名	默认值	描述
CLK_DIV	2	时钟分频比设置，有效值为 0~127

6.9.5. 例化模块

```
GTP_OSC_E3
#(
. CLK_DIV          (2                )
) U_OSC_E3(
        .CLKOUT    (CLKOUT      ),
        .EN_N      (EN          ),
        .RST_N     (RST_N       )
        );
```

6.9.6. 详细功能介绍

为用户提供可配置的输出时钟，输出的时钟可以作为全局时钟的输入源。

表 203 GTP_OSC_E3 分频系数与输出时钟关系

CLK_DIV	CLKOUT(MHz)
0	200/128
1	200/1
2	200/2
3	200/3
…	…
125	200/125
126	200/126
127	200/127

6.10. GTP_CRYSTAL

6.10.1. 支持器件

表 204 GTP_CRYSTAL 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

6.10.2. 功能介绍

GTP_CRYSTAL 的输出时钟可以作为全局时钟驱动逻辑单元，也可以作为 IO 时钟，其中端口 XTALA 和 XTALB 是差分信号。

6.10.3. 端口描述

表 205 GTP_CRYSTAL 的端口说明

端口信号	输入/输出	描述
EN_N	输入	使能信号，低电平有效
XTALA	输入	给晶体的一个输入管脚
XTALB	输入	给晶体的另个输入管脚，其中XTALA和XTALB为差分信号
CLKOUT	输出	输出时钟

6.10.4. 参数描述

表 206 GTP_CRYSTAL 的参数说明

参数	描述
CLK_EN	时钟使能配置，有效值为“TRUE”或“FALSE”。

6.10.5. 例化模块

```
GTP_CRYSTAL #(
    .CLK_EN     ("TRUE"     )     // "TRUE", "FALSE"
) u_crystal     (
    .CLKOUT     (CLKOUT     ),
    .XTALA      (XTALA      ),
    .XTALB      (XTALB      ),
    .EN_N       (EN_N       )
);
```

6.10.6. 详细功能介绍

当参数 CLK_EN 配置为 TRUE，EN_N 为 0 时的波形图如下。

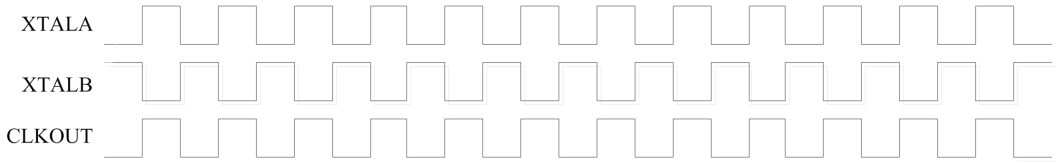


图 101 GTP_CRYSTAL 的波形图

6.11. GTP_PLL_E1

6.11.1. 支持器件

表 207 GTP_PLL_E1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	不支持	不支持	不支持	不支持

6.11.2. 功能介绍

主要实现分频，倍频，相位调整功能。该 GTP 支持输入时钟的动态选择，内外反馈模式，动态重配置，输出时钟 gate 功能。

6.11.3. 端口描述

表 208 GTP_PLL_E1 的端口说明

Port	方向	功能描述
CLKOUT0	输出	输出 divider0 分频输出时钟；
CLKOUT0_EXT	输出	输出 divider0 分频输出时钟，与 CLKOUT0 为同源时钟；
CLKOUT1	输出	输出 divider1 分频输出时钟；
CLKOUT2	输出	输出 divider2 分频输出时钟；
CLKOUT3	输出	输出 divider3 分频输出时钟；
CLKOUT4	输出	输出 divider4 分频输出时钟或输入时钟 bypass 输出；
CLKOUT5	输出	输出 divider0~4 分频输出 5 选 1 后时钟，由静态配置 CLKOUT5_SEL 选择；
CLKSWITCH_FLAG	输出	时钟自动切换标识，在时钟动态选择的自动模式下： 为 0 时，表示选择 CLKIN1 作为 PLL 基准时钟； 为 1 时，表示选择 CLKIN2 作为 PLL 基准时钟；
LOCK	输出	PLL 锁定输出信号；为 0 时，PLL 未锁定；为 1 时，PLL 已锁定；
CLKIN1	输入	输入时钟 1；
CLKIN2	输入	输入时钟 2；
CLKFB	输入	反馈时钟；
CLKIN_SEL	输入	输入时钟动态选择的手动选择端口；

Port	方向	功能描述
CLKIN_SEL_EN	输入	CLKIN_SEL 端口使能： 为 0 时，CLKIN_SEL 端口输入无效，动态选择的自动模式； 为 1 时，CLKIN_SEL 端口输入有效，动态选择的手动模式；
PFDEN	输入	PLL PFD 使能，高电平有效；
CLKOUT0_SYN	输入	输出时钟 CLKOUT0 GATE 控制，高电平有效；
CLKOUT0_EXT_SYN	输入	输出时钟 CLKOUT0_EXT GATE 控制，高电平有效；
CLKOUT1_SYN	输入	输出时钟 CLKOUT1 GATE 控制，高电平有效；
CLKOUT2_SYN	输入	输出时钟 CLKOUT2 GATE 控制，高电平有效；
CLKOUT3_SYN	输入	输出时钟 CLKOUT3 GATE 控制，高电平有效；
CLKOUT4_SYN	输入	输出时钟 CLKOUT4 GATE 控制，高电平有效；
CLKOUT5_SYN	输入	输出时钟 CLKOUT5 GATE 控制，高电平有效；
RATIOI	输入	输入 divider divider ratio 动态设置；
RATIO0	输入	输出 divider0 divider ratio 动态设置；
RATIO1	输入	输出 divider1 divider ratio 动态设置；
RATIO2	输入	输出 divider2 divider ratio 动态设置；
RATIO3	输入	输出 divider3 divider ratio 动态设置；
RATIO4	输入	输出 divider4 divider ratio 动态设置；
RATIOF	输入	反馈 divider divider ratio 动态设置；
DUTY0	输入	输出 divider0 duty cycle 动态设置；
DUTY1	输入	输出 divider1 duty cycle 动态设置；
DUTY2	输入	输出 divider2 duty cycle 动态设置；
DUTY3	输入	输出 divider3 duty cycle 动态设置；
DUTY4	输入	输出 divider4 duty cycle 动态设置；
DUTYF	输入	反馈 divider duty cycle 动态设置；
PHASE0	输入	输出 divider0 fine phase 动态设置；
PHASE1	输入	输出 divider1 fine phase 动态设置；
PHASE2	输入	输出 divider2 fine phase 动态设置；
PHASE3	输入	输出 divider3 fine phase 动态设置；
PHASE4	输入	输出 divider4 fine phase 动态设置；
PHASEF	输入	反馈 divider fine phase 动态设置；
CPHASE0	输入	输出 divider0 coarse phase 动态设置；
CPHASE1	输入	输出 divider1 coarse phase 动态设置；
CPHASE2	输入	输出 divider2 coarse phase 动态设置；
CPHASE3	输入	输出 divider3 coarse phase 动态设置；
CPHASE4	输入	输出 divider4 coarse phase 动态设置；
CPHASEF	输入	反馈 divider coarse phase 动态设置；
PLL_PWD	输入	Power Down，高电平有效；
RST	输入	全局复位信号，高电平有效；
RSTODIV_PHASE	输入	相位调整功能使能时，输出 divider0~4 复位信号，高电平有效； 相位调整功能关闭时，输出 divider0~4 受 RST 复位信号控制；

6.11.4. 参数描述

表 209 GTP_PLL_E1 的参数说明

参数名称	有效值	默认值	功能描述
DYNAMIC_RATIOI_EN	“FALSE”,“TRUE”	“FALSE”	输入 divider 分频动态控制使能；
STATIC_RATIOI	1~512	1	输入 divider 分频静态设置；
PFDEN_EN	“FALSE”,“TRUE”	“FALSE”	PLL PFDEN 信号设置
VCOCLK_DIV2	1’b0-1’b1	1’b0	PLL VCO 输出时钟 2 分频使能；
DYNAMIC_RATIO0_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider0 分频动态控制使能；
DYNAMIC_RATIO1_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider1 分频动态控制使能；
DYNAMIC_RATIO2_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider2 分频动态控制使能；
DYNAMIC_RATIO3_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider3 分频动态控制使能；
DYNAMIC_RATIO4_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider4 分频动态控制使能；
DYNAMIC_RATIOF_EN	“FALSE”,“TRUE”	“FALSE”	反馈 divider 分频动态控制使能；
STATIC_RATIO0	1~512	1	输出 divider0 分频静态设置；
STATIC_RATIO1	1~512	1	输出 divider1 分频静态设置；
STATIC_RATIO2	1~512	1	输出 divider2 分频静态设置；
STATIC_RATIO3	1~512	1	输出 divider3 分频静态设置；
STATIC_RATIO4	1~512	1	输出 divider4 分频静态设置；
STATIC_RATIOF	1~512	1	反馈 divider 分频静态设置；
DYNAMIC_DUTY0_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider0 duty 动态控制使能；
DYNAMIC_DUTY1_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider1 duty 动态控制使能；
DYNAMIC_DUTY2_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider2 duty 动态控制使能；
DYNAMIC_DUTY3_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider3 duty 动态控制使能；
DYNAMIC_DUTY4_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider4 duty 动态控制使能；
DYNAMIC_DUTYF_EN	“FALSE”,“TRUE”	“FALSE”	反馈 divider duty 动态控制使能；
STATIC_DUTY0	1~1022	1	输出 divider0 duty 静态设置；
STATIC_DUTY1	1~1022	1	输出 divider1 duty 静态设置；
STATIC_DUTY2	1~1022	1	输出 divider2 duty 静态设置；
STATIC_DUTY3	1~1022	1	输出 divider3 duty 静态设置；
STATIC_DUTY4	1~1022	1	输出 divider4 duty 静态设置；
STATIC_DUTYF	1~1022	1	反馈 divider duty 静态设置；
PHASE_ADJUST0_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider0 phase 调整使能；
PHASE_ADJUST1_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider1 phase 调整使能；
PHASE_ADJUST2_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider2 phase 调整使能；
PHASE_ADJUST3_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider3 phase 调整使能；
PHASE_ADJUST4_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider4 phase 调整使能；
DYNAMIC_PHASE0_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider0 phase 动态控制使能；
DYNAMIC_PHASE1_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider1 phase 动态控制使能；
DYNAMIC_PHASE2_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider2 phase 动态控制使能；
DYNAMIC_PHASE3_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider3 phase 动态控制使能；

参数名称	有效值	默认值	功能描述
DYNAMIC_PHASE4_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider4 phase 动态控制使能；
DYNAMIC_PHASEF_EN	“FALSE”,“TRUE”	“FALSE”	反馈 divider phase 动态控制使能；
STATIC_PHASE0	0~7	0	输出 divider0 fine phase 静态设置；
STATIC_PHASE1	0~7	0	输出 divider1 fine phase 静态设置；
STATIC_PHASE2	0~7	0	输出 divider2 fine phase 静态设置；
STATIC_PHASE3	0~7	0	输出 divider3 fine phase 静态设置；
STATIC_PHASE4	0~7	0	输出 divider4 fine phase 静态设置；
STATIC_PHASEF	0~7	0	反馈 divider fine phase 静态设置；
STATIC_CPHASE0	2~513	2	输出 divider0 coarse phase 静态设置；
STATIC_CPHASE1	2~513	2	输出 divider1 coarse phase 静态设置；
STATIC_CPHASE2	2~513	2	输出 divider2 coarse phase 静态设置；
STATIC_CPHASE3	2~513	2	输出 divider3 coarse phase 静态设置；
STATIC_CPHASE4	2~513	2	输出 divider4 coarse phase 静态设置；
STATIC_CPHASEF	2~513	2	反馈 divider coarse phase 静态设置；
CLK_CAS0_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider0 时钟级联使能；
CLK_CAS1_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider1 时钟级联使能；
CLK_CAS2_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider2 时钟级联使能；
CLK_CAS3_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider3 时钟级联使能；
CLK_CAS4_EN	“FALSE”,“TRUE”	“FALSE”	输出 divider4 时钟级联使能；
INTERNAL_FB	"DISABLE","ENABLE"	"ENABLE"	内部反馈 divider 选择设置；
EXTERNAL_FB	"CLKOUT0","CLKOUT1" "CLKOUT2","CLKOUT3" "CLKOUT4", "DISABLE"	"DISABLE"	外部反馈 divider 选择设置；
BANDWIDTH	“LOW”, “HIGH” “OPTIMIZED”	“OPTIMIZED”	带宽选择设置；
CLKOUT5_SEL	0~4	0	输出时钟 CLKOUT5 选择设置；
CLKIN_BYPASS_EN	“FALSE”,“TRUE”	“FALSE”	输入时钟 bypass 使能；
CLKOUT0_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT0_SYN 端口使能；
CLKOUT0_EXT_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT0_EXT_SYN 端口使能；

参数名称	有效值	默认值	功能描述
CLKOUT1_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT1_SYN 端口使能；
CLKOUT2_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT2_SYN 端口使能；
CLKOUT3_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT3_SYN 端口使能；
CLKOUT4_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT4_SYN 端口使能；
CLKOUT5_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT5_SYN 端口使能；
RSTODIV_PHASE_EN	“FALSE”,”TRUE”	“FALSE”	RSTODIV_PHASE 复位信号使能；
CLKIN_FREQ	5~625	50	输入时钟频率；
SIM_DEVICE	“PGL22G”,”PGL35ES”	“PGL22G”	器件选择设置；

6.11.5. 例化模块

输入为 50M，输出时钟 clkout0 50M，输出 pll_lock 信号，内反馈对应的例化如下：

```
GTP_PLL_E1 #(
    .CLKIN_FREQ(50),
    .PFDEN_EN("FALSE"),
    .VCOCLK_DIV2(1'b0),
    .DYNAMIC_RATIOI_EN("FALSE"),
    .DYNAMIC_RATIO0_EN("FALSE"),
    .DYNAMIC_RATIO1_EN("FALSE"),
    .DYNAMIC_RATIO2_EN("FALSE"),
    .DYNAMIC_RATIO3_EN("FALSE"),
    .DYNAMIC_RATIO4_EN("FALSE"),
    .DYNAMIC_RATIOF_EN("FALSE"),
    .STATIC_RATIOI(1),
    .STATIC_RATIO0(1),
    .STATIC_RATIO1(1),
    .STATIC_RATIO2(1),
    .STATIC_RATIO3(1),
    .STATIC_RATIO4(1),
    .STATIC_RATIOF(1),
    .DYNAMIC_DUTY0_EN("FALSE"),
    .DYNAMIC_DUTY1_EN("FALSE"),
    .DYNAMIC_DUTY2_EN("FALSE"),
    .DYNAMIC_DUTY3_EN("FALSE"),
```

.DYNAMIC_DUTY4_EN("FALSE"),

.DYNAMIC_DUTYF_EN("FALSE"),

.STATIC_DUTY0(2),

.STATIC_DUTY1(2),

.STATIC_DUTY2(2),

.STATIC_DUTY3(2),

.STATIC_DUTY4(2),

.STATIC_DUTYF(2),

.PHASE_ADJUST0_EN("FALSE"),

.PHASE_ADJUST1_EN("FALSE"),

.PHASE_ADJUST2_EN("FALSE"),

.PHASE_ADJUST3_EN("FALSE"),

.PHASE_ADJUST4_EN("FALSE"),

.DYNAMIC_PHASE0_EN("FALSE"),

.DYNAMIC_PHASE1_EN("FALSE"),

.DYNAMIC_PHASE2_EN("FALSE"),

.DYNAMIC_PHASE3_EN("FALSE"),

.DYNAMIC_PHASE4_EN("FALSE"),

.DYNAMIC_PHASEF_EN("FALSE"),

.STATIC_PHASE0(0),

.STATIC_PHASE1(0),

.STATIC_PHASE2(0),

.STATIC_PHASE3(0),

.STATIC_PHASE4(0),

.STATIC_PHASEF(0),

.STATIC_CPHASE0(2),

.STATIC_CPHASE1(2),

.STATIC_CPHASE2(2),

.STATIC_CPHASE3(2),

.STATIC_CPHASE4(2),

.STATIC_CPHASEF(2),

```
        .CLK_CAS0_EN("FALSE"),
        .CLK_CAS1_EN("FALSE"),
        .CLK_CAS2_EN("FALSE"),
        .CLK_CAS3_EN("FALSE"),
        .CLK_CAS4_EN("FALSE"),
        .CLKOUT5_SEL(0),
        .CLKIN_BYPASS_EN("FALSE"),
        .CLKOUT0_SYN_EN("FALSE"),
        .CLKOUT0_EXT_SYN_EN("FALSE"),
        .CLKOUT1_SYN_EN("FALSE"),
        .CLKOUT2_SYN_EN("FALSE"),
        .CLKOUT3_SYN_EN("FALSE"),
        .CLKOUT4_SYN_EN("FALSE"),
        .CLKOUT5_SYN_EN("FALSE"),
        .INTERNAL_FB("ENABLE"),
        .EXTERNAL_FB("DISABLE"),
        .BANDWIDTH("OPTIMIZED"),
        .RSTODIV_PHASE_EN("TRUE"),
        .SIM_DEVICE("PGL22G")
    ) <InstanceName> (
        .CPHASE0(),             // INPUT[9:0]
        .CPHASE1(),             // INPUT[9:0]
        .CPHASE2(),             // INPUT[9:0]
        .CPHASE3(),             // INPUT[9:0]
        .CPHASE4(),             // INPUT[9:0]
        .CPHASEF(),             // INPUT[9:0]
        .DUTY0(),               // INPUT[9:0]
        .DUTY1(),               // INPUT[9:0]
        .DUTY2(),               // INPUT[9:0]
        .DUTY3(),               // INPUT[9:0]
        .DUTY4(),               // INPUT[9:0]
```

```
.DUTYF(),              // INPUT[9:0]
.PHASE0(),             // INPUT[2:0]
.PHASE1(),             // INPUT[2:0]
.PHASE2(),             // INPUT[2:0]
.PHASE3(),             // INPUT[2:0]
.PHASE4(),             // INPUT[2:0]
.PHASEF(),             // INPUT[2:0]
.RATIO0(),             // INPUT[9:0]
.RATIO1(),             // INPUT[9:0]
.RATIO2(),             // INPUT[9:0]
.RATIO3(),             // INPUT[9:0]
.RATIO4(),             // INPUT[9:0]
.RATIOF(),             // INPUT[9:0]
.RATIOI(),             // INPUT[9:0]
.CLKOUT0(),            // OUTPUT
.CLKOUT0_EXT(),        // OUTPUT
.CLKOUT1(),            // OUTPUT
.CLKOUT2(),            // OUTPUT
.CLKOUT3(),            // OUTPUT
.CLKOUT4(),            // OUTPUT
.CLKOUT5(),            // OUTPUT
.CLKSWITCH_FLAG(), // OUTPUT
.LOCK(),               // OUTPUT
.CLKFB(),              // INPUT
.CLKIN1(),             // INPUT
.CLKIN2(),             // INPUT
.CLKIN_SEL(),          // INPUT
.CLKIN_SEL_EN(),       // INPUT
.CLKOUT0_EXT_SYN(),// INPUT
.CLKOUT0_SYN(),        // INPUT
.CLKOUT1_SYN(),        // INPUT
```

```
        .CLKOUT2_SYN(),      // INPUT
        .CLKOUT3_SYN(),      // INPUT
        .CLKOUT4_SYN(),      // INPUT
        .CLKOUT5_SYN(),      // INPUT
        .PFDEN(),            // INPUT
        .PLL_PWD(),          // INPUT
        .RST(),              // INPUT
        .RSTODIV_PHASE()     // INPUT
    );
```

6.11.6. 详细功能说明

参考《UG020004_Logos 系列 FPGA 时钟资源（Clock）用户指南》

6.12. GTP_PLL_E3

6.12.1. 支持器件

表 210 GTP_PLL_E3 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	不支持	支持	支持	支持	支持

6.12.2. 功能介绍

主要实现分频，倍频，相位调整功能。该 GTP 支持输入时钟的动态选择，内外反馈模式，动态重配置，输出时钟 gate 功能。

6.12.3. 端口描述

表 211 GTP_PLL_E3 端口描述

Port	方向	功能描述
CLKOUT0	输出	输出divider0分频输出时钟；
CLKOUT0_EXT	输出	输出divider0分频输出时钟，与CLKOUT0为同源时钟；
CLKOUT1	输出	输出divider1分频输出时钟；
CLKOUT2	输出	输出divider2分频输出时钟；
CLKOUT3	输出	输出divider3分频输出时钟；

CLKOUT4	输出	输出divider4分频输出时钟或输入时钟bypass输出；
CLKOUT5	输出	输出divider0~4分频输出5选1后时钟，由静态配置CLKOUT5_SEL选择；
CLKSWITCH_FLAG	输出	时钟自动切换标识，在时钟动态选择的自动模式下： 为0时，表示选择CLKIN1作为PLL基准时钟； 为1时，表示选择CLKIN2作为PLL基准时钟；
LOCK	输出	PLL锁定输出信号；为0时，PLL未锁定；为1时，PLL已锁定；
CLKIN1	输入	输入时钟1；
CLKIN2	输入	输入时钟2；
CLKFB	输入	反馈时钟；
CLKIN_SEL	输入	输入时钟选择端口；为0时，选择CLKIN1；为1时，CLKIN2；
CLKIN_SEL_EN	输入	CLKIN_SEL端口使能： 为0时，CLKIN_SEL端口输入无效，自动模式； 为1时，CLKIN_SEL端口输入有效，由CLKIN_SEL选择输入时钟；
PFDEN	输入	PLL PFD使能，高电平有效；
ICP_BASE	输入	Charge Pump电流基准设置
ICP_SEL[3:0]	输入	Charge Pump电流动态设置
LPFRES_SEL[2:0]	输入	Loop Filter电阻动态设置
CRIPPLE_SEL	输入	Loop Filter电阻动态设置
PHASE_SEL[2:0]	输入	选择PLL相应输出时钟进行相位调整； 3’b000：选择CLKOUT0， 3’b001：选择CLKOUT1， 3’b010：选择CLKOUT2， 3’b011：选择CLKOUT3； 3’b100：选择CLKOUT4； 3’b101：选择内部反馈时钟；
PHASE_DIR	输入	选择动态相位细调调整方向；1’b0：滞后，1’b1：超前；
PHASE_STEP_N	输入	动态相位细调调整toggle信号； 每toggle一次，输出时钟相位调整1/8Tvco；
LOAD_PHASE	输入	所选通道的当前相位细调值加载信号，脉冲信号，高电平有效；
RATIOI[9:0]	输入	输入divider divider ratio动态设置；

RATIOM[6:0]	输入	反馈M divider divider ratio动态设置；
RATIO0[9:0]	输入	输出divider0 divider ratio动态设置；
RATIO1[9:0]	输入	输出divider1 divider ratio动态设置；
RATIO2[9:0]	输入	输出divider2 divider ratio动态设置；
RATIO3[9:0]	输入	输出divider3 divider ratio动态设置；
RATIO4[9:0]	输入	输出divider4 divider ratio动态设置；
RATIOF[9:0]	输入	反馈F divider divider ratio动态设置；
DUTY0[9:0]	输入	输出divider0 duty cycle动态设置；
DUTY1[9:0]	输入	输出divider1 duty cycle动态设置；
DUTY2[9:0]	输入	输出divider2 duty cycle动态设置；
DUTY3[9:0]	输入	输出divider3 duty cycle动态设置；
DUTY4[9:0]	输入	输出divider4 duty cycle动态设置；
CLKOUT0_SYN	输入	输出时钟CLKOUT0 GATE控制，高电平有效；
CLKOUT0_EXT_SYN	输入	输出时钟CLKOUT0_EXT GATE控制，高电平有效；
CLKOUT1_SYN	输入	输出时钟CLKOUT1 GATE控制，高电平有效；
CLKOUT2_SYN	输入	输出时钟CLKOUT2 GATE控制，高电平有效；
CLKOUT3_SYN	输入	输出时钟CLKOUT3 GATE控制，高电平有效；
CLKOUT4_SYN	输入	输出时钟CLKOUT4 GATE控制，高电平有效；
CLKOUT5_SYN	输入	输出时钟CLKOUT5 GATE控制，高电平有效；
PLL_PWD	输入	Power Down，高电平有效；
RST	输入	全局复位信号，高电平有效；
RSTODIV	输入	输出divider0~4和FDIV复位信号，高电平有效；

6.12.4. 参数描述

表 212 GTP_PLL_E3 参数描述

参数名称	有效值	默认值	功能描述
CLKIN_FREQ	5~625	50	输入时钟频率
PFDEN_EN	“FALSE”,“TRUE”	“FALSE”	PLL PFDEN信号使能；
VCOCLK_DIV2	1’b0-1’b1	1’b0	PLL VCO输出时钟2分频使能；
DYNAMIC_RATIOI_EN	“FALSE”,“TRUE”	“FALSE”	输入divider分频动态控制使能；
DYNAMIC_RATIOM_EN	“FALSE”,“TRUE”	“FALSE”	反馈M divider分频动态控制使能；

DYNAMIC_RATIO0_EN	“FALSE”,“TRUE”	“FALSE”	输出divider0分频动态控制使能；
DYNAMIC_RATIO1_EN	“FALSE”,“TRUE”	“FALSE”	输出divider1分频动态控制使能；
DYNAMIC_RATIO2_EN	“FALSE”,“TRUE”	“FALSE”	输出divider2分频动态控制使能；
DYNAMIC_RATIO3_EN	“FALSE”,“TRUE”	“FALSE”	输出divider3分频动态控制使能；
DYNAMIC_RATIO4_EN	“FALSE”,“TRUE”	“FALSE”	输出divider4分频动态控制使能；
DYNAMIC_RATIOF_EN	“FALSE”,“TRUE”	“FALSE”	反馈F divider分频动态控制使能；
STATIC_RATIOI	1~512	1	输入divider分频静态设置；
STATIC_RATIOM	1~64	1	反馈M divider分频静态设置；
STATIC_RATIO0	1~512	1	输出divider0分频静态设置；
STATIC_RATIO1	1~512	1	输出divider1分频静态设置；
STATIC_RATIO2	1~512	1	输出divider2分频静态设置；
STATIC_RATIO3	1~512	1	输出divider3分频静态设置；
STATIC_RATIO4	1~512	1	输出divider4分频静态设置；
STATIC_RATIOF	1~512	1	反馈F divider分频静态设置；
DYNAMIC_DUTY0_EN	“FALSE”,“TRUE”	“FALSE”	输出divider0 duty动态控制使能；
DYNAMIC_DUTY1_EN	“FALSE”,“TRUE”	“FALSE”	输出divider1 duty动态控制使能；
DYNAMIC_DUTY2_EN	“FALSE”,“TRUE”	“FALSE”	输出divider2 duty动态控制使能；
DYNAMIC_DUTY3_EN	“FALSE”,“TRUE”	“FALSE”	输出divider3 duty动态控制使能；
DYNAMIC_DUTY4_EN	“FALSE”,“TRUE”	“FALSE”	输出divider4 duty动态控制使能；
STATIC_DUTY0	2~1022	2	输出divider0 duty静态设置；
STATIC_DUTY1	2~1022	2	输出divider1 duty静态设置；
STATIC_DUTY2	2~1022	2	输出divider2 duty静态设置；
STATIC_DUTY3	2~1022	2	输出divider3 duty静态设置；
STATIC_DUTY4	2~1022	2	输出divider4 duty静态设置；
STATIC_PHASE0	0~7	0	输出divider0 fine phase静态设置；
STATIC_PHASE1	0~7	0	输出divider1 fine phase静态设置；
STATIC_PHASE2	0~7	0	输出divider2 fine phase静态设置；
STATIC_PHASE3	0~7	0	输出divider3 fine phase静态设置；
STATIC_PHASE4	0~7	0	输出divider4 fine phase静态设置；
STATIC_PHASEF	0~7	0	反馈divider fine phase静态设置；

STATIC_CPHASE0	0~511	0	输出divider0 coarse phase静态设置；
STATIC_CPHASE1	0~511	0	输出divider1 coarse phase静态设置；
STATIC_CPHASE2	0~511	0	输出divider2 coarse phase静态设置；
STATIC_CPHASE3	0~511	0	输出divider3 coarse phase静态设置；
STATIC_CPHASE4	0~511	0	输出divider4 coarse phase静态设置；
STATIC_CPHASEF	0~511	0	反馈divider coarse phase静态设置；
CLK_CAS1_EN	“FALSE”,“TRUE”	“FALSE”	输出divider1时钟级联使能；
CLK_CAS2_EN	“FALSE”,“TRUE”	“FALSE”	输出divider2时钟级联使能；
CLK_CAS3_EN	“FALSE”,“TRUE”	“FALSE”	输出divider3时钟级联使能；
CLK_CAS4_EN	“FALSE”,“TRUE”	“FALSE”	输出divider4时钟级联使能；
CLKOUT5_SEL	0~4	0	输出时钟CLKOUT5选择设置；
CLKIN_BYPASS_EN	“FALSE”,“TRUE”	“FALSE”	输入时钟bypass使能；
CLKOUT0_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT0_SYN端口使能；
CLKOUT0_EXT_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT0_EXT_SYN端口使能；
CLKOUT1_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT1_SYN端口使能；
CLKOUT2_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT2_SYN端口使能；
CLKOUT3_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT3_SYN端口使能；
CLKOUT4_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT4_SYN端口使能；
CLKOUT5_SYN_EN	“FALSE”,”TRUE”	“FALSE”	CLKOUT5_SYN端口使能；
INTERNAL_FB	“DISABLE”,“ENABLE”	“ENABLE”	内部反馈divider选择设置；
EXTERNAL_FB	“CLKOUT0”,“CLKOUT1” “CLKOUT2”,“CLKOUT3” “CLKOUT4”,“DISABLE”	“DISABLE”	外部反馈divider选择设置；
DYNAMIC_LOOP_EN	“FALSE”,”TRUE”	“FALSE”	PLL环路带宽参数动态使能；
LOOP_MAPPING_EN	“FALSE”,”TRUE”	“FALSE”	PLL环路带宽参数自动映射使能；

BANDWIDTH	“LOW”, “HIGH” “OPTIMIZED”	“OPTIMIZED”	带宽选择设置；

6.12.5. 例化模块

输入为 50M，输出时钟 clkout0 50M，输出 pll_lock 信号，内反馈对应的例化如下：

```
GTP_PLL_E3 #(
.CLKIN_FREQ (50),
.PFDEN_EN ("FALSE"),
.VCOCLK_DIV2 (0),
.DYNAMIC_RATIOI_EN ("FALSE"),
.DYNAMIC_RATIOM_EN ("FALSE"),
.DYNAMIC_RATIO0_EN ("FALSE"),
.DYNAMIC_RATIO1_EN ("FALSE"),
.DYNAMIC_RATIO2_EN ("FALSE"),
.DYNAMIC_RATIO3_EN ("FALSE"),
.DYNAMIC_RATIO4_EN ("FALSE"),
.DYNAMIC_RATIOF_EN ("FALSE"),
.STATIC_RATIOI (2),
.STATIC_RATIOM (1),
.STATIC_RATIO0 (12),
.STATIC_RATIO1 (16),
.STATIC_RATIO2 (16),
.STATIC_RATIO3 (16),
.STATIC_RATIO4 (16),
.STATIC_RATIOF (24),
.DYNAMIC_DUTY0_EN ("FALSE"),
.DYNAMIC_DUTY1_EN ("FALSE"),
.DYNAMIC_DUTY2_EN ("FALSE"),
.DYNAMIC_DUTY3_EN ("FALSE"),
.DYNAMIC_DUTY4_EN ("FALSE"),
.STATIC_DUTY0 (12),
```

```
.STATIC_DUTY1 (16),
.STATIC_DUTY2 (16),
.STATIC_DUTY3 (16),
.STATIC_DUTY4 (16),
.STATIC_PHASE0 (0),
.STATIC_PHASE1 (0),
.STATIC_PHASE2 (0),
.STATIC_PHASE3 (0),
.STATIC_PHASE4 (0),
.STATIC_PHASEF (0),
.STATIC_CPHASE0 (0),
.STATIC_CPHASE1 (0),
.STATIC_CPHASE2 (0),
.STATIC_CPHASE3 (0),
.STATIC_CPHASE4 (0),
.STATIC_CPHASEF (0),
.CLK_CAS1_EN ("FALSE"),
.CLK_CAS2_EN ("FALSE"),
.CLK_CAS3_EN ("FALSE"),
.CLK_CAS4_EN ("FALSE"),
.CLKOUT5_SEL (0),
.CLKIN_BYPASS_EN ("FALSE"),
.CLKOUT0_SYN_EN ("FALSE"),
.CLKOUT0_EXT_SYN_EN ("FALSE"),
.CLKOUT1_SYN_EN ("FALSE"),
.CLKOUT2_SYN_EN ("FALSE"),
.CLKOUT3_SYN_EN ("FALSE"),
.CLKOUT4_SYN_EN ("FALSE"),
.CLKOUT5_SYN_EN ("FALSE"),
.INTERNAL_FB ("ENABLE"),
.EXTERNAL_FB ("DISABLE"),
```

```
.DYNAMIC_LOOP_EN ("FALSE"),
.LOOP_MAPPING_EN ("FALSE"),
.BANDWIDTH ("OPTIMIZED")
) u_pll_e3 (
.CLKOUT0 (clkout0),
.CLKOUT0_EXT (),
.CLKOUT1 (),
.CLKOUT2 (),
.CLKOUT3 (),
.CLKOUT4 (),
.CLKOUT5 (),
.CLKSWITCH_FLAG (),
.LOCK (pll_lock),
.CLKIN1 (clkin1),
.CLKIN2 (1'b0),
.CLKFB (),
.CLKIN_SEL (1'b0),
.CLKIN_SEL_EN (1'b0),
.PFDEN (),
.ICP_BASE (1'b0),
.ICP_SEL (4'b0),
.LPFRES_SEL (3'b0),
.CRIPPLE_SEL (1'b0),
.PHASE_SEL (3'b0),
.PHASE_DIR (1'b0),
.PHASE_STEP_N (1'b0),
.LOAD_PHASE (1'b0),
.RATIOM (7'b0),
.RATIOI (),
.RATIO0 (),
.RATIO1 (),
```

.RATIO2 (),

.RATIO3 (),

.RATIO4 (),

.RATIOF (),

.DUTY0 (),

.DUTY1 (),

.DUTY2 (),

.DUTY3 (),

.DUTY4 (),

.CLKOUT0_SYN (),

.CLKOUT0_EXT_SYN (),

.CLKOUT1_SYN (),

.CLKOUT2_SYN (),

.CLKOUT3_SYN (),

.CLKOUT4_SYN (),

.CLKOUT5_SYN (),

.PLL_PWD (1'b0),

.RST (1'b0),

.RSTODIV (1'b0)

6.12.6. 详细功能说明

参考《UG020004_Logos 系列 FPGA 时钟资源（Clock）用户指南》。

6.13. GTP_DLL

6.13.1. 支持器件

表 213 GTP_DLL 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

6.13.2. 功能介绍

GTP_DLL 的主要功能是动态锁定输入参考时钟的频率，输出该时钟周期的四分之一等效

delay step 数目，这个 Delay step 可实时追踪温度和电压的变化。其典型应用有 IO LOGIC 中的 DDC 将利用这个 delay step 完成输入信号（如输入 DQS）的 1/4 周期延迟，以正确采样数据；IO LOGIC 中的 DDC 将利用这个 delay step 产生 WCLK_DEL；时钟输入管脚中的 IOCLKDELY 用它来产生时钟延迟。

6.13.3. 端口描述

表 214 GTP_DLL 端口说明

端口信号	输入/输出	说明
CLKIN	输入	来自管脚的时钟输入
UPDATE_N	输入	请求更新DLL的delay step，低有效
RST	输入	高有效复位信号
PWD	输入	高有效power down信号
DELAY_STEP	输出	DLL输出的delay step
LOCK	输出	LOCK标志信号，高有效表示已经LOCK

6.13.4. 参数描述

表 215 GTP_DLL 参数说明

端口信号	输入/输出	说明
GRS_EN	参数	全局复位使能信号（芯片内部），默认值为“TRUE”
FAST_LOCK	参数	DLL LOCK模式选择，默认值为“TRUE”
DELAY_STEP_OFFSET	参数	静态微调DLL输出的DELAY_STEP，有效值为<-4, -3, -2, -1, 0, 1, 2, 3, 4>，默认值为<0>

6.13.5. 例化模块

```
GTP_DLL
#(
    . GRS_EN                ("TRUE" ),
    . FAST_LOCK             ("TRUE"),
    . DELAY_STEP_OFFSET    (    0    ) //-4, -3,-2, -1, 0, 1, 2, 3, 4
) GTP_DLL_INST(
    .CLKIN          (CLKIN          ),
    .UPDATE_N       (UPDATE_N       ),
    .RST            (RST            ),
    .PWD            (PWD            ),
    .LOCK           (LOCK           ),
    .DELAY_STEP     (DELAY_STEP     )
```

);

6.13.6. 详细功能介绍

GTP_DLL 将输入时钟 CLKIN 和延迟单元延时之间的关系转化为数字码，并根据相位差需求去对数字码进行比例控制，产生满足要求的码。

通过参数 DELAY_STEP_OFFSET 可设置不同的相位差，如下表所示。

表 216 不同配置下的相移列表

DELAY_STEP_OFFSET	相移
-4	45
-3	57
-2	68
-1	79
0	90
1	101
2	112
3	123
4	135

输入频率不同，产生的数字码也不同，对应关系如下表所示。

表 217 不同频率不同相移设置下的数字码列表

典型频率点（MHz）	DELAY_STEP								
	45 度	57 度	68 度	79 度	90 度	101 度	112 度	123 度	135 度
100	8'h32	8'h3E	8'h4B	8'h57	8'h64	8'h70	8'h7D	8'h89	8'h96
133	8'h25	8'h2F	8'h38	8'h42	8'h4B	8'h54	8'h5E	8'h67	8'h71
150	8'h21	8'h29	8'h32	8'h3A	8'h42	8'h4B	8'h53	8'h5B	8'h64
200	8'h19	8'h1F	8'h25	8'h2C	8'h32	8'h38	8'h3E	8'h45	8'h4B
267	8'h12	8'h17	8'h1C	8'h21	8'h25	8'h2A	8'h2F	8'h33	8'h38
300	8'h10	8'h15	8'h19	8'h1D	8'h21	8'h25	8'h29	8'h2E	8'h32
400	8'h0C	8'h0F	8'h13	8'h16	8'h19	8'h1C	8'h1F	8'h22	8'h25
500	8'h0A	8'h0C	8'h0F	8'h11	8'h14	8'h16	8'h19	8'h1B	8'h1E
533	8'h0A	8'h0B	8'h0E	8'h10	8'b12	8'h15	8'h17	8'h1A	8'h1C
600	8'h09	8'h0A	8'h0C	8'h0E	8'b10	8'h13	8'h15	8'h17	8'h19
625	8'h08	8'h0A	8'h0C	8'h0E	8'h10	8'h12	8'h14	8'h16	8'h18
667	8'h07	8'h09	8'h0B	8'h0D	8'h0F	8'h11	8'h13	8'h14	8'h16

以输入时钟 100MHz、相移 90°为例，GTP_DLL 波形图如下图所示。

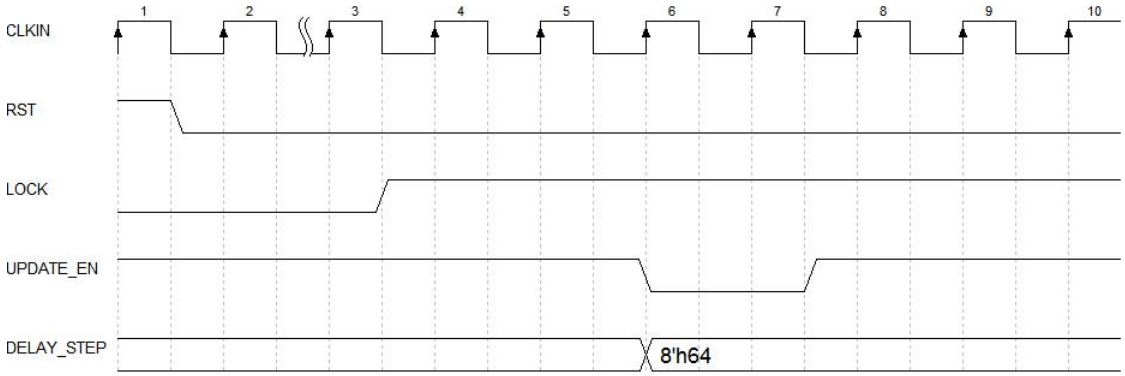


图 102 GTP_DLL 的波形图

7. 配置相关 GTP 使用说明

7.1. GTP_EFUSECODE 使用说明

7.1.1. 支持器件

表 218 GTP_EFUSECODE 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

7.1.2. 功能介绍

该 GTP 用来读取 EFUSECODE。

7.1.3. 端口描述

表 219 GTP_EFUSECODE 端口列表

端口名称	方向	功能描述
EFUSE_CODE[31:0]	输出	Efusecode 输出数据总线

7.1.4. 参数描述

表 220 GTP_EFUSECODE 参数列表

参数名称	参数类型	有效值	默认值	功能描述
SIM_EFUSE_VALUE	Binary	0~32’hffffffff ;	32’h12345678	Efusecode 值

7.1.5. 例化模板

```
GTP_EFUSECODE#(
.SIM_EFUSE_VALUE   (32'h12345678)
) GTP_EFUSECODE_inst(
.EFUSE_CODE            (efuse)
);
```

7.1.6. 详细功能说明

向用户并行输出 32bit 存储在 efuse 中的数据。

7.2. GTP_IPAL_E1 使用说明

7.2.1. 支持器件

表 221 GTP_IPAL_E1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

7.2.2. 功能介绍

通过 IPAL 接口，配置或回读位流。

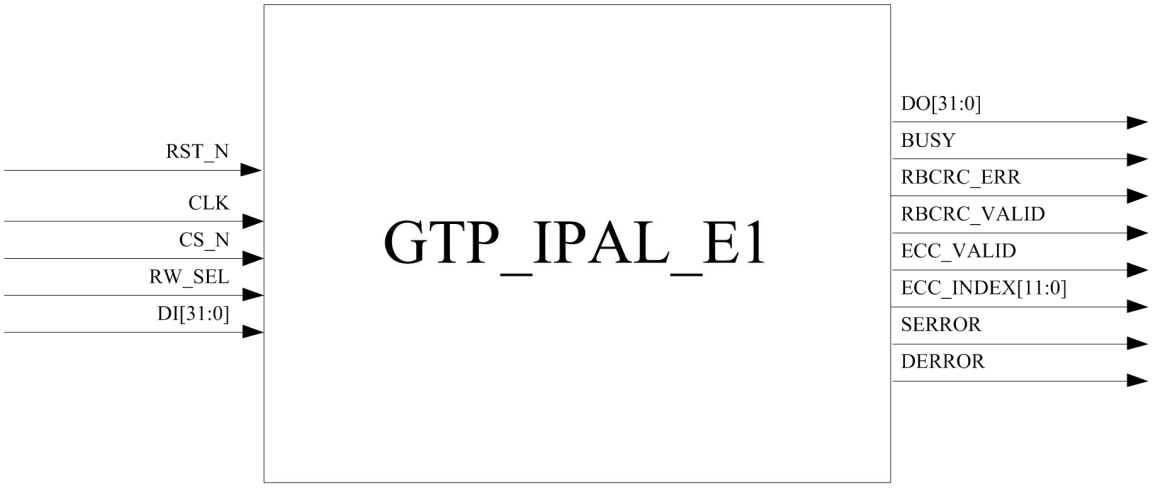


图 103 GTP_IPAL_E1 结构框图

7.2.3. 端口描述

表 222 GTP_IPAL_E1 端口列表

端口名称	方向	功能描述
RST_N	输入	复位
CLK	输入	时钟
CS_N	输入	片选使能信号
RW_SEL	输入	读写选择
BUSY	输出	回读，内部数据总线忙
DI	输入	输入数据
DO	输出	输出数据
RBCRC_ERR	输出	回读 crc 错误

RBCRC_VALID	输出	回读 crc 有效
ECC_VALID	输出	ECC 有效标志，脉冲信号，持续一个时钟周期
ECC_INDEX	输出	单 bit 错误的地址索引
SERROR	输出	SEU 检测的单比特错误标志
DERROR	输出	SEU 检测的双比特错误标志

7.2.4. 参数描述

表 223 GTP_IPAL_E1 参数列表

参数名称	参数类型	有效值	默认值	功能描述
IDCODE	binary	0~32'hffffffff	32'haaaa5555	芯片 IDCODE
DATA_WIDTH	String	X8 X16 X32	"X8"	位宽选择
SIM_DEVICE	String	"PGL25G"、"PGL50G"、"PGL50H'、"PGL100H"	"PGL25G"	器件选择

注：1.PGL25G、PGL20H 只支持 X8 、X16 不支持 X32。

2.SIM_DEVICE 的有效值表示该 GTP 支持这些器件的位流结构，用于 SEU IP。如果是 SEU 外的其他应用的话，还可以填 PGL 系列的任意一款器件，如 PGL12G、PGL22G、PGL20H 等。

7.2.5. 例化模板

```
GTP_IPAL_E1 #(
    .DATA_WIDTH("X8"),
    .IDCODE('b10101010101010100101010101010101),
    .SIM_DEVICE("PGL25G")
) GTP_IPAL_E1_inst (
    .DO(),              // OUTPUT[31:0]
    .ECC_INDEX(),   // OUTPUT[11:0]
    .DI(),              // INPUT[31:0]
    .BUSY(),            // OUTPUT
    .DERROR(),          // OUTPUT
    .ECC_VALID(),   // OUTPUT
    .RBCRC_ERR(),   // OUTPUT
    .RBCRC_VALID(),// OUTPUT
    .SERROR(),          // OUTPUT
    .CLK(),             // INPUT
    .CS_N(),            // INPUT
    .RST_N(),           // INPUT
```

```
    .RW_SEL()          // INPUT
);
```

7.2.6. 详细功能说明

用户可以通过例化该 GTP 进行内部从并行接口操作，也可以使用回读 CRC、SEU 接口很方便地进行回读 CRC 和 SEU 操作。这里主要对内部从并行接口进行介绍（RW_SEL 信号跳变只能在 CS_N 为高时进行）。

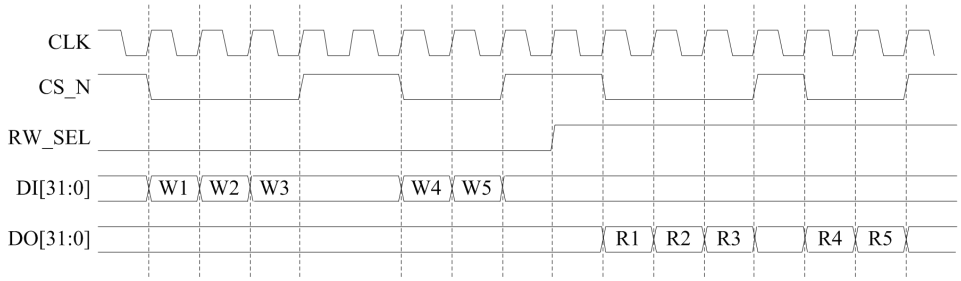


图 104 GTP_IPAL_E1 内部从并行接口读写接口时序

7.3. GTP_SCANCHAIN_E1 使用说明

7.3.1. 支持器件

表 224 GTP_SCANCHAIN_E1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

7.3.2. 功能介绍

通过 JTAG 接口，读取用户数据寄存器的值。

7.3.3. 端口描述

表 225 GTP_SCANCHAIN_E1 端口列表

端口名称	方向	功能描述
TDI	输入	Jtag 接口
TDO	输出	Jtag 接口
TMS	输入	Jtag 接口
TCK	输入	Jtag 接口

RST	输出	Jtag 软复位输出
CAPDR	输出	Jtag capturedr 状态指示
SHFTDR	输出	Shiftdr 状态指示
UPDR	输出	Updatedr 状态指示
JCLK	输出	Shiftdr 以为时钟
FLG_USER	输出	用户指令指示
TDI_USER	输出	用户寄存器数据输入
TDO_USER	输入	用户寄存器数据输出
JRTI	输出	Run/test idle 状态指示
TCK_USER	输出	Tck to user
TMS_USER	输出	Tms to user

7.3.4. 参数描述

表 226 GTP_SCANCHAIN_E1 参数列表

参数名称	参数类型	有效值	默认值	功能描述
IDCODE	Binary	0~32’hffffffff ;	32’haaaa5555	IDCODE
CHAIN_NUM	Integer	1、2、3、4	1	用户 DR 编号

7.3.5. 例化模板

```
GTP_SCANCHAIN
#(
    .IDCODE (32'haaaa5555),
    .CHAIN_NUM (1)
)
GTP_SCANCHAIN_inst(
    .TCK        (tck),
    .TDI        (tdi),
    .TMS        (tms),
    .TDO        (tdo),
    .CAPDR      (capture),
    .JCLK       (jclk),
    .RST        (rst),
    .FLG_USER   (flg_user),
    .SHFTDR     (shift),
    .TDI_USER   (tdi_user),
    .TMS_USER   (tms_user),
```

```
        .JRTI       (jrti),
        .UPDR       (update),
        .TDO_USER   (tdo_user)
    );
```

7.3.6. 详细功能说明

用户可读取芯片IDCODE；可通过JTAG接口分别读写多个用户逻辑数据寄存器值。

7.4. GTP_FLASHIF 使用说明

7.4.1. 支持器件

表 227 GTP_FLASHIF 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	不支持	不支持	不支持

7.4.2. 功能介绍

该模块为用户和内部或者外部 Flash 的时钟接口。

7.4.3. 端口描述

表 228 GTP_FLASHIF 端口列表

端口名称	方向	功能描述
EN_N	输入	用户使用内部 FLASH 标志，低有效
CLK	输入	用户内部 FLASH 时钟
CS_N	输入	用户内部 FLASH 片选,低有效
DIN	输入	用户内部 FLASH 数据总线输入
DOUT_EN_N	输入	内部 FLASH 数据总线输出使能，低有效
DOUT	输出	用户内部 FLASH 数据总线输出

7.4.4. 例化模板

```
GTP_FLASHIF GTP_FLASHIF_inst(
        .CLK        (clk ),
        .EN_N       (en_n),
        .CS_N       (cs_n),
        .DIN        (din),
        .DOUT_EN_N          (dout_en_n) ,
```

```
        .DOUT           (dout)
    );
```

7.5. GTP_UDID 使用说明

7.5.1. 支持器件

表 229 GTP_UDID 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

7.5.2. 功能介绍

该 GTP 用来读取 UDID CODE 值。

UID 长度为 64 位。

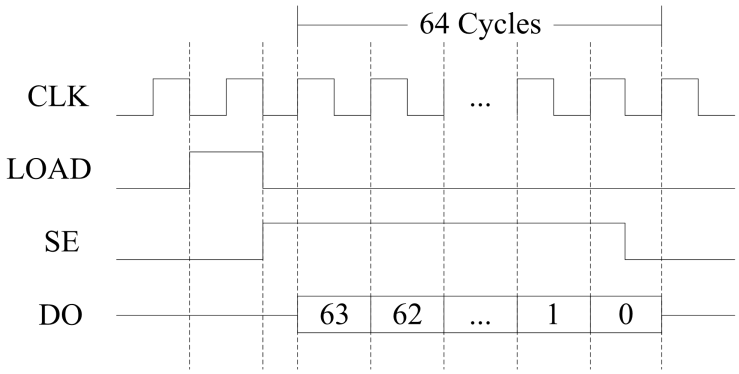


图 105 读 UID 时序

用户可以扩展 UID 的位宽。

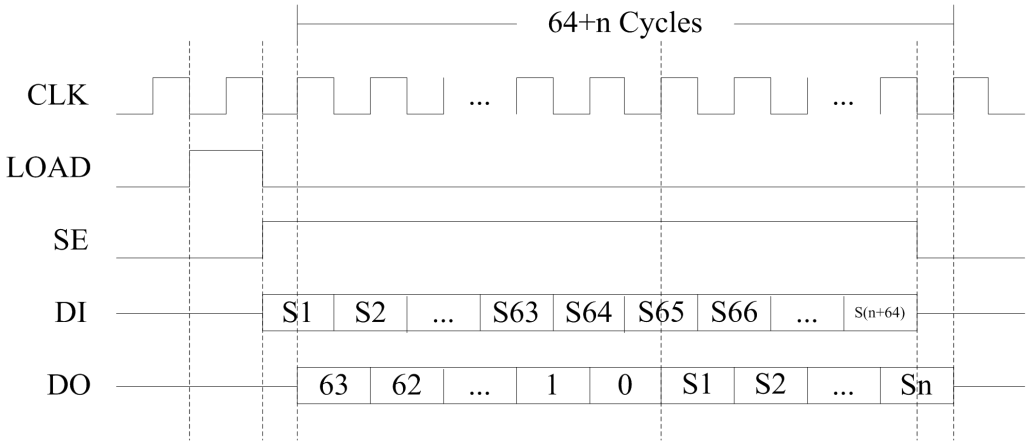


图 106 扩展 UID 位宽

7.5.3. 端口描述

表 230 GTP_UDID 端口列表

端口名称	方向	功能描述
DI	输入	串行数据输入
DO	输出	串行数据输出
SE	输入	数据移位使能
LOAD	输入	数据寄存器并行加载 UID CODE
CLK	输入	时钟

7.5.4. 参数描述

表 231 GTP_UDID 参数列表

参数名称	参数类型	有效值	默认值	功能描述
UDID_CODE	Binary	0~64'hffffffff ffffffff;	0	芯片身份标识码
UDID_WIDTH	integer	0~32'hffff_ffff	64	设置 UDID_CODE 的位宽

7.5.5. 例化模板

```
 GTP_UDID #(
        .UDID_WIDTH(64),
        .UDID_CODE('b0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000)
  ) GTP_UDID_inst (
        .DO(),    // OUTPUT
        .CLK(), // INPUT
        .DI(),    // INPUT
        .LOAD(),// INPUT
        .SE()     // INPUT
      );;
```

7.5.6. 详细功能说明

向用户串行输出64bit芯片身份标识识别码数据。

8. DDR GTP 使用说明

8.1. GTP_DDC_E1 使用说明

8.1.1. 支持器件

表 232 GTP_DDC_E1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

8.1.2. 功能介绍

为了支持 DDR/DDR2/DDR3 memory 接口，每个 IO Group 里面内置了相关的专用电路，对应 GTP_DDC_E1，结构框图如下：

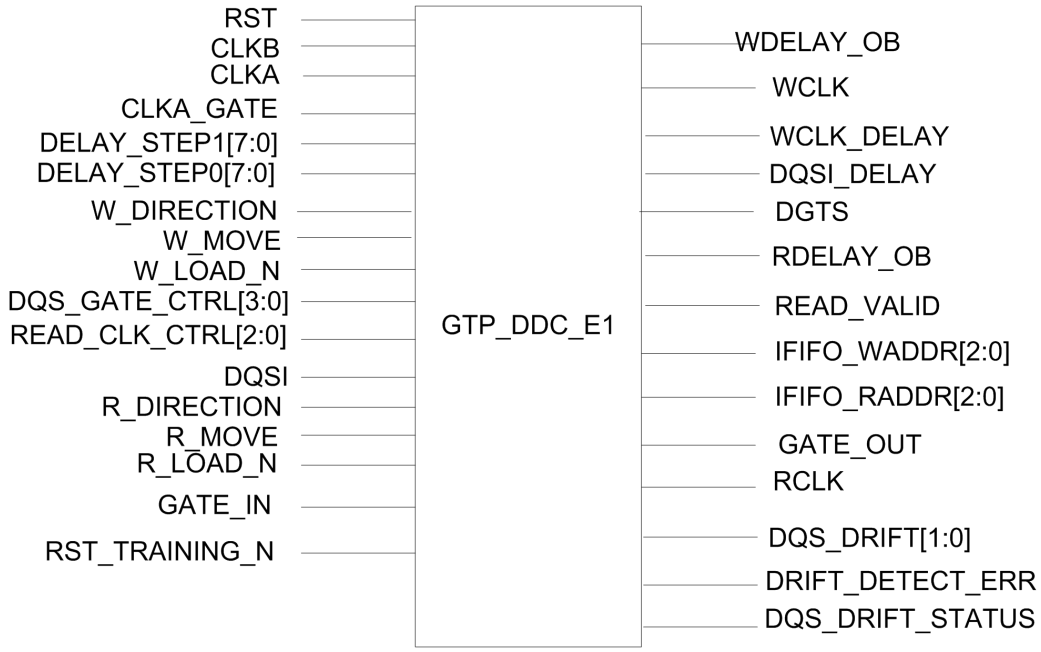


图 107 GTP_DDC_E1 结构框图

8.1.3. 端口描述

表 233 GTP_DDC_E1 端口列表

端口名称	方向	功能描述
RST	输入	本地reset，来自SRB。此外由于GTP中没有包含极性选择。因此在验证时，如果需要与grid device结果对比，则需要在GTP的该激励上添加极性控制。
CLKB	输入	系统时钟，可为Regional clock或者

		Global clock
CLKA	输入	专用的 IOCLK，在 DQS_DDC 实现中连接 IOCLK0，IOCLK1
CLKA_GATE	输入	用于GATE内部时钟
DELAY_STEP1	输入	来自 DLL 的 delay step
DELAY_STEP0	输入	针对 write leveling 的 delay step
W_DIRECTION	输入	写时钟延迟控制 code 调整方向控制
W_MOVE	输入	写时钟延迟控制 code 调整触发信号
W_LOAD_N	输入	写时钟延迟控制 code 写入
DQS_GATE_CTRL	输入	用于控制 DQS gate 时机
READ_CLK_CTRL	输入	读时钟控制信号
DQSI	输入	DQS 输入
R_DIRECTION	输入	读时钟延迟控制 code 调整方向控制
R_MOVE	输入	读时钟延迟控制 code 调整触发信号
R_LOAD_N	输入	读时钟延迟控制 code 写入
GATE_IN	输入	外部输入 gate 窗口
RST_TRAINING_N	输入	Training 结束后的复位信号，用于对 DGTS、读写指针生成和 READ_VALID 信号做复位控制。
WDELAY_OB	输出	write leveling delay 的 overflow
WCLK	输出	DQS Write clock
WCLK_DELAY	输出	Write clock 延迟 270 度
DQSI_DEL	输出	DQS read clock
DGTS	输出	DQS gate 状态标识，DQS_GATE_TRAINING_STATUS
GATE_OUT	输出	Gate 窗口输出
RDELAY_OB	输出	Read clock 的 overflow

DQS_DRIFT[1:0]	输出	DQS 延时变化检测信号，update 逻辑记录该信号并判断是否执行 gate 窗口位置 update 调整
DRIFT_DETECT_ERR	输出	DQS 延时变化检测出错标识，触发 PHY 执行 update training 操作
DQS_DRIFT_STATUS	输出	DQS 延时超出范围标识信号，拉高表示延时变化过大，从而触发 PHY 执行 update training 操作
READ_VALID	输出	读数据正确标志
IFIFO_WADDR	输出	IFIFO 写地址
IFIFO_RADDR	输出	IFIFO 读地址
RCLK	输出	提供给 IOL OSERDES 的串行时钟

8.1.4. 参数描述

表 234 GTP_DDC_E1 参数列表

参数名称	参数类型	有效值	默认值	功能描述
GRS_EN	string	" TRUE " " FALSE "	" TRUE "	全局复位信号使能
DDC_MODE	string	" FULL_RATE" " HALF_RATE " " QUAD_RATE "	" FULL_RATE"	DDC 工作模式
IFIFO_GENERIC	string	" TRUE " " FALSE "	" FALSE "	IFIFO 工作模式选择
WCLK_DELAY_OFFSET	interger	0~511	0	WCLK_DEL 偏移： 该参数的最高位为符号位 sign， sign=0，正向调整 sign=1，负向调整
DQSI_DELAY_OFFSET	interger	0~511	0	DQSI_DEL 偏移，功能同 WCLK_DELAY_OFFSET
CLKA_GATE_EN	string	" TRUE "	" FALSE "	GATE 使能

		" FALSE "		
R_DEALY_STEP_EN	string	" TRUE " " FALSE "	" TRUE "	DLL_STEP 使能
R_MOVE_EN	string	" TRUE " " FALSE "	" FALSE "	读延迟控制信号使能
W_MOVE_EN	string	" TRUE " " FALSE "	" FALSE "	针对 write leveling 的控制信号使能
R_EXTEND	string	" TRUE " " FALSE "	" FALSE "	write leveling 扩展
GATE_SEL	string	" TRUE " " FALSE "	" FALSE "	读时钟 gate 窗口来源选择
RADDR_INIT	string	0~7	0	RD_ADDR 初始值
WCLK_DELAY_SEL	string	" TRUE " " FALSE "	" FALSE "	WCLK_DELAY 来源选择：“FALSE”：此时为 WCLK 的 270 度相移；“TRUE”：此时为 WCLK 的 90 度相移；
RCLK_SEL	string	" TRUE " " FALSE "	" FALSE "	RCLK 来源选择：“FALSE”：CLKA/CLKB 取反；“TRUE”：CLKA/CLKB；

8.1.5. 例化模板

```
GTP_DDC_E1 #(
  .GRS_EN ("FALSE" ),
  .DDC_MODE ("QUAD_RATE"),
  .IFIFO_GENERIC ("FALSE" ),
  .WCLK_DELAY_OFFSET (0 ),
  .DQSI_DELAY_OFFSET (0 ),
  .CLKA_GATE_EN ("FALSE" ),
  .R_DELAY_STEP_EN ("TRUE" ),
  .R_MOVE_EN ("TRUE" ),
  .W_MOVE_EN ("FALSE" ),
  .R_EXTEND ("TRUE" ),
  .GATE_SEL ("TRUE" ),
```

```
  .RADDR_INIT (3'd0 )
) u_GTP_DDC_CMD(
  .WDELAY_OB ( ),
  .WCLK (wclk_cmd ),
  .WCLK_DELAY (),
  .RCLK (ioclk_cmd),
  .RDELAY_OB (),
  .DQSI_DELAY (),
  .DGTS (),
  .READ_VALID (),
  .GATE_OUT (),
  .IFIFO_WADDR (),
  .IFIFO_RADDR (),
  .DQS_DRIFT (),
  .DRIFT_DETECT_ERR (),
  .DQS_DRIFT_STATUS (),
  .RST (~reset_n_prim),
  .CLKB (rclk),//glck
  .CLKA (ioclk[0]),
  .CLKA_GATE (1'b1),
  .DELAY_STEP1 (dll_step),
  .DELAY_STEP0 (8'd0),
  .W_DIRECTION (1'b0),
  .W_MOVE (1'b0),
  .W_LOAD_N (1'b0),
  .DQS_GATE_CTRL (4'd0),
  .READ_CLK_CTRL (3'd0),
  .DQSI (),
  .GATE_IN (),
  .R_DIRECTION (1'b0),
  .R_MOVE (1'b0),
  .R_LOAD_N (1'b0),
  .RST_TRAINING_N (1'b1)
);
```

8.1.6. 详细功能说明

GTP_DDC_E1 主要实现以下功能：

1. 为DDR memory存储器产生写操作时钟；
2. 将读操作中的DQSI信号进行90度相移，以捕获输入数据，并产生gate正确标识信号DGTS；
3. 为core逻辑产生读操作有效判断信号；
4. 为输入FIFO产生读写指向操作信号；
5. 支持DDR3的wirte leveing过程。在quad/half rate模式时，支持时钟关断操作，且关断时间为

4个CLKA周期的整数倍

8.2. GTP_DDRC 使用说明

8.2.1. 支持器件

表 235 GTP_DDRC 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	不支持	不支持	不支持	不支持

8.2.2. 功能介绍

GTP_DDRC 主要实现 DDR 系统中内存控制器的功能，产生 DDR_PHY 所需 DFI 接口信号，控制 DDR_PHY 进行相应操作，从而控制 SDRAM 工作状态，完成数据读写等操作。

8.2.3. 端口描述

表 236 GTP_DDRC 端口列表

端口名称	方向	功能描述
CORE_DDRC_CORE_CLK	输入	ddrc时钟信号
CORE_DDRC_RST	输入	DDRC复位信号，低电平有效
ARESET_0	输入	AXI接口异步复位信号，低电平有效
ACLK_0	输入	AXI接口的时钟信号
AWID_0[7:0]	输入	AXI写地址ID，不同地址分组标记
AWADDR_0[31:0]	输入	AXI写地址，突发传输中第一次交易地址
AWLEN_0[7:0]	输入	AXI写burst length，突发传输中交易次数
AWSIZE_0[2:0]	输入	AXI写burst size，突发传输中每次交易数据大小
AWBURST_0[1:0]	输入	AXI写burst类型
AWLOCK_0	输入	AXI中锁定写交易操作，与atomic transfer相关
AWVALID_0	输入	AXI写地址有效
AWREADY_0	输出	AXI写地址已准备好，表明slave可以接收写地址
AWQOS_0[3:0]	输入	AXI端口写优先级，取值越大，优先级越高
AWURGENT_0	输入	AXI紧急写操作，使能后端口仲裁立刻进入写操作

端口名称	方向	功能描述
AWPOISON_0	输入	AXI无效写交易
WDATA_0[127:0]	输入	AXI写数据
WSTRB_0[15:0]	输入	AXI写数据选通信号
WLAST_0	输入	表明AXI突发传输中最后一次写交易
WVALID_0	输入	AXI写数据有效
WREADY_0	输出	表明slave可以接收写数据
BID_0[7:0]	输出	AXI写响应ID信号
BRESP_0[1:0]	输出	AXI写响应信号
BVALID_0	输出	AXI写响应有效
BREADY_0	输入	表明master可以接收写响应信号
ARID_0 [7:0]	输入	AXI读地址ID，不同地址分组标记
ARADDR_0[31:0]	输入	AXI读地址，突发传输中第一次交易地址
ARLEN_0 [7:0]	输入	AXI读burst length，突发传输中交易次数
ARSIZE_0 [2:0]	输入	AXI读burst size，突发传输中每次交易数据大小
ARBURST_0[1:0]	输入	AXI读burst类型
ARLOCK_0	输入	AXI中锁定读交易操作，与atomic transfer相关
ARVALID_0	输入	AXI读地址有效
ARREADY_0	输出	AXI读地址已准备好，表明slave可以接收读地址
ARQOS_0[3:0]	输入	AXI端口读优先级，取值越大，优先级越高
ARPOISON_0	输入	AXI无效读交易
RID_0[7:0]	输出	AXI读数据ID
RDATA_0[127:0]	输出	AXI读数据
RRESP_0[1:0]	输出	AXI读响应
RLAST_0	输出	表明AXI突发传输中最后一次读交易
RVALID_0	输出	AXI读数据有效
RREADY_0	输入	表明master可以接收读数据并响应
ARURGENT_0	输入	AXI紧急读操作，使能后端口仲裁立刻进入读操作
RAQ_PUSH_0	输出	写入AXI读地址FIFO的交易
RAQ_SPLIT_0	输出	写入AXI读地址FIFO的Wrap交易起始部分

端口名称	方向	功能描述
WAQ_PUSH_0	输出	写入AXI写地址FIFO的交易
WAQ_SPLIT_0	输出	写入AXI写地址FIFO的Wrap交易起始部分
ARESET_1	输入	AXI接口异步复位信号，低电平有效
ACLK_1	输入	AXI接口的时钟信号
AWID_1[7:0]	输入	AXI写地址ID，不同地址分组标记
AWADDR_1[31:0]	输入	AXI写地址，突发传输中第一次交易地址
AWLEN_1[7:0]	输入	AXI写burst length，突发传输中交易次数
AWSIZE_1[2:0]	输入	AXI写burst size，突发传输中每次交易数据大小
AWBURST_1[1:0]	输入	AXI写burst类型
AWLOCK_1	输入	AXI中锁定写交易操作，与atomic transfer相关
AWVALID_1	输入	AXI写地址有效
AWREADY_1	输出	AXI写地址已准备好，表明slave可以接收写地址
AWQOS_1[3:0]	输入	AXI端口写优先级，取值越大，优先级越高
AWURGENT_1	输入	AXI紧急写操作，使能后端口仲裁立刻进入写操作
AWPOISON_1	输入	AXI无效写交易
WDATA_1[63:0]	输入	AXI写数据
WSTRB_1[7:0]	输入	AXI写数据选通信号
WLAST_1	输入	表明AXI突发传输中最后一次写交易
WVALID_1	输入	AXI写数据有效
WREADY_1	输出	表明slave可以接收写数据
BID_1[7:0]	输出	AXI写响应ID信号
BRESP_1[1:0]	输出	AXI写响应信号
BVALID_1	输出	AXI写响应有效
BREADY_1	输入	表明master可以接收写响应信号
ARID_1[7:0]	输入	AXI读地址ID，不同地址分组标记
ARADDR_1[31:0]	输入	AXI读地址，突发传输中第一次交易地址
ARLEN_1 [7:0]	输入	AXI读burst length，突发传输中交易次数
ARSIZE_1[2:0]	输入	AXI读burst size，突发传输中每次交易数据大小

端口名称	方向	功能描述
ARBURST_1[1:0]	输入	AXI读burst类型
ARLOCK_1	输入	AXI中锁定读交易操作，与atomic transfer相关
ARVALID_1	输入	AXI读地址有效
ARREADY_1	输出	AXI读地址已准备好，表明slave可以接收读地址
ARQOS_1[3:0]	输入	AXI端口读优先级，取值越大，优先级越高
ARPOISON_1	输入	AXI无效读交易
RID_1[7:0]	输出	AXI读数据ID
RDATA_1[63:0]	输出	AXI读数据
RRESP_1[1:0]	输出	AXI读响应
RLAST_1	输出	表明AXI突发传输中最后一次读交易
RVALID_1	输出	AXI读数据有效
RREADY_1	输入	表明master可以接收读数据并响应
ARURGENT_1	输入	AXI紧急读操作，使能后端口仲裁立刻进入读操作
RAQ_PUSH_1	输出	写入AXI读地址FIFO的交易
RAQ_SPLIT_1	输出	写入AXI读地址FIFO的Wrap交易起始部分
WAQ_PUSH_1	输出	写入AXI写地址FIFO的交易
WAQ_SPLIT_1	输出	写入AXI写地址FIFO的Wrap交易起始部分
ARESET_2	输入	AXI接口异步复位信号，低电平有效
ACLK_2	输入	AXI接口的时钟信号
AWID_2[7:0]	输入	AXI写地址ID，不同地址分组标记
AWADDR_2[31:0]	输入	AXI写地址，突发传输中第一次交易地址
AWLEN_2[7:0]	输入	AXI写burst length，突发传输中交易次数
AWSIZE_2[2:0]	输入	AXI写burst size，突发传输中每次交易数据大小
AWBURST_2 [1:0]	输入	AXI写burst类型
AWLOCK_2	输入	AXI中锁定写交易操作，与atomic transfer相关
AWVALID_2	输入	AXI写地址有效
AWREADY_2	输出	AXI写地址已准备好，表明slave可以接收写地址
AWQOS_2[3:0]	输入	AXI端口写优先级，取值越大，优先级越高，

端口名称	方向	功能描述
AWURGENT_2	输入	AXI紧急写操作，使能后端口仲裁立刻进入写操作
AWPOISON_2	输入	AXI无效写交易
WDATA_2[63:0]	输入	AXI写数据
WSTRB_2[7:0]	输入	AXI写数据选通信号
WLAST_2	输入	表明AXI突发传输中最后一次写交易
WVALID_2	输入	AXI写数据有效
WREADY_2	输出	表明slave可以接收写数据
BID_2 [7:0]	输出	AXI写响应ID信号
BRESP_2 [1:0]	输出	AXI写响应信号
BVALID_2	输出	AXI写响应有效
BREADY_2	输入	表明master可以接收写响应信号
ARID_2 [7:0]	输入	AXI读地址ID，不同地址分组标记
ARADDR_2 [31:0]	输入	AXI读地址，突发传输中第一次交易地址
ARLEN_2 [7:0]	输入	AXI读burst length，突发传输中交易次数
ARSIZE_2 [2:0]	输入	AXI读burst size，突发传输中每次交易数据大小
ARBURST_2 [1:0]	输入	AXI读burst类型
ARLOCK_2	输入	AXI中锁定读交易操作，与atomic transfer相关
ARVALID_2	输入	AXI读地址有效
ARREADY_2	输出	AXI读地址已准备好，表明slave可以接收读地址
ARQOS_2[3:0]	输入	AXI端口读优先级，取值越大，优先级越高。
ARPOISON_2	输入	AXI无效读交易
RID_2[7:0]	输出	AXI读数据ID
RDATA_2[63:0]	输出	AXI读数据
RRESP_2 [1:0]	输出	AXI读响应
RLAST_2	输出	表明AXI突发传输中最后一次读交易
RVALID_2	输出	AXI读数据有效
RREADY_2	输入	表明master可以接收读数据并响应

端口名称	方向	功能描述
ARURGENT_2	输入	AXI紧急读操作，使能后端口仲裁立刻进入读操作
RAQ_PUSH_2	输出	写入AXI读地址FIFO的交易
RAQ_SPLIT_2	输出	写入AXI读地址FIFO的Wrap交易起始部分
WAQ_PUSH_2	输出	写入AXI写地址FIFO的交易
WAQ_SPLIT_2	输出	写入AXI写地址FIFO的Wrap交易起始部分
CSYSREQ_0	输入	AXI低功耗请求信号
CSYSACK_0	输出	AXI低功耗请求响应信号
CACTIVE_0	输出	表明AXI进入低功耗状态，低电平有效
CSYSREQ_1	输入	AXI低功耗请求信号
CSYSACK_1	输出	AXI低功耗请求响应信号
CACTIVE_1	输出	表明AXI进入低功耗状态，低电平有效
CSYSREQ_2	输入	AXI低功耗请求信号
CSYSACK_2	输出	AXI低功耗请求响应信号
CACTIVE_2	输出	表明AXI进入低功耗状态，低电平有效
CSYSREQ_DDRC	输入	DDRC低功耗请求信号
CSYSACK_DDRC	输出	DDRC低功耗请求响应信号
CACTIVE_DDRC	输出	表明已进入DDRC低功耗状态，低电平有效
PA_RMASK [2:0]	输入	屏蔽相应端口到 PA 中读地址请求。[2]对应 port2
PA_WMASK [2:0]	输入	屏蔽相应端口到PA中写地址请求。[2]对应port2
DFI_ADDRESS[31:0]	输出	送给PHY地址信号
DFI_BANK[5:0]	输出	DDRC送给PHY的BANK地址
DFI_CAS_N[1:0]	输出	CAS信号，低电平有效
DFI_RAS_N[1:0]	输出	RAS信号，低电平有效
DFI_WE_N[1:0]	输出	写使能信号，低电平有效
DFI_CKE[1:0]	输出	时钟使能信号
DFI_CS[1:0]	输出	片选信号，低电平有效
DFI_ODT[1:0]	输出	片上校准信号
DFI_RESET_N[1:0]	输出	送给phy复位memory信号，低电平有效，仅支持DDR3

端口名称	方向	功能描述
DFI_WRDATA[63:0]	输出	DFI写数据信号
DFI_WRDATA_MASK[7:0]	输出	DFI写数据位屏蔽信号
DFI_WRDATA_EN[3:0]	输出	DFI写数据使能
DFI_RDDATA[63:0]	输入	DFI读数据
DFI_RDDATA_EN[3:0]	输出	DFI读数据使能
DFI_RDDATA_VALID[3:0]	输入	DFI读数据有效信号
DFI_CTRLUPD_ACK	输入	DDRC更新请求响应信号
DFI_CTRLUPD_REQ	输出	DDRC更新请求信号
DFI_PHYUPD_REQ	输入	PHY更新请求信号
DFI_PHYUPD_TYPE[1:0]	输入	PHY更新类型选择信号
DFI_PHYUPD_ACK	输出	PHY更新请求响应信号
DFI_LP_REQ	输出	DFI低功耗请求信号
DFI_LP_WAKEUP[3:0]	输出	DFI低功耗唤醒时间
DFI_LP_ACK	输入	DFI低功耗请求响应信号
DFI_DRAM_CLK_DISABLE	输出	关闭SDRAM时钟信号
DFI_INIT_START	输出	PHY初始化开始
DFI_INIT_COMPLETE	输入	PHY初始化完成
DFI_FREQUENCY[4:0]	输出	表明系统运行频率
PCLK	输入	APB时钟信号
PRESET	输入	APB复位信号
PADDR[11:0]	输入	APB地址
PWDATA[31:0]	输入	APB写数据
PWRITE	输入	APB读写方向
PSEL	输入	APB选择信号
PENABLE	输入	APB使能信号
PREADY	输出	APB ready信号
PRDATA[31:0]	输出	APB读数据
PSLVERR	输出	APB错误信号

8.2.4. 例化模板

```
 GTP_DDRC u_ddrc(
.CORE_DDRC_CORE_CLK (core_clk),
.CORE_DDRC_RST (ddrc_ddrc_rst),
.ARESET_0 (ddrc_axi_reset0),
.ACLK_0 (aclk_0),
.AWID_0 (awid_0),
.AWADDR_0 (awaddr_0),
.AWLEN_0 (awlen_0),
.AWSIZE_0 (awsize_0),
.AWBURST_0 (awburst_0),
.AWLOCK_0 (awlock_0),
.AWVALID_0 (awvalid_0),
.AWREADY_0 (awready_0),
.AWURGENT_0 (awurgent_0),
.AWPOISON_0 (awpoison_0),
.WDATA_0 (wdata_0),
.WSTRB_0 (wstrb_0),
.WLAST_0 (wlast_0),
.WVALID_0 (wvalid_0),
.WREADY_0 (wready_0),
.BID_0 (bid_0),
.BRESP_0 (bresp_0),
.BVALID_0 (bvalid_0),
.BREADY_0 (bready_0),
.ARID_0 (arid_0),
.ARADDR_0 (araddr_0),
.ARLEN_0 (arlen_0),
.ARSIZE_0 (arsize_0),
.ARBURST_0 (arburst_0),
.ARLOCK_0 (arlock_0),
.ARVALID_0 (arvalid_0),
.ARREADY_0 (arready_0),
.ARPOISON_0 (arpoison_0),
.RID_0 (rid_0),
.RDATA_0 (rdata_0),
.RRESP_0 (rresp_0),
.RLAST_0 (rlast_0),
.RVALID_0 (rvalid_0),
.RREADY_0 (rready_0),
.ARURGENT_0 (arurgent_0),
.RAQ_PUSH_0 (raq_push_0),
.RAQ_SPLIT_0 (raq_split_0),
.WAQ_PUSH_0 (waq_push_0),
.WAQ_SPLIT_0 (waq_split_0),
.ARESET_1 (ddrc_axi_reset1),
```

.ACLK_1 (aclk_1),
.AWID_1 (awid_1),
.AWADDR_1 (awaddr_1),
.AWLEN_1 (awlen_1),
.AWSIZE_1 (awsize_1),
.AWBURST_1 (awburst_1),
.AWLOCK_1 (awlock_1),
.AWVALID_1 (awvalid_1),
.AWREADY_1 (awready_1),
.AWURGENT_1 (awurgent_1),
.AWPOISON_1 (awpoison_1),
.WDATA_1 (wdata_1),
.WSTRB_1 (wstrb_1),
.WLAST_1 (wlast_1),
.WVALID_1 (wvalid_1),
.WREADY_1 (wready_1),
.BID_1 (bid_1),
.BRESP_1 (bresp_1),
.BVALID_1 (bvalid_1),
.BREADY_1 (bready_1),
.ARID_1 (arid_1),
.ARADDR_1 (araddr_1),
.ARLEN_1 (arlen_1),
.ARSIZE_1 (arsize_1),
.ARBURST_1 (arburst_1),
.ARLOCK_1 (arlock_1),
.ARVALID_1 (arvalid_1),
.ARREADY_1 (arready_1),
.ARPOISON_1 (arpoison_1),
.RID_1 (rid_1),
.RDATA_1 (rdata_1),
.RRESP_1 (rresp_1),
.RLAST_1 (rlast_1),
.RVALID_1 (rvalid_1),
.RREADY_1 (rready_1),
.ARURGENT_1 (arurgent_1),
.RAQ_PUSH_1 (raq_push_1),
.RAQ_SPLIT_1 (raq_split_1),
.WAQ_PUSH_1 (waq_push_1),
.WAQ_SPLIT_1 (waq_split_1),
.ARESET_2 (ddrc_axi_reset2),
.ACLK_2 (aclk_2),
.AWID_2 (awid_2),
.AWADDR_2 (awaddr_2),
.AWLEN_2 (awlen_2),
.AWSIZE_2 (awsize_2),
.AWBURST_2 (awburst_2),

.AWLOCK_2 (awlock_2),
.AWVALID_2 (awvalid_2),
.AWREADY_2 (awready_2),
.AWURGENT_2 (awurgent_2),
.AWPOISON_2 (awpoison_2),
.WDATA_2 (wdata_2),
.WSTRB_2 (wstrb_2),
.WLAST_2 (wlast_2),
.WVALID_2 (wvalid_2),
.WREADY_2 (wready_2),
.BID_2 (bid_2),
.BRESP_2 (bresp_2),
.BVALID_2 (bvalid_2),
.BREADY_2 (bready_2),
.ARID_2 (arid_2),
.ARADDR_2 (araddr_2),
.ARLEN_2 (arlen_2),
.ARSIZE_2 (arsize_2),
.ARBURST_2 (arburst_2),
.ARLOCK_2 (arlock_2),
.ARVALID_2 (arvalid_2),
.ARREADY_2 (arready_2),
.ARPOISON_2 (arpoison_2),
.RID_2 (rid_2),
.RDATA_2 (rdata_2),
.RRESP_2 (rresp_2),
.RLAST_2 (rlast_2),
.RVALID_2 (rvalid_2),
.RREADY_2 (rready_2),
.ARURGENT_2 (arurgent_2),
.RAQ_PUSH_2 (raq_push_2),
.RAQ_SPLIT_2 (raq_split_2),
.WAQ_PUSH_2 (waq_push_2),
.WAQ_SPLIT_2 (waq_split_2),
.AWQOS_0 (awqos_0),
.ARQOS_0 (arqos_0),
.AWQOS_1 (awqos_1),
.ARQOS_1 (arqos_1),
.AWQOS_2 (awqos_2),
.ARQOS_2 (arqos_2),
.CSYSREQ_0 (csysreq_0),
.CSYSACK_0 (csysack_0),
.CACTIVE_0 (cactive_0),
.CSYSREQ_1 (csysreq_1),
.CSYSACK_1 (csysack_1),
.CACTIVE_1 (cactive_1),
.CSYSREQ_2 (csysreq_2),

.CSYSACK_2 (csysack_2),
.CACTIVE_2 (cactive_2),
.CSYSREQ_DDRC (csysreq_ddrc),
.CSYSACK_DDRC (csysack_ddrc),
.CACTIVE_DDRC (cactive_ddrc),
.PA_RMASK (pa_rmask),
.PA_WMASK (pa_wmask),
.DFI_ADDRESS (dfi_address),
.DFI_BANK (dfi_bank),
.DFI_CAS_N (dfi_cas_n),
.DFI_RAS_N (dfi_ras_n),
.DFI_WE_N (dfi_we_n),
.DFI_CKE (dfi_cke),
.DFI_CS (dfi_cs),
.DFI_ODT (dfi_odt),
.DFI_RESET_N (dfi_reset_n),
.DFI_WRDATA (dfi_wrdata),
.DFI_WRDATA_MASK (dfi_wrdata_mask),
.DFI_WRDATA_EN (dfi_wrdata_en),
.DFI_RDDATA (dfi_rddata),
.DFI_RDDATA_EN (dfi_rddata_en),
.DFI_RDDATA_VALID (dfi_rddata_valid),
.DFI_CTRLUPD_ACK (dfi_ctrlupd_ack),
.DFI_CTRLUPD_REQ (dfi_ctrlupd_req),
.DFI_DRAM_CLK_DISABLE (dfi_dram_clk_disable),
.DFI_INIT_COMPLETE (dfi_init_complete),
.DFI_INIT_START (dfi_init_start),
.DFI_FREQUENCY (dfi_frequency),
.DFI_PHYUPD_REQ (dfi_phyupd_req),
.DFI_PHYUPD_TYPE (dfi_phyupd_type),
.DFI_PHYUPD_ACK (dfi_phyupd_ack),
.DFI_LP_REQ (dfi_lp_req),
.DFI_LP_WAKEUP (dfi_lp_wakeup),
.DFI_LP_ACK (dfi_lp_ack),
.PCLK (pclk),
.PRESET (ddrc_preset),
.PADDR (ddrc_paddr),
.PWDATA (ddrc_pwdata),
.PWRITE (ddrc_pwrite),
.PSEL (ddrc_psel),
.PENABLE (ddrc_penable),
.PREADY (pready),
.PRDATA (prdata),
.PSLVERR (pslverr)

8.2.5. 详细功能说明

DDRC 将系统总线（AXI4）上的命令和数据转换成符合 DDR 协议的 DFI 接口的命令和数据。

8.2.6. GTP 使用方法

1. AXI DFI 相关接口信号需符合 AXI4、 DFI3.1 协议要求
2. DDRC 和 DDRPHY 需联合使用

8.3. GTP_DDRPHY 使用说明

8.3.1. 支持器件

表 237 GTP_DDRPHY 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	不支持	不支持	不支持	不支持

8.3.2. 功能介绍

PGL DDRPHY 与 PGL DDRC 配合使用，完成对 DDR 的 calibration 和 read-write 操作，具备的主要特征如下：

- 完全兼容DDRC（synopsys umctl2）的DFI接口；
- 支持SDRAM的类型：LPDDR，DDR2，DDR3和DDR3L；
- 支持DDR的最高频率达到1066Mbps；
- 支持DDRC与DDRPHY的频率比为1:2工作模式；
- 支持的数据位宽是16bit，在进行相应配置后也可以支持8bit,Burst length支持BL8和BL4；
- 仅支持单rank，不支持ECC功能；
- 提供debug接口，用于监控和控制DDR的calibration过程,提供APB接口，可以对PHY的寄存器进行配置。

8.3.3. 端口描述

表 238 GTP_DDRPHY 端口列表

端口名称	方向	功能描述
DFI_RDDATA[63:0]	输出	DFI读数据
DFI_RDDATA_VALID[3:0]	输出	DFI读数据有效信号
DFI_CTRLUPD_ACK	输出	DDRC 更新请求响应信号

DFI_PHYUPD_REQ	输出	PHY更新请求信号
DFI_PHYUPD_TYPE[1:0]	输出	PHY更新类型选择信号
DFI_INIT_COMPLETE	输出	PHY 初始化完成
DFI_LP_ACK	输出	DFI低功耗请求响应信号
DFI_ERROR	输出	表示 PHY 内部产生错误
DFI_ERROR_INFO[2:0]	输出	表示 PHY 产生的错误信息内容。
DFI_ADDRESS[31:0]	输入	发送给 PHY 的 DFI 地址信号.
DFI_BANK[5:0]	输入	送给 PHY 的 BANK 地址.
DFI_CAS_N[1:0]	输入	送给PHY的CAS信号，低电平有效
DFI_CKE[1:0]	输入	送给 PHY 的时钟使能信号
DFI_CS[1:0]	输入	送给 PHY 的片选信号，低电平有效
DFI_ODT[1:0]	输入	送给 PHY 的片上校准信号
DFI_RAS_N[1:0]	输入	送给 PHY 的 RAS 信号，低电平有效
DFI_WE_N[1:0]	输入	送给 PHY 的写使能信号，低电平有效
DFI_WRDATA[63:0]	输入	送给 PHY 的 DFI 写数据信号
DFI_WRDATA_EN[3:0]	输入	送给 PHY 的 DFI 写数据使能
DFI_WRDATA_MASK[7:0]	输入	送给 PHY 的 DFI 写数据位屏蔽信号
DFI_RDDATA_EN[3:0]	输入	送给 PHY 的 DFI 读数据使能
DFI_RESET_N[1:0]	输入	送给 phy 复位 memory 信号，低电平有效，仅支持 DDR3
DFI_CTRLUPD_REQ	输入	更新请求信号
DFI_PHYUPD_ACK	输入	送给PHY的PHY更新请求响应信号
DFI_DRAM_CLK_DISABLE	输入	送给 PHY 的关闭 SDRAM 时钟信号
DFI_INIT_START	输入	送给 PHY 初始化开始信号
DFI_FREQUENCY[4:0]	输入	送给 PHY 表明系统运行频率
DFI_LP_REQ	输入	送给 PHY 的 DFI 低功耗请求信号
DFI_LP_WAKEUP[3:0]	输入	送给 PHY 的 DFI 低功耗唤醒时间
PCLK	输入	APB时钟信号
PRESET	输入	APB复位信号

PADDR[11:0]	输入	APB地址
PWDATA[31:0]	输入	APB写数据
PWRITE	输入	APB读写方向
PSEL	输入	APB外设选择信号
PENABLE	输入	APB使能信号
PREADY	输出	APB ready信号
PRDATA[31:0]	输出	APB读数据
DDRPHY_CLKIN	输入	phy时钟信号
DDYPHY_RST	输入	phy 逻辑复位
DDRPHY_RST_REQ	输出	DDR PHY request DDR interface reset
DDRPHY_RST_ACK	输入	DDR interface reset completion flag from reset logic (Soft)
DDRPHY_UPDATE	输入	PHY Update 更新
DDRPHY_UPDATE_DONE	输出	PHY Update 更新完成
DDRPHY_WL_STEP_L[7:0]	输出	输出给DQS_DDC，用于write leveling training的code码（用于低8bit数据）
DDRPHY_WL_CTRL_L[2:0]	输出	用于控制DQS_DDC进行WCLK_DEL时钟延迟的微调（用于低8bit数据）
DDRPHY_RDQS_STEP_L[2:0]	输出	控制CLK_DQSI_DEL时钟延迟微调（用于低8bit数据）
DDRPHY_DQS_GATE_CTRL_L[1:0]	输出	对CLK_DQSI做gate操作的信号（用于低8bit数据）
DDRPHY_READ_CLK_CTRL_L[2:0]	输出	[2]:gate来源选择信号；[1:0]:gate training时钟源选择信号（用于低8bit数据）
DDRPHY_WL_OV_L	输入	CLK_W_DEL时钟延迟微调的裕量测试输出标识（用于低8bit数据
DDRPHY_DGTS_L	输入	gat窗口位置标识（用于低8bit数据

DDRPHY_READ_VALID_L	输入	来自DQS_DDC读有效信号
DDRPHY_DLL_STEP[7:0]	输入	DLL调节控制码
DDRPHY_RDEL_OV_L	输入	CLK_W_DEL时钟延迟微调的裕量测试输出标识,用于低8bit “1”表示溢出；“0”表示未溢出
DDRPHY_WEN_L[15:0]	输出	控制相应IOL为输出状态
DDRPHY_RDATA_L[31:0]	输入	来自IOL的低8bit读数据
DDRPHY_WDATA_L[31:0]	输出	到IOL的低8bit写数据
DDRPHY_WDQS_EN_L[1：0]	输出	控制相应IOL为输出状态
DDRPHY_WDQS_L[3:0]	输出	Data Strobe for write data. Centered in write data. For the x16, this corresponds to the data on DQ0-DQ7
DDRPHY_DM_L[3:0]	输出	Input Data Mask: DM is an input mask signal for write data. Input data is masked when DM is sampled HIGH coincident with that input data during a Write access
DDRPHY_WL_STEP_H[7:0]	输出	输出给DQS_DDC，用于write leveling training的code码（用于高8bit数据）
DDRPHY_WL_CTRL_H[2:0]	输出	用于控制DQS_DDC进行WCLK_DEL时钟延迟的微调（用于高8bit数据）
DDRPHY_RDQS_STEP_H[2:0]	输出	控制CLK_DQSI_DEL时钟延迟微调（用于高8bit数据）
DDRPHY_DQS_GATE_CTRL_H[1:0]	输出	对CLK_DQSI做gate操作的信号（用于高8bit数据）
DDRPHY_READ_CLK_CTRL_H[2:0]	输出	[2]:gate来源选择信号；[1:0]:gate training时钟源选择信号（用于高8bit数据）
DDRPHY_RDEL_OV_H	输入	CLK_W_DEL时钟延迟微调的裕量测试输出标识,用于高8bit “1”表示溢出；“0”表示未溢出
DDRPHY_WEN_H[15:0]	输入	控制高8bit daqIOL为输出状态

DDRPHY_WL_OV_H	输入	CLK_W_DEL时钟延迟微调的裕量测试输出标识（用于高8bit数据）
DDRPHY_DGTS_H	输入	gat窗口位置标识（用于高8bit数据）
DDRPHY_READ_VALID_H	输入	来自DQS_DDC读有效信号
DDRPHY_RDATA_H[31:0]	输入	来自IOL的高8bit读数据
DDRPHY_WDATA_H[31:0]	输出	到IOL的高8bit写数据
DDRPHY_DM_H[3:0]	输出	Input Data Mask: DM is an input mask signal for write data. Input data is masked when DM is sampled HIGH coincident with that input data during a Write access.
DDRPHY_WDQS_H[3:0]	输出	Data Strobe for write data. Centered in write data. For the x16, this corresponds to the data on DQ8-DQ15
DDRPHY_WDQS_EN_H[1：0]	输出	控制相应 IOL 为输出状态
DDRPHY_CA_EN[55:0]	输出	控制与命令相关 IOL 为输出状态,其中用于控制 ddrphy_dm_l 和 ddrphy_dm_h 输出状态的两个信号来源于 ddrphy_wen_l 和 ddrphy_wen_h,余下传输 CA 命令的 26 个 IOL tie 0（不包括 ddrphy_mem_reset）
DDRPHY_ADDR[63:0]	输出	PHY输出到SDRAM地址信号
DDRPHY_BA[11:0]	输出	SDRAM的Bank选择地址
DDRPHY_CK[3:0]	输出	Clock, All address and control input signals are sampled on the the positive edge of CK
DDRPHY_CKE[3:0]	输出	Clock Enable: CKE HIGH activates, and CKE Low deactivates
DDRPHY_RAS_N[3:0]	输出	RAS command
DDRPHY_CS_N[3:0]	输出	Chip Select: All commands are masked when CS# is registered HIGH

DDRPHY_CAS_N[3:0]	输出	CAS command
DDRPHY_WE_N[3:0]	输出	WE command
DDRPHY_ODT[3:0]	输出	On Die Termination: ODT (registered HIGH) enables termination resistance internal to the DDR3 SDRAM
DDRPHY_MEM_RST	输出	Active Low Asynchronous Reset
DDRPHY_GATEI_H	输出	对输入CLK_IO做关断操作，拉高时，经过三个CLK_IO下降沿后clk_io_out拉低为零。在QUAD/HALF RATE模式下，写时钟CLK_W拉低为零，CLK_W_DEL拉高，用于高8bit DQ
DDRPHY_GATEI_L	输出	对输入CLK_IO做关断操作，拉高时，经过三个CLK_IO下降沿后clk_io_out拉低为零。在QUAD/HALF RATE模式下，写时钟CLK_W拉低为零，CLK_W_DEL拉高, 用于低8bit DQ
DDRPHY_DQ_L[7:0]	输入	IOL 直接驱动输入, 对应传输 DQ[7:0]的 8 个 IOL 时IOL不做gearing操作，用于read training。
DDRPHY_DQ_H[7:0]	输入	IOL 直接驱动输入, 对应传输 DQ[15:8]的 8 个 IOL 时IOL不做gearing操作，用于read training。
DLL_UPDATE_ACK	输入	Dll_Update响应信号
DLL_UPDATE_REQ	输出	Dll_Update 请求信号
DDRPHY_UPDATE_TYPE[1:0]	输入	来自软逻辑的用户upate请求类型
DDRPHY_UPDATE_COMP_VAL_L[1:0]	输入	update操作时，低8bit gate移动的位置值，以1/4个cycle为单位
DDRPHY_UPDATE_COMP_DIR_L	输入	低8bit gate移动的方向指示 1’b0：gate向左移动； 1’b1：gate向右移动
DDRPHY_UPDATE_COMP_VAL_H[1:0]	输入	update操作时，高8bit gate移动的位置值，以1/4个cycle为单位
DDRPHY_UPDATE_COMP_DIR_H	输入	高8bit gate移动的方向指示

		1’b0：gate向左移动； 1’b1：gate向右移动
IOL_CE[59:0]	输出	60个IOL寄存器输入输出模式下寄存器时钟开启，内部tie1
IOL_CLK_SYS[59:0]	输出	60个IOL系统时钟,其中47个用到的IOL与core_ddrc_core_clk相同，剩下13个未使用IOL时钟tie1
IOL_LRS[59:0]	输出	47个使用到的IOL输入输出寄存器本地reset/set 13个未使用到IOL tie1
RST_DLL	输出	DLL local reset控制信号，极性可选择
UPDATE_N	输出	DLL输出码更新信号(低电平有效)，在DDR读写操作时候需要置高。最小信号电平宽度要求是两个DLL输入参考时钟周期的2倍。 ”1”: hold “0”: update
DLL_CLK_INPUT	输出	输入参考时钟，时钟源是来自SRB的regional clcok或来自 IO clock tree,DLL只在1:1模式下使用该信号，而现在是1:2模式，该信号tie 1(tie0/1都可以)
DLL_FREEZE	输出	DLL冻结控制 “1”: freeze, 输出CTRL_CODE、CTRL_CODE_T、LOCK、DIV_CLKIN_T信号保持不变； Fast_lock模式时，DIV_OSCCLK_T拉低； 非fast_lock模式时，DIV_OSCCLK_T保持不变； “0”: release，电路正常工作
DQS_RST[4:0]	输出	DQS local reset控制信号，极性可选择
DQS_RST_TRAINING_N[4:0]	输出	低电平有效复位信号，用于在dqs gate training结束时对DQSL的读写指针生成和读指针控制逻辑的复位信号复位

DQS_CLK_REGIONAL [4:0]	输出	来自HMEMC的regional clock时钟
DQS_GATEI[2:0]	输出	输入CLK_IO做关断操作，拉高时，经过三个CLK_IO下降沿后clk_io_out拉低为零。在QUAD/HALF RATE模式下，写时钟CLK_W拉低为零，CLK_W_DEL拉高
DQS_WL_STEP[23:0]	输出	用于 write leveling training 的 code 码 用于ca的3个dqs_ddc不做write leveling
DQS_WL_CTRL[8:0]	输出	CLK_W_DEL 时钟延迟的微调控制 [0]: 加载输入 code 码（DLL+Adjustment 的设置） [1]:利用该信号的下降沿和码位方向信号一起改变码字 [2]:码位方向控制，“0” 增加 码字,“1” 减少码字
DQS_DQS_GATE_CTRL[11:0]	输出	对CLK_DQSI做gate操作的信号，内部tie为0
DQS_DQS_GATE_CTRL_TF2[3:0]	输出	用于控制DQ两个DQS_DDC相对应信号高两bit未使用，需要tile1
DQS_READ_CLK_CTRL[8:0]	输出	内部 tie 为 0，控制 3 个用于 CMD 的 dqs_ddc
DQS_RDEL_CTRL[8:0]	输出	内部tie为1，控制3个用于CMD的dqs_ddc
IOL_TX_DATA_TF8[103:0]	输出	未使用的13个IOL TX_DATA tie 1
IOL_TX_DATA_TF4[183:0]	输出	除用于 reset 外已使用 46 个 IOL 都只用到 TX_DATA [3:0],剩余 bit 位 tie 1
IOL_TX_DATA_TF7[6:0]	输出	Mem_rst只使用TX_DATA[7]，因此剩余6bit tie1
IOL_IODLY_CTRL[179:0]	输出	60 个 IOL 都未使用需要 tie 1
IOL_MIPI_SW_DYN_I[59:0]	输出	60个IOL都未使用需要tie 1
IOL_TS_CTRL_TF4[51:0]	输出	13个未使用的IOL TS_CTRL需要tie 1
IOL_TS_CTRL_TF2[91:0]	输出	46 个使用到的 IOL ts_ctrl 只用低 2bit,剩余高 2bit tie 1
IOL_TS_CTRL_TF3[2:0]	输出	传输复位的IOL ts_ctrl只用[0],剩余高3bit tie

		1
MEM_RST_EN	输出	用于传输复位的IOL为输出状态
SRB_RST_DLL	输入	dll 复位信号，来自srb，由fpga core 来控制，高电平有效。
DLL_UPDATE_N	输入	穿过 HMEMC 输出给 DLL update 信号
SRB_DLL_FREEZE	输入	穿过HMEMC输出给DLL freeze信号
SRB_IOL_RST	输入	47个iol的复位信号，来自srb，由fpga core 来控制，高电平有效
SRB_DQS_RST	输入	5 个 dqs_ddc 的复位信号，来自 srb，由 fpga core 来控制，高电平有效。
SRB_DQS_RST_TRAINING	输入	控制 DQS_DDC 在 dqs gate training 结束时对 DQSL 的读写指针生成和读指针控制逻辑的复位信号复位,高电平有效。

8.3.4. 参数描述

表 239 GTP_DDRPHY 参数列表

参数名称	参数类型	有效值	默认值	功能描述
MR0_DDR3	binary	16'h0~16'hffff	16'h1108	Phy 缓存的 mr0_ddr3 值 每个寄存器中 bit 位具体含义请参考 DDR 协议
MR1_DDR3	binary	16'h0~16'hffff	16'h0001	Phy 缓存的 mr1_ddr3 值
MR2_DDR3	binary	16'h0~16'hffff	16'h0000	Phy 缓存的 mr2_ddr3 值
MR3_DDR3	binary	16'h0~16'hffff	16'h0000	Phy 缓存的 mr3_ddr3 值
MR_DDR2	binary	16'h0~16'hffff	16'h0100	Phy 缓存的 mr_ddr2 值
EMR1_DDR2	binary	16'h0~16'hffff	16'h0401	Phy 缓存的 emr1_ddr2 值
EMR2_DDR2	binary	16'h0~16'hffff	16'h0000	Phy 缓存的 emr2_ddr2 值
EMR3_DDR2	binary	16'h0~16'hffff	16'h0000	Phy 缓存的 emr3_ddr2 值
MR_LPDDR	binary	16'h0~16'hffff	16'h0003	Phy 缓存的 mr_lpddr 值
EMR_LPDDR	binary	16'h0~16'hffff	16'h0000	Phy 缓存的 emr_lpddr 值
TMRD	interger	0~3	0	tMRD 计数器值，若存在余数，则商+1，以上都以 phy

				cycle 为单位
TMOD	interger	0~7	0	tMOD 计数器值，若存在余数，则商+1，以上都以 phy cycle 为单位
TZQINIT	Interger	0~1023	0	tZQinit 计数器值，若存在余数，则商+1，以上都以 phy cycle 为单位
TRP	interger	0~7	0	Trp计数器值，若存在余数，则商+1，以上都以phy cycle 为单位
TXPR	interger	0~15	0	tXPR 计数器值，若存在余数，则商+1，以上都以 phy cycle 为单位
TRFC	Interger	0~255	0	Trfc 计数器值，若存在余数，则商+1，以上都以 phy cycle 为单位。
WL_EN	string	“TRUE” “FALSE”	“FALSE”	Write Leveling enble 1’b0 : disable, default. 1’b1: enable
DDR_TYPE	string	“DDR3”,”DDR2”，”LPDDR”	“DDR3”	DDR 类型
DATA_WIDTH	string	“16BIT”,”8BIT”	“16BIT”	“16BIT”:full data width ”8BIT”: half data width
DQS_GATE_MODE	binary	2’b00~2’b11	2’b00	Dqs_gate mode select 2,b00:通过数值确定gate边界，default； 2’b01:通过多余引脚环回，确定gate边界； 2’b10:通过改进后的dgts确定gate边界；
WRDATA_PATH_ADJ	string	“TRUE”	“FALSE”	写数据通路偏移调整

		“FALSE”		1’b0：数据不偏移，default; 1’b1：数据偏移
CTRL_PATH_ADJ	string	“TRUE” “FALSE”	“FALSE”	写命令通路偏移调整 1’b0：命令不偏移，default; 1’b1：命令偏移
WL_MAX_STEP	binary	8'h0~8'hff	8'h0	Write leveling 最大调整步数，范围：0~255
WL_MAX_CHECK	binary	5'h0~5'h11111	5'h0	Write leveling 最大调整次数，范围：0~31
MAN_WRLVL_DQS_L	string	“TRUE” “FALSE”	“FALSE”	手动调节低 8bit（dqsh） Write leveling enable 1’b0:disable 1’b1:enable
MAN_WRLVL_DQS_H	string	“TRUE” “FALSE”	“FALSE”	手动调节高 8bit（dqsh） Write leveling enable 1’b0:disable 1’b1:enable
WL_CTRL_L	binary	3'b000~3'b111	3’b000	低8bit（dqsl）Write leveling的step微调，使用说明参考DDC SPEC
WL_CTRL_H	binary	3’b000~3’b111	3’b000	高8bit（dqsh）Write leveling的step微调，使用说明参考DDC SPEC
INIT_READ_CLK_CTRL	binary	2’b00~2’b11	2’b00	给出 gate 初始 read_clk_ctrl 偏移量，由实际 SDRAM 决定。用于低 8bit DQ 2’b00:不偏移 2’b01:偏移 1/4 cycle 2’b10:偏移 1/2 cycle 2’b11:偏移 3/4cycle

INIT_READ_CLK_CTRL_H	binary	2'b00~2'b11	2'b00	给出 gate 初始 read_clk_ctrl 偏移量，由实际 SDRAM 决定。用于高 8bit DQ 2'b00:不偏移 2'b01:偏移 1/4 cycle 2'b10:偏移 1/2 cycle 2'b11:偏移 3/4cycle
INIT_SLIP_STEP	binary	4'h0~4'hf	4'h0	给出 gate 初始 dqs_gate_ctrl 初始位置， 取值范围：0~7, 用于低8bit DQ
INIT_SLIP_STEP_H	binary	4'h0~4'hf	4'h0	给出 gate 初始 dqs_gate_ctrl 初始位置， 取值范围：0~7,用于高8bit DQ。
FORCE_READ_CLK_CTRL_L	string	"TRUE" "FALSE"	"FALSE"	手动调节低 8bit（dqsl）gate 1'b0:disable 1'b1:enale
FORCE_READ_CLK_CTRL_H	string	"TRUE" "FALSE"	"FALSE"	手动调节高 8bit（dqsh）gate 1'b0:disable 1'b1:enale
STOP_WITH_ERROR	string	"TRUE" "FALSE"	"TRUE"	第一次 gate 失败后立刻退出 1'b0 : disable,default
DQGT_DEBUG	binary	1'b0~1'b1	1'b0	使 PHY 始终处于 gate 状态使能： 1'b0:disable,default 1'b1:enable
WRITE_DEBUG	binary	1'b0~1'b1	1'b0	PHY 做 DQS gate 时，连续发送数据使能 1'b0:disable，deafult

				1’b1:enable
RDEL_ADJ_MAX_RANG	binary	5’b00000~5’b11111	5’h0	rdel调整的最大次数，0~31
MIN_DQSI_WIN	binary	4’h0~4’hf	4’h0	dqsi的有效窗口大小，0~15
INIT_SAMP_POSITION	binary	8’h0~8’hff	8’h0	dqsi 初始采样点位置：0~255 用于低8bit DQ
INIT_SAMP_POSITION_H	binary	8’h0~8’hff	8’h0	dqsi 初始采样点位置：0~255 用于高8bit DQ
FORCE_SAMP_POSITION_L	string	“TRUE”“FALSE”	“FALSE”	强制低 8bit（dqsl）设置初始采样位置使能 1’b0:disable 1’b1:enable
FORCE_SAMP_POSITION_H	string	“TRUE” “FALSE”	“FALSE”	强制高 8bit（dqsl）设置初始采样位置使能 1’b0:disable 1’b1:enable
T200US	interger	0~131071	17’h0D2F0	以 phy clock cycle 为单位，给出定时 200us 时的定时计数器值，若存在余数，则商+1
T400NS	interger	0~127	0	1.DDR2 时，定时 400ns，以 phy cycle 计算 2.其他，default 为 0
T_LPDDR	binary	9’h0~9’h1ff	9’h0	[2:0]仅 ddr2/lpddr 时配置 [8:3]仅 lpddr 配置，以 phy cycle 为单位
RDEL_RD_CNT	binary	19’h 0~19’h7ffff	19’h 0	Rdel calbiration 时，发送的读操作次数， 仿真时，请将该值配成 2 的 16 次方

				实际上板调试时，需要根据情况，配置合适值
REF_CNT	binary	8’h00~8’hff	8’h00	[3:0] init_ref_cnt Initialization 时的刷新次数 [7:4] rd_ref_cnt read calibration 时的刷新次数 需要根据实际 sdram 来确定
APB_VLD	string	“TURE” “FALSE”	TURE	apb 配置的数据有效标示 1’b0:无效，default 1’b1：有效
TEST_PATTERN1	binary	128’h0000_0000_0000_0000_0000_0000_0000_0000~128’hffff_ffff_ffff_ffff_ffff_ffff_ffff_ffff	128’h0000_ffff_0000_ffff_0000_ffff_0000_ffff	测试 pattern1. dqs gate training pattern,
TEST_PATTERN2	binary	32’h0000_0000~32’hffff_ffff	32’h0000_0000	测试 pattern2,
TEST_PATTERN3	binary	32’h0000_0000~32’hffff_ffff	32’h0000_0000	测试 pattern3,
TXS	binary	8’h0~8’hff	8’h0	定时 txs 的计数器值，以 phy cycle 为单位
TRAIN_RST_TYPE	string	“TRUE” “FALSE”	FALSE	2 个版本之间切换使能： false：采用原始版本，只支持数据比对方式确定 gate 位置，同时 calibration 过程中复位时会导致 DDR 进入 Self – refresh 状态； true：采用修改后版本，只

				支持数据比对方式或 dgts 方式确定 gate 位置，同时 calibration 过程中复位时，DDR 无需进入 Self – refresh 状态；
WL_SETTING	binary	1’b0 ~1’b1	1’b1	Write leveling 策略选择。0：初始采样为 1 时结束；1：初始采样为 1 时继续。
WCLK_DEL_SEL	binary	1’b0 ~1’b1	1’b0	输出 DQ 信号的驱动时钟 WCLK_DEL 相位选择。0:270°；1:90°
INIT_WRLVL_STEP_L	binary	8’h00~8’hff	8’h00	Wl_en 为 0 时低八位 DQ 使用的固定wrlvl step
INIT_WRLVL_STEP_H	binary	8’h00~8’hff	8’h00	Wl_en 为 0 时高八位 DQ 使用的固定wrlvl step

8.3.5. 详细功能说明

- 完全兼容uMCTL2（synopsys）的DFI3.1接口；
- 支持SDRAM的类型：LPDDR，DDR2，DDR3和DDR3L；
- 支持DDR的最高频率达到1066Mbps；
- 仅支持MC与PHY的频率比是1:2；
- 支持的数据位宽是16/8；
- DDR3的支持BL8，DDR2和LPDDR支持BL4和BL8；
- 仅支持单rank；
- 不支持ECC功能；
- 提供debug接口，用于监控和控制DDR的calibration过程；
  提供 APB 接口，用于接收配置信息——phy 的 parameter；

8.4. GTP_IDDR_E2 使用说明

8.4.1. 支持器件

表 240 GTP_IDDR_E2 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H

是否支持该GTP	支持	支持	支持	支持	支持

8.4.2. 功能介绍

GTP_IDDR_E2 支持同沿 pipeline 的 DDR1TO2 速率转换。其结构框图如下所示：



图 108 GTP_IDDR_E2 结构框图

8.4.3. 端口描述

表 241 GTP_IDDR_E2 端口列表

端口名称	方向	宽度	功能描述
D	输入	1	数据输入
RS	输入	1	本地复位信号
CLK	输入	1	输入系统时钟
Q0	输出	1	数据输出
Q1	输出	1	数据输出

8.4.4. 参数描述

表 242 GTP_IDDR_E2 参数列表

参数名称	参数类型	有效值	默认值	功能描述
GRS_EN	String	“TRUE”“FALSE”	“TRUE”	全局复位使能
LRS_EN	String	“TRUE”“FALSE”	“FALSE”	局部复位使能

8.4.5. 例化模板

```
GTP_IDDR_E2#(
        .GRS_EN (“TRUE”),
        .LRS_EN (“FALSE”)
)
GTP_IDDR_E2_inst(
    .D    (d),
    .RS   (rs),
```

```
        .CLK (clk),
        .Q0    (q0),
        .Q1    (q1)
    );
```

8.5. GTP_ODDR_E2 使用说明

8.5.1. 支持器件

表 243 GTP_ODDR_E2 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

8.5.2. 功能介绍

GTP_ODDR_E2 支持同沿的 DDR2TO1 速率转换。其结构框图如下所示：



图 109 GTP_ODDR_E2 结构框图

8.5.3. 端口描述

表 244 GTP_ODDR_E2 端口列表

端口名称	方向	宽度	功能描述
RS	输入	1	本地复位/置位信号
CLK	输入	1	输入系统时钟
D0	输入	1	输入数据信号
D1	输入	1	输入数据信号
T	输入	1	选通数据输入
Q	输出	1	数据输出
TQ	输出	1	输出选通

8.5.4. 参数描述

表 245 GTP_ODDR_E2 参数列表

参数名称	参数类型	有效值	默认值	功能描述
GRS_EN	String	“TRUE”“FALSE”	“TRUE”	全局复位使能
LRS_EN	String	“TRUE”“FALSE”	“FALSE”	局部复位使能

8.5.5. 例化模板

```
GTP_ODDR_E2#
(
.GRS_EN (“TRUE”),
.LRS_EN (“TRUE”)
) GTP_ODDR_E2_INST (
. RS(RS),
.CLK (RCLK),
. D0(D0),
.D1(D1),
. T (T),
. Q (Q),
. TQ (TQ)
);
```

9. 其它 GTP 使用说明

9.1. GTP_GRS 使用说明

9.1.1. 支持器件

表 246 GTP_GRS 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

9.1.2. 功能介绍

该GTP用来控制全局复位信号，仿真时需例化该GTP。

9.1.3. 端口描述

表 247 GTP_GRS 端口列表

端口名称	方向	功能描述
GRS_N	输入	全局复位

9.1.4. 例化模板

```
GTP_GRS GRS_INST(
        .GRS_N (grs_n)
    );
```

9.2. GTP_START_E1 使用说明

9.2.1. 支持器件

表 248 GTP_START_E1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

9.2.2. 功能介绍

该 GTP 用来描述唤醒操作的全局信号释放过程。

9.2.3. 端口描述

表 249 GTP_START_E1 端口列表

端口名称	方向	功能描述
CLK	输入	唤醒时钟
GOE	输入	全局 IO 输出使能
GRS_N	输入	全局复位
GWE_N	输入	存储器写使能
WAKEUP_OVER	输出	唤醒结束标识信号

9.2.4. 例化模板

```
GTP_START GTP_START_inst(
    .CLK        (clk ),
    .GOE        (gouten),
    .GRS_N      (grs_n),
    .GWE        (gwe),
    .WAKEUP_OVER    (wakeup_over)
);
```

9.2.5. 详细功能说明

唤醒操作，全局信号的释放过程。

9.3. GTP_RES_CAL 使用说明

9.3.1. 支持器件

表 250 GTP_RES_CAL 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	不支持	不支持	不支持	不支持

9.3.2. 功能介绍

GTP_RES_CAL 描述用户模式时，进行 IO 的阻抗校准的输入、输出特性。

9.3.3. 端口描述

表 251 GTP_RES_CAL 端口列表

端口名称	方向	功能描述
CAL_REQ	输入	来自用户的校准请求信号，1 有效
CAL_DONE	输出	输出的校准完成信号，1 表示完成

9.3.4. 参数描述

表 252 GTP_RES_CAL 参数列表

参数名称	参数类型	有效值	默认值	功能描述
BANK_INDEX	String	L0,L1,L2,R0,R1,R2	L0	IO bank 位置

9.3.5. 例化模板

```
GTP_RES_CAL#(
        .BANK_INDEX (L0)
)
GTP_UDID _inst(
    .CAL_DONE  (cal_done),
    .CAL_REQ    (cal_req)
);
```

9.3.6. 详细功能说明

如下图所示（小写字母所示信号为中间信号）为输入输出信号的时序。初始时CAL_DONE=1，当CAL_REQ 进过一个pulse，便开始校准，CAL_DONE翻转为0，计数器计到1050个周期后，完成校准，CAL_DONE翻转为1。

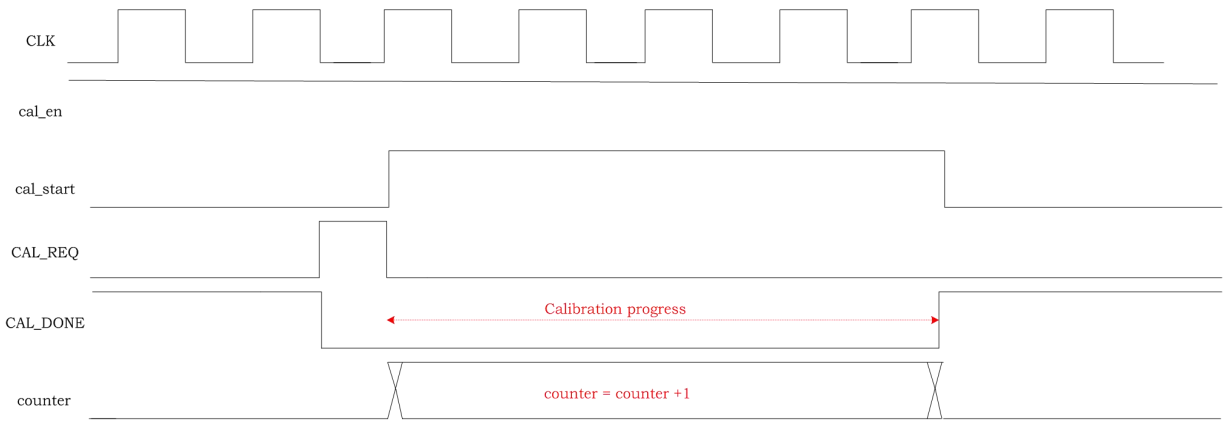


图 110 GTP_RES_CAL 校准模式时序

9.3.7. GTP 使用方法

GTP_RES_CAL直接map到RESCAL上。

9.4. GTP_APM_E1 使用说明

9.4.1. 支持器件

表 253 GTP_APM_E1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	支持	支持	支持

9.4.2. 功能介绍

GTP_APM_E1为逻辑运算单元，支持多种类型的逻辑运算，包括9bit和18bit的乘法运算，乘累加运算，通用乘加运算和FIR逻辑运算。并且具有可选的预加（Preadd）功能支持有符号数和无符号数，可选输入/输出寄存器，使能信号均为高有效，详细功能请参考《UG020003_Logos系列FPGA算术处理模块（APM）用户指南》。结构框图见下图。

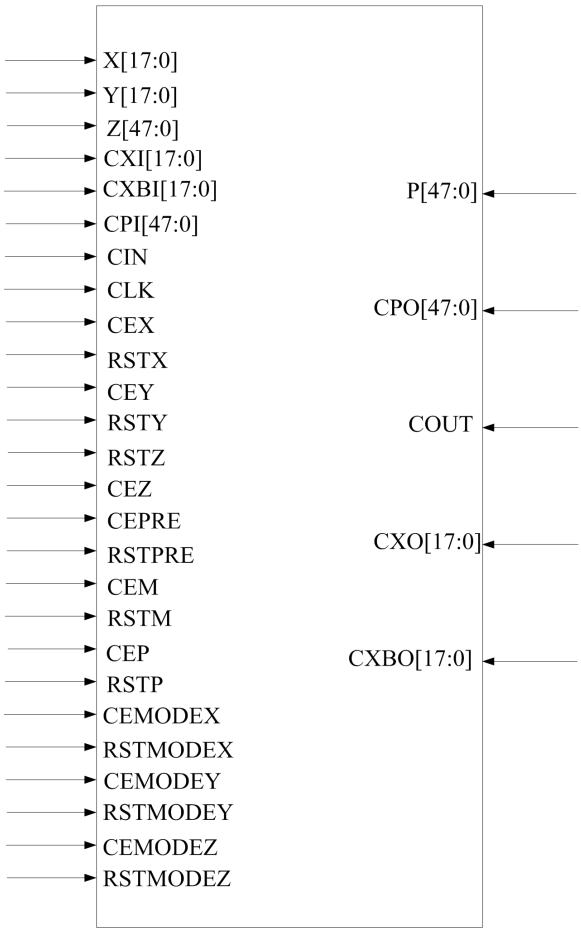


图 111 GTP_APM_E1 结构框图

9.4.3. 端口描述

表 254 GTP_APM_E1 端口列表

端口名称	方向	功能描述
X[17:0]	输入	并行数据输入 X
CXI[17:0]	输入	级联 X 输入
CXBI[17:0]	输入	级联 Z 输入
Y[17:0]	输入	并行数据输入 Y
Z[47:0]	输入	并行数据输入 Y
CPI[47:0]	输入	级联 P 输入
CIN	输入	级联 CIN 输入
MODEX	输入	APM 动态 X 端控制操作符
MODEY[2:0]	输入	APM 动态 Y 端控制操作符
MODEZ[3:0]	输入	APM 动态 Z 端控制操作符

CLK	输入	Clock input for DSP MU 0
CEX	输入	Clock X enable input
RSTX	输入	高有效的 X Reset input
CEY	输入	Clock Y enable input
RSTY	输入	高有效的 Y Reset input
CEZ	输入	Clock Z enable input
RSTZ	输入	高有效的 Z Reset input
CEPRE	输入	Clock PRE enable input
RSTPRE	输入	高有效的 PRE Reset input
CEM	输入	Clock M enable input
RSTM	输入	高有效的 M Reset input
CEP	输入	Clock P enable input
RSTP	输入	高有效的 P Reset input
CEMODEX	输入	Clock MODEX enable input
RSTMODEX	输入	高有效的 MODEX Reset input
CEMODEY	输入	Clock MODEY enable input
RSTMODEY	输入	高有效的 MODEY Reset input
CEMODEZ	输入	Clock MODEZ enable input
RSTMODEZ	输入	高有效的 MODEZ Reset input
P[47:0]	输出	并行数据输出
CPO[47:0]	输出	级联 P 输出
COUT	输出	级联 CIN 输出
CXO[17:0]	输出	级联 X 输出
CXBO[17:0]	输出	级联 Z 输出

9.4.4. 参数描述

表 255 GTP_APM_E1 参数列表

参数名	描述	默认值	设置值

GRS_EN	全局复位信号使能	TRUE	FALSE：不使能 TRUE：使能
ASYNC_RST	复位选择	0	0：同步复位 1：异步复位
X_SIGNED	X 的符号性	0	0：无符号 1：有符号
Y_SIGNED	Y 的符号性	0	0：无符号 1：有符号
USE_PREADD	preadder 选择	0	0：不使用 1：使用
USE_POSTADD	postadder 选择	0	0：不使用 1：使用
X_REG	X Register 选择	0	0：不使用 1：使用
CXO_REG	X 输出寄存器延迟	0	2’b00：0 个周期 2’b01：1 个周期 2’b10：2 个周期 2’b11：3 个周期
Y_REG	Y Register 选择	0	0：不使用 1：使用
Z_REG	Z Register 选择	0	0：不使用 1：使用
PREADD_REG	preadder Register 选择	0	0：不使用 1：使用
P_REG	postadder Register 选择	0	0：不使用 1：使用
MULT_REG	multiplier Register 选择	0	0：不使用 1：使用
MODEX_REG	MODEX Register 选择	0	0：不使用 1：使用

MODEY_REG	MODEY Register 选择	0	0：不使用 1：使用
MODEZ_REG	MODEZ Register 选择	0	0：不使用 1：使用
CPO_REG	PO、PCO Register 选择	0	0：不使用 1：使用
X_SEL	X 输入选择	0	0：X 1：XI
XB_SEL	级联 preadder 输入选择	0	2’b00：Zi[47:30] 2’b01：XBI 2’b10：XBI_reg 2’b11：XO
Z_INIT	Z 静态输入	0	用户自定义
USE_ACCLOW	累加器仅使用低 18 位反馈输入选择	0	0：不使用 1：使用
USE_SIMD	模式选择	0	0：模式 18 1：模式 9
CIN_SEL	postadder 进位输入选择	0	0：(MODEZ[3]|MODEY[2]) 1：CIN

控制APM工作模式的为MODEX，MODEY[2:0]，MODEZ[3:0]三组信号，它们的端口说明如下表所示：

表 256 GTP_APM_E1MODEX 列表

信号	说明
MODEX	preadder 加减选择　　0：加　　1：减
MODEZ[3]	对 Zmux 输出取负值，高有效
MODEZ[2:1]	Zmux 输入选择
MODEZ[0]	对上一级级联输出算术右移 18 位，高有效，用于实现大位宽乘法器如 36*36
MODEY[2]	对 Mult 输出取负值，高有效
MODEY[1]	乘法器输出到 Postadder，低有效
MODEY[0]	0：短路乘法器输出

	1：使用累加器反馈

Zmux输入有多种选择，具体由MODEZ[2:1]决定。如下表所示：

表 257 ZMUX 列表

MODEZ[2:1]	说明
00	“ZINIT”[47:0] 累加器初始化常数
01	累加器反馈
10	Z 输入端
11	上一级 APM 结果级联输出

9.4.5. 详细功能说明

9.4.5.1 乘法模式

如下图所示，APM配置成乘法模式时，其等效算术表达式为：

P=X*Y

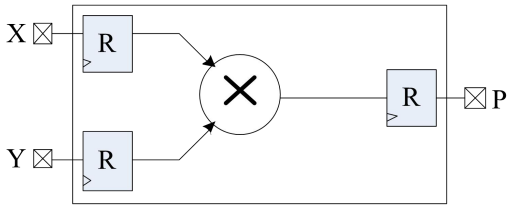


图 112 GTP_APM_E1 乘法模式应用示意图

如下图所示，使能APM中的Preadd Unit后，APM可配置成带预加的乘法模式，其算术表达式为：

P=Y*(X+/-Z)

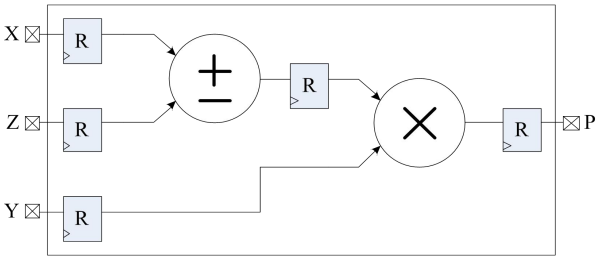


图 113 带预加的乘法模式应用示意图

其波形图如下图所示。

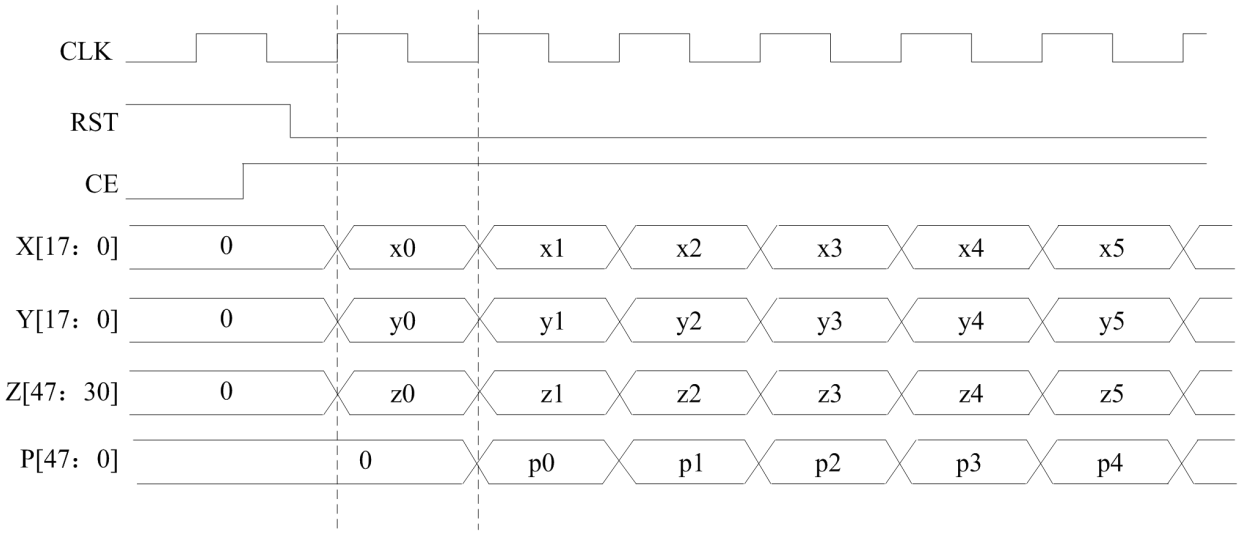


图 114 乘法模式的典型时序图

9.4.5.2 通用乘加模式

如下图所示，APM配置成通用乘加模式时，其算术表达式为：

$$P=X*Y +/- Z$$

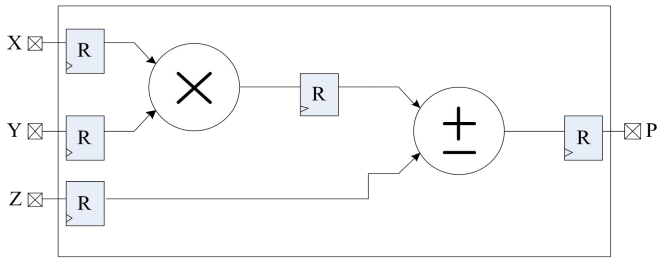


图 115 GTP_APM_E1 乘法模式应用示意图

波形图如下图所示，P在CEP有效时才能输出。

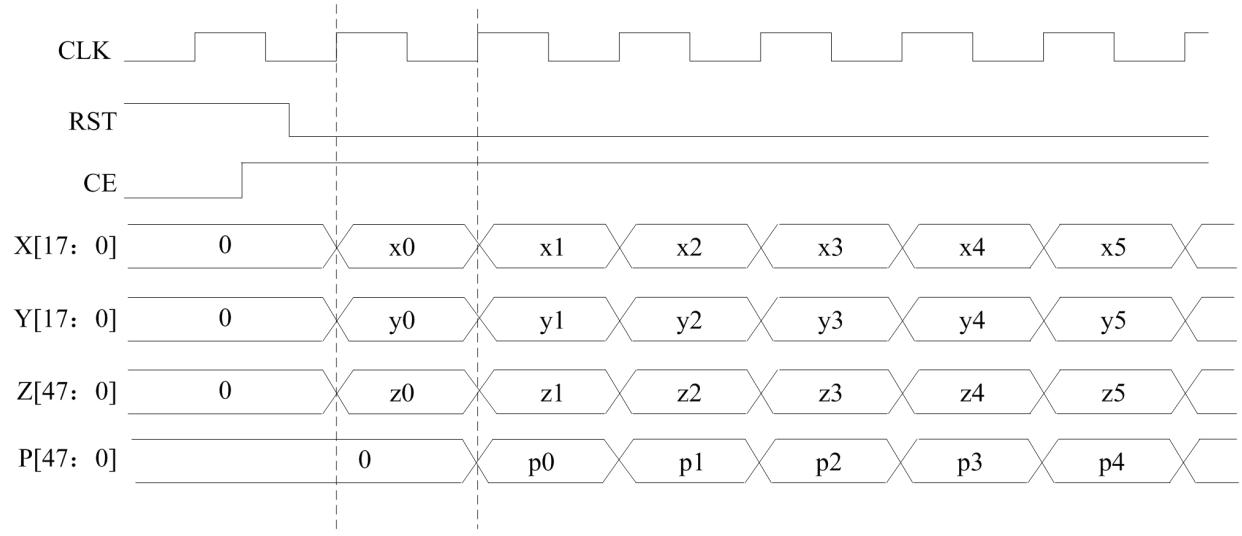


图 116 乘加模式的典型时序图

9.4.5.3 乘累加模式

如下图所示，APM 配置成乘累加模式时，其等效算术表达式为：

P=P +/- X*Y

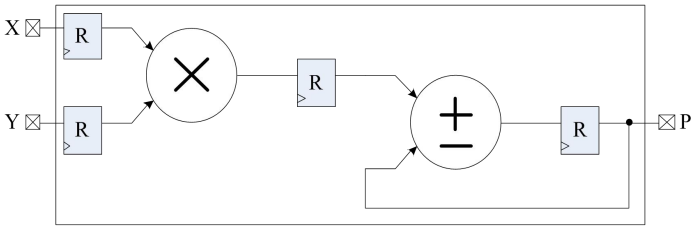


图 117 GTP_APM_E1 乘法模式应用示意图

如下图所示，使能 APM 中的 Preadd Unit 后，APM 可配置成带预加的乘累加模式，其算术表达式为：

P=P+/-Y*(X+/-Z)

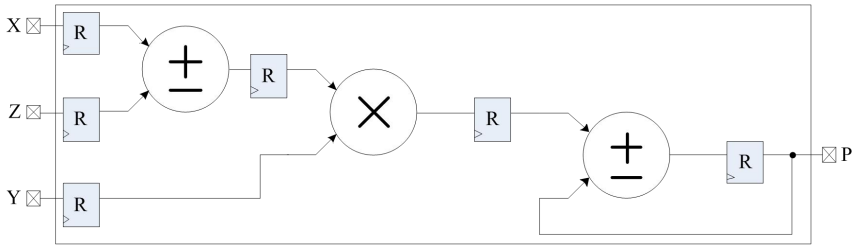


图 118 带预加的乘法模式应用示意图

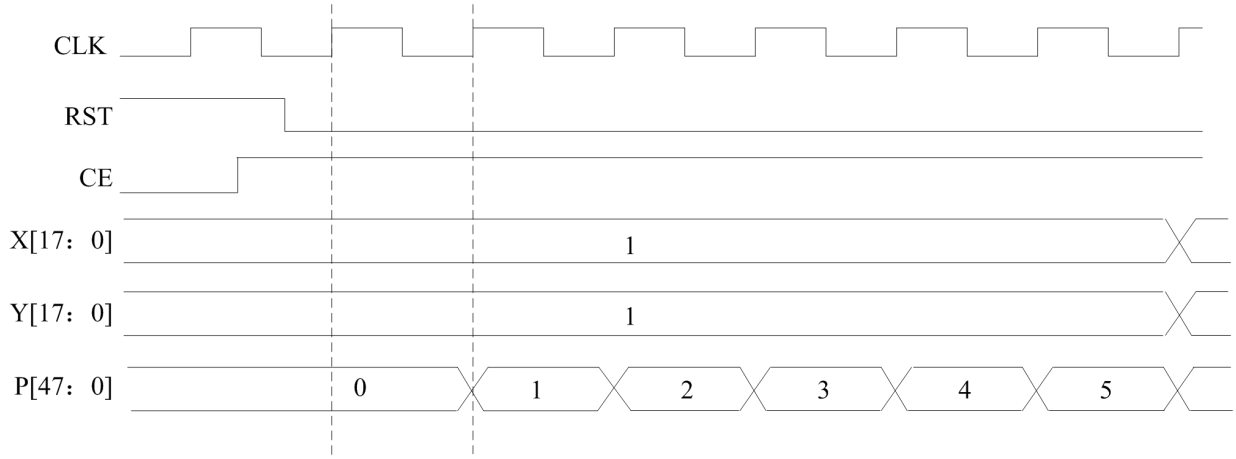


图 119 乘加模式的典型时序图

9.4.5.4 FIR 模式

通过 APM 级联也可实现 FIR 运算，典型的 FIR filter 可以描述为以下表达式：

$$y_n = \sum_{i=0}^{N-1} x_{n-i} h_i = x_n h_0 + x_{n-1} h_1 + \ldots + x_{n-N+1} h_{N-1}$$

x 为输入的数据流，y 为输出的数据流，h 为系数。下图为采用 24 个 APM 级联实现的 24 阶 Systolic FIR 运算的功能实现图。

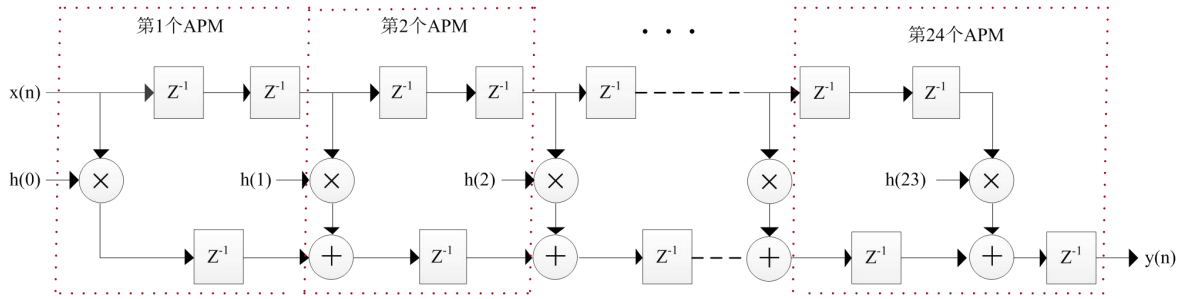


图 120 Systolic FIR 功能示意图

9.4.6. 例化模板

```
GTP_APM_E1  #(
    .GRS_EN                 ( "TRUE"    ),
    . X_SIGNED              (    0      ),
    . Y_SIGNED              (    0      ),
    . USE_POSTADD           (    0      ),
    . USE_PREADD            (    0      ),
    . PREADD_REG            (    0      ),
    . X_REG                 (    0      ),
    . CXO_REG               (    0      ),
    . Y_REG                 (    0      ),
    . Z_REG                 (    0      ),
    . MULT_REG              (    0      ),
    . P_REG                 (    0      ),
    . MODEX_REG             (    0      ),
    . MODEY_REG             (    0      ),
    . MODEZ_REG             (    0      ),
    . X_SEL                 (    0      ),
    . XB_SEL                (    0      ),
    . ASYNC_RST             (    0      ),
    . USE_SIMD              (    0      ),
    . Z_INIT                ( 48'h0000_0000_0000 ),
    . CPO_REG               (    0      ),
    . USE_ACCLOW            (    0      ),
    . CIN_SEL               (    0      )
)
GTP_APM_E1_inst(
    .P                      (    P           ),
    .CPO                    (    CPO         ),
    .COUT                   (    COUT        ),
    .CXO                    (    CXO             ),
    .CXBO                   (    CXBO        ),
    .X                      (    X           ),
    .CXI                    (    CXI         ),
    .CXBI                   (    CXBI        ),
```

```
        .Y              (   Y           ),
        .Z              (   Z           ),
        .CPI            (   CPI         ),
        .CIN            (   CIN         ),
        .MODEX          (   MODEX       ),
        .MODEY          (   MODEY       ),
        .MODEZ          (   MODEZ       ),
        .CLK            (   CLK         ),
        .RSTX           (   RSTX        ),
        .RSTY           (   RSTY        ),
        .RSTZ           (   RSTZ        ),
        .RSTM           (   RSTM        ),
        .RSTP           (   RSTP        ),
        .RSTPRE         (   RSTPRE          ),
        .RSTMODEX       (   RSTMODEX    ),
        .RSTMODEY       (   RSTMODEY    ),
        .RSTMODEZ       (   RSTMODEZ    ),
        .CEX            (   CEX         ),
        .CEY            (   CEY         ),
        .CEZ            (   CEZ         ),
        .CEM            (   CEM         ),
        .CEP            (   CEP         ),
        .CEPRE          (   CEPRE       ),
        .CEMODEX        (   CEMODEX     ),
        .CEMODEY        (   CEMODEY     ),
        .CEMODEZ        (   CEMODEZ     )
    );
```

9.5. GTP_HSST_E2 使用说明

9.5.1. 支持器件

表 258 GTP_HSST_E2 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	不支持	不支持	支持	支持	支持

9.5.2. 功能介绍

GTP_HSST_E2(High Speed Serial Transceiver)高速串行接口模块支持多种串行协议标准。主要特性包括：DataRate 速率可配置；参考时钟可灵活选择；收发通道可独立配置；Programmable Output Swing and De-emphasis；PMA Rx/Tx 支持 SSC；数据通道支持 8bit only，10 bit only，8b10b 8bit，16bit only，20bit only， 8b10b 16bit，32bit only，40bit only 以及 8b10b 32bit 模式；可支持 PCI Express GEN1，PCI Express GEN2，XAUI，千兆以太网，CPRI，SRIO 等协议。详细功

能及使用说明请参考《UG020013_Logos 系列 FPGA 高速串行收发器(HSSTLP)用户指南》。其结构简要框图如下所示：

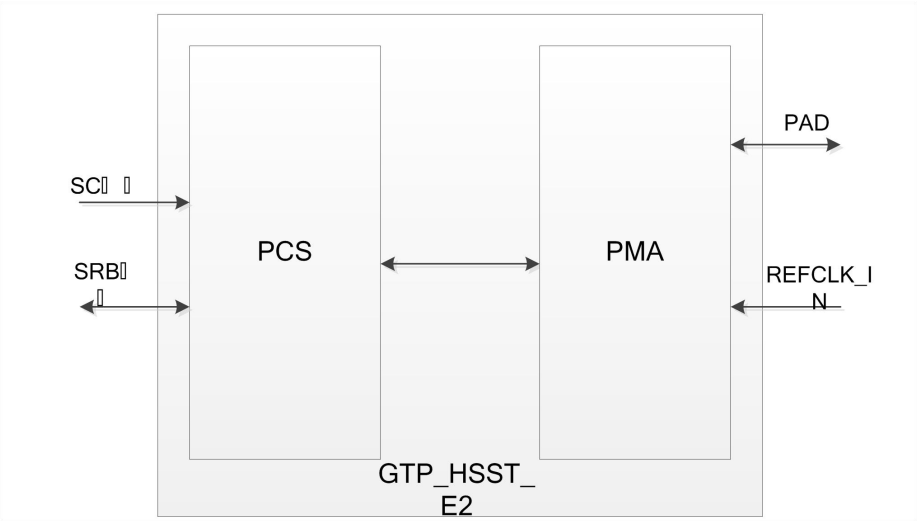


图 121 GTP_HSST_E2 结构框图

9.5.3. 端口描述

表 259 GTP_HSST_E2 端口列表

端口名称	宽度	方向	功能描述
P_REFCLKP_y	1	I	Positive of CML differential reference clock input
P_REFCLKN_y	1	I	Negative of CML differential reference clock input
P_PLL_TEST_y	1	O	Test signal pad for pll debugging
P_RX_SDPx	1	I	Positive of differential serial input(0.622Gb/s – 6.375Gb/s)
P_RX_SDNx	1	I	Negtive of differential serial input(0.622Gb/s – 6.375Gb/s)
P_TX_SDPx	1	O	Positive of differential serial output (0.622Gb/s – 6.375Gb/s)
P_TX_SDNx	1	O	Negative of differential serial output(0.622Gb/s – 6.375Gb/s)
P_RXx_CLK_FR_CORE	1	I	rclk of channel x,from CORE
P_RXx_CLK2_FR_CORE	1	I	rclk2 of channel x,from CORE
P_TXx_CLK_FR_CORE	1	I	tclk of channel x,from CORE
P_TXx_CLK_FR_CORE	1	I	tclk of channel x,from CORE

端口名称	宽度	方向	功能描述
P_HSST_RST	1	I	reset for the whole pcs,from HSST
P_PCS_RX_RST_x	1	I	reset for the rx channel x ,from HSST
P_PCS_TX_RST_x	1	I	reset for the tx channel x ,from HSST
P_PCS_CB_RST_x	1	I	Reset for the pcs_cb channel x
P_RXGEAR_SLIP_x	I	I	rx gear slip for channel x
P_CFG_CLK	1	I	cfg clk ,from CORE
P_CFG_RST	1	I	cfg reset ,from hsst_rst module
P_CFG_PSEL	1	I	Cfg select
P_CFG_ENABLE	1	I	cfg enable,from CORE
P_CFG_WRITE	1	I	cfg write,from CORE
P_CFG_ADDR[15:0]	16	I	cfg address,from CORE
P_CFG_WDATA[7:0]	8	I	cfg wdata,from CORE
P_TDATA_x[44:0]	45	I	transmite data of channel x,from CORE
P_PCS_WORD_ALIGN_EN[3:0]	4	I	align_en from CORE, for word align,used when word align working at outside state machine or CLK SLIP MODE, bit[0] for channel0,etc.
P_RX_POLARITY_INVERT[3:0]	4	I	polarity ctrl of pma_din ,bit[0] for channel 0,etc
P_CEB_ADETECT_EN[3:0]	4	I	channel bonding a detect enable signals, bit[0] for channel 0,etc
P_PCS_MCB_EXT_EN[3:0]	4	I	channel bonding outside state machine enable signals ,bit[0] for channel 0,etc
P_PCS_NEAREND_LOOP[3:0]	4	I	pcs near end loopback ctrl signals,bit[0] for channel 0,etc
P_PCS_FAREND_LOOP[3:0]	4	I	pcs far end loopback ctrl signals,bit[0] for channel 0,etc
P_PMA_NEAREND_PLOOP[3:0]	4	I	pma near end parallel loopback ctrl signals,bit[0] for channel 0,etc
P_PMA_NEAREND_SLOOP[3:0]	4	I	pma near end serial loopback ctrl signals,bit[0] for channel 0,etc

端口名称	宽度	方向	功能描述
P_PMA_FAREND_PLOOP[3:0]	4	I	pma far end parallel loopback ctrl signals,bit[0] for channel 0,etc
P_CFG_READY	1	O	cfg ready signal
P_CFG_RDATA[7:0]	8	O	cfg read data
P_CFG_INT	1	O	cfg int signal
P_PCS_RX_MCB_STATUS[3:0]	4	O	channel bonding state machine synced ,bit[0] for channel 0,etc
P_PCS_LSM_SYNCED[3:0]	4	O	word align state machine synced ,bit[0] for channel 0,etc
P_RDATA_x[46:0]	47	O	receive data to channel x
P_RCLK2FABRIC[x]	1	O	rclk to CORE,channel x
P_TCLK2FABRIC[x]	1	O	tclk to CORE,channel x
P_RESCAL_RST_I	1	I	rescal_rst输入
P_RESCAL_I_CODE_I[5:0]	6	I	rescal_code的输入
P_RESCAL_I_CODE_O[5:0]	6	O	rescal_code的输出
P_REFCK2CORE_y	1	O	reference clk to core, y is 0 or 1
P_PLL_REF_CLK_y	1	I	feed through signals to pll y,y is 0 or 1
P_PLL_RST_y	1	I	feed through signals to pma pll x,high active, y is 0 or 1
P_PLLPOWERDOWN_y	1	I	feed through signals to pma pll y
P_PLL_READY_y	1	O	pma_pll y lock to core
P_LANE_SYNC_y	1	I	Sync toggle signal
P_LANE_SYNC_EN_y	1	I	Sync toggle signal enable
P_RATE_CHANGE_TCLK_ON_y	1	I	rate change, to gate off tclk,for pll y, y is 0 or 1
P_LANE_PD_x	1	I	power down lane
P_LANE_RST_x	1	I	reset lane, high active
P_RX_LANE_PD_x	1	I	rx lane powerdown from core
P_RX_PMA_RST_x	1	I	reset lane rx module, high active
P_CTLE_ADP_RST _x	1	I	Reset for pma_ctle_adp of channel x

端口名称	宽度	方向	功能描述
P_RX_SIGDET_STATUS_x	1	O	rx_sigdet_status to core
P_RX_SATA_COMINIT_x	1	O	rx_sata_cominit to core
P_RX_SATA_COMWAKE_x	1	O	rx_sata_comwake to core
P_RX_LS_DATA_x	1	O	rx low speed data to core
P_RX_READY_x	1	O	receiver ready status,cdr align, to core
P_TEST_STATUS_x[19:0]	20	O	signals for test to core
P_TX_DEEMP_x[1:0]	2	I	transmitter de-emphasis from core
P_TX_LS_DATA_x	1	I	tx low speed data from core
P_TX_BEACON_EN_x	1	I	高有效且EI TX使能时，发送beacon使能信号
P_TX_SWING_x	1	I	tx_swing from core
P_TX_RXDET_REQ_x	1	I	tx_rxdet_req from core
P_TX_RATE_x[2:0]	3	I	tx rate control signal from core
P_TX_BUSWIDTH_x[2:0]	3	I	tx parallel data buswidth control from core
P_TX_MARGIN_x[2:0]	3	I	for pcie mode select transmitter voltage level ,from core
P_TX_RXDET_STATUS_x	1	O	receiver detection status to core
P_TX_PMA_RST_x	1	I	tx mocule reset, high active
P_TX_LANE_PD_x	1	I	tx mocule powerdown
P_RX_RATE_x[2:0]	3	I	rx rate control signal from core
P_RX_BUSWIDTH_x[2:0]	3	I	rx parallel data buswidth control from core
P_RX_HIGHZ_x	1	I	rx termination high z control signal from core
P_CA_ALIGN_RX[3:0]	4	O	clk aligner rx status,for 4 lanes
P_CA_ALIGN_TX[3:0]	4	O	clk aligner tx status,for 4 lanes
P_CIM_CLK_ALIGNER_RXx[7:0]	8	I	clk aligner rx setting step
P_CIM_CLK_ALIGNER_TXx[7:0]	8	I	clk aligner tx setting step
P_CIM_DYN_DLY_SEL_RXx	1	I	clk aligner rx dyn delay select
P_CIM_DYN_DLY_SEL_TXx	1	I	clk aligner tx dyn delay select
P_CIM_START_ALIGN_RXx	1	I	clk aligner rx start, rising edge active
P_CIM_START_ALIGN_TXx	1	I	clk aligner tx start, rising edge active

注：1.以上端口命名为“SIGNAL_NAME_y”的信号，表示每个 PLL 各有一个信号：

SIGNAL_NAME_0：For PLL 0

SIGNAL_NAME_1：For PLL 1

2.以上端口命名为“SIGNAL_NAME_x”的信号，每个 Channel 各有一个信号：

SIGNAL_NAME_0：For Channel 0

SIGNAL_NAME_1：For Channel 1

SIGNAL_NAME_2：For Channel 2

SIGNAL_NAME_3：For Channel 3

9.5.4. 参数描述

表 260 GTP_HSST_E2 参数列表

参数名称	参数类型	有效值	默认值	功能描述
PCS_CHx_BYPASS_WORD_ALIGN	<string>	"FALSE","TRUE"	FALSE	Bypass wordalign
PCS_CHx_BYPASS_DENC	<string>	"FALSE","TRUE"	FALSE	Bypass decoder
PCS_CHx_BYPASS_BONDING	<string>	"FALSE","TRUE"	FALSE	Bypass channel bonding
PCS_CHx_BYPASS_CTC	<string>	"FALSE","TRUE"	FALSE	Bypass ctc
PCS_CHx_BYPASS_GEAR	<string>	"FALSE","TRUE"	FALSE	Bypass rx gear
PCS_CHx_BYPASS_BRIDGE	<string>	"FALSE","TRUE"	FALSE	Bypass rx bridge unit
PCS_CHx_DATA_MODE	<string>	"X8", "X10", "X16", "X20"	X8	PCS Receiver data mode
PCS_CHx_RX_POLARITY_INV	<string>	"DELAY", "BIT_POLARITY_INVERION", "BIT_REVERSAL", "BOTH"	DELAY	极性反转和bit order反转

参数名称	参数类型	有效值	默认值	功能描述
PCS_CHx_ALIGN_MODE	<string>	"1GB", "10GB", "RAPIDIO", "OUTSIDE"	1GB	Word align state machine mode
PCS_CHx_SAMP_16B	<string>	"X16", "X20"	X16	bit order反转时用来区分16比特有效数据和20比特有效数据。 1: 数据为16bit 有效，注意16比特有效又分为16bonly和8bonly两种情况； 0: 数据为20bit 有效。
PCS_CHx_FARLP_PWR_REDUCTION	<string>	"FALSE","TRUE"	FALSE	高有效关闭far loop功能，降低power 1'b0: 对应于Parameter参数值"FALSE" 1'b1: 对应于Parameter参数值"TRUE"
PCS_CHx_COMMA_REG0	<interger>	0 to 1023	0	Word Alignment Comma字节定义0
PCS_CHx_COMMA_MASK	<interger>	0 to 1023	0	Word Alignment Comma Mask bit定义
PCS_CHx_CEB_MODE	<string>	"10GB", "RAPIDIO", "OUTSIDE"	10GB	Channel bomding state machine mode
PCS_CHx_CTC_MODE	<string>	"1SKIP", "2SKIP", "PCIE 2byte", "4SKIP"	"1SKIP"	CTC模式设置
PCS_CHx_A_REG	<interger>	0 to 255	0	Channel Bonding 使用的Align Pattern定义
PCS_CHx_GE_AUTO_EN	<string>	"FALSE","TRUE"	FALSE	高有效使能基于1 Gig Ethernet的/C/到/I2/的自动替换
PCS_CHx_SKIP_REG0	<interger>	0 to 1023	0	CTC使用的SKIP字符Byte 0
PCS_CHx_SKIP_REG1	<interger>	0 to 1023	0	CTC使用的SKIP字符Byte 1

参数名称	参数类型	有效值	默认值	功能描述
PCS_CHx_SKIP_REG2	<interger>	0 to 1023	0	CTC使用的SKIP字符Byte 2
PCS_CHx_SKIP_REG3	<interger>	0 to 1023	0	CTC使用的SKIP字符Byte 3
PCS_CHx_DEC_DUAL	<string>	"FALSE","TRUE"	FALSE	dual decoder or single decoder
PCS_CHx_SPLIT	<string>	"FALSE","TRUE"	FALSE	plit/union mode选择
PCS_CHx_FIFOFLAG_CTC	<string>	"FALSE","TRUE"	FALSE	Rx bridge unit中status优先级设置
PCS_CHx_COMMA_DET_MODE	<string>	"COMMA_PATTERN"，"RX_CLK_SLIP"	COMMA_PATTERN	Word Alignment模块中Alignment模式
PCS_CHx_ERRDETECT_SILENCE	<string>	"FALSE","TRUE"	FALSE	TRUE, k码解码出错将会强制覆盖，不会报错。 FALSE, k码解码出错将报错。
PCS_CHx_PMA_RCLK_POLINV	<string>	"PMA_RCLK"，"REVERSE_OF_PMA_RCLK"	PMA_RCLK	是否取反时钟
PCS_CHx_PCS_RCLK_SEL	<string>	"PMA_RCLK"，"PMA_TCLK" "MCB_RCLK"，"RCLK"	PMA_RCLK	PCS_RCLK选择
PCS_CHx_CB_RCLK_SEL	<string>	"PMA_RCLK"，"PMA_TCLK" "MCB_RCLK"，"RCLK"	PMA_RCLK	CB_RCLK选择
PCS_CHx_AFTER_CTC_RCLK_SEL_y	<string>	"PMA_RCLK"，"PMA_TCLK" "MCB_RCLK"，"RCLK2"	PMA_RCLK	AFTER_CTC_RCLK时钟源选择 2'b00:对应于Parameter参数值"PMA_RCLK"； 2'b01:对应于Parameter参数值"PMA_TCLK"；

参数名称	参数类型	有效值	默认值	功能描述
				2’b10:对应于Parameter参数值"MCB_RCLK"； 2’b11:对应于Parameter参数值"RCLK2"
PCS_CHx_RCLK_POLINV	<string>	"RCLK"，"REVERSE_OF_RCLK"	RCLK	是否取反时钟
PCS_CHx_BRIDGE_RCLK_SEL	<string>	"PMA_RCLK"，"PMA_TCLK"，"MCB_RCLK"，"RCLK"	PMA_RCLK	BRIDGE_RCLK选择
PCS_CHx_PCS_RCLK_EN	<string>	"FALSE","TRUE"	FALSE	PCS_RCLK enable
PCS_CHx_CB_RCLK_EN	<string>	"FALSE","TRUE"	FALSE	CB_RCLK enable
PCS_CHx_AFTER_CTC_RCLK_EN	<string>	"FALSE","TRUE"	FALSE	AFTER_CTC_RCLK enable
PCS_CHx_AFTER_CTC_RCLK_EN_GB_y	<string>	"FALSE","TRUE"	FALSE	PCS Receiver模块数据位宽选择之一 1'b0:输出after_ctc_rclkfabric； 1'b1: after_ctc_rclkfabric进行2分频输出；
PCS_CHx_PCS_RX_RSTN	<string>	"FALSE","TRUE"	FALSE	PCS_RX rstn
PCS_CHx_SLAVE	<string>	"MASTER"，"SLAVE"	MASTER	channel bonding设置
PCS_CHx_PCIE_SLAVE	<string>	"MASTER"，"SLAVE"	MASTER	基于PCI Express的channel bonding设置
PCS_CHx_RX_64B66B_67B	<string>	"NORMAL"，"64B_66B"	NORMAL	PCS Receiver模块数据位宽选择之一

参数名称	参数类型	有效值	默认值	功能描述
		"RESERVED" "2"		1'b0:输出after_ctc_rclkfabric； 1'b1: after_ctc_rclkfabric进行2分频输出；
PCS_CHx_RX_BRIDGE_CLK_POLINV	<string>	"RX_BRIDGE_CLK"， "REVERSE OF RX_BRIDGE_CLK"	RX_BRIDGE_CLK	是否取反时钟 1'b0: 对应于Parameter参数值"RX_BRIDGE_CLK"； 1'b1: reserved
PCS_CHx_PCS_CB_RSTN	<string>	"FALSE","TRUE"	FALSE	PCS CB复位寄存器，即cfg_pcs_cb_rstn,低有效. 1'b0: 对应于Parameter参数值"FALSE" 1'b1:对应于Parameter参数值"TRUE"；
PCS_CHx_TX_BRIDGE_GEAR_SEL	<string>	"FALSE","TRUE"	FALSE	PCS_TX_BRIDGE_GEAR选择 1’b0: 对应于Parameter参数值"FALSE"； 1’b1: 对应于Parameter参数值"TRUE"；
PCS_CHx_TX_BYPASS_BRIDGE_UINT	<string>	"FALSE","TRUE"	FALSE	Bypass tx bridge unit
PCS_CH0_TX_BYPASS_BRIDGE_FIFO	<string>	"FALSE","TRUE"	FALSE	Bypass tx_bridge_fifo
PCS_CHx_TX_BYPASS_GEAR	<string>	"FALSE","TRUE"	FALSE	Bypass tx gear
PCS_CHx_TX_BYPASS_ENC	<string>	"FALSE","TRUE"	FALSE	Bypass tx encoder
PCS_CHx_TX_BYPASS_BIT_SLIP	<string>	"FALSE","TRUE"	FALSE	Bypass tx bitslip
PCS_CHx_TX_GEAR_SPLIT	<string>	"FALSE","TRUE"	FALSE	Tx slit mode

参数名称	参数类型	有效值	默认值	功能描述
PCS_CHx_TX_DRIVE_REG_MODE	<string>	"NO_CHANGE", "EN_POLARIY_REV", "EN_BIT_REV", "EN_BOTH"	NO_CHANGE	Tx drive设置
PCS_CHx_TX_BIT_SLIP_CYCLES	<interger>	0 to 31	0	Tx bitslip数值设置
PCS_CHx_INT_TX_MASK_0	<string>	"FALSE","TRUE"	FALSE	Tx中断是否mask bit0
PCS_CHx_INT_TX_MASK_1	<string>	"FALSE","TRUE"	FALSE	Tx中断是否mask bit1
PCS_CHx_INT_TX_MASK_2	<string>	"FALSE","TRUE"	FALSE	Tx中断是否mask bit2
PCS_CHx_INT_TX_CLR_0	<string>	"FALSE","TRUE"	FALSE	清除发送通道中断状态寄存器bit0
PCS_CHx_INT_TX_CLR_1	<string>	"FALSE","TRUE"	FALSE	清除发送通道中断状态寄存器bit1
PCS_CHx_INT_TX_CLR_2	<string>	"FALSE","TRUE"	FALSE	清除发送通道中断状态寄存器bit2
PCS_CHx_TX_PMA_TCLK_POLINV	<string>	"PMA_TCLK"，"REVERSE_OF_PMA_TCLK"	PMA_TCLK	是否取反时钟
PCS_CHx_TX_PCS_CLK_EN_SEL	<string>	"FALSE","TRUE"	FALSE	TX_PCS_CLK enable
PCS_CHx_TX_BRIDGE_TCLK_SEL	<string>	" TCLK2"，"TCLK"	TCLK	TX_BRIDGE_TCLK选择
PCS_CHx_TX_TCLK_POLINV	<string>	"TCLK"，"REVERSE_OF_TCLK"	TCLK	是否取反时钟
PCS_CHx_TX_PCS_TCL	<string>	"TCLK"，	PMA_TC	时钟选择

参数名称	参数类型	有效值	默认值	功能描述
K_SEL		"PMA_TCLK"	LK	1'b0: pma_tclk 1'b1:tclk
PCS_CHx_TX_PCS_TX_RSTN	<string>	"FALSE","TRUE"	FALSE	PCS_tx rstn
PCS_CHx_TX_SLAVE	<string>	"MASTER"， "SLAVE"	MASTER	channel bonding设置
PCS_CHx_TX_GEAR_TCLK_EN_SEL	<string>	"FALSE","TRUE"	FALSE	时钟选择
PCS_CHx_DATA_WIDTH_MODE	<string>	"X20","X16", "X10", "X8"	X20	tx_drive_reg datawidth mode
PCS_CHx_TX_64B66B_67B	<string>	"NORMAL"， "64B_66B" "RESERVED" "2"	NORMAL	TX 64B66B_67B选择 2’b00: 对应于Parameter参数值 "NORMAL"； 2’b01: 对应于Parameter参数值 "64B_66B"； 2’b11: 对应于Parameter参数值 "64B_67B"；
PCS_CHx_TX_GEAR_TCLK_SEL	<string>	"TCLK2"， "PMA_TCLK"	MA_TCLK	时钟选择 1'b0: pma_tclk 1'b1:tclk2
PCS_CHx_TX_TCLK2FABRIC_SEL	<string>	"FALSE","TRUE"	FALSE	CLK2FABRIC选择
PCS_CHx_TX_OUTZZ	<string>	"FALSE","TRUE"	FALSE	内部数据格式，固定设置为 FALSE
PCS_CHx_ENC_DUAL	<string>	"FALSE","TRUE"	FALSE	dual encoder or single encoder
PCS_CHx_TX_BITSLIP_DATA_MODE	<string>	"X10","X20"	X10	Tx BITSLIP中的数据位宽模式
PCS_CHx_TX_BRIDGE_CLK_POLINV	<string>	"TX_BRIDGE_CLK" ,"REVERSE OF TX_BRIDGE_CLK"	TX_BRIDGE_CLK	TX_BRIDGE_CLK时钟选择 1’b0:对应于Parameter参数值 "TX_BRIDGE_CLK"；

参数名称	参数类型	有效值	默认值	功能描述
				1’b1:对应于Parameter参数值"REVERSE_OF_TX_BRIDGE_CLK"；
PCS_CHx_COMMA_REG1	<interger>	0 to 1023	0	Word Alignment Comma字节定义1
PCS_CHx_RAPID_IMAX	<interger>	0 to 7	0	Rapid IO Link State Machine 锁定状态的检测字节数
PCS_CHx_RAPID_VMIN_1	<interger>	0 to255	0	Rapid IO Link State Machine 退出状态的检测字节数，低8 bits
PCS_CHx_RAPID_VMIN_2	<interger>	0 to255	0	Rapid IO Link State Machine 退出状态的检测字节数，高8 bits
PCS_CHx_RX_PRBS_MODE	<string>	"DISABLE", "PRBS_7", "PRBS_15", "PRBS_23", "PRBS_31", "LONG_1", "LONG_0", "20UI", "D10_2", "PCIE"	DISABLE	接收侧PRBS Checker模式选择
PCS_CHx_RX_ERRCNT_CLR	<string>	"FALSE","TRUE"	FALSE	高有效清除PRBS checker中的错误计数
PCS_CH0_RX_PRBS_ERR_LPBK	<string>	"FALSE","TRUE"	FALSE	PRBS checker错误计数返回
PCS_CHx_TX_PRBS_MODE	<string>	"DISABLE", "PRBS_7", "PRBS_15", "PRBS_23", "PRBS_31",	DISABLE	发送侧PRBS模式选择

参数名称	参数类型	有效值	默认值	功能描述
		"LONG_1", "LONG_0", "20UI", "D10_2", "PCIE"		
PCS_CHx_TX_INSERT_ER	<string>	"FALSE","TRUE"	FALSE	由0~1的高脉冲可以在PRBS产生时插入错误码
PCS_CHx_ENABLE_PRBS_GEN	<string>	"FALSE","TRUE"	FALSE	PRBS 产生使能，高有效
PCS_CHx_ERR_CNT	<interger>	0 to255	0	PRBS checker中的错误计数
PCS_CHx_DEFAULT_RADDR	<interger>	0 to15	0	Channel bonding默认检测地址
PCS_CHx_MASTER_CHECK_OFFSET	<interger>	0 to15	0	主通道检测地址相对于通道默认检测地址的偏移值
PCS_CHx_DELAY_SET	<interger>	0 to15	0	通道延迟的拍数配置
PCS_CHx_SEACH_OFFSET	<string>	"20BIT"， "30BIT"， "40BIT"， "50BIT"， "60BIT"， "70BIT"	20BIT	Channel Bonding范围设置
PCS_CHx_CEB_RAPIDLS_MMAX	<interger>	0 to7	0	Channel Bonding RapidIO状态机使用的MMAX值
PCS_CHx_CTC_AFULL	<interger>	0 to31	0	CTC FIFO的Almost Full门限，默认需设置为5'd20
PCS_CHx_CTC_AEMPTY	<interger>	0 to31	0	CTC FIFO的Almost Empty门限，默认需设置为5'd12
PCS_CHx_CTC_CONTI_SKP_SET	<interger>	0 to1	0	Ctc连续add/delete设置
PCS_CHx_FAR_LOOP	<string>	"FALSE","TRUE"	FALSE	PCS远端环回使能，高有效

参数名称	参数类型	有效值	默认值	功能描述
PCS_CHx_NEAR_LOOP	<string>	"FALSE","TRUE"	FALSE	PCS近端环回使能，高有效
PCS_CHx_REG_TX2RX_PLOOP_EN	<string>	"FALSE","TRUE"	FALSE	PMA近端并行环回使能，高有效 1'b0：对应于Parameter参数值"FALSE "； 1'b1：对应于Parameter参数值"TRUE "
PCS_CHx_REG_TX2RX_SLOOP_EN	<string>	"FALSE","TRUE"	FALSE	PMA近端串行环回使能，高有效 1'b0：对应于Parameter参数值"FALSE "； 1'b1：对应于Parameter参数值"TRUE "
PCS_CHx_REG_RX2TX_PLOOP_EN	<string>	"FALSE","TRUE"	FALSE	PMA远端并行环回使能，高有效 1'b0：对应于Parameter参数值"FALSE "； 1'b1：对应于Parameter参数值"TRUE "
PCS_CHx_INT_RX_MASK_0	<string>	"FALSE","TRUE"	FALSE	接收通道中断状态寄存器bit0的掩码，高有效
PCS_CHx_INT_RX_MASK_1	<string>	"FALSE","TRUE"	FALSE	接收通道中断状态寄存器bit1的掩码，高有效
PCS_CHx_INT_RX_MASK_2	<string>	"FALSE","TRUE"	FALSE	接收通道中断状态寄存器bit2的掩码，高有效
PCS_CHx_INT_RX_MASK_3	<string>	"FALSE","TRUE"	FALSE	接收通道中断状态寄存器bit3的掩码，高有效
PCS_CHx_INT_RX_MASK_4	<string>	"FALSE","TRUE"	FALSE	接收通道中断状态寄存器bit4的掩码，高有效
PCS_CHx_INT_RX_MAS	<string>	"FALSE","TRUE"	FALSE	接收通道中断状态寄存器bit5的

参数名称	参数类型	有效值	默认值	功能描述
K_5				掩码，高有效
PCS_CHx_INT_RX_MASK_6	<string>	"FALSE","TRUE"	FALSE	接收通道中断状态寄存器bit6的掩码，高有效
PCS_CHx_INT_RX_MASK_7	<string>	"FALSE","TRUE"	FALSE	接收通道中断状态寄存器bit7的掩码，高有效
PCS_CHx_INT_RX_CLR_0	<string>	"FALSE","TRUE"	FALSE	清除接收通道中断状态寄存器bit0，高有效
PCS_CHx_INT_RX_CLR_1	<string>	"FALSE","TRUE"	FALSE	清除接收通道中断状态寄存器bit1，高有效
PCS_CHx_INT_RX_CLR_2	<string>	"FALSE","TRUE"	FALSE	清除接收通道中断状态寄存器bit2，高有效
PCS_CHx_INT_RX_CLR_3	<string>	"FALSE","TRUE"	FALSE	清除接收通道中断状态寄存器bit3，高有效
PCS_CHx_INT_RX_CLR_4	<string>	"FALSE","TRUE"	FALSE	清除接收通道中断状态寄存器bit4，高有效
PCS_CHx_INT_RX_CLR_5	<string>	"FALSE","TRUE"	FALSE	清除接收通道中断状态寄存器bit5，高有效
PCS_CHx_INT_RX_CLR_6	<string>	"FALSE","TRUE"	FALSE	清除接收通道中断状态寄存器bit6，高有效
PCS_CHx_INT_RX_CLR_7	<string>	"FALSE","TRUE"	FALSE	清除接收通道中断状态寄存器bit7，高有效
PCS_CHx_CA_RX	<interger>	0 to 255	0	Clk aligner Rx step setting
PCS_CHx_CA_TX	<interger>	0 to 255	0	Clk aligner Tx step setting
PCS_CHx_CA_DYN_DLY_EN_RX	<string>	"FALSE","TRUE"	FALSE	设为"FALSE"时，强制对应接收通道的CLK Aligner delay step为0； "TRUE"时，CLK Aligner delay step由PCS_CHx_CA_RX或P_CIM_CLK_ALIGNER_RXx[7:0]确定

参数名称	参数类型	有效值	默认值	功能描述
PCS_CHx_CA_DYN_DLY_EN_TX	<string>	"FALSE","TRUE"	FALSE	设为"FALSE"时，强制对应发送通道的CLK Aligner delay step为0； "TRUE"时，CLK Aligner delay step由PCS_CHx_CA_TXx或P_CIM_CLK_ALIGNER_TXx[7:0]确定
PCS_CHx_CA_DYN_DLY_SEL_RX	<string>	"FALSE","TRUE"	FALSE	设为"FALSE"时，关闭对应接收通道的CLK Aligner功能 设为"TRUE"时，使能对应接收通道的CLK Aligner功能
PCS_CHx_CA_DYN_DLY_SEL_TX	<string>	"FALSE","TRUE"	FALSE	设为"FALSE"时，关闭对应发送通道的CLK Aligner功能 设为"TRUE"时，使能对应发送通道的CLK Aligner功能
PCS_CHx_CA_RSTN_RX	<string>	"FALSE","TRUE"	FALSE	低有效复位对应接收通道的CLK Aligner内部逻辑
PCS_CHx_CA_RSTN_TX	<string>	"FALSE","TRUE"	FALSE	低有效复位对应发送通道的CLK Aligner内部逻辑
PCS_CHx_RXPRBS_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭RX PRBS功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值"POWER_REDUCTION"
PCS_CHx_WDALIGN_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭RX wordalign功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值

参数名称	参数类型	有效值	默认值	功能描述
				"POWER_REDUCTION"
PCS_CHx_RXDEC_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭RX decoder功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值"POWER_REDUCTION"
PCS_CHx_RXCB_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭RX channel bonding功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值"POWER_REDUCTION"
PCS_CHx_RXCTC_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭RX CTC功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值"POWER_REDUCTION"
PCS_CHx_RXGEAR_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭RX GEAR功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值"POWER_REDUCTION"
PCS_CHx_RXBRG_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭RX bridge功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值

参数名称	参数类型	有效值	默认值	功能描述
				"POWER_REDUCTION"
PCS_CHx_RXTEST_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭RX test_status功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值"POWER_REDUCTION"
PCS_CHx_TXBRG_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭TX bridge功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值"POWER_REDUCTION"
PCS_CHx_TXGEAR_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭TX GEAR功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值"POWER_REDUCTION"
PCS_CHx_TXENC_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭TX encoder功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值"POWER_REDUCTION"
PCS_CHx_TXBSLP_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭TX bitslip功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值

参数名称	参数类型	有效值	默认值	功能描述
				"POWER_REDUCTION"
PCS_CHx_TXPRBS_PWR_REDUCTION	<string>	"NORMAL","OWER_REDUCTION"	NORMAL	高有效关闭TX PRBS功能，降低power 1'b0：对应于Parameter参数值"NORMAL", 1'b1：对应于Parameter参数值"POWER_REDUCTION"
PCS_CHx_TXBRG_FULL_CHK_EN	<string>	"FALSE","TRUE"	FALSE	tx bridge unit fifo写满检测使能。 1'b0：对应于Parameter参数值"FALSE "； 1'b1：对应于Parameter参数值"TRUE ".
PCS_CHx_TXBRG_EMPTY_CHK_EN	<string>	"FALSE","TRUE"	FALSE	tx bridge unit fifo读空检测使能。 1'b0：对应于Parameter参数值"FALSE "； 1'b1：对应于Parameter参数值"TRUE ".
PCS_CHx_RXBRG_FULL_CHK_EN	<string>	"FALSE","TRUE"	FALSE	rx bridge unit fifo写满检测使能。 1'b0：对应于Parameter参数值"FALSE "； 1'b1：对应于Parameter参数值"TRUE ".
PCS_CHx_RXBRG_EMPTY_CHK_EN	<string>	"FALSE","TRUE"	FALSE	rx bridge unit fifo读空检测使能。 1'b0：对应于Parameter参数值"FALSE "； 1'b1：对应于Parameter参数值"TRUE ".
PCS_CHx_CTC_FULL_C	<string>	"FALSE","TRUE"	TRUE	rx ctc fifo写满检测使能。

参数名称	参数类型	有效值	默认值	功能描述
HK_EN				1'b0: 对应于Parameter参数值 "FALSE "； 1'b1: 对应于Parameter参数值 "TRUE ".
PCS_CHx_CTC_EMPTY_CHK_EN	<string>	"FALSE","TRUE"	TRUE	rx ctc fifo读空检测使能。 1'b0: 对应于Parameter参数值 "FALSE "； 1'b1: 对应于Parameter参数值 "TRUE ".
PCS_CHx_CEB_FULL_CHK_EN	<string>	"FALSE","TRUE"	FALSE	rx channel bonding fifo写满检测使能。 1'b0: 对应于Parameter参数值 "FALSE "； 1'b1: 对应于Parameter参数值 "TRUE ".
PCS_CHx_CEB_EMPTY_CHK_EN	<string>	"FALSE","TRUE"	FALSE	rx channel bonding fifo读空检测使能。 1'b0: 对应于Parameter参数值 "FALSE "； 1'b1: 对应于Parameter参数值 "TRUE ".
PCS_CHx_FLP_FULL_CHK_EN	<string>	"FALSE","TRUE"	TRUE	far loop fifo写满检测使能。 1'b0: 对应于Parameter参数值 "FALSE "； 1'b1: 对应于Parameter参数值 "TRUE ".
PCS_CHx_FLP_EMPTY_CHK_EN	<string>	"FALSE","TRUE"	TRUE	far loop fifo读空检测使能。 1'b0: 对应于Parameter参数值 "FALSE "； 1'b1: 对应于Parameter参数值

参数名称	参数类型	有效值	默认值	功能描述
				"TRUE ".
PMA_CHx_REG_RX_PD	<string>	"ON","OFF"	ON	receiver power down 1'b0: 对应于参数值"ON"， receiver is fully power on； 1'b1: 对应于参数值"OFF" receiver is fully power off。
PMA_CHx_REG_RX_PD_EN	<string>	"FALSE","TRUE"	FALSE	enable rx power down control from register 1'b0: 对应于参数值" DISABLE "， rx power down controlled by pma_rx_pd_x ； 1'b1: 对应于参数值" ENABLE" rx power down controlled by reg_rx_pd 。
PMA_CHx_REG_RX_CLKPATH_PD	<string>	"ON","OFF"	ON	1'b0: 对应于参数值"ON"， receiver datapath is power on； 1'b1: 对应于参数值"OFF" receiver datapath (preamp,slicer) is power off。
PMA_CHx_REG_RX_CLKPATH_PD_EN	<string>	"FALSE","TRUE"	FALSE	PLL reset control 1'b0: 对应于参数值" DISABLE "， clock path power down controlled by pma_rx_pd_x； 1'b1: 对应于参数值" ENABLE"， clock path power down controlled by

参数名称	参数类型	有效值	默认值	功能描述
				reg_rx_clkpath_pd 。
PMA_CHx_REG_RX_DATAPATH_PD	<string>	"ON","OFF"	ON	1'b0: 对应于参数值"ON"，receiver datapath is power on； 1'b1: 对应于参数值"OFF" receiver datapath (preamp,slicer) is power off。
PMA_CHx_REG_RX_DATAPATH_PD_EN	<string>	"FALSE","TRUE"	FALSE	enable the data path power down control from register 1'b0: 对应于参数值" DISABLE "， data path power down controlled by pma_rx_pd_x ； 1'b1: 对应于参数值" ENABLE" data path power down controlled by reg_rx_datapath_pd 。
PMA_CHx_REG_RX_SIGDET_PD	<string>	"ON","OFF"	ON	1'b0: 对应于参数值"ON"，receiver signal detect is power on； 1'b1: 对应于参数值"OFF" receiver signal detect is power off。
PMA_CHx_REG_RX_SIGDET_PD_EN	<string>	"FALSE","TRUE"	FALSE	enable rx signal detect power down control from register 1'b0: 对应于参数值" DISABLE "， rx signal detect power down controlled by pma_rx_pd_x ； 1'b1: 对应于参数值" ENABLE" rx signal detect power down

参数名称	参数类型	有效值	默认值	功能描述
				controlled by reg_rx_sigdet_pd
PMA_CHx_REG_RX_DCC_RST_N	<string>	"FALSE","TRUE"	TRUE	RX DCC reset register, low valid 1'b0: 对应于参数值"FALSE"， reset rx duty cycle correction； 1'b1: 对应于参数值"TRUE" normal operation of rx DCC。
PMA_CHx_REG_RX_DCC_RST_N_EN	<string>	"FALSE","TRUE"	FALSE	enable the DCC reset control from register 1'b0: 对应于参数 值"DISABLE"； cdr reset controlled by pma_rx_reset_n_x 1'b1: 对应于参数值"ENABLE" cdr reset controlled by reg_rx_dcc_rst_n。
PMA_CHx_REG_RX_CDR_RST_N	<string>	"FALSE","TRUE"	TRUE	CDR reset register, low valid 1'b0: 对应于参数值"FALSE"， reset CDR； 1'b1: 对应于参数值"TRUE"， normal operation of CDR。
PMA_CHx_REG_RX_CDR_RST_N_EN	<string>	"FALSE","TRUE"	FALSE	enable the DCC reset control from register 1'b0: 对应于参数 值"DISABLE"； cdr reset controlled by pma_rx_reset_n_x 1'b1: 对应于参数值"ENABLE" cdr reset controlled by reg_rx_dcc_rst_n。
PMA_CHx_REG_RX_SI	<string>	"FALSE","TRUE"	TRUE	rx signal detect reset register, low

参数名称	参数类型	有效值	默认值	功能描述
GDET_RST_N				valid 1'b0: 对应于参数值"FALSE"， reset sigdet, and the output rx_sigdet_status is 0； 1'b1: 对应于参数值"TRUE" sigdet reset controlled by reg_rx_sigdet_rst_n。
PMA_CHx_REG_RX_SIGDET_RST_N_EN	<string>	"FALSE","TRUE"	FALSE	Register value to overwrite rescal_done output 1'b0: 对应于参数值 "DISABLE"， resistor calibration is not done； 1'b1: 对应于参数值 "ENABLE"， resistor calibration is done。
PMA_CHx_REG_RXPCLK_SLIP	<string>	"FALSE","TRUE"	FALSE	rxpclk delay by 1 UI if reg_rxpclk_slip changes from 0 to 1 1'b0: 对应于参数值"FALSE"， 1'b1: 对应于参数值"TRUE"
PMA_CHx_REG_RXPCLK_SLIP_OW	<string>	"DISABLE","ENABLE"	DISABLE	enable rxpclk slip control from register 1'b0: 对应于参数值 "DISABLE"， rxpclk_slip controlled by pma_rxpclk_slip_x 1'b1: 对应于参数值"ENABLE" rxpclk_slip controlled by reg_rxpclk_slip_ow
PMA_CHx_REG_RX_PC	<string>	"FALSE","TRUE"	TRUE	enable the rx pclk switch reset

参数名称	参数类型	有效值	默认值	功能描述
LKSWITCH_RST_N				control from register 1'b0: 对应于参数值"DISABLE"， rx pclk switch block controlled by pma_rx_reset_n_x； 1'b1: 对应于参数值"ENABLE" rx pclk swithc controlled by reg_rx_pclkswitch_rst_n。
PMA_CHx_REG_RX_PCLKSWITCH_RST_N_EN	<string>	"FALSE","TRUE"	FALSE	enable the rx pclk switch reset control from register 1'b0: 对应于参数值"DISABLE"， rx pclk switch block controlled by pma_rx_reset_n_x； 1'b1: 对应于参数值"ENABLE" rx pclk swithc controlled by reg_rx_pclkswitch_rst_n。
PMA_CHx_REG_RX_PCLKSWITCH	<string>	"FALSE","TRUE"	FALSE	rx pclk switch control register 1'b0: 对应于参数值"FALSE"， refclk is selected as rxpclk； 1'b1: 对应于参数值"TRUE" cdr recovered pclk is selected as rxpclk。
PMA_CHx_REG_RX_PCLKSWITCH_EN	<string>	"FALSE","TRUE"	FALSE	enable the tx pclk switch control from register 1'b0: 对应于参数值"DISABLE"， rx pclk switch controlled by pll_ready； 1'b1: 对应于参数值"ENABLE"

参数名称	参数类型	有效值	默认值	功能描述
				rx pclk switch controlled by reg_rx_pclkswitch。
PMA_CHx_REG_RX_HIGHZ	<string>	"FALSE","TRUE"	FALSE	receiver termination res high z control register 1'b0: 对应于参数值"FALSE"，receiver in 50-ohm low-impedance mode； 1'b1: 对应于参数值"TRUE" receiver in highZ mode。
PMA_CHx_REG_RX_HIGHZ_EN	<string>	"FALSE","TRUE"	FALSE	enable the Rx highZ control from register 1'b0: 对应于参数值"DISABLE"，Rx highZ controlled by pma_rx_highz_x； 1'b1: 对应于参数值"ENABLE" Rx highZ controlled by rx_highz。
PMA_CHx_REG_RX_EQ_C_SET	<interger>	0 to 15	8	receiver EQ setting control for cap 0000~1111, default setting is 1000
PMA_CHx_REG_RX_EQ_R_SET	<interger>	0 to 15	8	receiver EQ setting control for resistor 0000~1111, default setting is 1000
PMA_CHx_REG_RX_BUSWIDTH	<string>	"8BIT","10BIT","16BIT","20BIT"	"20BIT"	Rx databus width: 可设置： 0："8BIT"； 1："10BIT"；

参数名称	参数类型	有效值	默认值	功能描述
				2："16BIT"； 3："20BIT" bit[2] is reserved.
PMA_CHx_REG_RX_BUSWIDTH_EN	<string>	"FALSE","TRUE"	FALSE	enable the Rx datawidth control from register 1'b0: 对应于参数值"DISABLE"， Rx rate controlled by pma_rx_buswidth_x[2:0] 1'b1: 对应于参数值"ENABLE"： Rx rate width controlled by reg_rx_width[2:0]
PMA_CHx_REG_RX_RATE	<string>	"DIV8","DIV4","DIV2","DIV1"	"DIV1"	Rx rate control register: 可设置： 0："DIV8",highest datarate /8； 1："DIV4",highest datarate /4； 2："DIV2",highest datarate /2； 3："DIV1",highest datarate bit[2] is reserved.
PMA_CHx_REG_RX_RATE_EN	<string>	"FALSE","TRUE"	FALSE	enable the Rx rate control from register 1'b0: 对应于参数值"DISABLE"， Rx rate controlled by pma_rx_rate_x[2:0] 1'b1: 对应于参数值"ENABLE"： Rx rate controlled by reg_rx_rate[2:0]

参数名称	参数类型	有效值	默认值	功能描述
PMA_CHx_REG_RX_RES_TRIM	<interger>	0 to 63	51	Rx termination resistor calibration register
PMA_CHx_REG_RX_RES_TRIM_EN	<string>	"FALSE","TRUE"	FALSE	Enable rx termination resistor calibration control from register 1'b0: 对应于参数值"DISABLE"， Rx res calibration controlled by pll_res_trim (internal, from pll) 1'b1: 对应于参数值"ENABLE" Rx res calibration controlled by reg_rx_res_trim
PMA_CHx_REG_RX_EQ_OFF	<string>	"FALSE","TRUE"	FALSE	1'b0: 对应于参数值"FALSE"， rx equalization is on 1'b1: 对应于参数值"TRUE" rx equalization is off.
PMA_CHx_REG_RX_PREAMP_IC	<interger>	0 to 4095	1367	rx preamp current dac
PMA_CHx_REG_RX_PCLK_EDGE_SEL	<string>	"POS_EDGE","NEG_EDGE"	POS_EDGE	rx pclk edge select 1'b0: 对应于参数值"POS_EDGE"， rx_pdata is clocked out to Tx on the falling edge of rxpclk 1'b1: 对应于参数值"NEG_EDGE" rx_pdata is clocked out to Tx on the rising edge of rxpclk.
PMA_CHx_REG_RX_PIBUF_IC	<interger>	0 to 3	2	rx pi buf bandwidth control logic
PMA_CHx_REG_RX_DCC_IC_RX	<interger>	0 to 3	3	rx dcc cml2cmos current dac control

参数名称	参数类型	有效值	默认值	功能描述
PMA_CHx_REG_RX_DCC_IC_TX	<interger>	0 to 3	3	tx dcc cml2cmos current dac control
PMA_CHx_REG_RX_ICTRL_TRX	<string>	"87_5PCT","100PCT","112_5PCT","125PCT"	"100PCT"	trx global bias current control 0："87_5PCT",87.5%； 1："100PCT",100%； 2："112_5PCT",112.5%； 3："125PCT",125%
PMA_CHx_REG_RX_ICTRL_SIGDET	<interger>	0 to 15	5	sigdet bias current control register, [1:0] to control sigdet module, [3:2] to control sigdet preamp module 00: sigdet bias current is 43.75uA 01: 50uA 10: 56.25uA 11: 62.5uA
PMA_CHx_REG_RX_ICTRL_PREAMP	<string>	"87_5PCT","100PCT","112_5PCT","125PCT"	"100PCT"	preamp bias current control 0："87_5PCT",87.5%； 1："100PCT",100%； 2："112_5PCT",112.5%； 3："125PCT",125%
PMA_CHx_REG_RX_ICTRL_SLICER	<string>	"87_5PCT","100PCT","112_5PCT","125PCT"	"100PCT"	slicer bias current control 0："87_5PCT",87.5%； 1："100PCT",100%； 2："112_5PCT",112.5%； 3："125PCT",125%
PMA_CHx_REG_RX_ICTRL_PIBUF	<string>	"87_5PCT","100PCT","112_5PCT","125PCT"	"100PCT"	phase interpolator pre-buffer bias current control 0："87_5PCT",87.5%； 1："100PCT",100%； 2："112_5PCT",112.5%；

参数名称	参数类型	有效值	默认值	功能描述
				3："125PCT",125%
PMA_CHx_REG_RX_ICTRL_PI	<string>	"87_5PCT","100PCT","112_5PCT","125PCT"	"100PCT"	phase interpolator path bias current control 0："87_5PCT",87.5%； 1："100PCT",100%； 2："112_5PCT",112.5%； 3："125PCT",125%
PMA_CHx_REG_RX_ICTRL_DCC	<string>	"87_5PCT","100PCT","112_5PCT","125PCT"	"100PCT"	receiver DCC bias current control 0："87_5PCT",87.5%； 1："100PCT",100%； 2："112_5PCT",112.5%； 3："125PCT",125%
PMA_CHx_REG_RX_ICTRL_PREDRV	<string>	"87_5PCT","100PCT","112_5PCT","125PCT"	"100PCT"	tx predriver bias current control 0："87_5PCT",87.5%； 1："100PCT",100%； 2："112_5PCT",112.5%； 3："125PCT",125%
PMA_CHx_REG_TX_RATE	<string>	"DIV8","DIV4","DIV2","DIV1"	DIV1	Tx rate control register : 可设置： 0："DIV8",highest datarate /8； 1："DIV4",highest datarate /4； 2："DIV2",highest datarate /2； 3："DIV1",highest datarate bit[2] is reserved.
PMA_CHx_REG_TX_RATE_EN	<string>	"FALSE","TRUE"	FALSE	enable the tx rate control from register 1'b0: 对应于参数值 "DISABLE"， tx rate controlled by pma_tx_rate_x[2:0]

参数名称	参数类型	有效值	默认值	功能描述
				1'b1: 对应于参数值"ENABLE"; tx rate controlled by reg_rx_rate[2:0]
PMA_CHx_REG_RX_TX 2RX_PLPBK_RST_N	<string>	"FALSE","TRUE"	TRUE	rx tx2rx parallel loop back block reset register, low valid 1'b0: 对应于参数值"FALSE"，reset tx2rx parallel loop back block 1'b1: 对应于参数值"TRUE"; normal operation
PMA_CHx_REG_RX_TX 2RX_PLPBK_RST_N_EN	<string>	"FALSE","TRUE"	FALSE	enable the tx2rx parallel loop back reset control from register 1'b0: 对应于参数值"DISABLE"，sigdet reset controlled by pma_rx_reset_n_x 1'b1: 对应于参数值"ENABLE"; sigdet reset controlled by reg_rx_tx2rx_plpbk_rst_n
PMA_CHx_REG_RX_TX 2RX_PLPBK_EN	<string>	"FALSE","TRUE"	FALSE	tx2rx paralled loopback enable register 1'b0: 对应于参数值"DISABLE"，tx2rx paralled loopback is off 1'b1: 对应于参数值"ENABLE"; tx2rx paralled loopback is on
PMA_CHx_REG_TXCLK _SEL	<string>	"PLL","RXCLK"	PLL	enable the Rx rate control from register 1'b0: 对应于参数值"PLL"，select pll clock as txclk ;

参数名称	参数类型	有效值	默认值	功能描述
				1'b1: 对应于参数值"RXCLK"： select rxclk as txclk
PMA_CHx_REG_RX_DATA_POLARITY	<string>	"NORMAL","REVERSE"	NORMAL	rx data polarity control register, default is 0 1'b0: 对应于参数值"NORMAL"， reverse the rx parallel output polarity ; 1'b1: 对应于参数值"REVERSE", normal output
PMA_CHx_REG_RX_ERR_INSERT	<string>	"FALSE","TRUE"	FALSE	reg_rx_err_insert rising edge is used to generate on clock cycle pulse, default is 0 1'b0: 对应于参数值"FALSE"; 1'b1: 对应于参数值"TRUE",
PMA_CHx_REG_UDP_CHK_EN	<string>	"FALSE","TRUE"	FALSE	BIST UDP checker enable 1'b0: 对应于参数值"DISABLE"， UDP checker is disabled; 1'b1: 对应于参数值"ENABLE", UDP checker is enabled
PMA_CHx_REG_PRBS_SEL	<string>	"PRBS7","PRBS15","PRBS23","PRBS31"	PRBS7	PRBS pattern select : 可设置： 0："PRBS7",PRBS7 2^7-1, X7+X6+1 (default)； 1："PRBS15",PRBS15 2^15-1, X15+X14+1； 2："PRBS23",PRBS23 2^23-1, X23+X18+1；

参数名称	参数类型	有效值	默认值	功能描述
				3："PRBS31",PRBS31 2^31-1, X31+X28+1
PMA_CHx_REG_PRBS_CHK_EN	<string>	"FALSE","TRUE"	FALSE	BIST PRBS checker enable 1'b0：对应于参数值"DISABLE"， BIST checker is disabled; 1'b1：对应于参数值"ENABLE" BIST checker is enabled
PMA_CHx_REG_PRBS_CHK_WIDTH_SEL	<string>	"8BIT","10BIT","16BIT","20BIT"	20BIT	bist checker input data bus width selection 0："8BIT",highest datarate /8； 1："10BIT",highest datarate /4； 2："16BIT",highest datarate /2； 3："20BIT",highest datarate
PMA_CHx_REG_BIST_CHK_PAT_SEL	<string>	"PRBS","CONSTANT"	PRBS	BIST constant pattern or PRBS pattern selection 1'b0：对应于参数值"PRBS"， PRBS pattern ; 1'b1：对应于参数值 "CONSTANT", constant pattern
PMA_CHx_REG_LOAD_ERR_CNT	<string>	"DISABLE","ENABLE"	DISABLE	enable to load BIST checker error counter to 32bit status 1'b0：对应于参数值 "DISABLE"， 1-->0: 32bit status holds last loaded error counter value, and error counter continues without being affected; 1'b1：对应于参数值"ENABLE",

参数名称	参数类型	有效值	默认值	功能描述
				0-->1: error counter value is loaded, and error counter is cleared to 0
PMA_CHx_REG_CHK_COUNTER_EN	<string>	"FALSE","TRUE"	FALSE	BIST UDP checker enable 1'b0: 对应于参数值"DISABLE"， bist prbs checker error counter enable ; 1'b1: 对应于参数值"ENABLE", bist prbs checker error counter disable, and clear the bist error counter value
PMA_CHx_REG_CDR_PROP_GAIN	<interger>	0 to 7	5	PRBS pattern select : 可设置： 0：0, 1/2^12； 1：1, 1/2^11； 2：2, 1/2^10； 3：3, 1/2^9； 4：4，1/2^8； 5：5，1/2^7； 6：6，1/2^6； 7：7，1/2^5
PMA_CHx_REG_CDR_PROP_TURBO_GAIN	<interger>	0 to 7	6	turbo proportional gain control: (refer to the PI control value) : 可设置： 0：0, 1/2^12； 1：1, 1/2^11； 2：2, 1/2^10； 3：3, 1/2^9； 4：4，1/2^8；

参数名称	参数类型	有效值	默认值	功能描述
				5：5，1/2^7； 6：6，1/2^6； 7：7，1/2^5
PMA_CHx_REG_CDR_INT_GAIN	<interger>	0 to 7	5	integral gain control: (refer to the PI control value): 可设置： 0：0, 1/2^17； 1：1, 1/2^16； 2：2, 1/2^15； 3：3, 1/2^14； 4：4，1/2^13； 5：5，1/2^12； 6：6，1/2^11； 7：7，1/2^10
PMA_CHx_REG_CDR_INT_TURBO_GAIN	<interger>	0 to 7	6	turbo integral gain control: (refer to the PI control value) : 可设置： 0：1/2^17； 1：1/2^16； 2：1/2^15； 3：1/2^14； 4：1/2^13； 5：1/2^12； 6：1/2^11； 7：1/2^10
PMA_CHx_REG_CDR_INT_SAT_MAX	<interger>	0 to 1023	992	Set the maximum integral path saturation value. (2's complement) Default value is 1000ppm. MSB is sign bit and LSB is

参数名称	参数类型	有效值	默认值	功能描述
				1/2^11(62.5ppm).
PMA_CHx_REG_CDR_INT_SAT_MIN	<interger>	0 to 1023	32	Set the minimum integral path saturation value.(2's complement). Default value is -1000ppm. MSB is sign bit and LSB is 1/2^11(62.5ppm).
PMA_CHx_REG_CDR_INT_RST	<string>	"FALSE","TRUE"	FALSE	cdr integral path reset, high valid 1'b0: 对应于参数值"FALSE"，disable. 1'b1: 对应于参数值"TRUE"：reset the integral path.
PMA_CHx_REG_CDR_INT_RST_OW	<string>	"DISABLE","ENABLE"	DISABLE	cdr integral path reset overwrite, high valid 1'b0: 对应于参数值"DISABLE"，disable； 1'b1: 对应于参数值"ENABLE" reset the integral path。
PMA_CHx_REG_CDR_PROP_RST	<string>	"FALSE","TRUE"	FALSE	cdr proportional path reset, high valid 1'b0: 对应于参数值"FALSE"，disable； 1'b1: 对应于参数值"TRUE" reset the proportional path。
PMA_CHx_REG_CDR_PROP_RST_OW	<string>	"DISABLE","ENABLE"	DISABLE	cdr proportional path reset overwrite, high valid 1'b0: 对应于参数值"DISABLE"，disable；

参数名称	参数类型	有效值	默认值	功能描述
				1'b1: 对应于参数值"ENABLE" reset the proportional path using reg_cdr_prop_rst。
PMA_CHx_REG_CDR_LOCK_RST	<string>	"FALSE","TRUE"	FALSE	cdr lock counter reset, high valid 1'b0: 对应于参数值"FALSE"，disable； 1'b1: 对应于参数值"TRUE" reset the lock counter。
PMA_CHx_REG_CDR_LOCK_RST_OW	<string>	"DISABLE","ENABLE"	DISABLE	cdr lock counter reset overwrite, high valid. 1'b0: 对应于参数值"DISABLE"，disable； 1'b1: 对应于参数值"ENABLE" reset the lock counter using reg_cdr_lock_rst。
PMA_CHx_REG_CDR_RX_PI_FORCE_SEL	<interger>	0 to 1	0	cdr rx PI control force value selection 1'b0：0，rx pi ctrl using the accumulator output 1'b1：1，rx pi ctrl using the force data
PMA_CHx_REG_CDR_RX_PI_FORCE_D	<interger>	0 to 255	0	cdr rx PI control force data.
PMA_CHx_REG_CDR_LOCK_TIMER	<string>	"0_8U","1_2U","1_6U","2_4U","3_2U","4_8U","12_8U","25_6U"	1_2U	cdr lock detection pseudo timer control.(for 800MHz clock): 可设置： 0："0_32U", 0.32u 256cycles； 1："0_665U", 0.665u 512cycles； 2："1_28U", 1.28u 1028cycles；

参数名称	参数类型	有效值	默认值	功能描述
				3："1_96U", 1.96u 1536cycles； 4："2_6U"，2.6u 2048cycles； 5："5_2U"，5.2u 4096cycles； 6："7_8U"，7.8u 6144cycles； 7："10_5U"，10.5u 8192cycles
PMA_CHx_REG_CDR_TURBO_MODE_TIMER	<interger>	0 to 3	1	0："DIV1", turbo_mode time is equal to the lock time； 1："DIV2", turbo_mode time is 1/2 of the lock time； 2："DIV4", turbo_mode time is 1/4 of the lock time； 3："DIV8", turbo_mode time is 1/8 of the lock time.
PMA_CHx_REG_CDR_LOCK_VAL	<string>	"FALSE","TRUE"	FALSE	cdr lock detection (cdr_ready) overwrite value 1'b0: 对应于参数值"FALSE"，disable. 1'b1: 对应于参数值"TRUE"：reset the integral path.
PMA_CHx_REG_CDR_LOCK_OW	<string>	"DISABLE", "ENABLE"	DISABLE	cdr lock detection (cdr_ready) overwrite enable 1'b0: 对应于参数值"DISABLE"，disable； 1'b1: 对应于参数值"ENABLE" cdr_lock is set by reg_cdr_lock_val。
PMA_CHx_REG_CDR_INT_SAT_DET_EN	<string>	"FALSE", "TRUE"	TRUE	Integral path saturation detection enable: 1'b0: 对应于参数值

参数名称	参数类型	有效值	默认值	功能描述
				"DISABLE"， disable； 1'b1：对应于参数值"ENABLE" reset the integral path
PMA_CHx_REG_CDR_SAT_DET_STATUS_EN	<string>	"FALSE", "TRUE"	FALSE	Enable cdr saturation status detection: 1'b0：对应于参数值 "DISABLE"， disable； 1'b1：对应于参数值"ENABLE" enable
PMA_CHx_REG_CDR_SAT_DET_STATUS_RESET_EN	<string>	"FALSE", "TRUE"	FALSE	When the cdr saturation is detected, reset the cdr: 1'b0：对应于参数值 "DISABLE"， disable； 1'b1：对应于参数值"ENABLE" enable
PMA_CHx_REG_CDR_PI_CTRL_RST	<string>	"FALSE", "TRUE"	FALSE	cdr pictrl reset register 1'b0：对应于参数值"FALSE"， 1'b1：对应于参数值"TRUE"；
PMA_CHx_REG_CDR_PI_CTRL_RST_OW	<string>	"DISABLE", "ENABLE"	DISABLE	cdr pi control reset overwrite.Active high. 1'b0：对应于参数值 "DISABLE"， disable； 1'b1：对应于参数值"ENABLE" enable
PMA_CHx_REG_CDR_SAT_DET_RST	<string>	"FALSE", "TRUE"	FALSE	cdr saturation detection reset, Active high.

参数名称	参数类型	有效值	默认值	功能描述
				1'b0: 对应于参数值"FALSE"，Disable 1'b1: 对应于参数值"TRUE"：reset the sat_det module.
PMA_CHx_REG_CDR_SAT_DET_RST_OW	<string>	"DISABLE", "ENABLE"	DISABLE	cdr saturation detection reset overwrite. Active high. 1'b0: 对应于参数值"DISABLE"，disable； 1'b1: 对应于参数值"ENABLE" reset the sat_det using reg_cdr_sat_det_rst
PMA_CHx_REG_CDR_SAT_DET_STICKY_RST	<string>	"FALSE", "TRUE"	FALSE	cdr saturation detection status sticky reset overwrite. Active high. 1'b0: 对应于参数值"DISABLE"，disable； 1'b1: 对应于参数值"ENABLE" reset the sat_det status sticky using reg_cdr_sat_det_sticky_rst
PMA_CHx_REG_CDR_SAT_DET_STICKY_RST_OW	<string>	"DISABLE", "ENABLE"	DISABLE	cdr saturation detection status sticky reset overwrite. Active high. 1'b0: 对应于参数值"DISABLE"，disable； 1'b1: 对应于参数值"ENABLE" reset the sat_det status

参数名称	参数类型	有效值	默认值	功能描述
				sticky using reg_cdr_sat_det_sticky_rst
PMA_CHx_REG_CDR_SIGDET_STATUS_DIS	<string>	"FALSE", "TRUE"	FALSE	Disable the sigdet_status. 1'b0: 对应于参数值"FALSE"，cdr reset when sigdet_status is low. 1'b1: 对应于参数值"TRUE" cdr donot response to sigdet_status transition
PMA_CHx_REG_CDR_SAT_DET_TIMER	<interger>	0 to 3	2	0: 0. 64 out ot 1024 cycles saturation detected is reckoned as saturation. 1: 1. 128 out ot 1024 cycles saturation detected is reckoned as saturation. 2: 2. 256 out ot 1024 cycles saturation detected is reckoned as saturation. 3: 3. 512 out ot 1024 cycles saturation detected is reckoned as saturation.
PMA_CHx_REG_CDR_SAT_DET_STATUS_VAL	<string>	"FALSE", "TRUE"	FALSE	register for cdr saturation detection status overwrite. 1'b0: 对应于参数值"FALSE"， 1'b1: 对应于参数值"TRUE"：
PMA_CHx_REG_CDR_SAT_DET_STATUS_OW	<string>	"DISABLE", "ENABLE"	DISABLE	cdr saturation detection status overwrite enabel. 1'b0: 对应于参数值 "DISABLE"，disable；

参数名称	参数类型	有效值	默认值	功能描述
				1'b1: 对应于参数值"ENABLE" enable. cdr sat_det status using reg_cdr_sat_det_status_val
PMA_CHx_REG_CDR_TURBO_MODE_EN	<string>	"FALSE", "TRUE"	TRUE	cdr turbo mode enable: 1'b0: 对应于参数值 "DISABLE"， disable； 1'b1: 对应于参数值"ENABLE" enable
PMA_CHx_REG_CDR_STATUS_RADDR_INIT	<interger>	0 to 7	0	Initial read address value of cdr status fifo
PMA_CHx_REG_CDR_STATUS_FIFO_EN	<string>	"FALSE", "TRUE"	TRUE	Enable signal of cdr status fifo 1'b0: 对应于参数值 "DISABLE"， disable； 1'b1: 对应于参数值"ENABLE" enable.
PMA_CHx_REG_PMA_TEST_SEL	<interger>	0 to 1	0	Select which signal connected to fifo for test 3'b000: 0，cdr pictrl value 3'b001: 1，cdr int data Others reserved 000
PMA_CHx_REG_OOB_COMWAKE_GAP_MIN	<interger>	0 to 63	3	Minimum length threshold for a COMWAKE signal gap.(4/75M~53ns)
PMA_CHx_REG_OOB_COMWAKE_GAP_MAX	<interger>	0 to 63	11	Maximum length threshold for a COMWAKE signal gap.(12/75M~160ns)
PMA_CHx_REG_OOB_COMINIT_GAP_MIN	<interger>	0 to 255	15	Minimum length threshold for a COMINIT signal

参数名称	参数类型	有效值	默认值	功能描述
				gap.(16/75M~213ns)
PMA_CHx_REG_OOB_COMINIT_GAP_MAX	<interger>	0 to 255	35	Maximum length threshold for a COMINIT signal gap.(36/75M~480ns)
PMA_CHx_REG_RX_PIBUF_IC_TX	<interger>	0 to 3	1	current control register of pi buffer for tx
PMA_CHx_REG_COMWAKE_STATUS_CLEAR	<interger>	0 to 1	0	Clear pma_rx_sata_comwake status, high active. When comwake need clear, this register should be written twice with first time “1” (clear enabled) and second time “0” (clear disabled, waiting for new detection).
PMA_CHx_REG_COMINIT_STATUS_CLEAR	<interger>	0 to 1	0	Clear pma_rx_sata_cominit status, high active. When cominit need clear, this register should be written twice with first time “1” (clear enabled) and second time “0” (clear disabled, waiting for new detection).
PMA_CHx_REG_RX_SYNC_RST_N_EN	<string>	"FALSE", "TRUE"	FALSE	rx sync module reset overwrite 0: rx sync reset controlled by internal logic 1: rx sync reset controlled by reg_rx_sync_rst_n
PMA_CHx_REG_RX_SYNC_RST_N	<string>	"FALSE", "TRUE"	TRUE	rx sync module reset register
PMA_CHx_REG_RX_SATA_COMINIT_OW	<string>	"DISABLE", "ENABLE"	DISABLE	rx COMINIT signal in sata mode overwrite: 1'b0: 对应于参数值

参数名称	参数类型	有效值	默认值	功能描述
				"DISABLE"， use FSM output signal； 1'b1：对应于参数值"ENABLE" use reg_rx_sata_cominit
PMA_CHx_REG_RX_SATA_COMINIT	<string>	"FALSE", "TRUE"	FALSE	1'b0：对应于参数值"FALSE"， COMINIT signal is not detected. 1'b1：对应于参数值"TRUE"： COMINIT signal is detected
PMA_CHx_REG_RX_SATA_COMWAKE_OW	<string>	"DISABLE", "ENABLE"	DISABLE	rx COMWAKE signal in sata mode overwrite: 1'b0：对应于参数值 "DISABLE"， use FSM output signal； 1'b1：对应于参数值"ENABLE" use reg_rx_sata_cominit
PMA_CHx_REG_RX_SATA_COMWAKE	<string>	"FALSE", "TRUE"	FALSE	1'b0：对应于参数值"FALSE"， COMWAKE signal is not detected. 1'b1：对应于参数值"TRUE" COMWAKE signal is detected
PMA_CHx_REG_RX_DCC_DISABLE	<string>	"DISABLE", "ENABLE"	ENABLE	rx dcc disable control: 1'b0：对应于参数值"ENABLE"， tx dcc is enable.； 1'b1：对应于参数值"DISABLE" tx dcc is disable.
PMA_CHx_REG_TX_DCC_DISABLE	<string>	"DISABLE", "ENABLE"	ENABLE	tx dcc disable control: 1'b0：对应于参数值"ENABLE

参数名称	参数类型	有效值	默认值	功能描述
				", tx dcc is enable.; 1'b1: 对应于参数值"DISABLE " tx dcc is disable.
PMA_CHx_REG_RX_SLIP_SEL_EN	<string>	"FALSE", "TRUE"	FALSE	overwrite enable signal of reg_rx_slip_sel
PMA_CHx_REG_RX_SLIP_SEL	<interger>	0 to 15	0	slip output selection signal based on counter value of slip control
PMA_CHx_REG_RX_SLIP_EN	<string>	"FALSE", "TRUE"	FALSE	1: bypass cdr demux 0: cdr demux works normally
PMA_CHx_REG_RX_SIGDET_STATUS_SEL	<interger>	0 to 7	5	rx signal detection status signals selection: 0: channel 1 (comparator based signal detector) 1: channel 2 (comparator based signal detector+ check window filter) 2: channel 3 (slicer based signal detector) 3: channel1 || channel2 4: channel1 || channel3 5: channel1 || channel2|| channel3 6: channel2 || channel3 7: channel 1
PMA_CHx_REG_RX_SIGDET_FSM_RST_N	<string>	"FALSE", "TRUE"	TRUE	rx signal status post process fsm resetn: 1'b0: 对应于参数值"FALSE"，active. 1'b1: 对应于参数值"TRUE"：

参数名称	参数类型	有效值	默认值	功能描述
				normal mode, reset by pma_por_n & pma_rx_reset_n
PMA_CHx_REG_RX_SIGDET_STATUS_OW	<string>	"DISABLE", "ENABLE"	DISABLE	rx signal detection status overwrite: 1'b0: 对应于参数值"DISABLE"，use rx_sigdet_status from analog； 1'b1: 对应于参数值"ENABLE" use reg_rx_sigdet_status.
PMA_CHx_REG_RX_SIGDET_STATUS	<string>	"FALSE", "TRUE"	FALSE	rx signal detection status: 1'b0: 对应于参数值"FALSE"，no signal is detected. 1'b1: 对应于参数值"TRUE"：input signal is detected
PMA_CHx_REG_RX_SIGDET_VTH	<string>	"12_5MV", "25MV", "37_5MV", "50MV", "62_5MV", "75MV", "87_5MV", "100MV"	50MV	rx signal detect single end peak-peak voltage threshold control: 7: "100MV". 100mV 6: "87_5MV". 87.5mV 5: "75MV". 75mV 4: "62_5MV". 62.5mV 3: "50MV". 50mV 2: "37_5MV". 37.5mV 1: "25MV". 25mV 0: "12_5MV". 12.5mV Bit[3] is reserved.
PMA_CHx_REG_RX_SIGDET_GRM	<interger>	0 to 3	0	glitch remove setting for rx signal detection. 0: 0.

参数名称	参数类型	有效值	默认值	功能描述
				1: 1. 2: 2. 3: 3.
PMA_CHx_REG_RX_SIGDET_PULSE_EXT	<string>	"DISABLE", "ENABLE"	DISABLE	signal pulse extention enable for rx signal detection. 1'b0: 对应于参数值"DISABLE"，use rx_sigdet_status from analog； 1'b1: 对应于参数值"ENABLE" use reg_rx_sigdet_status.
PMA_CHx_REG_RX_SIGDET_CH2_SEL	<interger>	0 to 1	0	rx signal detect channel 2 input signal selection. 1'b0: 0，the RC filtered comparison signal is sent to detect channel 2. 1'b1: 1 the comparison signal is directly sent to signal detect channel 2
PMA_CHx_REG_RX_SIGDET_CH2_CHK_WINDOW	<interger>	0 to 31	3	channel 2 sigdet filter check window is 1 cycle to 32 cycles configurable. 00000: 1 refclk clock cycle …. 11111: 32 refclk clock cycles
PMA_CHx_REG_RX_SIGDET_CHK_WINDOW_EN	<string>	"FALSE", "TRUE"	TRUE	1'b0: 对应于参数值"DISABLE"， 1'b1: 对应于参数值"ENABLE" if high, sigdet fsm use chk window to filter sigdet otherwise

参数名称	参数类型	有效值	默认值	功能描述
				send out directly.
PMA_CHx_REG_RX_SIGDET_NOSIG_COUNT_SETTING	<interger>	0 to 7	4	counter of consecutive clock cycls during which no signal is detected. 000: nosig_count_timeout=8'd0 001: nosig_count_timeout=8'd3 010: nosig_count_timeout=8'd7 011: nosig_count_timeout=8'd15 100: nosig_count_timeout=8'd31 101: nosig_count_timeout=8'd63 110: nosig_count_timeout=8'd127 111: nosig_count_timeout=8'd255
PMA_CHx_REG_RX_SIGDET_OOB_DET_COUNT_VAL	<interger>	0 to 31	0	counter value of sigdet in oob detection mode
PMA_CHx_REG_SLIP_FIFO_INV_EN	<string>	"FALSE", "TRUE"	FALSE	owerwrite enable signal of reg_slip_fifo_in
PMA_CHx_REG_SLIP_FIFO_INV	<string>	"POS_EDGE", "NEG_EDGE"	POS_EDGE	Slip fifo clock domain edge select signal. 0: slip clock domain uses rising edge 1: slip clock domain uses falling edge
PMA_CHx_REG_RX_SIGDET_4OOB_DET_SEL	<interger>	0 to 7	7	Description is the same as pma_rx_reg_i[252:250]. It's for oob application. rx signal detection status signals selection:

参数名称	参数类型	有效值	默认值	功能描述
				000: channel 1 (comparator based signal detector) 001: channel 2 (comparator based signal detector+ check window filter) 010: channel 3 (slicer based signal detector) 011: channel1 || channel2 100: channel1 || channel3 101: channel1 || channel2 || channel3 || channel4 110: channel2 || channel3 111: channel 4
PMA_CHx_REG_RX_SIGDET_IC_I	<interger>	0 to 15	10	rx signal detector mux and comparator current control.
PMA_CHx_REG_RX_OOB_DETECTOR_RESET_N_OW	<string>	"DISABLE", "ENABLE"	DISABLE	Enable the oob detector reset control from register 1'b0: 对应于参数值"DISABLE"，oob detector reset controlled by pma_rx_reset_n&pma_por_n 1'b1: 对应于参数值"ENABLE" oob detector reset controlled by reg_rx_oob_detector_reset_n
PMA_CHx_REG_RX_OOB_DETECTOR_RESET_N	<string>	"FALSE", "TRUE"	FALSE	rx oob detector reset register, low effect 1'b0: 对应于参数值"FALSE"，reset rx oob detector 1'b1: 对应于参数值"TRUE"：normal operation

参数名称	参数类型	有效值	默认值	功能描述
PMA_CHx_REG_RX_OOB_DETECTOR_PD_OW	<string>	"DISABLE", "ENABLE"	DISABLE	rx oob detector powerdown overwrite: 1'b0: 对应于参数值 "DISABLE"， use FSM output signal 1'b1: 对应于参数值"ENABLE" use reg_rx_oob_detector_pd
PMA_CHx_REG_RX_OOB_DETECTOR_PD	<string>	"ON","OFF"	ON	rx oob detector powerdown overwrite: 1'b0: 对应于参数值 "DISABLE"， use FSM output signal 1'b1: 对应于参数值"ENABLE" use reg_rx_oob_detector_pd
PMA_CHx_REG_RX_TERM_CM_CTRL	<string>	"5DIV7", "2DIV3", "5DIV6", "4DIV5"	5DIV7	Rx termination common mode voltage control. 0: "5DIV7". 5/6*vdd 1: "2DIV3". 2/3*vdd 2: "5DIV6". 5/6*vdd 3: "4DIV5". 4/5*vdd
PMA_CHx_REG_TX_PD	<string>	"ON","OFF"	ON	enable tx power down control from register 1'b0: 对应于参数值" DISABLE "， tx power down controlled by pma_tx_pd_x ； 1'b1: 对应于参数值" ENABLE" tx power down controlled by reg_tx_pd 。
PMA_CHx_REG_TX_PD	<string>	"DISABLE",	DISABLE	enable tx power down control

参数名称	参数类型	有效值	默认值	功能描述
_OW		"ENABLE"		from register 1'b0: 对应于参数值" DISABLE ", tx power down controlled by pma_tx_pd_x ； 1'b1: 对应于参数值" ENABLE" tx power down controlled by reg_tx_pd 。
PMA_CHx_REG_TX_CLKPATH_PD	<string>	"ON","OFF"	ON	enable the clock path power down control from register: 1'b0: 对应于参数值" DISABLE ", clock path power down controlled by pma_tx_pd_x； 1'b1: 对应于参数值" ENABLE"， clock path power down controlled by reg_tx_clkpath_pd 。
PMA_CHx_REG_TX_CLKPATH_PD_OW	<string>	"DISABLE", "ENABLE"	DISABLE	enable the clock path power down control from register: 1'b0: 对应于参数值" DISABLE ", clock path power down controlled by pma_tx_pd_x； 1'b1: 对应于参数值" ENABLE"， clock path power down controlled by reg_tx_clkpath_pd 。

参数名称	参数类型	有效值	默认值	功能描述
PMA_CHx_REG_TX_BEACON_TIMER_SEL	<integer>	0 to 3	0	tx beacon timer selection register
PMA_CHx_REG_TX_RXDET_REQ_OW	<string>	"DISABLE", "ENABLE"	DISABLE	rxdet request enable register overwrite 1'b0: 对应于参数值" DISABLE ", use pma pin； 1'b1: 对应于参数值" ENABLE" use tx_reg value 。
PMA_CHx_REG_TX_RXDET_REQ	<string>	"FALSE", "TRUE"	FALSE	rxdet request register 1'b0: 对应于参数值"FALSE"， 1'b1: 对应于参数值"TRUE"
PMA_CHx_REG_TX_BEACON_EN_OW	<string>	"DISABLE", "ENABLE"	DISABLE	beacon enable register overwrite 1'b0: 对应于参数值 "DISABLE"； use pma_tx_beacon_en_x pin 1'b1: 对应于参数值"ENABLE" use reg_tx_beacon_en。
PMA_CHx_REG_TX_BEACON_EN	<string>	"FALSE", "TRUE"	FALSE	register beacon enable 1'b0: 对应于参数值 "DISABLE"； no beacon 1'b1: 对应于参数值"ENABLE" beacon mode。
PMA_CHx_REG_TX_EI_EN_OW	<string>	"DISABLE", "ENABLE"	DISABLE	EI enable register overwrite 1'b0: 对应于参数值 "DISABLE"； use pma_tx_ei_en_x pin 1'b1: 对应于参数值"ENABLE"

参数名称	参数类型	有效值	默认值	功能描述
				use reg_tx_ei_en。
PMA_CHx_REG_TX_EI_EN	<string>	"FALSE", "TRUE"	FALSE	register EI enable 1'b0: 对应于参数值"DISABLE"，no EI； 1'b1: 对应于参数值"ENABLE"，EI mode
PMA_CHx_REG_TX_RES_CAL_EN	<string>	"FALSE", "TRUE"	FALSE	tx res dac register enable 1'b0: 对应于参数值"DISABLE"，res dac ctrl is from register； 1'b1: 对应于参数值"ENABLE" res dac ctrl is from pll res calibration。
PMA_CHx_REG_TX_RES_CAL	<integer>	0 to 63	51	register tx res ctrl code default is 50ohm (default 110011)。
PMA_CHx_REG_TX_BIAS_CAL_EN	<string>	"FALSE", "TRUE"	FALSE	tx driver 200uA bias (rx) ctrl dac register enable 1'b0: 对应于参数值"DISABLE"，driver 200uA bias dac ctrl is from register； 1'b1: 对应于参数值"ENABLE" driver 200uAbias dac ctrl is from pll res calibration。
PMA_CHx_REG_TX_BIAS_CTRL	<integer>	0 to 63	48	register tx driver 200uA bias ctrl code ipn200_drv=200uA,full bias
PMA_CHx_REG_TX_RXDET_TIMER_SEL	<string>	"3CYCLE", "12CYCLE",	12CYCLE	register enable rx detect sample waiting time control

参数名称	参数类型	有效值	默认值	功能描述
		"24CYCLE", "36CYCLE"		可设置： 0："3CYCLE"； 1："12CYCLE"； 2："24CYCLE"； 3："36CYCLE"
PMA_CHx_REG_TX_SYNC_OW	<string>	"DISABLE", "ENABLE"	DISABLE	tx sync register overwrite 1'b0: 对应于参数值 "DISABLE"， use pma_lane_sync pin 1'b1: 对应于参数值 "ENABLE"： use reg_tx_sync
PMA_CHx_REG_TX_SYNC	<string>	"DISABLE", "ENABLE"	DISABLE	register tx sync enable 1'b0: 对应于参数值 "DISABLE"， no sync 1'b1: 对应于参数值 "ENABLE"： no sync
PMA_CHx_REG_TX_PD_POST	<string>	"ON","OFF"	OFF	Overwrite of reg_tx_pd_post 1'b0: 对应于参数值 "DISABLE"， overwrite disabled 1'b1: 对应于参数值 "ENABLE"： overwrite with reg_tx_pd_post
PMA_CHx_REG_TX_PD_POST_OW	<string>	"DISABLE", "ENABLE"	DISABLE	Overwrite of reg_tx_pd_post 1'b0: 对应于参数值 "DISABLE"， overwrite disabled

参数名称	参数类型	有效值	默认值	功能描述
				1'b1: 对应于参数值 "ENABLE"： overwrite with reg_tx_pd_post
PMA_CHx_REG_TX_RESET_N_OW	<string>	"DISABLE", "ENABLE"	DISABLE	tx reset n register overwrite 1'b0: 对应于参数值 "DISABLE"， use pma_tx_reset_n_x pin 1'b1: 对应于参数值 "ENABLE"： use reg_tx_reset_n
PMA_CHx_REG_TX_RESET_N	<string>	"FALSE", "TRUE"	TRUE	register tx reset n enable 1'b0: 对应于参数值"FALSE"， tx reset 1'b1: 对应于参数值"TRUE"： tx not reset
PMA_CHx_REG_TX_DCC_RESET_N_OW	<string>	"DISABLE", "ENABLE"	DISABLE	tx dcc reset n register overwrite 1'b0: 对应于参数值 "DISABLE"， use pma_tx_reset_n_x pin 1'b1: 对应于参数值 "ENABLE"： use reg_tx_reset_n
PMA_CHx_REG_TX_DCC_RESET_N	<string>	"FALSE", "TRUE"	TRUE	register tx dcc reset n enable 1'b0: 对应于参数值"FALSE"， tx dcc reset 1'b1: 对应于参数值"TRUE"： tx dcc not reset
PMA_CHx_REG_TX_BUSWIDTH_OW	<string>	"DISABLE", "ENABLE"	DISABLE	tx buswidth register overwrite 1'b0: 对应于参数值 "DISABLE"，

参数名称	参数类型	有效值	默认值	功能描述
				use pma_tx_buswidth_x[2:0] 1'b1：对应于参数值 "ENABLE"： use reg_tx_buswidth[2:0]
PMA_CHx_REG_TX_BUSWIDTH	<string>	"8BIT", "10BIT", "16BIT", "20BIT"	20BIT	tx buswidth control 0："8BIT"； 1："10BIT"； 2："16BIT"； 3："20BIT" bit[2] is reserved.
PMA_CHx_REG_PLL_READY_OW	<string>	"DISABLE", "ENABLE"	DISABLE	tx pll ready register overwrite 1'b0：对应于参数值 "DISABLE"， use pll_ready pin 1'b1：对应于参数值 "ENABLE"： use reg_pll_ready
PMA_CHx_REG_PLL_READY	<string>	"FALSE", "TRUE"	TRUE	register pll ready enable 1'b0：对应于参数值"FALSE"， pll not ready 1'b1：对应于参数值"TRUE"： pll ready
PMA_CHx_REG_TX_PCLK_SW_OW	<string>	"DISABLE", "ENABLE"	DISABLE	txpclk switch register overwrite 0: use internal logic to control txpclk switch 1: use reg_tx_pclk_sw
PMA_CHx_REG_TX_PCLK_SW	<string>	"FALSE", "TRUE"	TRUE	txpclk switch register 0: select refclk as txpclk 1: normal txpclk

参数名称	参数类型	有效值	默认值	功能描述
PMA_CHx_REG_EI_PCLK_DELAY_SEL	<integer>	0 to 3	0	Control tx_main_pre_z, tx_post_pre_z, tx_main_bdata, tx_post_bdata delay relative with normal parallel data. 00: 0 pclk cycle, the four signals are synchronized with parallel data. 01: -1 pclk cycle, the four signals are 1 cycle earlier than parallel data. 01: +1 pclk cycle, the four signals are 1 cycle later than parallel data. 11: reserved
PMA_CHx_REG_TX_DRV01_DAC0	<integer>	0 to 63	0	post cursor tx bias ctrl code setting0 (1x=3.125uA) 000000:0 000001:1x … 111111:63x
PMA_CHx_REG_TX_DRV01_DAC1	<integer>	0 to 63	10	post cursor tx bias ctrl code setting1 (1x=3.125uA) 000000:0 000001:1x … 111111:63x -3.5dB
PMA_CHx_REG_TX_DRV01_DAC2	<integer>	0 to 63	16	post cursor tx bias ctrl code setting2 (1x=3.125uA) 000000:0

参数名称	参数类型	有效值	默认值	功能描述
				000001:1x … 111111:63x -6dB
PMA_CHx_REG_TX_DRV00_DAC0	<integer>	0 to 63	63	main cursor tx bias ctrl code setting0 (1x=3.125uA) 000000:0 000001:1x … 111111:63x(total=196.875uA) 0dB
PMA_CHx_REG_TX_DRV00_DAC1	<integer>	0 to 63	53	main cursor tx bias ctrl code setting0 (1x=3.125uA) 000000:0 000001:1x … 111111:63x(total=196.875uA) -3.5dB
PMA_CHx_REG_TX_DRV00_DAC2	<integer>	0 to 63	48	main cursor tx bias ctrl code setting0 (1x=3.125uA) 000000:0 000001:1x … 111111:63x(total=196.875uA) -6dB
PMA_CHx_REG_TX_AMP0	<integer>	0 to 63	8	tx full swing bias register control (1x=2.08uA) 000000:0 000001:1x ….. 111111:63x (131uA)

参数名称	参数类型	有效值	默认值	功能描述
PMA_CHx_REG_TX_AMP1	<integer>	0 to 63	16	tx full swing bias register control (1x=2.08uA) 000000:0 000001:1x ….. 111111:63x (131uA)
PMA_CHx_REG_TX_AMP2	<integer>	0 to 63	32	tx full swing bias register control (1x=2.08uA) 000000:0 000001:1x ….. 111111:63x (131uA)
PMA_CHx_REG_TX_AMP3	<integer>	0 to 63	48	tx full swing bias register control (1x=2.08uA) 000000:0 000001:1x ….. 111111:63x (131uA)
PMA_CHx_REG_TX_AMP4	<integer>	0 to 63	56	tx full swing bias register control (1x=2.08uA) 000000:0 000001:1x ….. 111111:63x (131uA)
PMA_CHx_REG_TX_MARGIN	<integer>	0 to 7	0	tx_margin control register
PMA_CHx_REG_TX_MARGIN_OW	<string>	"DISABLE", "ENABLE"	DISABLE	overwrite signal of tx_margin 1'b0: 对应于参数值 "DISABLE"， tx_margin is controlled by reg_tx_margin 1'b1: 对应于参数值"ENABLE"; tx_margin is controlled by pma_tx_margin

参数名称	参数类型	有效值	默认值	功能描述
PMA_CHx_REG_TX_DEEMP	<integer>	0 to 3	0	tx_deemp control register
PMA_CHx_REG_TX_DEEMP_OW	<string>	"DISABLE", "ENABLE"	DISABLE	overwrite signal of tx_deemp 1'b0: 对应于参数值"DISABLE"，tx_deemp is controlled by reg_tx_deemp 1'b1: 对应于参数值"ENABLE"; tx_deemp is controlled by pma_tx_deemp
PMA_CHx_REG_TX_SWING	<string>	"FALSE", "TRUE"	FALSE	tx_swing control register 1'b0: 对应于参数值"FALSE"， 1'b1: 对应于参数值"TRUE";
PMA_CHx_REG_TX_SWING_OW	<string>	"DISABLE", "ENABLE"	DISABLE	overwrite signal of tx_swing. 1'b0: 对应于参数值"DISABLE"，tx_swing is controlled by reg_tx_swing 1'b1: 对应于参数值"ENABLE"; tx_swing is controlled by pma_tx_swing
PMA_CHx_REG_TX_RXDET_THRESHOLD	<string>	"50MV", "75MV", "100MV", 125MV"	100MV	Tx rxdet threshold select 0："50MV". vdda-50mv； 1："75MV". vdda-75mv； 2："100MV". vdda-100mv (default)； 3："125MV". vdda-125mv
PMA_CHx_REG_TX_BEACON_OSC_CTRL	<integer>	0 to 7	4	beacon osc frequnce ctrl 1**: 18M 0**: 36M

参数名称	参数类型	有效值	默认值	功能描述
PMA_CHx_REG_TX_PREDRV_DAC	<integer>	0 to 3	1	tx pre-driver bias setting
PMA_CHx_REG_TX_PREDRV_CM_CTRL	<integer>	0 to 3	1	tx predriver common mode ctrl
PMA_CHx_REG_TX_TX2RX_SLPBACK_EN	<string>	"FALSE", "TRUE"	FALSE	Tx to rx serial loopback enable. 1'b0: 对应于参数值"DISABLE"， serial loopback disabled 1'b1: 对应于参数值"ENABLE"; serial loopback enabled
PMA_CHx_REG_TX_PCLK_EDGE_SEL	<string>	"POS_EDGE", "NEG_EDGE"	POS_EDGE	1'b0: 对应于参数值"POS_EDGE"， retime in positive clk edge at 40bit data input 1'b1: 对应于参数值"NEG_EDGE"; retime in negtive clk edge at 40bit data input
PMA_CHx_REG_TX_RXDET_STATUS_OW	<string>	"DISABLE", "ENABLE"	DISABLE	register rxdetect status force enable. 1'b0: 对应于参数值"DISABLE"， rxdetect_status = tx_reg[278] 1'b1: 对应于参数值"ENABLE"; from detection
PMA_CHx_REG_TX_RXDET_STATUS	<string>	"FALSE", "TRUE"	TRUE	rxdetect status force value 1'b0: 对应于参数值"FALSE"， rx not detected, floating 1'b1: 对应于参数值"TRUE" rx detected, 50-ohm termination

参数名称	参数类型	有效值	默认值	功能描述
PMA_CHx_REG_TX_PRBS_GEN_EN	<string>	"FALSE", "TRUE"	FALSE	PRBS generator enable 1'b0: 对应于参数值" DISABLE ", PRBS clock is gated to 0； 1'b1: 对应于参数值" ENABLE" PRBS clock is on。
PMA_CHx_REG_TX_PRBS_GEN_WIDTH_SEL	<string>	"8BIT", "10BIT", "16BIT", "20BIT"	20BIT	PRBS width select 0: 8BIT 1: 10BIT 2: 16BIT 3: 20BIT
PMA_CHx_REG_TX_PRBS_SEL	<string>	"PRBS7", "PRBS15", "PRBS23", "PRBS31"	PRBS7	PRBS pattern select 0: PRBS7 1: PRBS15 2: PRBS23 3: PRBS31
PMA_CHx_REG_TX_UDP_DATA	<integer>	0 to 1048575	256773	constant 20-bit pattern, default = K28.5
PMA_CHx_REG_TX_FIFO_RST_N	<string>	"FALSE", "TRUE"	FALSE	fifo reset signal to reset fifo. 1'b0: 对应于参数值" FALSE", fifo is rst; 1'b1: 对应于参数值"TRUE" not rst。
PMA_CHx_REG_TX_FIFO_WP_CTRL	<integer>	0 to 7	2	fifo write probe to ctrl distance write probe to read probe
PMA_CHx_REG_TX_FIFO_EN	<string>	"FALSE", "TRUE"	FALSE	reg_fifo_en is 0 ,fifo txpclk is gated. 1'b0: 对应于参数值" DISABLE ", txpclk is gated off during

参数名称	参数类型	有效值	默认值	功能描述
				rate change； 1'b1：对应于参数值" ENABLE" txpclk is on。
PMA_CHx_REG_TX_DATA_MUX_SEL	<integer>	0 to 3	2	parallel data mux select to select different data source. 0: data from pcs 1: prbs bist data 2: udp data 3: rx loopback data
PMA_CHx_REG_TX_ERR_INSERT	<string>	"FALSE", "TRUE"	FALSE	tx error insertion, add an error at reg_tx_err_ins riseedge in txpdata_order[0]. 1'b0：对应于参数值" FALSE"； 1'b1：对应于参数值"TRUE"。
PMA_CHx_REG_TX_SATA_EN	<string>	"FALSE", "TRUE"	FALSE	SATA mode enable register, This register is combined with pma_tx_ei_en_x to determine whether normal EI mode or OOB_EI mode valid only for SATA is active. 1'b0：对应于参数值" DISABLE"， SATA mode is disabled； 1'b1：对应于参数值" ENABLE" SATA mode is enabled。
PMA_CHx_REG_RATE_CHANGE_TXPCLK_ON_OW	<string>	"DISABLE", "ENABLE"	DISABLE	rate change txpclk on overwrite signal 1'b0：对应于参数值" DISABLE"， rate change txpclk on is

参数名称	参数类型	有效值	默认值	功能描述
				controlled by pma_rate_change_txpclk_on； 1'b1：对应于参数值" ENABLE" rate change txpclk on is controlled by reg_rate_change_txpclk_on。
PMA_CHx_REG_RATE_ CHANGE_TXPCLK_ON	<string>	"DISABLE", "ENABLE"	ENABLE	txpclk enable signal when rate changes 1'b0：对应于参数值" DISABLE "， txpclk is gated off during rate change； 1'b1：对应于参数值" ENABLE" txpclk is on。
PMA_CHx_REG_TX_PU LLUP_DAC0	<integer>	0 to 31	8	register tx pullup dac ctrl code, 00111:1.4mA 01000:1.6mA …… 11111:6.2mA
PMA_CHx_REG_TX_PU LLUP_DAC1	<integer>	0 to 31	8	register tx pullup dac ctrl code, 对应reg_tx_pullup_dac1[2:0]位
PMA_CHx_REG_TX_PU LLUP_DAC2	<integer>	0 to 31	8	register tx pullup dac ctrl code 00111:1.4mA 01000:1.6mA …… 11111:6.2mA
PMA_CHx_REG_TX_PU LLUP_DAC3	<integer>	0 to 31	8	register tx pullup dac ctrl code， 对应reg_tx_pullup_dac3[0]位。
PMA_CHx_REG_TX_OO B_DELAY_SEL	<integer>	0 to 15	0	Controls the oob ei enable signal delay steps to compensate data

参数名称	参数类型	有效值	默认值	功能描述
				latency. 0000: 38UI 0001: 39UI 001x: 40UI 0100: 58UI 0101: 59UI 011x: 60UI 1x00: 78UI 1x01: 79UI 1x1x: 80UI
PMA_CHx_REG_TX_POLARITY	<string>	"NORMAL", "REVERSE"	NORMAL	invert tx parallel data: 1'b0: 对应于参数值" NORMAL ", txpdata_out[19:0]=txpdata_in[19:0]； 1'b1: 对应于参数值" REVERSE". txpdata_out[19:0]=~txpdata_in[19:0]。
PMA_CHx_REG_TX_SLPBK_AMP	<integer>	0 to 3	1	serial loopback amplitude adjustion
PMA_CHx_REG_TX_LS_MODE_EN	<string>	"FALSE", "TRUE"	FALSE	low-speed mode enable signal,低速模式使能。 1'b0: 对应于参数值 "DISABLE"， 低速模式不使能； 1'b1: 对应于参数值 "ENABLE"，

参数名称	参数类型	有效值	默认值	功能描述
				低速模式使能。
PMA_CHx_REG_TX_JTAG_MODE_EN_OW	<string>	"DISABLE", "ENABLE"	DISABLE	tx jtag mode enable overwrite signal 1'b0: 对应于参数值"DISABLE"，tx jtag mode is controlled by pma_tx_jtag_mode_en； 1'b1: 对应于参数值"ENABLE"，tx jtag mode is controlled by reg_tx_jtag_mode_en。
PMA_CHx_REG_TX_JTAG_MODE_EN	<string>	"FALSE", "TRUE"	FALSE	tx jtag mode enable register 1'b0: 对应于参数值"DISABLE"; 1'b1: 对应于参数值"ENABLE"。
PMA_CHx_REG_RX_JTAG_MODE_EN_OW	<string>	"DISABLE", "ENABLE"	DISABLE	rx jtag mode enable overwrite signal 1'b0: 对应于参数值"DISABLE", rx jtag mode is controlled by pma_rx_jtag_mode_en; 1'b1: 对应于参数值"ENABLE", rx jtag mode is controlled by reg_rx_jtag_mode_en。
PMA_CHx_REG_RX_JTAG_MODE_EN	<string>	"FALSE", "TRUE"	FALSE	rx jtag mode enable register 1'b0: 对应于参数值"DISABLE";

参数名称	参数类型	有效值	默认值	功能描述
				1'b1: 对应于参数值 "ENABLE"。
PMA_CHx_REG_RX_JTAG_OE	<string>	"DISABLE", "ENABLE"	DISABLE	rx jtag module enable register 0: disabled; 1: enable
PMA_CHx_REG_RX_ACJTAG_VHYSTSE	<integer>	0 to 7	0	rx acjtag hysteresis voltage control register
PMA_CHx_REG_TX_FBCLK_FAR_EN	<string>	"FALSE", "TRUE"	FALSE	tx far-end dcc fbclk path enable register 0: near-end dcc fbclk path is selected; 1: far-end dcc fbclk path is selected.
PMA_CHx_REG_RX_TERM_MODE_CTRL	<integer>	0 to 7	6	REG_RX_TERM_MODE_CTRL
PMA_CHx_REG_PLPBK_TXPCLK_EN	<string>	"FALSE", "TRUE"	TRUE	txpclk enable register in parallel loopback mode 0: rxpclk is used in parallel loopback mode to guarantee no FIFO overflow 1: txpclk is used (normal working mode).
PMA_CHx_REG_TX_609_600	<integer>		0	add for pma tx
PMA_CHx_REG_RX_CDR_617_610	<integer>		0	add for cdr use
PMA_CHx_REG_RX_CDR_623_618	<integer>		0	add for cdr use
PMA_CHx_REG_RX_631_624	<integer>		0	add for pma rx

参数名称	参数类型	有效值	默认值	功能描述
PMA_CHx_REG_RX_639_632	<integer>		0	add for pma rx
PMA_CHx_REG_RX_647_640	<integer>		0	add for pma rx
PMA_CHx_REG_RX_655_648	<integer>		0	add for pma rx
PMA_CHx_REG_RX_659_656	<integer>		0	add for pma rx
PMA_CHx_CFG_LANE_POWERUP	<string>	"ON","OFF"	OFF	cfg_lane_powerup，Lane上电配置。 1'b0: 对应于参数值"OFF"； 1'b1: 对应于参数值"ON".
PMA_CHx_CFG_PMA_POR_N	<string>	"FALSE", "TRUE"	FALSE	Lane复位配置。 1'b0: 对应于参数值"FALSE"； 1'b1: 对应于参数值"TRUE".
PMA_CHx_CFG_RX_LANE_POWERUP	<string>	"ON","OFF"	OFF	Lane的Rx通道上电配置。 1'b0: 对应于参数值"OFF"； 1'b1: 对应于参数值"ON".
PMA_CHx_CFG_RX_PMA_RSTN	<string>	"FALSE", "TRUE"	FALSE	Lane的Rx通道复位配置。 1'b0: 对应于参数值"FALSE"； 1'b1: 对应于参数值"TRUE".
PMA_CHx_CFG_TX_LANE_POWERUP	<string>	"ON","OFF"	OFF	Lane的Tx通道上电配置。 1'b0: 对应于参数值"OFF"； 1'b1: 对应于参数值"ON".
PMA_CHx_CFG_TX_PMA_RSTN	<string>	"FALSE", "TRUE"	FALSE	Lane的Tx通道复位配置。 1'b0: 对应于参数值"FALSE"； 1'b1: 对应于参数值"TRUE".
PMA_CHx_REG_RESERVED_48_45	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESER	<integer>	0 to 1	0	reserved

参数名称	参数类型	有效值	默认值	功能描述
VED_69				
PMA_CHx_REG_RESERVED_77_76	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_171_164	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_175_172	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_190	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_233_232	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_235_234	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_241_240	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_285_283	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_286	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_295	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_298	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_332_325	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_340_333	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_348_341	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESER	<integer>	0 to 1	0	reserved

参数名称	参数类型	有效值	默认值	功能描述
VED_354_349				
PMA_CHx_REG_RESERVED_373	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_376	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_452	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_502_499	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_506_505	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_550_549	<integer>	0 to 1	0	reserved
PMA_CHx_REG_RESERVED_556_552	<integer>	0 to 1	0	reserved
PMA_PLLy_REG_PLL_POWERDOWN_OW	<string>	"DISABLE", "ENABLE"	DISABLE	PLL powerdown control enable 1'b0: 对应于参数值" DISABLE ", Powerdown由端口控制； 1'b1: 对应于参数值" ENABLE" Powerdown由寄存器控制。
PMA_PLLy_REG_PLL_POWERDOWN	<string>	"ON","OFF"	ON	PLL powerdown control 1'b0: 对应于参数值"ON"，not powerdown； 1'b1: 对应于参数值"OFF"，powerdown.
PMA_PLLy_REG_PLL_RESET_N_OW	<string>	"DISABLE", "ENABLE"	DISABLE	PLL reset control enable 1'b0: 对应于参数值" DISABLE ",

参数名称	参数类型	有效值	默认值	功能描述
				reset由端口控制； 1'b1：对应于参数值" ENABLE" reset由寄存器控制。
PMA_PLLy_REG_PLL_RESET_N	<string>	"FALSE", "TRUE"	TRUE	PLL reset control 1'b0：对应于参数值"FALSE"， 低复位； 1'b1：对应于参数值"TRUE"，不 复位。
PMA_PLLy_REG_PLL_READY_OW	<string>	"DISABLE", "ENABLE"	DISABLE	PLL ready register control 1'b0：对应于参数值"FALSE"， PLL is not ready； 1'b1：对应于参数值"TRUE"， PLL is ready。
PMA_PLLy_REG_PLL_READY	<string>	"FALSE", "TRUE"	FALSE	PLL ready register control 1'b0：对应于参数值"FALSE"， PLL is not ready； 1'b1：对应于参数值"TRUE"， PLL is ready。
PMA_PLLy_REG_LANE_SYNC_OW	<string>	"DISABLE", "ENABLE"	DISABLE	lane sync control enable 1'b0：对应于参数值" DISABLE "， ready由端口控制； 1'b1：对应于参数值" ENABLE" ready由寄存器控制。
PMA_PLLy_REG_LANE_SYNC	<string>	"FALSE", "TRUE"	FALSE	PLL ready register control，tx lanes sync up once lane sync turns 0 to 1。
PMA_PLLy_REG_LOCKDET_REPEAT	<string>	"DISABLE", "ENABLE"	DISABLE	lock detection repeat enable register 1'b0：对应于参数值

参数名称	参数类型	有效值	默认值	功能描述
				"DISABLE"， lock detection modue works one time； 1'b1：对应于参数值"ENABLE" lock detection module will work continuously。
PMA_PLLy_REG_RESCAL_I_CODE_PMA	<string>	"DISABLE", "ENABLE"	DISABLE	resistor calibration input code is from pma or not. 1'b0：对应于参数值"DISABLE"，rescal code is not from pma； 1'b1：对应于参数值"ENABLE"，rescal code is from pma。
PMA_PLLy_REG_RESCAL_RESET_N_OW	<string>	"DISABLE", "ENABLE"	DISABLE	Register reset_n overwrite 1'b0：对应于参数值"DISABLE"， rescal_reset_n由端口控制； 1'b1：对应于参数值"ENABLE"， rescal_reset_n由寄存器控制。
PMA_PLLy_REG_RESCAL_RESET_N	<string>	"FALSE", "TRUE"	FALSE	Register reset_n value 1'b0：对应于参数值"FALSE"，低复位； 1'b1：对应于参数值"TRUE"，不复位。
PMA_PLLy_REG_RESCAL_DONE_OW	<string>	"DISABLE", "ENABLE"	DISABLE	Register overwrite enable for rescal_done 1'b0：对应于参数值

参数名称	参数类型	有效值	默认值	功能描述
				"DISABLE"， rescal_done由实际状态控制； 1'b1：对应于参数值"ENABLE" rescal_done由寄存器控制。
PMA_PLLy_REG_RESCAL_DONE	<string>	"FALSE", "TRUE"	FALSE	Register value to overwrite rescal_done output 1'b0：对应于参数值"FALSE"，resistor calibration is not done； 1'b1：对应于参数值"TRUE"，resistor calibration is done。
PMA_PLLy_REG_RESCAL_CODE_OW	<string>	"DISABLE", "ENABLE"	DISABLE	rescal code overwrite 1'b0：对应于参数值"DISABLE"， current control offset code for calibration. 1'b1：对应于参数值"ENABLE"： rescal_i_tune<4:0> = reg_rescal_i_offset<4:0>, rescal_r_code<5:0>={reg_rescal_onchip_small, reg_rescal_r_offset<4:0>}
PMA_PLLy_REG_LDO_VREF_SEL	<integer>	0 to 3	2	ldo output voltage control register 00: ldo output voltage – 1.05V 01: ldo output voltage – 1.1V 10: ldo output voltage – 1.15V 11: ldo output voltage – 1.2V
PMA_PLLy_REG_BIAS_	<integer>	0 to 3	1	bias current control of vco replica

参数名称	参数类型	有效值	默认值	功能描述
VCOREP_C				module 00: bias current is 43.75uA 01: bias current is 50uA 10: bias current is 56.25uA 11: bias current is 62.5uA
PMA_PLLy_REG_RESCAL_I_CODE	<integer>	0 to 63	32	If rescal_i_code is not in the range of [reg_i_ctrl_min, reg_i_ctrl_max], force rescal_i_code=reg_rescal_i_code
PMA_PLLy_REG_RESCAL_ONCHIP_SMALL_OW	<string>	"DISABLE", "ENABLE"	DISABLE	Register overwrite enable for rescal_onchip_small 1'b0: 对应于参数值 "DISABLE"，rescal_onchip_small is used (default). 1'b1: 对应于参数值 "ENABLE"：rescal_onchip_small is ignored. Resistor calibration uses: rescal_int_r_small=reg_rescal_onchip_small_val
PMA_PLLy_REG_RESCAL_ONCHIP_SMALL	<integer>	0 to 1	0	Register value to overwrite rescal_onchip_small. 0: on-chip resistor is bigger than off-chip (default); 1: on-chip resistor is smaller than off-chip.

参数名称	参数类型	有效值	默认值	功能描述
PMA_PLLy_REG_JTAG_OE	<string>	"DISABLE", "ENABLE"	DISABLE	refclk jtag mode enable register 0: refclk jtag is diabled. 1: refclk jtag is enabled.
PMA_PLLy_REG_JTAG_AC_MODE	<string>	"DISABLE", "ENABLE"	DISABLE	refclk acjtag mode enable register
PMA_PLLy_REG_JTAG_VHYSTSEL	<integer>	0 to 7	0	refclk jtag hysteresis voltage selection
PMA_PLLy_REG_PLL_LOCKDET_EN_OW	<string>	"DISABLE", "ENABLE"	DISABLE	Register overwrite control for pll_lockdet_en。 1'b0: 对应于参数值 "DISABLE"，pll_lockdet enable is controlled by pll_lockdet_en(default)； 1'b1: 对应于参数值"ENABLE" pll_lockdet_en is ignored. pll_lockdet enable is controlled by reg_pll_lockdet_en。
PMA_PLLy_REG_PLL_LOCKDET_EN	<string>	"FALSE", "TRUE"	FALSE	Register value for overwriting pll_lockdet_en。 1'b0: 对应于参数值 "DISABLE"，pll_lockdet is disabled (default)； 1'b1: 对应于参数值"ENABLE" pll_lockdet is enabled.
PMA_PLLy_REG_PLL_LOCKDET_RESET_N_OW	<string>	"DISABLE", "ENABLE"	DISABLE	Register overwrite control for pll_lockdet_reset_n。 1'b0: 对应于参数值 "DISABLE"，

参数名称	参数类型	有效值	默认值	功能描述
				pll_lockdet reset_n is controlled by pll_ lockdet_reset_n(default)； 1'b1：对应于参数值"ENABLE" pll_lockdet_reset_n is ignored. pll_lockdet reset_n is controlled by reg_pll_lockdet_reset_n.
PMA_PLLy_REG_PLL_LOCKDET_RESET_N	<string>	"FALSE", "TRUE"	FALSE	Register value for overwriting pll_lockdet_reset_n。 1'b0：对应于参数值" FALSE "， pll_lockdet is reset (default)； 1'b1：对应于参数值"TRUE" pll_lockdet is normal.
PMA_PLLy_REG_PLL_LOCKED_OW	<string>	"DISABLE", "ENABLE"	DISABLE	Register overwrite control for pll_locked。 1'b0：对应于参数值 "DISABLE"， pll_locked controlled by actual lock detection(default)； 1'b1：对应于参数值"ENABLE" pll_lock mode is controlled by reg_pll_locked.
PMA_PLLy_REG_PLL_LOCKED	<string>	"FALSE", "TRUE"	FALSE	Register value for overwriting pll_locked status。 1'b0：对应于参数值" FALSE "， PLL lock is not acquired yet；

参数名称	参数类型	有效值	默认值	功能描述
				1'b1：对应于参数值"TRUE" PLL lock is acquired.
PMA_PLLy_REG_PLL_LOCKED_STICKY_CLEAR	<string>	"FALSE", "TRUE"	FALSE	Sticky bit clear for reg_pll_locked_sticky_status。 1'b0：对应于参数值" FALSE "，reg_pll_locked_sticky_status remains its value； 1'b1：对应于参数值"TRUE" reg_pll_locked_sticky_status=0.
PMA_PLLy_REG_PLL_UNLOCKED_STICKY_CLEAR	<string>	"FALSE", "TRUE"	FALSE	Sticky bit clear for reg_pll_unlocked_sticky_ Status。 1'b0：对应于参数值" FALSE "，reg_pll_unlocked_sticky_status remains its value； 1'b1：对应于参数值"TRUE" reg_pll_unlocked_sticky_status=0
PMA_PLLy_REG_NOFBCLK_STICKY_CLEAR	<string>	"FALSE", "TRUE"	FALSE	1'b0：对应于参数值" FALSE "，reg_nofbclk_sticky_status not cleared； 1'b1：对应于参数值"TRUE" reg_nofbclk_sticky_status is cleared.
PMA_PLLy_REG_PLL_L	<integer>	0 to 7	7	Counter value for reference clock

参数名称	参数类型	有效值	默认值	功能描述
OCKDET_REFCT				time out value of lock detect. 0: 2^7 1: 2^8 2: 2^9 3: 2^10 4: 2^11 5: 2^12 6: 2^14 7: 2^16 (default)
PMA_PLLy_REG_PLL_LOCKDET_FBCT	<integer>	0 to 7	7	counter value for feedback clock time out value of lock detect 0: 2^7 1: 2^8 2: 2^9 3: 2^10 4: 2^11 5: 2^12 6: 2^14 7: 2^16 (default)
PMA_PLLy_REG_PLL_LOCKDET_LOCKCT	<integer>	0 to 7	4	Register value of lock detection threshold for count difference between ref_clk & fb_clk counters. 0: 2 1: 4 2: 8 3: 16 4: 32(default) 5: 64 6: 128

参数名称	参数类型	有效值	默认值	功能描述
				7: 256
PMA_PLLy_REG_PLL_LOCKDET_ITER	<integer>	0 to 7	3	Register value of number of iterations of consecutive successful one-time lock detection before set pll_lock. 0: 1 1: 2 2: 4 3: 8(default) 4: 16 5: 32 6: 64 7: 128
PMA_PLLy_REG_PLL_UNLOCKDET_ITER	<integer>	0 to 3	2	Register value of number of iterations of lock detection attemps before set pll_unlock. 0: 63 1: 127 2: 255(default) 3: 1023
PMA_PLLy_REG_PD_VCO	<string>	"ON","OFF"	ON	VCO power down control 1'b0: 对应于参数值"ON"，VCO is not powered down(default)； 1'b1: 对应于参数值"OFF" VCO is powered down。
PMA_PLLy_REG_FBCLK_TEST_EN	<string>	"FALSE", "TRUE"	FALSE	feedback clock to test signal enable 1'b0: 对应于参数值"DISABLE"，

参数名称	参数类型	有效值	默认值	功能描述
				fbclk to test is disabled (default)； 1'b1：对应于参数值"ENABLE" fbclk to test is enabled。
PMA_PLLy_REG_REFCLK_TEST_EN	<string>	"FALSE", "TRUE"	FALSE	refclk to test port enable 1'b0：对应于参数值 "DISABLE"， refclk to test port is disabled (default)； 1'b1：对应于参数值"ENABLE" refclk to test port is enabled。
PMA_PLLy_REG_TEST_SEL	<integer>	0 to 20	0	test signal selection 00000: vssa 00001: vdda 00010: vdda25/3 00011: vc_cap (ring:1, lc: 1/2) 00100: biasp_test (ring:1, lc: 1/2) 00101: biasp_fb_test (ring:1, lc: 1/2) 00110: vc_dummy (ring:1, lc: 1/2) 00111: lcvco_amp 01000: lcvco_amp_ref 01001: rnposab 1k 01010: rnposab 2k 01011: rpposab 1k 01100: rpposab 2k 01101: rhrpo 1k 01110: rhrpo 2k

参数名称	参数类型	有效值	默认值	功能描述
				01111: reserved 10000: test_fbclk 10001: test_refclk 10010: test_pll_fsm 10011: test_trx 10100: test_lcvco_amp_det
PMA_PLLy_REG_TEST_V_EN	<string>	"FALSE", "TRUE"	FALSE	reg_test_v_en=test_v_en, test signal voltage enable. 1'b0: 对应于参数值"DISABLE"，test signal of voltage is disabled(default)； 1'b1: 对应于参数值"ENABLE" test signal of voltage is enabled。
PMA_PLLy_REG_TEST_SIG_HALF_EN	<string>	"FALSE", "TRUE"	FALSE	test_sig_half_en=reg_test_sig_hal_en, test signal voltage half enable 1'b0: 对应于参数值"DISABLE"，test signal voltage out is 1:1 (default)； 1'b1: 对应于参数值"ENABLE" test signal voltage out is 1:2。
PMA_PLLy_REG_TEST_FSM	<integer>	0 to 3	0	FSM test signal select 00: test_fsm=pll_ready 01: test_fsm=rescal_en 10: test_fsm=pll_locked 11: test_fsm=pll_unlocked

参数名称	参数类型	有效值	默认值	功能描述
PMA_PLLy_REG_REFCLK_OUT_PD	<string>	"ON","OFF"	OFF	power down register of output refclk when refclk populating 0: refclk output buffer power on to deliver local refclk to others pll(s). 1: refclk output buffer is power off.
PMA_PLLy_REG_BGR_STARTUP_EN	<string>	"FALSE", "TRUE"	FALSE	bgr_startup_en = reg_bgr_startup_en, bandgap startup enable 1'b0: 对应于参数值"DISABLE"，bandgap startup circuit is disabled (default)； 1'b1: 对应于参数值"ENABLE" bandgap startup circuit is enabled。
PMA_PLLy_REG_BGR_STARTUP	<string>	"FALSE", "TRUE"	FALSE	bgr_startup = reg_bgr_startup, bandgap startup control 1'b0: 对应于参数值"FALSE"，bandgap is not in start-up (default)； 1'b1: 对应于参数值"TRUE" bandgap is start-up。
PMA_PLLy_REG_PD_BGR	<string>	"ON","OFF"	ON	bandgap powerdown control 1'b0: 对应于参数值"ON"，bgr is power on； 1'b1: 对应于参数值"OFF" bgr is power off。

参数名称	参数类型	有效值	默认值	功能描述
PMA_PLLy_REG_REFCLK_TERM_VCM_EN	<string>	"FALSE", "TRUE"	TRUE	refclk termination common mode voltage selection register 0: refclk termination common mode voltage follows input signal when DC coupling. 1: refclk termination common voltage is connected to internal vcm.
PMA_PLLy_REG_FBDIVA_5_EN	<string>	"FALSE", "TRUE"	TRUE	fbdiva_5_en = reg_fbdiva_5_en feedback divider A div-5 enable 1'b0: 对应于参数值"DISABLE"，对应1/4； 1'b1: 对应于参数值"ENABLE"对应1/5(default)。
PMA_PLLy_REG_FBDIVB	<integer>	0 to 2	1	feedback divider B 00: 1/2 (default) 01: 1/5 10: 1/4
PMA_PLLy_REG_RESET_N_PFDQP_OW	<string>	"DISABLE", "ENABLE"	DISABLE	PFD reset_n enable 1'b0: 对应于参数值"DISABLE"，reset_n_pfdqp is controlled by FSM (default)； 1'b1: 对应于参数值"ENABLE" reset_n_pfdqp = reg_reset_n_pfdqp。
PMA_PLLy_REG_RESET_N_PFDQP	<string>	"FALSE", "TRUE"	FALSE	PFD/QP reset_n register control 1'b0: 对应于参数值"FALSE"，

参数名称	参数类型	有效值	默认值	功能描述
				PFD/QP is in reset (default)； 1'b1：对应于参数值"TRUE" PFD/QP is in normal mode。
PMA_PLLy_REG_QPCURRENT	<integer>	0 to 15	12	charge-pump current setting
PMA_PLLy_REG_VC_FORCE_EN	<string>	"FALSE", "TRUE"	FALSE	vc_force enable register control 1'b0：对应于参数值"DISABLE"， vc force is disabled (default)； 1'b1：对应于参数值"ENABLE" vc force is enabled。
PMA_PLLy_REG_VCRESET_C_RING	<integer>	0 to 31	24	vc reset value for ring VCO band calibration 11000: 400uA x 1.6k = 640mV (ring VCO)
PMA_PLLy_REG_LPF_R_C	<integer>	0 to 15	0	lpf_r_c = reg_lpf_r_c LPF resistor control 0000 0010: 1k
PMA_PLLy_REG_LPF_TR_C	<integer>	0 to 15	2	lpf_tr_c = reg_lpf_tr_c LPF resistor control 0000 0010: 1k
PMA_PLLy_REG_PD_BIAS	<string>	"ON","OFF"	ON	pd_bias_b = not(reg_pd_bias + pma_pll_powerdown + pma_pll_iddq) pd_bias = reg_pd_bias PLL bias power down control 1'b0：对应于参数值"ON"，

参数名称	参数类型	有效值	默认值	功能描述
				PLL bias is powered up(default)； 1'b1：对应于参数值"OFF" PLL bias is powered down。
PMA_PLLy_REG_ICTRL_PLL	<integer>	0 to 3	1	ictrl_pll = reg_ictrl_pll bias control for PLL
PMA_PLLy_REG_BIAS_QP	<integer>	0 to 3	1	bias_qp = reg_bias_qp bias control for charge-pump
PMA_PLLy_REG_BIAS_LANE_SYNC	<integer>	0 to 3	1	lane sync module bias current control register 00: 43.75uA 01: 50uA 10: 56.25uA 11: 62.5uA
PMA_PLLy_REG_BIAS_CLKBUFS1	<integer>	0 to 3	1	bias_clkbufs1=reg_bias_clkbufs1 bias control for CML buffer stage 1
PMA_PLLy_REG_TXPCLK_SEL	<integer>	0 to 3	0	When performing tx rate change, PLL need txpclk to sample input rate change singal. While some lane(s) may power down, so this register is to choose one lane which is power on. 00: select txpclk of lane0; 01: select txpclk of lane1; 10: select txpclk of lane2; 11: select txpclk of lane3;
PMA_PLLy_REG_BIAS_CLKBUFS3	<integer>	0 to 3	1	bias_clkbufs3=reg_bias_clkbufs3 bias control for CML buffer stage 3

参数名称	参数类型	有效值	默认值	功能描述
PMA_PLLy_REG_LANE_SYNC_EN	<string>	"FALSE", "TRUE"	FALSE	lane_sync_en register value. 0: lane sync is disabled. 1: lane sync is enabled.
PMA_PLLy_REG_LANE_SYNC_EN_OW	<string>	"DISABLE", "ENABLE"	DISABLE	lane_sync_en overwrite signal. 0: lane_sync_en is controlled by input signal. 1: lane_sync_en is controlled by reg_lane_sync_en.
PMA_PLLy_REG_BIAS_D2S	<integer>	0 to 3	1	bias_d2s=reg_bias_d2s bias control for D2S in feedback path
PMA_PLLy_REG_BIAS_REFD2S_C	<integer>	0 to 3	1	bias_refd2s_c = reg_bias_refd2s_c bias control for reference clock D2S
PMA_PLLy_REG_BIAS_VCRST_C	<integer>	0 to 3	1	bias_vcrst_c = reg_bias_vcrst_c bias control for vcrst
PMA_PLLy_REG_BIAS_REFBUF_C	<integer>	0 to 3	1	bias_refbuf_c=reg_bias_refbuf_c bias control for rererence clock CML buffer input
PMA_PLLy_REG_CLKBUFS1_C	<integer>	0 to 3	1	clkbufs1_c=reg_clkbufs1_c CML clock buffer stage1 tail control
PMA_PLLy_REG_CLKBUFS2_C	<integer>	0 to 7	6	clkbufs2_c=reg_clkbufs2_c CML clock buffer stage2 tail control
PMA_PLLy_REG_CLKBUFS3_C	<integer>	0 to 7	6	clkbufs3_c=reg_clkbufs3_c CML clock buffer stage3 tail control
PMA_PLLy_REG_CLKB	<integer>	0 to 3	1	clkbufs4_c=reg_clkbufs4_c

参数名称	参数类型	有效值	默认值	功能描述
UFS4_C				CML clock buffer stage4 tail control
PMA_PLLy_REG_PLL_REFCLK_CML_SEL	<integer>	0 to 3	0	Refclk can be selected from 2 pairs input and adjacent populated refclk. Configuration register to select which refclk. 00: refclk is from local pad. 01: refclk is from pad of another PLL in the same quad. 10: refclk is from pad of adjacent quad. 11: reserved
PMA_PLLy_REG_REFCLK_SEL	<string>	"FALSE", "TRUE"	FALSE	select refclk is from pad or from digital internal 1'b0: 对应于参数值"FALSE"，refclk is from pad； 1'b1: 对应于参数值"TRUE" refclk is from digital。
PMA_PLLy_REG_RESCAL_R_CODE_SIGN	<string>	"FALSE", "TRUE"	TRUE	The highest bit of rescal_r_code 1'b0: 对应于参数值"FALSE"; 1'b1: 对应于参数值"TRUE"
PMA_PLLy_REG_PLL_UNLOCKED_OW	<string>	"DISABLE", "ENABLE"	DISABLE	Register overwrite control for pll_unlocked 1'b0: 对应于参数值"DISABLE"，pll_unlocked controlled by actual lock detection(default)； 1'b1: 对应于参数值"ENABLE" pll_lock mode is controlled by

参数名称	参数类型	有效值	默认值	功能描述
				reg_pll_unlocked。
PMA_PLLy_REG_PLL_UNLOCKED	<string>	"FALSE", "TRUE"	FALSE	Register value for overwriting pll_un locked status 1'b0: 对应于参数值"FALSE"，PLL unlock is detected； 1'b1: 对应于参数值"TRUE" PLL unlock is not detected。
PMA_PLLy_REG_PLL_LOCKDET_MODE	<string>	"FALSE", "TRUE"	FALSE	Select lockdet mode 1'b0: 对应于参数值"FALSE"; 1'b1: 对应于参数值"TRUE"
PMA_PLLy_REG_PLL_CLKBUF_PD_LEFT	<string>	"ON","OFF"	ON	Power down signal of left clock buffer. VCO output driver is divided into two parts, each driving two lanes. 1'b0: 对应于参数值"ON"; 1'b1: 对应于参数值"OFF".
PMA_PLLy_REG_PLL_CLKBUF_PD_RIGHT	<string>	"ON","OFF"	ON	Power down signal of right clock buffer.VCO output driver is divided into two parts, each driving two lanes.When lanes working at different datarates,PLL0’s left buffer power down and PLL1’s left buffer power on. When 4 lanes work at same datarate, PLL0 left and right buffer power on while PLL1 left buffer power down. 1'b0: 对应于参数值"ON";

参数名称	参数类型	有效值	默认值	功能描述
				1'b1: 对应于参数值"OFF".
PMA_PLLy_REG_RESCAL_EN	<string>	"FALSE", "TRUE"	FALSE	Register rescal_enable value 0: rescal disabled (default) 1: rescal enabled
PMA_PLLy_REG_RESCAL_I_CODE_VAL	<integer>	0 to 63	0	reg_rescal_i_code_val[3:0]
PMA_PLLy_REG_RESCAL_I_CODE_OW	<integer>	0 to 1	0	overwrite control of resistor calibration input code 0: rescal input code is from pma_rescal_i_code_i 1: rescal input code is from reg_rescal_i_code_val
PMA_PLLy_REG_RESCAL_ITER_VALID_SEL	<integer>	0 to 3	0	iteration number of valid resistor calibration (valid means calibration result is in reasonable range) 00: 64 01: 32 10: 16 11: 8
PMA_PLLy_REG_RESCAL_WAIT_SEL	<integer>	0 to 1	0	time interval of rescal_i_tune update 0: 32 refclk period 1: 64 refclk period
PMA_PLLy_REG_I_CTRL_MAX	<integer>	0 to 63	45	reg_i_ctrl_max[5:0]
PMA_PLLy_REG_I_CTRL_MIN	<integer>	0 to 63	19	reg_i_ctrl_min[5:0]
PMA_PLLy_REG_RESERVED_167_160	<integer>	0 to 255	0	[7:2]:0; [1]:带隙基准电路bgr11_ring

参数名称	参数类型	有效值	默认值	功能描述
				bypass enable [0]:pll_refcl_div2_en_o
PMA_PLLy_REG_RESERVED_175_168	<integer>	0 to 255	0	reserved
PMA_PLLy_REG_RESERVED_183_176	<integer>	0 to 255	0	reserved
PMA_PLLy_REG_RESERVED_191_184	<integer>	0 to 255	0	reserved
PARM_CFG_HSST_RSTN	<string>	"FALSE", "TRUE"	FALSE	配置PLL的cfg_pll_rstn 1'b0: 对应于参数值"FALSE"，低复位； 1'b1: 对应于参数值"TRUE"正常，不复位。
PARM_PLLy_POWERUP	<string>	"ON","OFF"	OFF	配置PLL的cfg_pllpowerup 1'b0: 对应于参数值"ON"; 1'b1: 对应于参数值"OFF".
PARM_PLLy_RSTN	<string>	"FALSE", "TRUE"	FALSE	配置cfg_hsst_rstn 1'b0: 对应于参数值"FALSE"，低复位； 1'b1: 对应于参数值"TRUE"正常，不复位。
PARM_CFG_RSTN	<string>	"FALSE", "TRUE"	FALSE	配置PARM_CFG_RSTN 1'b0: 对应于参数值"FALSE"，低复位； 1'b1: 对应于参数值"TRUE"正常，不复位。
PARM_GRSN_DIS	<string>	"FALSE", "TRUE"	FALSE	配置PARM_GRSN_DIS 1'b0: 对应于参数值"FALSE"，低复位； 1'b1: 对应于参数值"TRUE"

参数名称	参数类型	有效值	默认值	功能描述
				正常，不复位。

注：1.以上参数命名为“CHx”的信号，每个 Channel 各有一个信号：

CH0：For Channel 0

CH1：For Channel 1

CH2：For Channel 2

CH3：For Channel 3

2.以上参数命名为 PLLy”的信号，每个 PLL 各有一个信号：

PLL0: For PLL0

PLL1: For PLL1

9.5.5. 例化模板

```
GTP_HSST_E2 #(
    .PCS_CH0_BYPASS_WORD_ALIGN("FALSE"),
    .PCS_CH0_BYPASS_DENC("FALSE"),
    .PCS_CH0_BYPASS_BONDING("FALSE"),
    .PCS_CH0_BYPASS_CTC("FALSE"),
    .PCS_CH0_BYPASS_GEAR("FALSE"),
    .PCS_CH0_BYPASS_BRIDGE("FALSE"),
    .PCS_CH0_BYPASS_BRIDGE_FIFO("FALSE"),
    .PCS_CH0_DATA_MODE("X8"),
    .PCS_CH0_RX_POLARITY_INV("DELAY"),
    .PCS_CH0_ALIGN_MODE("1GB"),
    .PCS_CH0_SAMP_16B("X16"),
    .PCS_CH0_FARLP_PWR_REDUCTION("FALSE"),
    .PCS_CH0_COMMA_REG0(0),
    .PCS_CH0_COMMA_MASK(0),
    .PCS_CH0_CEB_MODE("10GB"),
    .PCS_CH0_CTC_MODE("1SKIP"),
    .PCS_CH0_A_REG(0),
    .PCS_CH0_GE_AUTO_EN("FALSE"),
    .PCS_CH0_SKIP_REG0(0),
    .PCS_CH0_SKIP_REG1(0),
    .PCS_CH0_SKIP_REG2(0),
    .PCS_CH0_SKIP_REG3(0),
    .PCS_CH0_DEC_DUAL("FALSE"),
    .PCS_CH0_SPLIT("FALSE"),
    .PCS_CH0_FIFOFLAG_CTC("FALSE"),
    .PCS_CH0_COMMA_DET_MODE("COMMA_PATTERN"),
    .PCS_CH0_ERRDETECT_SILENCE("FALSE"),
    .PCS_CH0_PMA_RCLK_POLINV("PMA_RCLK"),
```

```
.PCS_CH0_PCS_RCLK_SEL("PMA_RCLK"),
.PCS_CH0_CB_RCLK_SEL("PMA_RCLK"),
.PCS_CH0_AFTER_CTC_RCLK_SEL("PMA_RCLK"),
.PCS_CH0_AFTER_CTC_RCLK_SEL_1("PMA_RCLK"),
.PCS_CH0_RCLK_POLINV("RCLK"),
.PCS_CH0_BRIDGE_RCLK_SEL("PMA_RCLK"),
.PCS_CH0_PCS_RCLK_EN("FALSE"),
.PCS_CH0_CB_RCLK_EN("FALSE"),
.PCS_CH0_AFTER_CTC_RCLK_EN("FALSE"),
.PCS_CH0_AFTER_CTC_RCLK_EN_GB("FALSE"),
.PCS_CH0_AFTER_CTC_RCLK_EN_GB_1("FALSE"),
.PCS_CH0_PCS_RX_RSTN("FALSE"),
.PCS_CH0_SLAVE("MASTER"),
.PCS_CH0_PCIE_SLAVE("MASTER"),
.PCS_CH0_RX_64B66B_67B("NORMAL"),
.PCS_CH0_RX_BRIDGE_CLK_POLINV("RX_BRIDGE_CLK"),
.PCS_CH0_PCS_CB_RSTN("FALSE"),
.PCS_CH0_TX_BRIDGE_GEAR_SEL("FALSE"),
.PCS_CH0_TX_BYPASS_BRIDGE_UINT("FALSE"),
.PCS_CH0_TX_BYPASS_BRIDGE_FIFO("FALSE"),
.PCS_CH0_TX_BYPASS_GEAR("FALSE"),
.PCS_CH0_TX_BYPASS_ENC("FALSE"),
.PCS_CH0_TX_BYPASS_BIT_SLIP("FALSE"),
.PCS_CH0_TX_GEAR_SPLIT("FALSE"),
.PCS_CH0_TX_DRIVE_REG_MODE("NO_CHANGE"),
.PCS_CH0_TX_BIT_SLIP_CYCLES(0),
.PCS_CH0_INT_TX_MASK_0("FALSE"),
.PCS_CH0_INT_TX_MASK_1("FALSE"),
.PCS_CH0_INT_TX_MASK_2("FALSE"),
.PCS_CH0_INT_TX_CLR_0("FALSE"),
.PCS_CH0_INT_TX_CLR_1("FALSE"),
.PCS_CH0_INT_TX_CLR_2("FALSE"),
.PCS_CH0_TX_PMA_TCLK_POLINV("PMA_TCLK"),
.PCS_CH0_TX_PCS_CLK_EN_SEL("FALSE"),
.PCS_CH0_TX_BRIDGE_TCLK_SEL("TCLK"),
.PCS_CH0_TX_TCLK_POLINV("TCLK"),
.PCS_CH0_TX_PCS_TCLK_SEL("PMA_TCLK"),
.PCS_CH0_TX_PCS_TX_RSTN("FALSE"),
.PCS_CH0_TX_SLAVE("MASTER"),
.PCS_CH0_TX_GEAR_TCLK_EN_SEL("FALSE"),
.PCS_CH0_DATA_WIDTH_MODE("X20"),
.PCS_CH0_TX_64B66B_67B("NORMAL"),
.PCS_CH0_TX_GEAR_TCLK_SEL("PMA_TCLK"),
.PCS_CH0_TX_TCLK2FABRIC_SEL("FALSE"),
.PCS_CH0_TX_OUTZZ("FALSE"),
.PCS_CH0_ENC_DUAL("FALSE"),
.PCS_CH0_TX_BITSLIP_DATA_MODE("X10"),
```

```
.PCS_CH0_TX_BRIDGE_CLK_POLINV("TX_BRIDGE_CLK"),
.PCS_CH0_COMMA_REG1(0),
.PCS_CH0_RAPID_IMAX(0),
.PCS_CH0_RAPID_VMIN_1(0),
.PCS_CH0_RAPID_VMIN_2(0),
.PCS_CH0_RX_PRBS_MODE("DISABLE"),
.PCS_CH0_RX_ERRCNT_CLR("FALSE"),
.PCS_CH0_RX_PRBS_ERR_LPBK("FALSE"),
.PCS_CH0_TX_PRBS_MODE("DISABLE"),
.PCS_CH0_TX_INSERT_ER("FALSE"),
.PCS_CH0_ENABLE_PRBS_GEN("FALSE"),
.PCS_CH0_ERR_CNT(0),
.PCS_CH0_DEFAULT_RADDR(0),
.PCS_CH0_MASTER_CHECK_OFFSET(0),
.PCS_CH0_DELAY_SET(0),
.PCS_CH0_SEACH_OFFSET("20BIT"),
.PCS_CH0_CEB_RAPIDLS_MMAX(0),
.PCS_CH0_CTC_AFULL(0),
.PCS_CH0_CTC_AEMPTY(0),
.PCS_CH0_CTC_CONTI_SKP_SET(0),
.PCS_CH0_FAR_LOOP("FALSE"),
.PCS_CH0_NEAR_LOOP("FALSE"),
.PCS_CH0_REG_TX2RX_PLOOP_EN("FALSE"),
.PCS_CH0_REG_TX2RX_SLOOP_EN("FALSE"),
.PCS_CH0_REG_RX2TX_PLOOP_EN("FALSE"),
.PCS_CH0_INT_RX_MASK_0("FALSE"),
.PCS_CH0_INT_RX_MASK_1("FALSE"),
.PCS_CH0_INT_RX_MASK_2("FALSE"),
.PCS_CH0_INT_RX_MASK_3("FALSE"),
.PCS_CH0_INT_RX_MASK_4("FALSE"),
.PCS_CH0_INT_RX_MASK_5("FALSE"),
.PCS_CH0_INT_RX_MASK_6("FALSE"),
.PCS_CH0_INT_RX_MASK_7("FALSE"),
.PCS_CH0_INT_RX_CLR_0("FALSE"),
.PCS_CH0_INT_RX_CLR_1("FALSE"),
.PCS_CH0_INT_RX_CLR_2("FALSE"),
.PCS_CH0_INT_RX_CLR_3("FALSE"),
.PCS_CH0_INT_RX_CLR_4("FALSE"),
.PCS_CH0_INT_RX_CLR_5("FALSE"),
.PCS_CH0_INT_RX_CLR_6("FALSE"),
.PCS_CH0_INT_RX_CLR_7("FALSE"),
.PCS_CH0_CA_RSTN_RX("FALSE"),
.PCS_CH0_CA_DYN_DLY_EN_RX("FALSE"),
.PCS_CH0_CA_DYN_DLY_SEL_RX("FALSE"),
.PCS_CH0_CA_RX(0),
.PCS_CH0_CA_RSTN_TX("FALSE"),
.PCS_CH0_CA_DYN_DLY_EN_TX("FALSE"),
```

```
.PCS_CH0_CA_DYN_DLY_SEL_TX("FALSE"),
.PCS_CH0_CA_TX(0),
.PCS_CH0_RXPRBS_PWR_REDUCTION("NORMAL"),
.PCS_CH0_WDALIGN_PWR_REDUCTION("NORMAL"),
.PCS_CH0_RXDEC_PWR_REDUCTION("NORMAL"),
.PCS_CH0_RXCB_PWR_REDUCTION("NORMAL"),
.PCS_CH0_RXCTC_PWR_REDUCTION("NORMAL"),
.PCS_CH0_RXGEAR_PWR_REDUCTION("NORMAL"),
.PCS_CH0_RXBRG_PWR_REDUCTION("NORMAL"),
.PCS_CH0_RXTEST_PWR_REDUCTION("NORMAL"),
.PCS_CH0_TXBRG_PWR_REDUCTION("NORMAL"),
.PCS_CH0_TXGEAR_PWR_REDUCTION("NORMAL"),
.PCS_CH0_TXENC_PWR_REDUCTION("NORMAL"),
.PCS_CH0_TXBSLP_PWR_REDUCTION("NORMAL"),
.PCS_CH0_TXPRBS_PWR_REDUCTION("NORMAL"),
.PCS_CH0_TXBRG_FULL_CHK_EN("FALSE"),
.PCS_CH0_TXBRG_EMPTY_CHK_EN("FALSE"),
.PCS_CH0_RXBRG_FULL_CHK_EN("FALSE"),
.PCS_CH0_RXBRG_EMPTY_CHK_EN("FALSE"),
.PCS_CH0_CTC_FULL_CHK_EN("TRUE"),
.PCS_CH0_CTC_EMPTY_CHK_EN("TRUE"),
.PCS_CH0_CEB_FULL_CHK_EN("FALSE"),
.PCS_CH0_CEB_EMPTY_CHK_EN("FALSE"),
.PCS_CH0_FLP_FULL_CHK_EN("TRUE"),
.PCS_CH0_FLP_EMPTY_CHK_EN("TRUE"),
.PCS_CH1_BYPASS_WORD_ALIGN("FALSE"),
.PCS_CH1_BYPASS_DENC("FALSE"),
.PCS_CH1_BYPASS_BONDING("FALSE"),
.PCS_CH1_BYPASS_CTC("FALSE"),
.PCS_CH1_BYPASS_GEAR("FALSE"),
.PCS_CH1_BYPASS_BRIDGE("FALSE"),
.PCS_CH1_BYPASS_BRIDGE_FIFO("FALSE"),
.PCS_CH1_DATA_MODE("X8"),
.PCS_CH1_RX_POLARITY_INV("DELAY"),
.PCS_CH1_ALIGN_MODE("1GB"),
.PCS_CH1_SAMP_16B("X16"),
.PCS_CH1_FARLP_PWR_REDUCTION("FALSE"),
.PCS_CH1_COMMA_REG0(0),
.PCS_CH1_COMMA_MASK(0),
.PCS_CH1_CEB_MODE("10GB"),
.PCS_CH1_CTC_MODE("1SKIP"),
.PCS_CH1_A_REG(0),
.PCS_CH1_GE_AUTO_EN("FALSE"),
.PCS_CH1_SKIP_REG0(0),
.PCS_CH1_SKIP_REG1(0),
.PCS_CH1_SKIP_REG2(0),
.PCS_CH1_SKIP_REG3(0),
```

```
.PCS_CH1_DEC_DUAL("FALSE"),
.PCS_CH1_SPLIT("FALSE"),
.PCS_CH1_FIFOFLAG_CTC("FALSE"),
.PCS_CH1_COMMA_DET_MODE("COMMA_PATTERN"),
.PCS_CH1_ERRDETECT_SILENCE("FALSE"),
.PCS_CH1_PMA_RCLK_POLINV("PMA_RCLK"),
.PCS_CH1_PCS_RCLK_SEL("PMA_RCLK"),
.PCS_CH1_CB_RCLK_SEL("PMA_RCLK"),
.PCS_CH1_AFTER_CTC_RCLK_SEL("PMA_RCLK"),
.PCS_CH1_AFTER_CTC_RCLK_SEL_1("PMA_RCLK"),
.PCS_CH1_RCLK_POLINV("RCLK"),
.PCS_CH1_BRIDGE_RCLK_SEL("PMA_RCLK"),
.PCS_CH1_PCS_RCLK_EN("FALSE"),
.PCS_CH1_CB_RCLK_EN("FALSE"),
.PCS_CH1_AFTER_CTC_RCLK_EN("FALSE"),
.PCS_CH1_AFTER_CTC_RCLK_EN_GB("FALSE"),
.PCS_CH1_AFTER_CTC_RCLK_EN_GB_1("FALSE"),
.PCS_CH1_PCS_RX_RSTN("FALSE"),
.PCS_CH1_SLAVE("MASTER"),
.PCS_CH1_PCIE_SLAVE("MASTER"),
.PCS_CH1_RX_64B66B_67B("NORMAL"),
.PCS_CH1_RX_BRIDGE_CLK_POLINV("RX_BRIDGE_CLK"),
.PCS_CH1_PCS_CB_RSTN("FALSE"),
.PCS_CH1_TX_BRIDGE_GEAR_SEL("FALSE"),
.PCS_CH1_TX_BYPASS_BRIDGE_UINT("FALSE"),
.PCS_CH1_TX_BYPASS_BRIDGE_FIFO("FALSE"),
.PCS_CH1_TX_BYPASS_GEAR("FALSE"),
.PCS_CH1_TX_BYPASS_ENC("FALSE"),
.PCS_CH1_TX_BYPASS_BIT_SLIP("FALSE"),
.PCS_CH1_TX_GEAR_SPLIT("FALSE"),
.PCS_CH1_TX_DRIVE_REG_MODE("NO_CHANGE"),
.PCS_CH1_TX_BIT_SLIP_CYCLES(0),
.PCS_CH1_INT_TX_MASK_0("FALSE"),
.PCS_CH1_INT_TX_MASK_1("FALSE"),
.PCS_CH1_INT_TX_MASK_2("FALSE"),
.PCS_CH1_INT_TX_CLR_0("FALSE"),
.PCS_CH1_INT_TX_CLR_1("FALSE"),
.PCS_CH1_INT_TX_CLR_2("FALSE"),
.PCS_CH1_TX_PMA_TCLK_POLINV("PMA_TCLK"),
.PCS_CH1_TX_PCS_CLK_EN_SEL("FALSE"),
.PCS_CH1_TX_BRIDGE_TCLK_SEL("TCLK"),
.PCS_CH1_TX_TCLK_POLINV("TCLK"),
.PCS_CH1_TX_PCS_TCLK_SEL("PMA_TCLK"),
.PCS_CH1_TX_PCS_TX_RSTN("FALSE"),
.PCS_CH1_TX_SLAVE("MASTER"),
.PCS_CH1_TX_GEAR_TCLK_EN_SEL("FALSE"),
.PCS_CH1_DATA_WIDTH_MODE("X20"),
```

```
.PCS_CH1_TX_64B66B_67B("NORMAL"),
.PCS_CH1_TX_GEAR_TCLK_SEL("PMA_TCLK"),
.PCS_CH1_TX_TCLK2FABRIC_SEL("FALSE"),
.PCS_CH1_TX_OUTZZ("FALSE"),
.PCS_CH1_ENC_DUAL("FALSE"),
.PCS_CH1_TX_BITSLIP_DATA_MODE("X10"),
.PCS_CH1_TX_BRIDGE_CLK_POLINV("TX_BRIDGE_CLK"),
.PCS_CH1_COMMA_REG1(0),
.PCS_CH1_RAPID_IMAX(0),
.PCS_CH1_RAPID_VMIN_1(0),
.PCS_CH1_RAPID_VMIN_2(0),
.PCS_CH1_RX_PRBS_MODE("DISABLE"),
.PCS_CH1_RX_ERRCNT_CLR("FALSE"),
.PCS_CH1_RX_PRBS_ERR_LPBK("FALSE"),
.PCS_CH1_TX_PRBS_MODE("DISABLE"),
.PCS_CH1_TX_INSERT_ER("FALSE"),
.PCS_CH1_ENABLE_PRBS_GEN("FALSE"),
.PCS_CH1_ERR_CNT(0),
.PCS_CH1_DEFAULT_RADDR(0),
.PCS_CH1_MASTER_CHECK_OFFSET(0),
.PCS_CH1_DELAY_SET(0),
.PCS_CH1_SEACH_OFFSET("20BIT"),
.PCS_CH1_CEB_RAPIDLS_MMAX(0),
.PCS_CH1_CTC_AFULL(0),
.PCS_CH1_CTC_AEMPTY(0),
.PCS_CH1_CTC_CONTI_SKP_SET(0),
.PCS_CH1_FAR_LOOP("FALSE"),
.PCS_CH1_NEAR_LOOP("FALSE"),
.PCS_CH1_REG_TX2RX_PLOOP_EN("FALSE"),
.PCS_CH1_REG_TX2RX_SLOOP_EN("FALSE"),
.PCS_CH1_REG_RX2TX_PLOOP_EN("FALSE"),
.PCS_CH1_INT_RX_MASK_0("FALSE"),
.PCS_CH1_INT_RX_MASK_1("FALSE"),
.PCS_CH1_INT_RX_MASK_2("FALSE"),
.PCS_CH1_INT_RX_MASK_3("FALSE"),
.PCS_CH1_INT_RX_MASK_4("FALSE"),
.PCS_CH1_INT_RX_MASK_5("FALSE"),
.PCS_CH1_INT_RX_MASK_6("FALSE"),
.PCS_CH1_INT_RX_MASK_7("FALSE"),
.PCS_CH1_INT_RX_CLR_0("FALSE"),
.PCS_CH1_INT_RX_CLR_1("FALSE"),
.PCS_CH1_INT_RX_CLR_2("FALSE"),
.PCS_CH1_INT_RX_CLR_3("FALSE"),
.PCS_CH1_INT_RX_CLR_4("FALSE"),
.PCS_CH1_INT_RX_CLR_5("FALSE"),
.PCS_CH1_INT_RX_CLR_6("FALSE"),
.PCS_CH1_INT_RX_CLR_7("FALSE"),
```

```
.PCS_CH1_CA_RSTN_RX("FALSE"),
.PCS_CH1_CA_DYN_DLY_EN_RX("FALSE"),
.PCS_CH1_CA_DYN_DLY_SEL_RX("FALSE"),
.PCS_CH1_CA_RX(0),
.PCS_CH1_CA_RSTN_TX("FALSE"),
.PCS_CH1_CA_DYN_DLY_EN_TX("FALSE"),
.PCS_CH1_CA_DYN_DLY_SEL_TX("FALSE"),
.PCS_CH1_CA_TX(0),
.PCS_CH1_RXPRBS_PWR_REDUCTION("NORMAL"),
.PCS_CH1_WDALIGN_PWR_REDUCTION("NORMAL"),
.PCS_CH1_RXDEC_PWR_REDUCTION("NORMAL"),
.PCS_CH1_RXCB_PWR_REDUCTION("NORMAL"),
.PCS_CH1_RXCTC_PWR_REDUCTION("NORMAL"),
.PCS_CH1_RXGEAR_PWR_REDUCTION("NORMAL"),
.PCS_CH1_RXBRG_PWR_REDUCTION("NORMAL"),
.PCS_CH1_RXTEST_PWR_REDUCTION("NORMAL"),
.PCS_CH1_TXBRG_PWR_REDUCTION("NORMAL"),
.PCS_CH1_TXGEAR_PWR_REDUCTION("NORMAL"),
.PCS_CH1_TXENC_PWR_REDUCTION("NORMAL"),
.PCS_CH1_TXBSLP_PWR_REDUCTION("NORMAL"),
.PCS_CH1_TXPRBS_PWR_REDUCTION("NORMAL"),
.PCS_CH1_TXBRG_FULL_CHK_EN("FALSE"),
.PCS_CH1_TXBRG_EMPTY_CHK_EN("FALSE"),
.PCS_CH1_RXBRG_FULL_CHK_EN("FALSE"),
.PCS_CH1_RXBRG_EMPTY_CHK_EN("FALSE"),
.PCS_CH1_CTC_FULL_CHK_EN("TRUE"),
.PCS_CH1_CTC_EMPTY_CHK_EN("TRUE"),
.PCS_CH1_CEB_FULL_CHK_EN("FALSE"),
.PCS_CH1_CEB_EMPTY_CHK_EN("FALSE"),
.PCS_CH1_FLP_FULL_CHK_EN("TRUE"),
.PCS_CH1_FLP_EMPTY_CHK_EN("TRUE"),
.PCS_CH2_BYPASS_WORD_ALIGN("FALSE"),
.PCS_CH2_BYPASS_DENC("FALSE"),
.PCS_CH2_BYPASS_BONDING("FALSE"),
.PCS_CH2_BYPASS_CTC("FALSE"),
.PCS_CH2_BYPASS_GEAR("FALSE"),
.PCS_CH2_BYPASS_BRIDGE("FALSE"),
.PCS_CH2_BYPASS_BRIDGE_FIFO("FALSE"),
.PCS_CH2_DATA_MODE("X8"),
.PCS_CH2_RX_POLARITY_INV("DELAY"),
.PCS_CH2_ALIGN_MODE("1GB"),
.PCS_CH2_SAMP_16B("X16"),
.PCS_CH2_FARLP_PWR_REDUCTION("FALSE"),
.PCS_CH2_COMMA_REG0(0),
.PCS_CH2_COMMA_MASK(0),
.PCS_CH2_CEB_MODE("10GB"),
.PCS_CH2_CTC_MODE("1SKIP"),
```

```
.PCS_CH2_A_REG(0),
.PCS_CH2_GE_AUTO_EN("FALSE"),
.PCS_CH2_SKIP_REG0(0),
.PCS_CH2_SKIP_REG1(0),
.PCS_CH2_SKIP_REG2(0),
.PCS_CH2_SKIP_REG3(0),
.PCS_CH2_DEC_DUAL("FALSE"),
.PCS_CH2_SPLIT("FALSE"),
.PCS_CH2_FIFOFLAG_CTC("FALSE"),
.PCS_CH2_COMMA_DET_MODE("COMMA_PATTERN"),
.PCS_CH2_ERRDETECT_SILENCE("FALSE"),
.PCS_CH2_PMA_RCLK_POLINV("PMA_RCLK"),
.PCS_CH2_PCS_RCLK_SEL("PMA_RCLK"),
.PCS_CH2_CB_RCLK_SEL("PMA_RCLK"),
.PCS_CH2_AFTER_CTC_RCLK_SEL("PMA_RCLK"),
.PCS_CH2_AFTER_CTC_RCLK_SEL_1("PMA_RCLK"),
.PCS_CH2_RCLK_POLINV("RCLK"),
.PCS_CH2_BRIDGE_RCLK_SEL("PMA_RCLK"),
.PCS_CH2_PCS_RCLK_EN("FALSE"),
.PCS_CH2_CB_RCLK_EN("FALSE"),
.PCS_CH2_AFTER_CTC_RCLK_EN("FALSE"),
.PCS_CH2_AFTER_CTC_RCLK_EN_GB("FALSE"),
.PCS_CH2_AFTER_CTC_RCLK_EN_GB_1("FALSE"),
.PCS_CH2_PCS_RX_RSTN("FALSE"),
.PCS_CH2_SLAVE("MASTER"),
.PCS_CH2_PCIE_SLAVE("MASTER"),
.PCS_CH2_RX_64B66B_67B("NORMAL"),
.PCS_CH2_RX_BRIDGE_CLK_POLINV("RX_BRIDGE_CLK"),
.PCS_CH2_PCS_CB_RSTN("FALSE"),
.PCS_CH2_TX_BRIDGE_GEAR_SEL("FALSE"),
.PCS_CH2_TX_BYPASS_BRIDGE_UINT("FALSE"),
.PCS_CH2_TX_BYPASS_BRIDGE_FIFO("FALSE"),
.PCS_CH2_TX_BYPASS_GEAR("FALSE"),
.PCS_CH2_TX_BYPASS_ENC("FALSE"),
.PCS_CH2_TX_BYPASS_BIT_SLIP("FALSE"),
.PCS_CH2_TX_GEAR_SPLIT("FALSE"),
.PCS_CH2_TX_DRIVE_REG_MODE("NO_CHANGE"),
.PCS_CH2_TX_BIT_SLIP_CYCLES(0),
.PCS_CH2_INT_TX_MASK_0("FALSE"),
.PCS_CH2_INT_TX_MASK_1("FALSE"),
.PCS_CH2_INT_TX_MASK_2("FALSE"),
.PCS_CH2_INT_TX_CLR_0("FALSE"),
.PCS_CH2_INT_TX_CLR_1("FALSE"),
.PCS_CH2_INT_TX_CLR_2("FALSE"),
.PCS_CH2_TX_PMA_TCLK_POLINV("PMA_TCLK"),
.PCS_CH2_TX_PCS_CLK_EN_SEL("FALSE"),
.PCS_CH2_TX_BRIDGE_TCLK_SEL("TCLK"),
```

```
.PCS_CH2_TX_TCLK_POLINV("TCLK"),
.PCS_CH2_TX_PCS_TCLK_SEL("PMA_TCLK"),
.PCS_CH2_TX_PCS_TX_RSTN("FALSE"),
.PCS_CH2_TX_SLAVE("MASTER"),
.PCS_CH2_TX_GEAR_TCLK_EN_SEL("FALSE"),
.PCS_CH2_DATA_WIDTH_MODE("X20"),
.PCS_CH2_TX_64B66B_67B("NORMAL"),
.PCS_CH2_TX_GEAR_TCLK_SEL("PMA_TCLK"),
.PCS_CH2_TX_TCLK2FABRIC_SEL("FALSE"),
.PCS_CH2_TX_OUTZZ("FALSE"),
.PCS_CH2_ENC_DUAL("FALSE"),
.PCS_CH2_TX_BITSLIP_DATA_MODE("X10"),
.PCS_CH2_TX_BRIDGE_CLK_POLINV("TX_BRIDGE_CLK"),
.PCS_CH2_COMMA_REG1(0),
.PCS_CH2_RAPID_IMAX(0),
.PCS_CH2_RAPID_VMIN_1(0),
.PCS_CH2_RAPID_VMIN_2(0),
.PCS_CH2_RX_PRBS_MODE("DISABLE"),
.PCS_CH2_RX_ERRCNT_CLR("FALSE"),
.PCS_CH2_RX_PRBS_ERR_LPBK("FALSE"),
.PCS_CH2_TX_PRBS_MODE("DISABLE"),
.PCS_CH2_TX_INSERT_ER("FALSE"),
.PCS_CH2_ENABLE_PRBS_GEN("FALSE"),
.PCS_CH2_ERR_CNT(0),
.PCS_CH2_DEFAULT_RADDR(0),
.PCS_CH2_MASTER_CHECK_OFFSET(0),
.PCS_CH2_DELAY_SET(0),
.PCS_CH2_SEACH_OFFSET("20BIT"),
.PCS_CH2_CEB_RAPIDLS_MMAX(0),
.PCS_CH2_CTC_AFULL(0),
.PCS_CH2_CTC_AEMPTY(0),
.PCS_CH2_CTC_CONTI_SKP_SET(0),
.PCS_CH2_FAR_LOOP("FALSE"),
.PCS_CH2_NEAR_LOOP("FALSE"),
.PCS_CH2_REG_TX2RX_PLOOP_EN("FALSE"),
.PCS_CH2_REG_TX2RX_SLOOP_EN("FALSE"),
.PCS_CH2_REG_RX2TX_PLOOP_EN("FALSE"),
.PCS_CH2_INT_RX_MASK_0("FALSE"),
.PCS_CH2_INT_RX_MASK_1("FALSE"),
.PCS_CH2_INT_RX_MASK_2("FALSE"),
.PCS_CH2_INT_RX_MASK_3("FALSE"),
.PCS_CH2_INT_RX_MASK_4("FALSE"),
.PCS_CH2_INT_RX_MASK_5("FALSE"),
.PCS_CH2_INT_RX_MASK_6("FALSE"),
.PCS_CH2_INT_RX_MASK_7("FALSE"),
.PCS_CH2_INT_RX_CLR_0("FALSE"),
.PCS_CH2_INT_RX_CLR_1("FALSE"),
```

```
.PCS_CH2_INT_RX_CLR_2("FALSE"),
.PCS_CH2_INT_RX_CLR_3("FALSE"),
.PCS_CH2_INT_RX_CLR_4("FALSE"),
.PCS_CH2_INT_RX_CLR_5("FALSE"),
.PCS_CH2_INT_RX_CLR_6("FALSE"),
.PCS_CH2_INT_RX_CLR_7("FALSE"),
.PCS_CH2_CA_RSTN_RX("FALSE"),
.PCS_CH2_CA_DYN_DLY_EN_RX("FALSE"),
.PCS_CH2_CA_DYN_DLY_SEL_RX("FALSE"),
.PCS_CH2_CA_RX(0),
.PCS_CH2_CA_RSTN_TX("FALSE"),
.PCS_CH2_CA_DYN_DLY_EN_TX("FALSE"),
.PCS_CH2_CA_DYN_DLY_SEL_TX("FALSE"),
.PCS_CH2_CA_TX(0),
.PCS_CH2_RXPRBS_PWR_REDUCTION("NORMAL"),
.PCS_CH2_WDALIGN_PWR_REDUCTION("NORMAL"),
.PCS_CH2_RXDEC_PWR_REDUCTION("NORMAL"),
.PCS_CH2_RXCB_PWR_REDUCTION("NORMAL"),
.PCS_CH2_RXCTC_PWR_REDUCTION("NORMAL"),
.PCS_CH2_RXGEAR_PWR_REDUCTION("NORMAL"),
.PCS_CH2_RXBRG_PWR_REDUCTION("NORMAL"),
.PCS_CH2_RXTEST_PWR_REDUCTION("NORMAL"),
.PCS_CH2_TXBRG_PWR_REDUCTION("NORMAL"),
.PCS_CH2_TXGEAR_PWR_REDUCTION("NORMAL"),
.PCS_CH2_TXENC_PWR_REDUCTION("NORMAL"),
.PCS_CH2_TXBSLP_PWR_REDUCTION("NORMAL"),
.PCS_CH2_TXPRBS_PWR_REDUCTION("NORMAL"),
.PCS_CH2_TXBRG_FULL_CHK_EN("FALSE"),
.PCS_CH2_TXBRG_EMPTY_CHK_EN("FALSE"),
.PCS_CH2_RXBRG_FULL_CHK_EN("FALSE"),
.PCS_CH2_RXBRG_EMPTY_CHK_EN("FALSE"),
.PCS_CH2_CTC_FULL_CHK_EN("TRUE"),
.PCS_CH2_CTC_EMPTY_CHK_EN("TRUE"),
.PCS_CH2_CEB_FULL_CHK_EN("FALSE"),
.PCS_CH2_CEB_EMPTY_CHK_EN("FALSE"),
.PCS_CH2_FLP_FULL_CHK_EN("TRUE"),
.PCS_CH2_FLP_EMPTY_CHK_EN("TRUE"),
.PCS_CH3_BYPASS_WORD_ALIGN("FALSE"),
.PCS_CH3_BYPASS_DENC("FALSE"),
.PCS_CH3_BYPASS_BONDING("FALSE"),
.PCS_CH3_BYPASS_CTC("FALSE"),
.PCS_CH3_BYPASS_GEAR("FALSE"),
.PCS_CH3_BYPASS_BRIDGE("FALSE"),
.PCS_CH3_BYPASS_BRIDGE_FIFO("FALSE"),
.PCS_CH3_DATA_MODE("X8"),
.PCS_CH3_RX_POLARITY_INV("DELAY"),
.PCS_CH3_ALIGN_MODE("1GB"),
```

```
.PCS_CH3_SAMP_16B("X16"),
.PCS_CH3_FARLP_PWR_REDUCTION("FALSE"),
.PCS_CH3_COMMA_REG0(0),
.PCS_CH3_COMMA_MASK(0),
.PCS_CH3_CEB_MODE("10GB"),
.PCS_CH3_CTC_MODE("1SKIP"),
.PCS_CH3_A_REG(0),
.PCS_CH3_GE_AUTO_EN("FALSE"),
.PCS_CH3_SKIP_REG0(0),
.PCS_CH3_SKIP_REG1(0),
.PCS_CH3_SKIP_REG2(0),
.PCS_CH3_SKIP_REG3(0),
.PCS_CH3_DEC_DUAL("FALSE"),
.PCS_CH3_SPLIT("FALSE"),
.PCS_CH3_FIFOFLAG_CTC("FALSE"),
.PCS_CH3_COMMA_DET_MODE("COMMA_PATTERN"),
.PCS_CH3_ERRDETECT_SILENCE("FALSE"),
.PCS_CH3_PMA_RCLK_POLINV("PMA_RCLK"),
.PCS_CH3_PCS_RCLK_SEL("PMA_RCLK"),
.PCS_CH3_CB_RCLK_SEL("PMA_RCLK"),
.PCS_CH3_AFTER_CTC_RCLK_SEL("PMA_RCLK"),
.PCS_CH3_AFTER_CTC_RCLK_SEL_1("PMA_RCLK"),
.PCS_CH3_RCLK_POLINV("RCLK"),
.PCS_CH3_BRIDGE_RCLK_SEL("PMA_RCLK"),
.PCS_CH3_PCS_RCLK_EN("FALSE"),
.PCS_CH3_CB_RCLK_EN("FALSE"),
.PCS_CH3_AFTER_CTC_RCLK_EN("FALSE"),
.PCS_CH3_AFTER_CTC_RCLK_EN_GB("FALSE"),
.PCS_CH3_AFTER_CTC_RCLK_EN_GB_1("FALSE"),
.PCS_CH3_PCS_RX_RSTN("FALSE"),
.PCS_CH3_SLAVE("MASTER"),
.PCS_CH3_PCIE_SLAVE("MASTER"),
.PCS_CH3_RX_64B66B_67B("NORMAL"),
.PCS_CH3_RX_BRIDGE_CLK_POLINV("RX_BRIDGE_CLK"),
.PCS_CH3_PCS_CB_RSTN("FALSE"),
.PCS_CH3_TX_BRIDGE_GEAR_SEL("FALSE"),
.PCS_CH3_TX_BYPASS_BRIDGE_UINT("FALSE"),
.PCS_CH3_TX_BYPASS_BRIDGE_FIFO("FALSE"),
.PCS_CH3_TX_BYPASS_GEAR("FALSE"),
.PCS_CH3_TX_BYPASS_ENC("FALSE"),
.PCS_CH3_TX_BYPASS_BIT_SLIP("FALSE"),
.PCS_CH3_TX_GEAR_SPLIT("FALSE"),
.PCS_CH3_TX_DRIVE_REG_MODE("NO_CHANGE"),
.PCS_CH3_TX_BIT_SLIP_CYCLES(0),
.PCS_CH3_INT_TX_MASK_0("FALSE"),
.PCS_CH3_INT_TX_MASK_1("FALSE"),
.PCS_CH3_INT_TX_MASK_2("FALSE"),
```

```
.PCS_CH3_INT_TX_CLR_0("FALSE"),
.PCS_CH3_INT_TX_CLR_1("FALSE"),
.PCS_CH3_INT_TX_CLR_2("FALSE"),
.PCS_CH3_TX_PMA_TCLK_POLINV("PMA_TCLK"),
.PCS_CH3_TX_PCS_CLK_EN_SEL("FALSE"),
.PCS_CH3_TX_BRIDGE_TCLK_SEL("TCLK"),
.PCS_CH3_TX_TCLK_POLINV("TCLK"),
.PCS_CH3_TX_PCS_TCLK_SEL("PMA_TCLK"),
.PCS_CH3_TX_PCS_TX_RSTN("FALSE"),
.PCS_CH3_TX_SLAVE("MASTER"),
.PCS_CH3_TX_GEAR_TCLK_EN_SEL("FALSE"),
.PCS_CH3_DATA_WIDTH_MODE("X20"),
.PCS_CH3_TX_64B66B_67B("NORMAL"),
.PCS_CH3_TX_GEAR_TCLK_SEL("PMA_TCLK"),
.PCS_CH3_TX_TCLK2FABRIC_SEL("FALSE"),
.PCS_CH3_TX_OUTZZ("FALSE"),
.PCS_CH3_ENC_DUAL("FALSE"),
.PCS_CH3_TX_BITSLIP_DATA_MODE("X10"),
.PCS_CH3_TX_BRIDGE_CLK_POLINV("TX_BRIDGE_CLK"),
.PCS_CH3_COMMA_REG1(0),
.PCS_CH3_RAPID_IMAX(0),
.PCS_CH3_RAPID_VMIN_1(0),
.PCS_CH3_RAPID_VMIN_2(0),
.PCS_CH3_RX_PRBS_MODE("DISABLE"),
.PCS_CH3_RX_ERRCNT_CLR("FALSE"),
.PCS_CH3_RX_PRBS_ERR_LPBK("FALSE"),
.PCS_CH3_TX_PRBS_MODE("DISABLE"),
.PCS_CH3_TX_INSERT_ER("FALSE"),
.PCS_CH3_ENABLE_PRBS_GEN("FALSE"),
.PCS_CH3_ERR_CNT(0),
.PCS_CH3_DEFAULT_RADDR(0),
.PCS_CH3_MASTER_CHECK_OFFSET(0),
.PCS_CH3_DELAY_SET(0),
.PCS_CH3_SEACH_OFFSET("20BIT"),
.PCS_CH3_CEB_RAPIDLS_MMAX(0),
.PCS_CH3_CTC_AFULL(0),
.PCS_CH3_CTC_AEMPTY(0),
.PCS_CH3_CTC_CONTI_SKP_SET(0),
.PCS_CH3_FAR_LOOP("FALSE"),
.PCS_CH3_NEAR_LOOP("FALSE"),
.PCS_CH3_REG_TX2RX_PLOOP_EN("FALSE"),
.PCS_CH3_REG_TX2RX_SLOOP_EN("FALSE"),
.PCS_CH3_REG_RX2TX_PLOOP_EN("FALSE"),
.PCS_CH3_INT_RX_MASK_0("FALSE"),
.PCS_CH3_INT_RX_MASK_1("FALSE"),
.PCS_CH3_INT_RX_MASK_2("FALSE"),
.PCS_CH3_INT_RX_MASK_3("FALSE"),
```

```
.PCS_CH3_INT_RX_MASK_4("FALSE"),
.PCS_CH3_INT_RX_MASK_5("FALSE"),
.PCS_CH3_INT_RX_MASK_6("FALSE"),
.PCS_CH3_INT_RX_MASK_7("FALSE"),
.PCS_CH3_INT_RX_CLR_0("FALSE"),
.PCS_CH3_INT_RX_CLR_1("FALSE"),
.PCS_CH3_INT_RX_CLR_2("FALSE"),
.PCS_CH3_INT_RX_CLR_3("FALSE"),
.PCS_CH3_INT_RX_CLR_4("FALSE"),
.PCS_CH3_INT_RX_CLR_5("FALSE"),
.PCS_CH3_INT_RX_CLR_6("FALSE"),
.PCS_CH3_INT_RX_CLR_7("FALSE"),
.PCS_CH3_CA_RSTN_RX("FALSE"),
.PCS_CH3_CA_DYN_DLY_EN_RX("FALSE"),
.PCS_CH3_CA_DYN_DLY_SEL_RX("FALSE"),
.PCS_CH3_CA_RX(0),
.PCS_CH3_CA_RSTN_TX("FALSE"),
.PCS_CH3_CA_DYN_DLY_EN_TX("FALSE"),
.PCS_CH3_CA_DYN_DLY_SEL_TX("FALSE"),
.PCS_CH3_CA_TX(0),
.PCS_CH3_RXPRBS_PWR_REDUCTION("NORMAL"),
.PCS_CH3_WDALIGN_PWR_REDUCTION("NORMAL"),
.PCS_CH3_RXDEC_PWR_REDUCTION("NORMAL"),
.PCS_CH3_RXCB_PWR_REDUCTION("NORMAL"),
.PCS_CH3_RXCTC_PWR_REDUCTION("NORMAL"),
.PCS_CH3_RXGEAR_PWR_REDUCTION("NORMAL"),
.PCS_CH3_RXBRG_PWR_REDUCTION("NORMAL"),
.PCS_CH3_RXTEST_PWR_REDUCTION("NORMAL"),
.PCS_CH3_TXBRG_PWR_REDUCTION("NORMAL"),
.PCS_CH3_TXGEAR_PWR_REDUCTION("NORMAL"),
.PCS_CH3_TXENC_PWR_REDUCTION("NORMAL"),
.PCS_CH3_TXBSLP_PWR_REDUCTION("NORMAL"),
.PCS_CH3_TXPRBS_PWR_REDUCTION("NORMAL"),
.PCS_CH3_TXBRG_FULL_CHK_EN("FALSE"),
.PCS_CH3_TXBRG_EMPTY_CHK_EN("FALSE"),
.PCS_CH3_RXBRG_FULL_CHK_EN("FALSE"),
.PCS_CH3_RXBRG_EMPTY_CHK_EN("FALSE"),
.PCS_CH3_CTC_FULL_CHK_EN("TRUE"),
.PCS_CH3_CTC_EMPTY_CHK_EN("TRUE"),
.PCS_CH3_CEB_FULL_CHK_EN("FALSE"),
.PCS_CH3_CEB_EMPTY_CHK_EN("FALSE"),
.PCS_CH3_FLP_FULL_CHK_EN("TRUE"),
.PCS_CH3_FLP_EMPTY_CHK_EN("TRUE"),
.PMA_CH0_REG_RX_PD("ON"),
.PMA_CH0_REG_RX_PD_EN("FALSE"),
.PMA_CH0_REG_RX_CLKPATH_PD("ON"),
.PMA_CH0_REG_RX_CLKPATH_PD_EN("FALSE"),
```

```
.PMA_CH0_REG_RX_DATAPATH_PD("ON"),
.PMA_CH0_REG_RX_DATAPATH_PD_EN("FALSE"),
.PMA_CH0_REG_RX_SIGDET_PD("ON"),
.PMA_CH0_REG_RX_SIGDET_PD_EN("FALSE"),
.PMA_CH0_REG_RX_DCC_RST_N("TRUE"),
.PMA_CH0_REG_RX_DCC_RST_N_EN("FALSE"),
.PMA_CH0_REG_RX_CDR_RST_N("TRUE"),
.PMA_CH0_REG_RX_CDR_RST_N_EN("FALSE"),
.PMA_CH0_REG_RX_SIGDET_RST_N("TRUE"),
.PMA_CH0_REG_RX_SIGDET_RST_N_EN("FALSE"),
.PMA_CH0_REG_RXPCLK_SLIP("FALSE"),
.PMA_CH0_REG_RXPCLK_SLIP_OW("DISABLE"),
.PMA_CH0_REG_RX_PCLKSWITCH_RST_N("TRUE"),
.PMA_CH0_REG_RX_PCLKSWITCH_RST_N_EN("FALSE"),
.PMA_CH0_REG_RX_PCLKSWITCH("FALSE"),
.PMA_CH0_REG_RX_PCLKSWITCH_EN("FALSE"),
.PMA_CH0_REG_RX_HIGHZ("FALSE"),
.PMA_CH0_REG_RX_HIGHZ_EN("FALSE"),
.PMA_CH0_REG_RX_EQ_C_SET(8),
.PMA_CH0_REG_RX_EQ_R_SET(8),
.PMA_CH0_REG_RX_BUSWIDTH("20BIT"),
.PMA_CH0_REG_RX_BUSWIDTH_EN("FALSE"),
.PMA_CH0_REG_RX_RATE("DIV1"),
.PMA_CH0_REG_RX_RATE_EN("FALSE"),
.PMA_CH0_REG_RX_RES_TRIM(51),
.PMA_CH0_REG_RX_RES_TRIM_EN("FALSE"),
.PMA_CH0_REG_RX_EQ_OFF("FALSE"),
.PMA_CH0_REG_RX_PREAMP_IC(1367),
.PMA_CH0_REG_RX_PCLK_EDGE_SEL("POS_EDGE"),
.PMA_CH0_REG_RX_PIBUF_IC(2),
.PMA_CH0_REG_RX_DCC_IC_RX(3),
.PMA_CH0_REG_RX_DCC_IC_TX(3),
.PMA_CH0_REG_RX_ICTRL_TRX("100PCT"),
.PMA_CH0_REG_RX_ICTRL_SIGDET(5),
.PMA_CH0_REG_RX_ICTRL_PREAMP("100PCT"),
.PMA_CH0_REG_RX_ICTRL_SLICER("100PCT"),
.PMA_CH0_REG_RX_ICTRL_PIBUF("100PCT"),
.PMA_CH0_REG_RX_ICTRL_PI("100PCT"),
.PMA_CH0_REG_RX_ICTRL_DCC("100PCT"),
.PMA_CH0_REG_RX_ICTRL_PREDRV("100PCT"),
.PMA_CH0_REG_TX_RATE("DIV1"),
.PMA_CH0_REG_TX_RATE_EN("FALSE"),
.PMA_CH0_REG_RX_TX2RX_PLPBK_RST_N("TRUE"),
.PMA_CH0_REG_RX_TX2RX_PLPBK_RST_N_EN("FALSE"),
.PMA_CH0_REG_RX_TX2RX_PLPBK_EN("FALSE"),
.PMA_CH0_REG_TXCLK_SEL("PLL"),
.PMA_CH0_REG_RX_DATA_POLARITY("NORMAL"),
```

```
.PMA_CH0_REG_RX_ERR_INSERT("FALSE"),
.PMA_CH0_REG_UDP_CHK_EN("FALSE"),
.PMA_CH0_REG_PRBS_SEL("PRBS7"),
.PMA_CH0_REG_PRBS_CHK_EN("FALSE"),
.PMA_CH0_REG_PRBS_CHK_WIDTH_SEL("20BIT"),
.PMA_CH0_REG_BIST_CHK_PAT_SEL("PRBS"),
.PMA_CH0_REG_LOAD_ERR_CNT("DISABLE"),
.PMA_CH0_REG_CHK_COUNTER_EN("FALSE"),
.PMA_CH0_REG_CDR_PROP_GAIN(5),
.PMA_CH0_REG_CDR_PROP_TURBO_GAIN(6),
.PMA_CH0_REG_CDR_INT_GAIN(5),
.PMA_CH0_REG_CDR_INT_TURBO_GAIN(6),
.PMA_CH0_REG_CDR_INT_SAT_MAX(992),
.PMA_CH0_REG_CDR_INT_SAT_MIN(32),
.PMA_CH0_REG_CDR_INT_RST("FALSE"),
.PMA_CH0_REG_CDR_INT_RST_OW("DISABLE"),
.PMA_CH0_REG_CDR_PROP_RST("FALSE"),
.PMA_CH0_REG_CDR_PROP_RST_OW("DISABLE"),
.PMA_CH0_REG_CDR_LOCK_RST("FALSE"),
.PMA_CH0_REG_CDR_LOCK_RST_OW("DISABLE"),
.PMA_CH0_REG_CDR_RX_PI_FORCE_SEL(0),
.PMA_CH0_REG_CDR_RX_PI_FORCE_D(0),
.PMA_CH0_REG_CDR_LOCK_TIMER("1_2U"),
.PMA_CH0_REG_CDR_TURBO_MODE_TIMER(1),
.PMA_CH0_REG_CDR_LOCK_VAL("FALSE"),
.PMA_CH0_REG_CDR_LOCK_OW("DISABLE"),
.PMA_CH0_REG_CDR_INT_SAT_DET_EN("TRUE"),
.PMA_CH0_REG_CDR_SAT_DET_STATUS_EN("FALSE"),
.PMA_CH0_REG_CDR_SAT_DET_STATUS_RESET_EN("FALSE"),
.PMA_CH0_REG_CDR_PI_CTRL_RST("FALSE"),
.PMA_CH0_REG_CDR_PI_CTRL_RST_OW("DISABLE"),
.PMA_CH0_REG_CDR_SAT_DET_RST("FALSE"),
.PMA_CH0_REG_CDR_SAT_DET_RST_OW("DISABLE"),
.PMA_CH0_REG_CDR_SAT_DET_STICKY_RST("FALSE"),
.PMA_CH0_REG_CDR_SAT_DET_STICKY_RST_OW("DISABLE"),
.PMA_CH0_REG_CDR_SIGDET_STATUS_DIS("FALSE"),
.PMA_CH0_REG_CDR_SAT_DET_TIMER(2),
.PMA_CH0_REG_CDR_SAT_DET_STATUS_VAL("FALSE"),
.PMA_CH0_REG_CDR_SAT_DET_STATUS_OW("DISABLE"),
.PMA_CH0_REG_CDR_TURBO_MODE_EN("TRUE"),
.PMA_CH0_REG_CDR_STATUS_RADDR_INIT(0),
.PMA_CH0_REG_CDR_STATUS_FIFO_EN("TRUE"),
.PMA_CH0_REG_PMA_TEST_SEL(0),
.PMA_CH0_REG_OOB_COMWAKE_GAP_MIN(3),
.PMA_CH0_REG_OOB_COMWAKE_GAP_MAX(11),
.PMA_CH0_REG_OOB_COMINIT_GAP_MIN(15),
.PMA_CH0_REG_OOB_COMINIT_GAP_MAX(35),
```

```
.PMA_CH0_REG_RX_PIBUF_IC_TX(1),
.PMA_CH0_REG_COMWAKE_STATUS_CLEAR(0),
.PMA_CH0_REG_COMINIT_STATUS_CLEAR(0),
.PMA_CH0_REG_RX_SYNC_RST_N_EN("FALSE"),
.PMA_CH0_REG_RX_SYNC_RST_N("TRUE"),
.PMA_CH0_REG_RX_SATA_COMINIT_OW("DISABLE"),
.PMA_CH0_REG_RX_SATA_COMINIT("FALSE"),
.PMA_CH0_REG_RX_SATA_COMWAKE_OW("DISABLE"),
.PMA_CH0_REG_RX_SATA_COMWAKE("FALSE"),
.PMA_CH0_REG_RX_DCC_DISABLE("ENABLE"),
.PMA_CH0_REG_TX_DCC_DISABLE("ENABLE"),
.PMA_CH0_REG_RX_SLIP_SEL_EN("FALSE"),
.PMA_CH0_REG_RX_SLIP_SEL(0),
.PMA_CH0_REG_RX_SLIP_EN("FALSE"),
.PMA_CH0_REG_RX_SIGDET_STATUS_SEL(5),
.PMA_CH0_REG_RX_SIGDET_FSM_RST_N("TRUE"),
.PMA_CH0_REG_RX_SIGDET_STATUS_OW("DISABLE"),
.PMA_CH0_REG_RX_SIGDET_STATUS("FALSE"),
.PMA_CH0_REG_RX_SIGDET_VTH("50MV"),
.PMA_CH0_REG_RX_SIGDET_GRM(0),
.PMA_CH0_REG_RX_SIGDET_PULSE_EXT("DISABLE"),
.PMA_CH0_REG_RX_SIGDET_CH2_SEL(0),
.PMA_CH0_REG_RX_SIGDET_CH2_CHK_WINDOW(3),
.PMA_CH0_REG_RX_SIGDET_CHK_WINDOW_EN("TRUE"),
.PMA_CH0_REG_RX_SIGDET_NOSIG_COUNT_SETTING(4),
.PMA_CH0_REG_RX_SIGDET_OOB_DET_COUNT_VAL(0),
.PMA_CH0_REG_SLIP_FIFO_INV_EN("FALSE"),
.PMA_CH0_REG_SLIP_FIFO_INV("POS_EDGE"),
.PMA_CH0_REG_RX_SIGDET_4OOB_DET_SEL(7),
.PMA_CH0_REG_RX_SIGDET_IC_I(10),
.PMA_CH0_REG_RX_OOB_DETECTOR_RESET_N_OW("DISABLE"),
.PMA_CH0_REG_RX_OOB_DETECTOR_RESET_N("FALSE"),
.PMA_CH0_REG_RX_OOB_DETECTOR_PD_OW("DISABLE"),
.PMA_CH0_REG_RX_OOB_DETECTOR_PD("ON"),
.PMA_CH0_REG_RX_TERM_CM_CTRL("5DIV7"),
.PMA_CH0_REG_TX_PD("ON"),
.PMA_CH0_REG_TX_PD_OW("DISABLE"),
.PMA_CH0_REG_TX_CLKPATH_PD("ON"),
.PMA_CH0_REG_TX_CLKPATH_PD_OW("DISABLE"),
.PMA_CH0_REG_TX_BEACON_TIMER_SEL(0),
.PMA_CH0_REG_TX_RXDET_REQ_OW("DISABLE"),
.PMA_CH0_REG_TX_RXDET_REQ("FALSE"),
.PMA_CH0_REG_TX_BEACON_EN_OW("DISABLE"),
.PMA_CH0_REG_TX_BEACON_EN("FALSE"),
.PMA_CH0_REG_TX_EI_EN_OW("DISABLE"),
.PMA_CH0_REG_TX_EI_EN("FALSE"),
.PMA_CH0_REG_TX_RES_CAL_EN("FALSE"),
```

```
.PMA_CH0_REG_TX_RES_CAL(51),
.PMA_CH0_REG_TX_BIAS_CAL_EN("FALSE"),
.PMA_CH0_REG_TX_BIAS_CTRL(48),
.PMA_CH0_REG_TX_RXDET_TIMER_SEL("12CYCLE"),
.PMA_CH0_REG_TX_SYNC_OW("DISABLE"),
.PMA_CH0_REG_TX_SYNC("DISABLE"),
.PMA_CH0_REG_TX_PD_POST("OFF"),
.PMA_CH0_REG_TX_PD_POST_OW("DISABLE"),
.PMA_CH0_REG_TX_RESET_N_OW("DISABLE"),
.PMA_CH0_REG_TX_RESET_N("TRUE"),
.PMA_CH0_REG_TX_DCC_RESET_N_OW("DISABLE"),
.PMA_CH0_REG_TX_DCC_RESET_N("TRUE"),
.PMA_CH0_REG_TX_BUSWIDTH_OW("DISABLE"),
.PMA_CH0_REG_TX_BUSWIDTH("20BIT"),
.PMA_CH0_REG_PLL_READY_OW("DISABLE"),
.PMA_CH0_REG_PLL_READY("TRUE"),
.PMA_CH0_REG_TX_PCLK_SW_OW("DISABLE"),
.PMA_CH0_REG_TX_PCLK_SW("TRUE"),
.PMA_CH0_REG_EI_PCLK_DELAY_SEL(0),
.PMA_CH0_REG_TX_DRV01_DAC0(0),
.PMA_CH0_REG_TX_DRV01_DAC1(10),
.PMA_CH0_REG_TX_DRV01_DAC2(16),
.PMA_CH0_REG_TX_DRV00_DAC0(63),
.PMA_CH0_REG_TX_DRV00_DAC1(53),
.PMA_CH0_REG_TX_DRV00_DAC2(48),
.PMA_CH0_REG_TX_AMP0(8),
.PMA_CH0_REG_TX_AMP1(16),
.PMA_CH0_REG_TX_AMP2(32),
.PMA_CH0_REG_TX_AMP3(48),
.PMA_CH0_REG_TX_AMP4(56),
.PMA_CH0_REG_TX_MARGIN(0),
.PMA_CH0_REG_TX_MARGIN_OW("DISABLE"),
.PMA_CH0_REG_TX_DEEMP(0),
.PMA_CH0_REG_TX_DEEMP_OW("DISABLE"),
.PMA_CH0_REG_TX_SWING("FALSE"),
.PMA_CH0_REG_TX_SWING_OW("DISABLE"),
.PMA_CH0_REG_TX_RXDET_THRESHOLD("100MV"),
.PMA_CH0_REG_TX_BEACON_OSC_CTRL(4),
.PMA_CH0_REG_TX_PREDRV_DAC(1),
.PMA_CH0_REG_TX_PREDRV_CM_CTRL(1),
.PMA_CH0_REG_TX_TX2RX_SLPBACK_EN("FALSE"),
.PMA_CH0_REG_TX_PCLK_EDGE_SEL("POS_EDGE"),
.PMA_CH0_REG_TX_RXDET_STATUS_OW("DISABLE"),
.PMA_CH0_REG_TX_RXDET_STATUS("TRUE"),
.PMA_CH0_REG_TX_PRBS_GEN_EN("FALSE"),
.PMA_CH0_REG_TX_PRBS_GEN_WIDTH_SEL("20BIT"),
.PMA_CH0_REG_TX_PRBS_SEL("PRBS7"),
```

```
.PMA_CH0_REG_TX_UDP_DATA(256773),
.PMA_CH0_REG_TX_FIFO_RST_N("FALSE"),
.PMA_CH0_REG_TX_FIFO_WP_CTRL(2),
.PMA_CH0_REG_TX_FIFO_EN("FALSE"),
.PMA_CH0_REG_TX_DATA_MUX_SEL(0),
.PMA_CH0_REG_TX_ERR_INSERT("FALSE"),
.PMA_CH0_REG_TX_SATA_EN("FALSE"),
.PMA_CH0_REG_RATE_CHANGE_TXPCLK_ON_OW("DISABLE"),
.PMA_CH0_REG_RATE_CHANGE_TXPCLK_ON("ENABLE"),
.PMA_CH0_REG_TX_PULLUP_DAC0(8),
.PMA_CH0_REG_TX_PULLUP_DAC1(8),
.PMA_CH0_REG_TX_PULLUP_DAC2(8),
.PMA_CH0_REG_TX_PULLUP_DAC3(8),
.PMA_CH0_REG_TX_OOB_DELAY_SEL(0),
.PMA_CH0_REG_TX_POLARITY("NORMAL"),
.PMA_CH0_REG_TX_SLPBK_AMP(1),
.PMA_CH0_REG_TX_LS_MODE_EN("FALSE"),
.PMA_CH0_REG_TX_JTAG_MODE_EN_OW("DISABLE"),
.PMA_CH0_REG_TX_JTAG_MODE_EN("FALSE"),
.PMA_CH0_REG_RX_JTAG_MODE_EN_OW("DISABLE"),
.PMA_CH0_REG_RX_JTAG_MODE_EN("FALSE"),
.PMA_CH0_REG_RX_JTAG_OE("DISABLE"),
.PMA_CH0_REG_RX_ACJTAG_VHYSTSE(0),
.PMA_CH0_REG_TX_FBCLK_FAR_EN("FALSE"),
.PMA_CH0_REG_RX_TERM_MODE_CTRL(6),
.PMA_CH0_REG_PLPBK_TXPCLK_EN("TRUE"),
.PMA_CH0_REG_TX_609_600(0),
.PMA_CH0_REG_RX_CDR_617_610(0),
.PMA_CH0_REG_RX_CDR_623_618(0),
.PMA_CH0_REG_RX_631_624(0),
.PMA_CH0_REG_RX_639_632(0),
.PMA_CH0_REG_RX_647_640(0),
.PMA_CH0_REG_RX_655_648(0),
.PMA_CH0_REG_RX_659_656(0),
.PMA_CH0_CFG_LANE_POWERUP("OFF"),
.PMA_CH0_CFG_PMA_POR_N("FALSE"),
.PMA_CH0_CFG_RX_LANE_POWERUP("OFF"),
.PMA_CH0_CFG_RX_PMA_RSTN("FALSE"),
.PMA_CH0_CFG_TX_LANE_POWERUP("OFF"),
.PMA_CH0_CFG_TX_PMA_RSTN("FALSE"),
.PMA_CH0_CFG_CTLE_ADP_RSTN("TRUE"),
.PMA_CH0_REG_RESERVED_48_45(0),
.PMA_CH0_REG_RESERVED_69(0),
.PMA_CH0_REG_RESERVED_77_76(0),
.PMA_CH0_REG_RESERVED_171_164(0),
.PMA_CH0_REG_RESERVED_175_172(0),
.PMA_CH0_REG_RESERVED_190(0),
```

```
.PMA_CH0_REG_RESERVED_233_232(0),
.PMA_CH0_REG_RESERVED_235_234(0),
.PMA_CH0_REG_RESERVED_241_240(0),
.PMA_CH0_REG_RESERVED_285_283(0),
.PMA_CH0_REG_RESERVED_286(0),
.PMA_CH0_REG_RESERVED_295(0),
.PMA_CH0_REG_RESERVED_298(0),
.PMA_CH0_REG_RESERVED_332_325(0),
.PMA_CH0_REG_RESERVED_340_333(0),
.PMA_CH0_REG_RESERVED_348_341(0),
.PMA_CH0_REG_RESERVED_354_349(0),
.PMA_CH0_REG_RESERVED_373(0),
.PMA_CH0_REG_RESERVED_376(0),
.PMA_CH0_REG_RESERVED_452(0),
.PMA_CH0_REG_RESERVED_502_499(0),
.PMA_CH0_REG_RESERVED_506_505(0),
.PMA_CH0_REG_RESERVED_550_549(0),
.PMA_CH0_REG_RESERVED_556_552(0),
.PMA_CH1_REG_RX_PD("ON"),
.PMA_CH1_REG_RX_PD_EN("FALSE"),
.PMA_CH1_REG_RX_CLKPATH_PD("ON"),
.PMA_CH1_REG_RX_CLKPATH_PD_EN("FALSE"),
.PMA_CH1_REG_RX_DATAPATH_PD("ON"),
.PMA_CH1_REG_RX_DATAPATH_PD_EN("FALSE"),
.PMA_CH1_REG_RX_SIGDET_PD("ON"),
.PMA_CH1_REG_RX_SIGDET_PD_EN("FALSE"),
.PMA_CH1_REG_RX_DCC_RST_N("TRUE"),
.PMA_CH1_REG_RX_DCC_RST_N_EN("FALSE"),
.PMA_CH1_REG_RX_CDR_RST_N("TRUE"),
.PMA_CH1_REG_RX_CDR_RST_N_EN("FALSE"),
.PMA_CH1_REG_RX_SIGDET_RST_N("TRUE"),
.PMA_CH1_REG_RX_SIGDET_RST_N_EN("FALSE"),
.PMA_CH1_REG_RXPCLK_SLIP("FALSE"),
.PMA_CH1_REG_RXPCLK_SLIP_OW("DISABLE"),
.PMA_CH1_REG_RX_PCLKSWITCH_RST_N("TRUE"),
.PMA_CH1_REG_RX_PCLKSWITCH_RST_N_EN("FALSE"),
.PMA_CH1_REG_RX_PCLKSWITCH("FALSE"),
.PMA_CH1_REG_RX_PCLKSWITCH_EN("FALSE"),
.PMA_CH1_REG_RX_HIGHZ("FALSE"),
.PMA_CH1_REG_RX_HIGHZ_EN("FALSE"),
.PMA_CH1_REG_RX_EQ_C_SET(8),
.PMA_CH1_REG_RX_EQ_R_SET(8),
.PMA_CH1_REG_RX_BUSWIDTH("20BIT"),
.PMA_CH1_REG_RX_BUSWIDTH_EN("FALSE"),
.PMA_CH1_REG_RX_RATE("DIV1"),
.PMA_CH1_REG_RX_RATE_EN("FALSE"),
.PMA_CH1_REG_RX_RES_TRIM(51),
```

```
.PMA_CH1_REG_RX_RES_TRIM_EN("FALSE"),
.PMA_CH1_REG_RX_EQ_OFF("FALSE"),
.PMA_CH1_REG_RX_PREAMP_IC(1367),
.PMA_CH1_REG_RX_PCLK_EDGE_SEL("POS_EDGE"),
.PMA_CH1_REG_RX_PIBUF_IC(2),
.PMA_CH1_REG_RX_DCC_IC_RX(3),
.PMA_CH1_REG_RX_DCC_IC_TX(3),
.PMA_CH1_REG_RX_ICTRL_TRX("100PCT"),
.PMA_CH1_REG_RX_ICTRL_SIGDET(5),
.PMA_CH1_REG_RX_ICTRL_PREAMP("100PCT"),
.PMA_CH1_REG_RX_ICTRL_SLICER("100PCT"),
.PMA_CH1_REG_RX_ICTRL_PIBUF("100PCT"),
.PMA_CH1_REG_RX_ICTRL_PI("100PCT"),
.PMA_CH1_REG_RX_ICTRL_DCC("100PCT"),
.PMA_CH1_REG_RX_ICTRL_PREDRV("100PCT"),
.PMA_CH1_REG_TX_RATE("DIV1"),
.PMA_CH1_REG_TX_RATE_EN("FALSE"),
.PMA_CH1_REG_RX_TX2RX_PLPBK_RST_N("TRUE"),
.PMA_CH1_REG_RX_TX2RX_PLPBK_RST_N_EN("FALSE"),
.PMA_CH1_REG_RX_TX2RX_PLPBK_EN("FALSE"),
.PMA_CH1_REG_TXCLK_SEL("PLL"),
.PMA_CH1_REG_RX_DATA_POLARITY("NORMAL"),
.PMA_CH1_REG_RX_ERR_INSERT("FALSE"),
.PMA_CH1_REG_UDP_CHK_EN("FALSE"),
.PMA_CH1_REG_PRBS_SEL("PRBS7"),
.PMA_CH1_REG_PRBS_CHK_EN("FALSE"),
.PMA_CH1_REG_PRBS_CHK_WIDTH_SEL("20BIT"),
.PMA_CH1_REG_BIST_CHK_PAT_SEL("PRBS"),
.PMA_CH1_REG_LOAD_ERR_CNT("DISABLE"),
.PMA_CH1_REG_CHK_COUNTER_EN("FALSE"),
.PMA_CH1_REG_CDR_PROP_GAIN(5),
.PMA_CH1_REG_CDR_PROP_TURBO_GAIN(6),
.PMA_CH1_REG_CDR_INT_GAIN(5),
.PMA_CH1_REG_CDR_INT_TURBO_GAIN(6),
.PMA_CH1_REG_CDR_INT_SAT_MAX(992),
.PMA_CH1_REG_CDR_INT_SAT_MIN(32),
.PMA_CH1_REG_CDR_INT_RST("FALSE"),
.PMA_CH1_REG_CDR_INT_RST_OW("DISABLE"),
.PMA_CH1_REG_CDR_PROP_RST("FALSE"),
.PMA_CH1_REG_CDR_PROP_RST_OW("DISABLE"),
.PMA_CH1_REG_CDR_LOCK_RST("FALSE"),
.PMA_CH1_REG_CDR_LOCK_RST_OW("DISABLE"),
.PMA_CH1_REG_CDR_RX_PI_FORCE_SEL(0),
.PMA_CH1_REG_CDR_RX_PI_FORCE_D(0),
.PMA_CH1_REG_CDR_LOCK_TIMER("1_2U"),
.PMA_CH1_REG_CDR_TURBO_MODE_TIMER(1),
.PMA_CH1_REG_CDR_LOCK_VAL("FALSE"),
```

```
.PMA_CH1_REG_CDR_LOCK_OW("DISABLE"),
.PMA_CH1_REG_CDR_INT_SAT_DET_EN("TRUE"),
.PMA_CH1_REG_CDR_SAT_DET_STATUS_EN("FALSE"),
.PMA_CH1_REG_CDR_SAT_DET_STATUS_RESET_EN("FALSE"),
.PMA_CH1_REG_CDR_PI_CTRL_RST("FALSE"),
.PMA_CH1_REG_CDR_PI_CTRL_RST_OW("DISABLE"),
.PMA_CH1_REG_CDR_SAT_DET_RST("FALSE"),
.PMA_CH1_REG_CDR_SAT_DET_RST_OW("DISABLE"),
.PMA_CH1_REG_CDR_SAT_DET_STICKY_RST("FALSE"),
.PMA_CH1_REG_CDR_SAT_DET_STICKY_RST_OW("DISABLE"),
.PMA_CH1_REG_CDR_SIGDET_STATUS_DIS("FALSE"),
.PMA_CH1_REG_CDR_SAT_DET_TIMER(2),
.PMA_CH1_REG_CDR_SAT_DET_STATUS_VAL("FALSE"),
.PMA_CH1_REG_CDR_SAT_DET_STATUS_OW("DISABLE"),
.PMA_CH1_REG_CDR_TURBO_MODE_EN("TRUE"),
.PMA_CH1_REG_CDR_STATUS_RADDR_INIT(0),
.PMA_CH1_REG_CDR_STATUS_FIFO_EN("TRUE"),
.PMA_CH1_REG_PMA_TEST_SEL(0),
.PMA_CH1_REG_OOB_COMWAKE_GAP_MIN(3),
.PMA_CH1_REG_OOB_COMWAKE_GAP_MAX(11),
.PMA_CH1_REG_OOB_COMINIT_GAP_MIN(15),
.PMA_CH1_REG_OOB_COMINIT_GAP_MAX(35),
.PMA_CH1_REG_RX_PIBUF_IC_TX(1),
.PMA_CH1_REG_COMWAKE_STATUS_CLEAR(0),
.PMA_CH1_REG_COMINIT_STATUS_CLEAR(0),
.PMA_CH1_REG_RX_SYNC_RST_N_EN("FALSE"),
.PMA_CH1_REG_RX_SYNC_RST_N("TRUE"),
.PMA_CH1_REG_RX_SATA_COMINIT_OW("DISABLE"),
.PMA_CH1_REG_RX_SATA_COMINIT("FALSE"),
.PMA_CH1_REG_RX_SATA_COMWAKE_OW("DISABLE"),
.PMA_CH1_REG_RX_SATA_COMWAKE("FALSE"),
.PMA_CH1_REG_RX_DCC_DISABLE("ENABLE"),
.PMA_CH1_REG_TX_DCC_DISABLE("ENABLE"),
.PMA_CH1_REG_RX_SLIP_SEL_EN("FALSE"),
.PMA_CH1_REG_RX_SLIP_SEL(0),
.PMA_CH1_REG_RX_SLIP_EN("FALSE"),
.PMA_CH1_REG_RX_SIGDET_STATUS_SEL(5),
.PMA_CH1_REG_RX_SIGDET_FSM_RST_N("TRUE"),
.PMA_CH1_REG_RX_SIGDET_STATUS_OW("DISABLE"),
.PMA_CH1_REG_RX_SIGDET_STATUS("FALSE"),
.PMA_CH1_REG_RX_SIGDET_VTH("50MV"),
.PMA_CH1_REG_RX_SIGDET_GRM(0),
.PMA_CH1_REG_RX_SIGDET_PULSE_EXT("DISABLE"),
.PMA_CH1_REG_RX_SIGDET_CH2_SEL(0),
.PMA_CH1_REG_RX_SIGDET_CH2_CHK_WINDOW(3),
.PMA_CH1_REG_RX_SIGDET_CHK_WINDOW_EN("TRUE"),
.PMA_CH1_REG_RX_SIGDET_NOSIG_COUNT_SETTING(4),
```

```
.PMA_CH1_REG_RX_SIGDET_OOB_DET_COUNT_VAL(0),
.PMA_CH1_REG_SLIP_FIFO_INV_EN("FALSE"),
.PMA_CH1_REG_SLIP_FIFO_INV("POS_EDGE"),
.PMA_CH1_REG_RX_SIGDET_4OOB_DET_SEL(7),
.PMA_CH1_REG_RX_SIGDET_IC_I(10),
.PMA_CH1_REG_RX_OOB_DETECTOR_RESET_N_OW("DISABLE"),
.PMA_CH1_REG_RX_OOB_DETECTOR_RESET_N("FALSE"),
.PMA_CH1_REG_RX_OOB_DETECTOR_PD_OW("DISABLE"),
.PMA_CH1_REG_RX_OOB_DETECTOR_PD("ON"),
.PMA_CH1_REG_RX_TERM_CM_CTRL("5DIV7"),
.PMA_CH1_REG_TX_PD("ON"),
.PMA_CH1_REG_TX_PD_OW("DISABLE"),
.PMA_CH1_REG_TX_CLKPATH_PD("ON"),
.PMA_CH1_REG_TX_CLKPATH_PD_OW("DISABLE"),
.PMA_CH1_REG_TX_BEACON_TIMER_SEL(0),
.PMA_CH1_REG_TX_RXDET_REQ_OW("DISABLE"),
.PMA_CH1_REG_TX_RXDET_REQ("FALSE"),
.PMA_CH1_REG_TX_BEACON_EN_OW("DISABLE"),
.PMA_CH1_REG_TX_BEACON_EN("FALSE"),
.PMA_CH1_REG_TX_EI_EN_OW("DISABLE"),
.PMA_CH1_REG_TX_EI_EN("FALSE"),
.PMA_CH1_REG_TX_RES_CAL_EN("FALSE"),
.PMA_CH1_REG_TX_RES_CAL(51),
.PMA_CH1_REG_TX_BIAS_CAL_EN("FALSE"),
.PMA_CH1_REG_TX_BIAS_CTRL(48),
.PMA_CH1_REG_TX_RXDET_TIMER_SEL("12CYCLE"),
.PMA_CH1_REG_TX_SYNC_OW("DISABLE"),
.PMA_CH1_REG_TX_SYNC("DISABLE"),
.PMA_CH1_REG_TX_PD_POST("OFF"),
.PMA_CH1_REG_TX_PD_POST_OW("DISABLE"),
.PMA_CH1_REG_TX_RESET_N_OW("DISABLE"),
.PMA_CH1_REG_TX_RESET_N("TRUE"),
.PMA_CH1_REG_TX_DCC_RESET_N_OW("DISABLE"),
.PMA_CH1_REG_TX_DCC_RESET_N("TRUE"),
.PMA_CH1_REG_TX_BUSWIDTH_OW("DISABLE"),
.PMA_CH1_REG_TX_BUSWIDTH("20BIT"),
.PMA_CH1_REG_PLL_READY_OW("DISABLE"),
.PMA_CH1_REG_PLL_READY("TRUE"),
.PMA_CH1_REG_TX_PCLK_SW_OW("DISABLE"),
.PMA_CH1_REG_TX_PCLK_SW("TRUE"),
.PMA_CH1_REG_EI_PCLK_DELAY_SEL(0),
.PMA_CH1_REG_TX_DRV01_DAC0(0),
.PMA_CH1_REG_TX_DRV01_DAC1(10),
.PMA_CH1_REG_TX_DRV01_DAC2(16),
.PMA_CH1_REG_TX_DRV00_DAC0(63),
.PMA_CH1_REG_TX_DRV00_DAC1(53),
.PMA_CH1_REG_TX_DRV00_DAC2(48),
```

```
.PMA_CH1_REG_TX_AMP0(8),
.PMA_CH1_REG_TX_AMP1(16),
.PMA_CH1_REG_TX_AMP2(32),
.PMA_CH1_REG_TX_AMP3(48),
.PMA_CH1_REG_TX_AMP4(56),
.PMA_CH1_REG_TX_MARGIN(0),
.PMA_CH1_REG_TX_MARGIN_OW("DISABLE"),
.PMA_CH1_REG_TX_DEEMP(0),
.PMA_CH1_REG_TX_DEEMP_OW("DISABLE"),
.PMA_CH1_REG_TX_SWING("FALSE"),
.PMA_CH1_REG_TX_SWING_OW("DISABLE"),
.PMA_CH1_REG_TX_RXDET_THRESHOLD("100MV"),
.PMA_CH1_REG_TX_BEACON_OSC_CTRL(4),
.PMA_CH1_REG_TX_PREDRV_DAC(1),
.PMA_CH1_REG_TX_PREDRV_CM_CTRL(1),
.PMA_CH1_REG_TX_TX2RX_SLPBACK_EN("FALSE"),
.PMA_CH1_REG_TX_PCLK_EDGE_SEL("POS_EDGE"),
.PMA_CH1_REG_TX_RXDET_STATUS_OW("DISABLE"),
.PMA_CH1_REG_TX_RXDET_STATUS("TRUE"),
.PMA_CH1_REG_TX_PRBS_GEN_EN("FALSE"),
.PMA_CH1_REG_TX_PRBS_GEN_WIDTH_SEL("20BIT"),
.PMA_CH1_REG_TX_PRBS_SEL("PRBS7"),
.PMA_CH1_REG_TX_UDP_DATA(256773),
.PMA_CH1_REG_TX_FIFO_RST_N("FALSE"),
.PMA_CH1_REG_TX_FIFO_WP_CTRL(2),
.PMA_CH1_REG_TX_FIFO_EN("FALSE"),
.PMA_CH1_REG_TX_DATA_MUX_SEL(0),
.PMA_CH1_REG_TX_ERR_INSERT("FALSE"),
.PMA_CH1_REG_TX_SATA_EN("FALSE"),
.PMA_CH1_REG_RATE_CHANGE_TXPCLK_ON_OW("DISABLE"),
.PMA_CH1_REG_RATE_CHANGE_TXPCLK_ON("ENABLE"),
.PMA_CH1_REG_TX_PULLUP_DAC0(8),
.PMA_CH1_REG_TX_PULLUP_DAC1(8),
.PMA_CH1_REG_TX_PULLUP_DAC2(8),
.PMA_CH1_REG_TX_PULLUP_DAC3(8),
.PMA_CH1_REG_TX_OOB_DELAY_SEL(0),
.PMA_CH1_REG_TX_POLARITY("NORMAL"),
.PMA_CH1_REG_TX_SLPBK_AMP(1),
.PMA_CH1_REG_TX_LS_MODE_EN("FALSE"),
.PMA_CH1_REG_TX_JTAG_MODE_EN_OW("DISABLE"),
.PMA_CH1_REG_TX_JTAG_MODE_EN("FALSE"),
.PMA_CH1_REG_RX_JTAG_MODE_EN_OW("DISABLE"),
.PMA_CH1_REG_RX_JTAG_MODE_EN("FALSE"),
.PMA_CH1_REG_RX_JTAG_OE("DISABLE"),
.PMA_CH1_REG_RX_ACJTAG_VHYSTSE(0),
.PMA_CH1_REG_TX_FBCLK_FAR_EN("FALSE"),
.PMA_CH1_REG_RX_TERM_MODE_CTRL(6),
```

```
.PMA_CH1_REG_PLPBK_TXPCLK_EN("TRUE"),
.PMA_CH1_REG_TX_609_600(0),
.PMA_CH1_REG_RX_CDR_617_610(0),
.PMA_CH1_REG_RX_CDR_623_618(0),
.PMA_CH1_REG_RX_631_624(0),
.PMA_CH1_REG_RX_639_632(0),
.PMA_CH1_REG_RX_647_640(0),
.PMA_CH1_REG_RX_655_648(0),
.PMA_CH1_REG_RX_659_656(0),
.PMA_CH1_CFG_LANE_POWERUP("OFF"),
.PMA_CH1_CFG_PMA_POR_N("FALSE"),
.PMA_CH1_CFG_RX_LANE_POWERUP("OFF"),
.PMA_CH1_CFG_RX_PMA_RSTN("FALSE"),
.PMA_CH1_CFG_TX_LANE_POWERUP("OFF"),
.PMA_CH1_CFG_TX_PMA_RSTN("FALSE"),
.PMA_CH1_CFG_CTLE_ADP_RSTN("TRUE"),
.PMA_CH1_REG_RESERVED_48_45(0),
.PMA_CH1_REG_RESERVED_69(0),
.PMA_CH1_REG_RESERVED_77_76(0),
.PMA_CH1_REG_RESERVED_171_164(0),
.PMA_CH1_REG_RESERVED_175_172(0),
.PMA_CH1_REG_RESERVED_190(0),
.PMA_CH1_REG_RESERVED_233_232(0),
.PMA_CH1_REG_RESERVED_235_234(0),
.PMA_CH1_REG_RESERVED_241_240(0),
.PMA_CH1_REG_RESERVED_285_283(0),
.PMA_CH1_REG_RESERVED_286(0),
.PMA_CH1_REG_RESERVED_295(0),
.PMA_CH1_REG_RESERVED_298(0),
.PMA_CH1_REG_RESERVED_332_325(0),
.PMA_CH1_REG_RESERVED_340_333(0),
.PMA_CH1_REG_RESERVED_348_341(0),
.PMA_CH1_REG_RESERVED_354_349(0),
.PMA_CH1_REG_RESERVED_373(0),
.PMA_CH1_REG_RESERVED_376(0),
.PMA_CH1_REG_RESERVED_452(0),
.PMA_CH1_REG_RESERVED_502_499(0),
.PMA_CH1_REG_RESERVED_506_505(0),
.PMA_CH1_REG_RESERVED_550_549(0),
.PMA_CH1_REG_RESERVED_556_552(0),
.PMA_CH2_REG_RX_PD("ON"),
.PMA_CH2_REG_RX_PD_EN("FALSE"),
.PMA_CH2_REG_RX_CLKPATH_PD("ON"),
.PMA_CH2_REG_RX_CLKPATH_PD_EN("FALSE"),
.PMA_CH2_REG_RX_DATAPATH_PD("ON"),
.PMA_CH2_REG_RX_DATAPATH_PD_EN("FALSE"),
.PMA_CH2_REG_RX_SIGDET_PD("ON"),
```

```
.PMA_CH2_REG_RX_SIGDET_PD_EN("FALSE"),
.PMA_CH2_REG_RX_DCC_RST_N("TRUE"),
.PMA_CH2_REG_RX_DCC_RST_N_EN("FALSE"),
.PMA_CH2_REG_RX_CDR_RST_N("TRUE"),
.PMA_CH2_REG_RX_CDR_RST_N_EN("FALSE"),
.PMA_CH2_REG_RX_SIGDET_RST_N("TRUE"),
.PMA_CH2_REG_RX_SIGDET_RST_N_EN("FALSE"),
.PMA_CH2_REG_RXPCLK_SLIP("FALSE"),
.PMA_CH2_REG_RXPCLK_SLIP_OW("DISABLE"),
.PMA_CH2_REG_RX_PCLKSWITCH_RST_N("TRUE"),
.PMA_CH2_REG_RX_PCLKSWITCH_RST_N_EN("FALSE"),
.PMA_CH2_REG_RX_PCLKSWITCH("FALSE"),
.PMA_CH2_REG_RX_PCLKSWITCH_EN("FALSE"),
.PMA_CH2_REG_RX_HIGHZ("FALSE"),
.PMA_CH2_REG_RX_HIGHZ_EN("FALSE"),
.PMA_CH2_REG_RX_EQ_C_SET(8),
.PMA_CH2_REG_RX_EQ_R_SET(8),
.PMA_CH2_REG_RX_BUSWIDTH("20BIT"),
.PMA_CH2_REG_RX_BUSWIDTH_EN("FALSE"),
.PMA_CH2_REG_RX_RATE("DIV1"),
.PMA_CH2_REG_RX_RATE_EN("FALSE"),
.PMA_CH2_REG_RX_RES_TRIM(51),
.PMA_CH2_REG_RX_RES_TRIM_EN("FALSE"),
.PMA_CH2_REG_RX_EQ_OFF("FALSE"),
.PMA_CH2_REG_RX_PREAMP_IC(1367),
.PMA_CH2_REG_RX_PCLK_EDGE_SEL("POS_EDGE"),
.PMA_CH2_REG_RX_PIBUF_IC(2),
.PMA_CH2_REG_RX_DCC_IC_RX(3),
.PMA_CH2_REG_RX_DCC_IC_TX(3),
.PMA_CH2_REG_RX_ICTRL_TRX("100PCT"),
.PMA_CH2_REG_RX_ICTRL_SIGDET(5),
.PMA_CH2_REG_RX_ICTRL_PREAMP("100PCT"),
.PMA_CH2_REG_RX_ICTRL_SLICER("100PCT"),
.PMA_CH2_REG_RX_ICTRL_PIBUF("100PCT"),
.PMA_CH2_REG_RX_ICTRL_PI("100PCT"),
.PMA_CH2_REG_RX_ICTRL_DCC("100PCT"),
.PMA_CH2_REG_RX_ICTRL_PREDRV("100PCT"),
.PMA_CH2_REG_TX_RATE("DIV1"),
.PMA_CH2_REG_TX_RATE_EN("FALSE"),
.PMA_CH2_REG_RX_TX2RX_PLPBK_RST_N("TRUE"),
.PMA_CH2_REG_RX_TX2RX_PLPBK_RST_N_EN("FALSE"),
.PMA_CH2_REG_RX_TX2RX_PLPBK_EN("FALSE"),
.PMA_CH2_REG_TXCLK_SEL("PLL"),
.PMA_CH2_REG_RX_DATA_POLARITY("NORMAL"),
.PMA_CH2_REG_RX_ERR_INSERT("FALSE"),
.PMA_CH2_REG_UDP_CHK_EN("FALSE"),
.PMA_CH2_REG_PRBS_SEL("PRBS7"),
```

```
.PMA_CH2_REG_PRBS_CHK_EN("FALSE"),
.PMA_CH2_REG_PRBS_CHK_WIDTH_SEL("20BIT"),
.PMA_CH2_REG_BIST_CHK_PAT_SEL("PRBS"),
.PMA_CH2_REG_LOAD_ERR_CNT("DISABLE"),
.PMA_CH2_REG_CHK_COUNTER_EN("FALSE"),
.PMA_CH2_REG_CDR_PROP_GAIN(5),
.PMA_CH2_REG_CDR_PROP_TURBO_GAIN(6),
.PMA_CH2_REG_CDR_INT_GAIN(5),
.PMA_CH2_REG_CDR_INT_TURBO_GAIN(6),
.PMA_CH2_REG_CDR_INT_SAT_MAX(992),
.PMA_CH2_REG_CDR_INT_SAT_MIN(32),
.PMA_CH2_REG_CDR_INT_RST("FALSE"),
.PMA_CH2_REG_CDR_INT_RST_OW("DISABLE"),
.PMA_CH2_REG_CDR_PROP_RST("FALSE"),
.PMA_CH2_REG_CDR_PROP_RST_OW("DISABLE"),
.PMA_CH2_REG_CDR_LOCK_RST("FALSE"),
.PMA_CH2_REG_CDR_LOCK_RST_OW("DISABLE"),
.PMA_CH2_REG_CDR_RX_PI_FORCE_SEL(0),
.PMA_CH2_REG_CDR_RX_PI_FORCE_D(0),
.PMA_CH2_REG_CDR_LOCK_TIMER("1_2U"),
.PMA_CH2_REG_CDR_TURBO_MODE_TIMER(1),
.PMA_CH2_REG_CDR_LOCK_VAL("FALSE"),
.PMA_CH2_REG_CDR_LOCK_OW("DISABLE"),
.PMA_CH2_REG_CDR_INT_SAT_DET_EN("TRUE"),
.PMA_CH2_REG_CDR_SAT_DET_STATUS_EN("FALSE"),
.PMA_CH2_REG_CDR_SAT_DET_STATUS_RESET_EN("FALSE"),
.PMA_CH2_REG_CDR_PI_CTRL_RST("FALSE"),
.PMA_CH2_REG_CDR_PI_CTRL_RST_OW("DISABLE"),
.PMA_CH2_REG_CDR_SAT_DET_RST("FALSE"),
.PMA_CH2_REG_CDR_SAT_DET_RST_OW("DISABLE"),
.PMA_CH2_REG_CDR_SAT_DET_STICKY_RST("FALSE"),
.PMA_CH2_REG_CDR_SAT_DET_STICKY_RST_OW("DISABLE"),
.PMA_CH2_REG_CDR_SIGDET_STATUS_DIS("FALSE"),
.PMA_CH2_REG_CDR_SAT_DET_TIMER(2),
.PMA_CH2_REG_CDR_SAT_DET_STATUS_VAL("FALSE"),
.PMA_CH2_REG_CDR_SAT_DET_STATUS_OW("DISABLE"),
.PMA_CH2_REG_CDR_TURBO_MODE_EN("TRUE"),
.PMA_CH2_REG_CDR_STATUS_RADDR_INIT(0),
.PMA_CH2_REG_CDR_STATUS_FIFO_EN("TRUE"),
.PMA_CH2_REG_PMA_TEST_SEL(0),
.PMA_CH2_REG_OOB_COMWAKE_GAP_MIN(3),
.PMA_CH2_REG_OOB_COMWAKE_GAP_MAX(11),
.PMA_CH2_REG_OOB_COMINIT_GAP_MIN(15),
.PMA_CH2_REG_OOB_COMINIT_GAP_MAX(35),
.PMA_CH2_REG_RX_PIBUF_IC_TX(1),
.PMA_CH2_REG_COMWAKE_STATUS_CLEAR(0),
.PMA_CH2_REG_COMINIT_STATUS_CLEAR(0),
```

```
.PMA_CH2_REG_RX_SYNC_RST_N_EN("FALSE"),
.PMA_CH2_REG_RX_SYNC_RST_N("TRUE"),
.PMA_CH2_REG_RX_SATA_COMINIT_OW("DISABLE"),
.PMA_CH2_REG_RX_SATA_COMINIT("FALSE"),
.PMA_CH2_REG_RX_SATA_COMWAKE_OW("DISABLE"),
.PMA_CH2_REG_RX_SATA_COMWAKE("FALSE"),
.PMA_CH2_REG_RX_DCC_DISABLE("ENABLE"),
.PMA_CH2_REG_TX_DCC_DISABLE("ENABLE"),
.PMA_CH2_REG_RX_SLIP_SEL_EN("FALSE"),
.PMA_CH2_REG_RX_SLIP_SEL(0),
.PMA_CH2_REG_RX_SLIP_EN("FALSE"),
.PMA_CH2_REG_RX_SIGDET_STATUS_SEL(5),
.PMA_CH2_REG_RX_SIGDET_FSM_RST_N("TRUE"),
.PMA_CH2_REG_RX_SIGDET_STATUS_OW("DISABLE"),
.PMA_CH2_REG_RX_SIGDET_STATUS("FALSE"),
.PMA_CH2_REG_RX_SIGDET_VTH("50MV"),
.PMA_CH2_REG_RX_SIGDET_GRM(0),
.PMA_CH2_REG_RX_SIGDET_PULSE_EXT("DISABLE"),
.PMA_CH2_REG_RX_SIGDET_CH2_SEL(0),
.PMA_CH2_REG_RX_SIGDET_CH2_CHK_WINDOW(3),
.PMA_CH2_REG_RX_SIGDET_CHK_WINDOW_EN("TRUE"),
.PMA_CH2_REG_RX_SIGDET_NOSIG_COUNT_SETTING(4),
.PMA_CH2_REG_RX_SIGDET_OOB_DET_COUNT_VAL(0),
.PMA_CH2_REG_SLIP_FIFO_INV_EN("FALSE"),
.PMA_CH2_REG_SLIP_FIFO_INV("POS_EDGE"),
.PMA_CH2_REG_RX_SIGDET_4OOB_DET_SEL(7),
.PMA_CH2_REG_RX_SIGDET_IC_I(10),
.PMA_CH2_REG_RX_OOB_DETECTOR_RESET_N_OW("DISABLE"),
.PMA_CH2_REG_RX_OOB_DETECTOR_RESET_N("FALSE"),
.PMA_CH2_REG_RX_OOB_DETECTOR_PD_OW("DISABLE"),
.PMA_CH2_REG_RX_OOB_DETECTOR_PD("ON"),
.PMA_CH2_REG_RX_TERM_CM_CTRL("5DIV7"),
.PMA_CH2_REG_TX_PD("ON"),
.PMA_CH2_REG_TX_PD_OW("DISABLE"),
.PMA_CH2_REG_TX_CLKPATH_PD("ON"),
.PMA_CH2_REG_TX_CLKPATH_PD_OW("DISABLE"),
.PMA_CH2_REG_TX_BEACON_TIMER_SEL(0),
.PMA_CH2_REG_TX_RXDET_REQ_OW("DISABLE"),
.PMA_CH2_REG_TX_RXDET_REQ("FALSE"),
.PMA_CH2_REG_TX_BEACON_EN_OW("DISABLE"),
.PMA_CH2_REG_TX_BEACON_EN("FALSE"),
.PMA_CH2_REG_TX_EI_EN_OW("DISABLE"),
.PMA_CH2_REG_TX_EI_EN("FALSE"),
.PMA_CH2_REG_TX_RES_CAL_EN("FALSE"),
.PMA_CH2_REG_TX_RES_CAL(51),
.PMA_CH2_REG_TX_BIAS_CAL_EN("FALSE"),
.PMA_CH2_REG_TX_BIAS_CTRL(48),
```

```
.PMA_CH2_REG_TX_RXDET_TIMER_SEL("12CYCLE"),
.PMA_CH2_REG_TX_SYNC_OW("DISABLE"),
.PMA_CH2_REG_TX_SYNC("DISABLE"),
.PMA_CH2_REG_TX_PD_POST("OFF"),
.PMA_CH2_REG_TX_PD_POST_OW("DISABLE"),
.PMA_CH2_REG_TX_RESET_N_OW("DISABLE"),
.PMA_CH2_REG_TX_RESET_N("TRUE"),
.PMA_CH2_REG_TX_DCC_RESET_N_OW("DISABLE"),
.PMA_CH2_REG_TX_DCC_RESET_N("TRUE"),
.PMA_CH2_REG_TX_BUSWIDTH_OW("DISABLE"),
.PMA_CH2_REG_TX_BUSWIDTH("20BIT"),
.PMA_CH2_REG_PLL_READY_OW("DISABLE"),
.PMA_CH2_REG_PLL_READY("TRUE"),
.PMA_CH2_REG_TX_PCLK_SW_OW("DISABLE"),
.PMA_CH2_REG_TX_PCLK_SW("TRUE"),
.PMA_CH2_REG_EI_PCLK_DELAY_SEL(0),
.PMA_CH2_REG_TX_DRV01_DAC0(0),
.PMA_CH2_REG_TX_DRV01_DAC1(10),
.PMA_CH2_REG_TX_DRV01_DAC2(16),
.PMA_CH2_REG_TX_DRV00_DAC0(63),
.PMA_CH2_REG_TX_DRV00_DAC1(53),
.PMA_CH2_REG_TX_DRV00_DAC2(48),
.PMA_CH2_REG_TX_AMP0(8),
.PMA_CH2_REG_TX_AMP1(16),
.PMA_CH2_REG_TX_AMP2(32),
.PMA_CH2_REG_TX_AMP3(48),
.PMA_CH2_REG_TX_AMP4(56),
.PMA_CH2_REG_TX_MARGIN(0),
.PMA_CH2_REG_TX_MARGIN_OW("DISABLE"),
.PMA_CH2_REG_TX_DEEMP(0),
.PMA_CH2_REG_TX_DEEMP_OW("DISABLE"),
.PMA_CH2_REG_TX_SWING("FALSE"),
.PMA_CH2_REG_TX_SWING_OW("DISABLE"),
.PMA_CH2_REG_TX_RXDET_THRESHOLD("100MV"),
.PMA_CH2_REG_TX_BEACON_OSC_CTRL(4),
.PMA_CH2_REG_TX_PREDRV_DAC(1),
.PMA_CH2_REG_TX_PREDRV_CM_CTRL(1),
.PMA_CH2_REG_TX_TX2RX_SLPBACK_EN("FALSE"),
.PMA_CH2_REG_TX_PCLK_EDGE_SEL("POS_EDGE"),
.PMA_CH2_REG_TX_RXDET_STATUS_OW("DISABLE"),
.PMA_CH2_REG_TX_RXDET_STATUS("TRUE"),
.PMA_CH2_REG_TX_PRBS_GEN_EN("FALSE"),
.PMA_CH2_REG_TX_PRBS_GEN_WIDTH_SEL("20BIT"),
.PMA_CH2_REG_TX_PRBS_SEL("PRBS7"),
.PMA_CH2_REG_TX_UDP_DATA(256773),
.PMA_CH2_REG_TX_FIFO_RST_N("FALSE"),
.PMA_CH2_REG_TX_FIFO_WP_CTRL(2),
```

```
.PMA_CH2_REG_TX_FIFO_EN("FALSE"),
.PMA_CH2_REG_TX_DATA_MUX_SEL(0),
.PMA_CH2_REG_TX_ERR_INSERT("FALSE"),
.PMA_CH2_REG_TX_SATA_EN("FALSE"),
.PMA_CH2_REG_RATE_CHANGE_TXPCLK_ON_OW("DISABLE"),
.PMA_CH2_REG_RATE_CHANGE_TXPCLK_ON("ENABLE"),
.PMA_CH2_REG_TX_PULLUP_DAC0(8),
.PMA_CH2_REG_TX_PULLUP_DAC1(8),
.PMA_CH2_REG_TX_PULLUP_DAC2(8),
.PMA_CH2_REG_TX_PULLUP_DAC3(8),
.PMA_CH2_REG_TX_OOB_DELAY_SEL(0),
.PMA_CH2_REG_TX_POLARITY("NORMAL"),
.PMA_CH2_REG_TX_SLPBK_AMP(1),
.PMA_CH2_REG_TX_LS_MODE_EN("FALSE"),
.PMA_CH2_REG_TX_JTAG_MODE_EN_OW("DISABLE"),
.PMA_CH2_REG_TX_JTAG_MODE_EN("FALSE"),
.PMA_CH2_REG_RX_JTAG_MODE_EN_OW("DISABLE"),
.PMA_CH2_REG_RX_JTAG_MODE_EN("FALSE"),
.PMA_CH2_REG_RX_JTAG_OE("DISABLE"),
.PMA_CH2_REG_RX_ACJTAG_VHYSTSE(0),
.PMA_CH2_REG_TX_FBCLK_FAR_EN("FALSE"),
.PMA_CH2_REG_RX_TERM_MODE_CTRL(6),
.PMA_CH2_REG_PLPBK_TXPCLK_EN("TRUE"),
.PMA_CH2_REG_TX_609_600(0),
.PMA_CH2_REG_RX_CDR_617_610(0),
.PMA_CH2_REG_RX_CDR_623_618(0),
.PMA_CH2_REG_RX_631_624(0),
.PMA_CH2_REG_RX_639_632(0),
.PMA_CH2_REG_RX_647_640(0),
.PMA_CH2_REG_RX_655_648(0),
.PMA_CH2_REG_RX_659_656(0),
.PMA_CH2_CFG_LANE_POWERUP("OFF"),
.PMA_CH2_CFG_PMA_POR_N("FALSE"),
.PMA_CH2_CFG_RX_LANE_POWERUP("OFF"),
.PMA_CH2_CFG_RX_PMA_RSTN("FALSE"),
.PMA_CH2_CFG_TX_LANE_POWERUP("OFF"),
.PMA_CH2_CFG_TX_PMA_RSTN("FALSE"),
.PMA_CH2_CFG_CTLE_ADP_RSTN("TRUE"),
.PMA_CH2_REG_RESERVED_48_45(0),
.PMA_CH2_REG_RESERVED_69(0),
.PMA_CH2_REG_RESERVED_77_76(0),
.PMA_CH2_REG_RESERVED_171_164(0),
.PMA_CH2_REG_RESERVED_175_172(0),
.PMA_CH2_REG_RESERVED_190(0),
.PMA_CH2_REG_RESERVED_233_232(0),
.PMA_CH2_REG_RESERVED_235_234(0),
.PMA_CH2_REG_RESERVED_241_240(0),
```

```
.PMA_CH2_REG_RESERVED_285_283(0),
.PMA_CH2_REG_RESERVED_286(0),
.PMA_CH2_REG_RESERVED_295(0),
.PMA_CH2_REG_RESERVED_298(0),
.PMA_CH2_REG_RESERVED_332_325(0),
.PMA_CH2_REG_RESERVED_340_333(0),
.PMA_CH2_REG_RESERVED_348_341(0),
.PMA_CH2_REG_RESERVED_354_349(0),
.PMA_CH2_REG_RESERVED_373(0),
.PMA_CH2_REG_RESERVED_376(0),
.PMA_CH2_REG_RESERVED_452(0),
.PMA_CH2_REG_RESERVED_502_499(0),
.PMA_CH2_REG_RESERVED_506_505(0),
.PMA_CH2_REG_RESERVED_550_549(0),
.PMA_CH2_REG_RESERVED_556_552(0),
.PMA_CH3_REG_RX_PD("ON"),
.PMA_CH3_REG_RX_PD_EN("FALSE"),
.PMA_CH3_REG_RX_CLKPATH_PD("ON"),
.PMA_CH3_REG_RX_CLKPATH_PD_EN("FALSE"),
.PMA_CH3_REG_RX_DATAPATH_PD("ON"),
.PMA_CH3_REG_RX_DATAPATH_PD_EN("FALSE"),
.PMA_CH3_REG_RX_SIGDET_PD("ON"),
.PMA_CH3_REG_RX_SIGDET_PD_EN("FALSE"),
.PMA_CH3_REG_RX_DCC_RST_N("TRUE"),
.PMA_CH3_REG_RX_DCC_RST_N_EN("FALSE"),
.PMA_CH3_REG_RX_CDR_RST_N("TRUE"),
.PMA_CH3_REG_RX_CDR_RST_N_EN("FALSE"),
.PMA_CH3_REG_RX_SIGDET_RST_N("TRUE"),
.PMA_CH3_REG_RX_SIGDET_RST_N_EN("FALSE"),
.PMA_CH3_REG_RXPCLK_SLIP("FALSE"),
.PMA_CH3_REG_RXPCLK_SLIP_OW("DISABLE"),
.PMA_CH3_REG_RX_PCLKSWITCH_RST_N("TRUE"),
.PMA_CH3_REG_RX_PCLKSWITCH_RST_N_EN("FALSE"),
.PMA_CH3_REG_RX_PCLKSWITCH("FALSE"),
.PMA_CH3_REG_RX_PCLKSWITCH_EN("FALSE"),
.PMA_CH3_REG_RX_HIGHZ("FALSE"),
.PMA_CH3_REG_RX_HIGHZ_EN("FALSE"),
.PMA_CH3_REG_RX_EQ_C_SET(8),
.PMA_CH3_REG_RX_EQ_R_SET(8),
.PMA_CH3_REG_RX_BUSWIDTH("20BIT"),
.PMA_CH3_REG_RX_BUSWIDTH_EN("FALSE"),
.PMA_CH3_REG_RX_RATE("DIV1"),
.PMA_CH3_REG_RX_RATE_EN("FALSE"),
.PMA_CH3_REG_RX_RES_TRIM(51),
.PMA_CH3_REG_RX_RES_TRIM_EN("FALSE"),
.PMA_CH3_REG_RX_EQ_OFF("FALSE"),
.PMA_CH3_REG_RX_PREAMP_IC(1367),
```

```
.PMA_CH3_REG_RX_PCLK_EDGE_SEL("POS_EDGE"),
.PMA_CH3_REG_RX_PIBUF_IC(2),
.PMA_CH3_REG_RX_DCC_IC_RX(3),
.PMA_CH3_REG_RX_DCC_IC_TX(3),
.PMA_CH3_REG_RX_ICTRL_TRX("100PCT"),
.PMA_CH3_REG_RX_ICTRL_SIGDET(5),
.PMA_CH3_REG_RX_ICTRL_PREAMP("100PCT"),
.PMA_CH3_REG_RX_ICTRL_SLICER("100PCT"),
.PMA_CH3_REG_RX_ICTRL_PIBUF("100PCT"),
.PMA_CH3_REG_RX_ICTRL_PI("100PCT"),
.PMA_CH3_REG_RX_ICTRL_DCC("100PCT"),
.PMA_CH3_REG_RX_ICTRL_PREDRV("100PCT"),
.PMA_CH3_REG_TX_RATE("DIV1"),
.PMA_CH3_REG_TX_RATE_EN("FALSE"),
.PMA_CH3_REG_RX_TX2RX_PLPBK_RST_N("TRUE"),
.PMA_CH3_REG_RX_TX2RX_PLPBK_RST_N_EN("FALSE"),
.PMA_CH3_REG_RX_TX2RX_PLPBK_EN("FALSE"),
.PMA_CH3_REG_TXCLK_SEL("PLL"),
.PMA_CH3_REG_RX_DATA_POLARITY("NORMAL"),
.PMA_CH3_REG_RX_ERR_INSERT("FALSE"),
.PMA_CH3_REG_UDP_CHK_EN("FALSE"),
.PMA_CH3_REG_PRBS_SEL("PRBS7"),
.PMA_CH3_REG_PRBS_CHK_EN("FALSE"),
.PMA_CH3_REG_PRBS_CHK_WIDTH_SEL("20BIT"),
.PMA_CH3_REG_BIST_CHK_PAT_SEL("PRBS"),
.PMA_CH3_REG_LOAD_ERR_CNT("DISABLE"),
.PMA_CH3_REG_CHK_COUNTER_EN("FALSE"),
.PMA_CH3_REG_CDR_PROP_GAIN(5),
.PMA_CH3_REG_CDR_PROP_TURBO_GAIN(6),
.PMA_CH3_REG_CDR_INT_GAIN(5),
.PMA_CH3_REG_CDR_INT_TURBO_GAIN(6),
.PMA_CH3_REG_CDR_INT_SAT_MAX(992),
.PMA_CH3_REG_CDR_INT_SAT_MIN(32),
.PMA_CH3_REG_CDR_INT_RST("FALSE"),
.PMA_CH3_REG_CDR_INT_RST_OW("DISABLE"),
.PMA_CH3_REG_CDR_PROP_RST("FALSE"),
.PMA_CH3_REG_CDR_PROP_RST_OW("DISABLE"),
.PMA_CH3_REG_CDR_LOCK_RST("FALSE"),
.PMA_CH3_REG_CDR_LOCK_RST_OW("DISABLE"),
.PMA_CH3_REG_CDR_RX_PI_FORCE_SEL(0),
.PMA_CH3_REG_CDR_RX_PI_FORCE_D(0),
.PMA_CH3_REG_CDR_LOCK_TIMER("1_2U"),
.PMA_CH3_REG_CDR_TURBO_MODE_TIMER(1),
.PMA_CH3_REG_CDR_LOCK_VAL("FALSE"),
.PMA_CH3_REG_CDR_LOCK_OW("DISABLE"),
.PMA_CH3_REG_CDR_INT_SAT_DET_EN("TRUE"),
.PMA_CH3_REG_CDR_SAT_DET_STATUS_EN("FALSE"),
```

```
.PMA_CH3_REG_CDR_SAT_DET_STATUS_RESET_EN("FALSE"),
.PMA_CH3_REG_CDR_PI_CTRL_RST("FALSE"),
.PMA_CH3_REG_CDR_PI_CTRL_RST_OW("DISABLE"),
.PMA_CH3_REG_CDR_SAT_DET_RST("FALSE"),
.PMA_CH3_REG_CDR_SAT_DET_RST_OW("DISABLE"),
.PMA_CH3_REG_CDR_SAT_DET_STICKY_RST("FALSE"),
.PMA_CH3_REG_CDR_SAT_DET_STICKY_RST_OW("DISABLE"),
.PMA_CH3_REG_CDR_SIGDET_STATUS_DIS("FALSE"),
.PMA_CH3_REG_CDR_SAT_DET_TIMER(2),
.PMA_CH3_REG_CDR_SAT_DET_STATUS_VAL("FALSE"),
.PMA_CH3_REG_CDR_SAT_DET_STATUS_OW("DISABLE"),
.PMA_CH3_REG_CDR_TURBO_MODE_EN("TRUE"),
.PMA_CH3_REG_CDR_STATUS_RADDR_INIT(0),
.PMA_CH3_REG_CDR_STATUS_FIFO_EN("TRUE"),
.PMA_CH3_REG_PMA_TEST_SEL(0),
.PMA_CH3_REG_OOB_COMWAKE_GAP_MIN(3),
.PMA_CH3_REG_OOB_COMWAKE_GAP_MAX(11),
.PMA_CH3_REG_OOB_COMINIT_GAP_MIN(15),
.PMA_CH3_REG_OOB_COMINIT_GAP_MAX(35),
.PMA_CH3_REG_RX_PIBUF_IC_TX(1),
.PMA_CH3_REG_COMWAKE_STATUS_CLEAR(0),
.PMA_CH3_REG_COMINIT_STATUS_CLEAR(0),
.PMA_CH3_REG_RX_SYNC_RST_N_EN("FALSE"),
.PMA_CH3_REG_RX_SYNC_RST_N("TRUE"),
.PMA_CH3_REG_RX_SATA_COMINIT_OW("DISABLE"),
.PMA_CH3_REG_RX_SATA_COMINIT("FALSE"),
.PMA_CH3_REG_RX_SATA_COMWAKE_OW("DISABLE"),
.PMA_CH3_REG_RX_SATA_COMWAKE("FALSE"),
.PMA_CH3_REG_RX_DCC_DISABLE("ENABLE"),
.PMA_CH3_REG_TX_DCC_DISABLE("ENABLE"),
.PMA_CH3_REG_RX_SLIP_SEL_EN("FALSE"),
.PMA_CH3_REG_RX_SLIP_SEL(0),
.PMA_CH3_REG_RX_SLIP_EN("FALSE"),
.PMA_CH3_REG_RX_SIGDET_STATUS_SEL(5),
.PMA_CH3_REG_RX_SIGDET_FSM_RST_N("TRUE"),
.PMA_CH3_REG_RX_SIGDET_STATUS_OW("DISABLE"),
.PMA_CH3_REG_RX_SIGDET_STATUS("FALSE"),
.PMA_CH3_REG_RX_SIGDET_VTH("50MV"),
.PMA_CH3_REG_RX_SIGDET_GRM(0),
.PMA_CH3_REG_RX_SIGDET_PULSE_EXT("DISABLE"),
.PMA_CH3_REG_RX_SIGDET_CH2_SEL(0),
.PMA_CH3_REG_RX_SIGDET_CH2_CHK_WINDOW(3),
.PMA_CH3_REG_RX_SIGDET_CHK_WINDOW_EN("TRUE"),
.PMA_CH3_REG_RX_SIGDET_NOSIG_COUNT_SETTING(4),
.PMA_CH3_REG_RX_SIGDET_OOB_DET_COUNT_VAL(0),
.PMA_CH3_REG_SLIP_FIFO_INV_EN("FALSE"),
.PMA_CH3_REG_SLIP_FIFO_INV("POS_EDGE"),
```

```
.PMA_CH3_REG_RX_SIGDET_4OOB_DET_SEL(7),
.PMA_CH3_REG_RX_SIGDET_IC_I(10),
.PMA_CH3_REG_RX_OOB_DETECTOR_RESET_N_OW("DISABLE"),
.PMA_CH3_REG_RX_OOB_DETECTOR_RESET_N("FALSE"),
.PMA_CH3_REG_RX_OOB_DETECTOR_PD_OW("DISABLE"),
.PMA_CH3_REG_RX_OOB_DETECTOR_PD("ON"),
.PMA_CH3_REG_RX_TERM_CM_CTRL("5DIV7"),
.PMA_CH3_REG_TX_PD("ON"),
.PMA_CH3_REG_TX_PD_OW("DISABLE"),
.PMA_CH3_REG_TX_CLKPATH_PD("ON"),
.PMA_CH3_REG_TX_CLKPATH_PD_OW("DISABLE"),
.PMA_CH3_REG_TX_BEACON_TIMER_SEL(0),
.PMA_CH3_REG_TX_RXDET_REQ_OW("DISABLE"),
.PMA_CH3_REG_TX_RXDET_REQ("FALSE"),
.PMA_CH3_REG_TX_BEACON_EN_OW("DISABLE"),
.PMA_CH3_REG_TX_BEACON_EN("FALSE"),
.PMA_CH3_REG_TX_EI_EN_OW("DISABLE"),
.PMA_CH3_REG_TX_EI_EN("FALSE"),
.PMA_CH3_REG_TX_RES_CAL_EN("FALSE"),
.PMA_CH3_REG_TX_RES_CAL(51),
.PMA_CH3_REG_TX_BIAS_CAL_EN("FALSE"),
.PMA_CH3_REG_TX_BIAS_CTRL(48),
.PMA_CH3_REG_TX_RXDET_TIMER_SEL("12CYCLE"),
.PMA_CH3_REG_TX_SYNC_OW("DISABLE"),
.PMA_CH3_REG_TX_SYNC("DISABLE"),
.PMA_CH3_REG_TX_PD_POST("OFF"),
.PMA_CH3_REG_TX_PD_POST_OW("DISABLE"),
.PMA_CH3_REG_TX_RESET_N_OW("DISABLE"),
.PMA_CH3_REG_TX_RESET_N("TRUE"),
.PMA_CH3_REG_TX_DCC_RESET_N_OW("DISABLE"),
.PMA_CH3_REG_TX_DCC_RESET_N("TRUE"),
.PMA_CH3_REG_TX_BUSWIDTH_OW("DISABLE"),
.PMA_CH3_REG_TX_BUSWIDTH("20BIT"),
.PMA_CH3_REG_PLL_READY_OW("DISABLE"),
.PMA_CH3_REG_PLL_READY("TRUE"),
.PMA_CH3_REG_TX_PCLK_SW_OW("DISABLE"),
.PMA_CH3_REG_TX_PCLK_SW("TRUE"),
.PMA_CH3_REG_EI_PCLK_DELAY_SEL(0),
.PMA_CH3_REG_TX_DRV01_DAC0(0),
.PMA_CH3_REG_TX_DRV01_DAC1(10),
.PMA_CH3_REG_TX_DRV01_DAC2(16),
.PMA_CH3_REG_TX_DRV00_DAC0(63),
.PMA_CH3_REG_TX_DRV00_DAC1(53),
.PMA_CH3_REG_TX_DRV00_DAC2(48),
.PMA_CH3_REG_TX_AMP0(8),
.PMA_CH3_REG_TX_AMP1(16),
.PMA_CH3_REG_TX_AMP2(32),
```

```
.PMA_CH3_REG_TX_AMP3(48),
.PMA_CH3_REG_TX_AMP4(56),
.PMA_CH3_REG_TX_MARGIN(0),
.PMA_CH3_REG_TX_MARGIN_OW("DISABLE"),
.PMA_CH3_REG_TX_DEEMP(0),
.PMA_CH3_REG_TX_DEEMP_OW("DISABLE"),
.PMA_CH3_REG_TX_SWING("FALSE"),
.PMA_CH3_REG_TX_SWING_OW("DISABLE"),
.PMA_CH3_REG_TX_RXDET_THRESHOLD("100MV"),
.PMA_CH3_REG_TX_BEACON_OSC_CTRL(4),
.PMA_CH3_REG_TX_PREDRV_DAC(1),
.PMA_CH3_REG_TX_PREDRV_CM_CTRL(1),
.PMA_CH3_REG_TX_TX2RX_SLPBACK_EN("FALSE"),
.PMA_CH3_REG_TX_PCLK_EDGE_SEL("POS_EDGE"),
.PMA_CH3_REG_TX_RXDET_STATUS_OW("DISABLE"),
.PMA_CH3_REG_TX_RXDET_STATUS("TRUE"),
.PMA_CH3_REG_TX_PRBS_GEN_EN("FALSE"),
.PMA_CH3_REG_TX_PRBS_GEN_WIDTH_SEL("20BIT"),
.PMA_CH3_REG_TX_PRBS_SEL("PRBS7"),
.PMA_CH3_REG_TX_UDP_DATA(256773),
.PMA_CH3_REG_TX_FIFO_RST_N("FALSE"),
.PMA_CH3_REG_TX_FIFO_WP_CTRL(2),
.PMA_CH3_REG_TX_FIFO_EN("FALSE"),
.PMA_CH3_REG_TX_DATA_MUX_SEL(0),
.PMA_CH3_REG_TX_ERR_INSERT("FALSE"),
.PMA_CH3_REG_TX_SATA_EN("FALSE"),
.PMA_CH3_REG_RATE_CHANGE_TXPCLK_ON_OW("DISABLE"),
.PMA_CH3_REG_RATE_CHANGE_TXPCLK_ON("ENABLE"),
.PMA_CH3_REG_TX_PULLUP_DAC0(8),
.PMA_CH3_REG_TX_PULLUP_DAC1(8),
.PMA_CH3_REG_TX_PULLUP_DAC2(8),
.PMA_CH3_REG_TX_PULLUP_DAC3(8),
.PMA_CH3_REG_TX_OOB_DELAY_SEL(0),
.PMA_CH3_REG_TX_POLARITY("NORMAL"),
.PMA_CH3_REG_TX_SLPBK_AMP(1),
.PMA_CH3_REG_TX_LS_MODE_EN("FALSE"),
.PMA_CH3_REG_TX_JTAG_MODE_EN_OW("DISABLE"),
.PMA_CH3_REG_TX_JTAG_MODE_EN("FALSE"),
.PMA_CH3_REG_RX_JTAG_MODE_EN_OW("DISABLE"),
.PMA_CH3_REG_RX_JTAG_MODE_EN("FALSE"),
.PMA_CH3_REG_RX_JTAG_OE("DISABLE"),
.PMA_CH3_REG_RX_ACJTAG_VHYSTSE(0),
.PMA_CH3_REG_TX_FBCLK_FAR_EN("FALSE"),
.PMA_CH3_REG_RX_TERM_MODE_CTRL(6),
.PMA_CH3_REG_PLPBK_TXPCLK_EN("TRUE"),
.PMA_CH3_REG_TX_609_600(0),
.PMA_CH3_REG_RX_CDR_617_610(0),
```

```
.PMA_CH3_REG_RX_CDR_623_618(0),
.PMA_CH3_REG_RX_631_624(0),
.PMA_CH3_REG_RX_639_632(0),
.PMA_CH3_REG_RX_647_640(0),
.PMA_CH3_REG_RX_655_648(0),
.PMA_CH3_REG_RX_659_656(0),
.PMA_CH3_CFG_LANE_POWERUP("OFF"),
.PMA_CH3_CFG_PMA_POR_N("FALSE"),
.PMA_CH3_CFG_RX_LANE_POWERUP("OFF"),
.PMA_CH3_CFG_RX_PMA_RSTN("FALSE"),
.PMA_CH3_CFG_TX_LANE_POWERUP("OFF"),
.PMA_CH3_CFG_TX_PMA_RSTN("FALSE"),
.PMA_CH3_CFG_CTLE_ADP_RSTN("TRUE"),
.PMA_CH3_REG_RESERVED_48_45(0),
.PMA_CH3_REG_RESERVED_69(0),
.PMA_CH3_REG_RESERVED_77_76(0),
.PMA_CH3_REG_RESERVED_171_164(0),
.PMA_CH3_REG_RESERVED_175_172(0),
.PMA_CH3_REG_RESERVED_190(0),
.PMA_CH3_REG_RESERVED_233_232(0),
.PMA_CH3_REG_RESERVED_235_234(0),
.PMA_CH3_REG_RESERVED_241_240(0),
.PMA_CH3_REG_RESERVED_285_283(0),
.PMA_CH3_REG_RESERVED_286(0),
.PMA_CH3_REG_RESERVED_295(0),
.PMA_CH3_REG_RESERVED_298(0),
.PMA_CH3_REG_RESERVED_332_325(0),
.PMA_CH3_REG_RESERVED_340_333(0),
.PMA_CH3_REG_RESERVED_348_341(0),
.PMA_CH3_REG_RESERVED_354_349(0),
.PMA_CH3_REG_RESERVED_373(0),
.PMA_CH3_REG_RESERVED_376(0),
.PMA_CH3_REG_RESERVED_452(0),
.PMA_CH3_REG_RESERVED_502_499(0),
.PMA_CH3_REG_RESERVED_506_505(0),
.PMA_CH3_REG_RESERVED_550_549(0),
.PMA_CH3_REG_RESERVED_556_552(0),
.PMA_PLL0_REG_PLL_POWERDOWN_OW("DISABLE"),
.PMA_PLL0_REG_PLL_POWERDOWN("ON"),
.PMA_PLL0_REG_PLL_RESET_N_OW("DISABLE"),
.PMA_PLL0_REG_PLL_RESET_N("TRUE"),
.PMA_PLL0_REG_PLL_READY_OW("DISABLE"),
.PMA_PLL0_REG_PLL_READY("FALSE"),
.PMA_PLL0_REG_LANE_SYNC_OW("DISABLE"),
.PMA_PLL0_REG_LANE_SYNC("FALSE"),
.PMA_PLL0_REG_LOCKDET_REPEAT("DISABLE"),
.PMA_PLL0_REG_RESCAL_I_CODE_PMA("DISABLE"),
```

```
.PMA_PLL0_REG_RESCAL_RESET_N_OW("DISABLE"),
.PMA_PLL0_REG_RESCAL_RESET_N("FALSE"),
.PMA_PLL0_REG_RESCAL_DONE_OW("DISABLE"),
.PMA_PLL0_REG_RESCAL_DONE("FALSE"),
.PMA_PLL0_REG_RESCAL_CODE_OW("DISABLE"),
.PMA_PLL0_REG_LDO_VREF_SEL(2),
.PMA_PLL0_REG_BIAS_VCOREP_C(1),
.PMA_PLL0_REG_RESCAL_I_CODE(32),
.PMA_PLL0_REG_RESCAL_ONCHIP_SMALL_OW("DISABLE"),
.PMA_PLL0_REG_RESCAL_ONCHIP_SMALL(0),
.PMA_PLL0_REG_JTAG_OE("DISABLE"),
.PMA_PLL0_REG_JTAG_AC_MODE("DISABLE"),
.PMA_PLL0_REG_JTAG_VHYSTSEL(0),
.PMA_PLL0_REG_PLL_LOCKDET_EN_OW("DISABLE"),
.PMA_PLL0_REG_PLL_LOCKDET_EN("FALSE"),
.PMA_PLL0_REG_PLL_LOCKDET_RESET_N_OW("DISABLE"),
.PMA_PLL0_REG_PLL_LOCKDET_RESET_N("FALSE"),
.PMA_PLL0_REG_PLL_LOCKED_OW("DISABLE"),
.PMA_PLL0_REG_PLL_LOCKED("FALSE"),
.PMA_PLL0_REG_PLL_LOCKED_STICKY_CLEAR("FALSE"),
.PMA_PLL0_REG_PLL_UNLOCKED_STICKY_CLEAR("FALSE"),
.PMA_PLL0_REG_NOFBCLK_STICKY_CLEAR("FALSE"),
.PMA_PLL0_REG_PLL_LOCKDET_REFCT(7),
.PMA_PLL0_REG_PLL_LOCKDET_FBCT(7),
.PMA_PLL0_REG_PLL_LOCKDET_LOCKCT(4),
.PMA_PLL0_REG_PLL_LOCKDET_ITER(3),
.PMA_PLL0_REG_PLL_UNLOCKDET_ITER(2),
.PMA_PLL0_REG_PD_VCO("ON"),
.PMA_PLL0_REG_FBCLK_TEST_EN("FALSE"),
.PMA_PLL0_REG_REFCLK_TEST_EN("FALSE"),
.PMA_PLL0_REG_TEST_SEL(0),
.PMA_PLL0_REG_TEST_V_EN("FALSE"),
.PMA_PLL0_REG_TEST_SIG_HALF_EN("FALSE"),
.PMA_PLL0_REG_TEST_FSM(0),
.PMA_PLL0_REG_REFCLK_OUT_PD("OFF"),
.PMA_PLL0_REG_BGR_STARTUP_EN("FALSE"),
.PMA_PLL0_REG_BGR_STARTUP("FALSE"),
.PMA_PLL0_REG_PD_BGR("ON"),
.PMA_PLL0_REG_REFCLK_TERM_VCM_EN("TRUE"),
.PMA_PLL0_REG_FBDIVA_5_EN("TRUE"),
.PMA_PLL0_REG_FBDIVB(1),
.PMA_PLL0_REG_RESET_N_PFDQP_OW("DISABLE"),
.PMA_PLL0_REG_RESET_N_PFDQP("FALSE"),
.PMA_PLL0_REG_QPCURRENT(12),
.PMA_PLL0_REG_VC_FORCE_EN("FALSE"),
.PMA_PLL0_REG_VCRESET_C_RING(24),
.PMA_PLL0_REG_LPF_R_C(0),
```

```
.PMA_PLL0_REG_LPF_TR_C(2),
.PMA_PLL0_REG_PD_BIAS("ON"),
.PMA_PLL0_REG_ICTRL_PLL(1),
.PMA_PLL0_REG_BIAS_QP(1),
.PMA_PLL0_REG_BIAS_LANE_SYNC(1),
.PMA_PLL0_REG_BIAS_CLKBUFS1(1),
.PMA_PLL0_REG_TXPCLK_SEL(0),
.PMA_PLL0_REG_BIAS_CLKBUFS3(1),
.PMA_PLL0_REG_LANE_SYNC_EN("FALSE"),
.PMA_PLL0_REG_LANE_SYNC_EN_OW("DISABLE"),
.PMA_PLL0_REG_BIAS_D2S(1),
.PMA_PLL0_REG_BIAS_REFD2S_C(1),
.PMA_PLL0_REG_BIAS_VCRST_C(1),
.PMA_PLL0_REG_BIAS_REFBUF_C(1),
.PMA_PLL0_REG_CLKBUFS1_C(1),
.PMA_PLL0_REG_CLKBUFS2_C(6),
.PMA_PLL0_REG_CLKBUFS3_C(6),
.PMA_PLL0_REG_CLKBUFS4_C(1),
.PMA_PLL0_REG_PLL_REFCLK_CML_SEL(0),
.PMA_PLL0_REG_REFCLK_SEL("FALSE"),
.PMA_PLL0_REG_RESCAL_R_CODE_SIGN("TRUE"),
.PMA_PLL0_REG_PLL_UNLOCKED_OW("DISABLE"),
.PMA_PLL0_REG_PLL_UNLOCKED("FALSE"),
.PMA_PLL0_REG_PLL_LOCKDET_MODE("FALSE"),
.PMA_PLL0_REG_PLL_CLKBUF_PD_LEFT("ON"),
.PMA_PLL0_REG_PLL_CLKBUF_PD_RIGHT("ON"),
.PMA_PLL0_REG_RESCAL_EN("FALSE"),
.PMA_PLL0_REG_RESCAL_I_CODE_VAL(0),
.PMA_PLL0_REG_RESCAL_I_CODE_OW(0),
.PMA_PLL0_REG_RESCAL_ITER_VALID_SEL(0),
.PMA_PLL0_REG_RESCAL_WAIT_SEL(0),
.PMA_PLL0_REG_I_CTRL_MAX(45),
.PMA_PLL0_REG_I_CTRL_MIN(19),
.PMA_PLL0_REG_RESERVED_167_160(0),
.PMA_PLL0_REG_RESERVED_175_168(0),
.PMA_PLL0_REG_RESERVED_183_176(0),
.PMA_PLL0_REG_RESERVED_191_184(0),
.PARM_CFG_HSST_RSTN("FALSE"),
.PARM_PLL0_POWERUP("OFF"),
.PARM_PLL0_RSTN("FALSE"),
.PMA_PLL1_REG_PLL_POWERDOWN_OW("DISABLE"),
.PMA_PLL1_REG_PLL_POWERDOWN("ON"),
.PMA_PLL1_REG_PLL_RESET_N_OW("DISABLE"),
.PMA_PLL1_REG_PLL_RESET_N("TRUE"),
.PMA_PLL1_REG_PLL_READY_OW("DISABLE"),
.PMA_PLL1_REG_PLL_READY("FALSE"),
.PMA_PLL1_REG_LANE_SYNC_OW("DISABLE"),
```

```
.PMA_PLL1_REG_LANE_SYNC("FALSE"),
.PMA_PLL1_REG_LOCKDET_REPEAT("DISABLE"),
.PMA_PLL1_REG_RESCAL_I_CODE_PMA("DISABLE"),
.PMA_PLL1_REG_RESCAL_RESET_N_OW("DISABLE"),
.PMA_PLL1_REG_RESCAL_RESET_N("FALSE"),
.PMA_PLL1_REG_RESCAL_DONE_OW("DISABLE"),
.PMA_PLL1_REG_RESCAL_DONE("FALSE"),
.PMA_PLL1_REG_RESCAL_CODE_OW("DISABLE"),
.PMA_PLL1_REG_LDO_VREF_SEL(2),
.PMA_PLL1_REG_BIAS_VCOREP_C(1),
.PMA_PLL1_REG_RESCAL_I_CODE(32),
.PMA_PLL1_REG_RESCAL_ONCHIP_SMALL_OW("DISABLE"),
.PMA_PLL1_REG_RESCAL_ONCHIP_SMALL(0),
.PMA_PLL1_REG_JTAG_OE("DISABLE"),
.PMA_PLL1_REG_JTAG_AC_MODE("DISABLE"),
.PMA_PLL1_REG_JTAG_VHYSTSEL(0),
.PMA_PLL1_REG_PLL_LOCKDET_EN_OW("DISABLE"),
.PMA_PLL1_REG_PLL_LOCKDET_EN("FALSE"),
.PMA_PLL1_REG_PLL_LOCKDET_RESET_N_OW("DISABLE"),
.PMA_PLL1_REG_PLL_LOCKDET_RESET_N("FALSE"),
.PMA_PLL1_REG_PLL_LOCKED_OW("DISABLE"),
.PMA_PLL1_REG_PLL_LOCKED("FALSE"),
.PMA_PLL1_REG_PLL_LOCKED_STICKY_CLEAR("FALSE"),
.PMA_PLL1_REG_PLL_UNLOCKED_STICKY_CLEAR("FALSE"),
.PMA_PLL1_REG_NOFBCLK_STICKY_CLEAR("FALSE"),
.PMA_PLL1_REG_PLL_LOCKDET_REFCT(7),
.PMA_PLL1_REG_PLL_LOCKDET_FBCT(7),
.PMA_PLL1_REG_PLL_LOCKDET_LOCKCT(4),
.PMA_PLL1_REG_PLL_LOCKDET_ITER(3),
.PMA_PLL1_REG_PLL_UNLOCKDET_ITER(2),
.PMA_PLL1_REG_PD_VCO("ON"),
.PMA_PLL1_REG_FBCLK_TEST_EN("FALSE"),
.PMA_PLL1_REG_REFCLK_TEST_EN("FALSE"),
.PMA_PLL1_REG_TEST_SEL(0),
.PMA_PLL1_REG_TEST_V_EN("FALSE"),
.PMA_PLL1_REG_TEST_SIG_HALF_EN("FALSE"),
.PMA_PLL1_REG_TEST_FSM(0),
.PMA_PLL1_REG_REFCLK_OUT_PD("OFF"),
.PMA_PLL1_REG_BGR_STARTUP_EN("FALSE"),
.PMA_PLL1_REG_BGR_STARTUP("FALSE"),
.PMA_PLL1_REG_PD_BGR("ON"),
.PMA_PLL1_REG_REFCLK_TERM_VCM_EN("TRUE"),
.PMA_PLL1_REG_FBDIVA_5_EN("TRUE"),
.PMA_PLL1_REG_FBDIVB(1),
.PMA_PLL1_REG_RESET_N_PFDQP_OW("DISABLE"),
.PMA_PLL1_REG_RESET_N_PFDQP("FALSE"),
.PMA_PLL1_REG_QPCURRENT(12),
```

```
        .PMA_PLL1_REG_VC_FORCE_EN("FALSE"),
        .PMA_PLL1_REG_VCRESET_C_RING(24),
        .PMA_PLL1_REG_LPF_R_C(0),
        .PMA_PLL1_REG_LPF_TR_C(2),
        .PMA_PLL1_REG_PD_BIAS("ON"),
        .PMA_PLL1_REG_ICTRL_PLL(1),
        .PMA_PLL1_REG_BIAS_QP(1),
        .PMA_PLL1_REG_BIAS_LANE_SYNC(1),
        .PMA_PLL1_REG_BIAS_CLKBUFS1(1),
        .PMA_PLL1_REG_TXPCLK_SEL(0),
        .PMA_PLL1_REG_BIAS_CLKBUFS3(1),
        .PMA_PLL1_REG_LANE_SYNC_EN("FALSE"),
        .PMA_PLL1_REG_LANE_SYNC_EN_OW("DISABLE"),
        .PMA_PLL1_REG_BIAS_D2S(1),
        .PMA_PLL1_REG_BIAS_REFD2S_C(1),
        .PMA_PLL1_REG_BIAS_VCRST_C(1),
        .PMA_PLL1_REG_BIAS_REFBUF_C(1),
        .PMA_PLL1_REG_CLKBUFS1_C(1),
        .PMA_PLL1_REG_CLKBUFS2_C(6),
        .PMA_PLL1_REG_CLKBUFS3_C(6),
        .PMA_PLL1_REG_CLKBUFS4_C(1),
        .PMA_PLL1_REG_PLL_REFCLK_CML_SEL(0),
        .PMA_PLL1_REG_REFCLK_SEL("FALSE"),
        .PMA_PLL1_REG_RESCAL_R_CODE_SIGN("TRUE"),
        .PMA_PLL1_REG_PLL_UNLOCKED_OW("DISABLE"),
        .PMA_PLL1_REG_PLL_UNLOCKED("FALSE"),
        .PMA_PLL1_REG_PLL_LOCKDET_MODE("FALSE"),
        .PMA_PLL1_REG_PLL_CLKBUF_PD_LEFT("ON"),
        .PMA_PLL1_REG_PLL_CLKBUF_PD_RIGHT("ON"),
        .PMA_PLL1_REG_RESCAL_EN("FALSE"),
        .PMA_PLL1_REG_RESCAL_I_CODE_VAL(0),
        .PMA_PLL1_REG_RESCAL_I_CODE_OW(0),
        .PMA_PLL1_REG_RESCAL_ITER_VALID_SEL(0),
        .PMA_PLL1_REG_RESCAL_WAIT_SEL(0),
        .PMA_PLL1_REG_I_CTRL_MAX(45),
        .PMA_PLL1_REG_I_CTRL_MIN(19),
        .PMA_PLL1_REG_RESERVED_167_160(0),
        .PMA_PLL1_REG_RESERVED_175_168(0),
        .PMA_PLL1_REG_RESERVED_183_176(0),
        .PMA_PLL1_REG_RESERVED_191_184(0),
        .PARM_PLL1_POWERUP("OFF"),
        .PARM_PLL1_RSTN("FALSE"),
        .PARM_GRSN_DIS("FALSE"),
        .PARM_CFG_RSTN("FALSE")
    ) GTP_HSST_E2_inst (
        .P_CA_ALIGN_RX(),              // OUTPUT[3:0]
        .P_CA_ALIGN_TX(),              // OUTPUT[3:0]
```

```
.P_CFG_RDATA(),                 // OUTPUT[7:0]
.P_PCS_LSM_SYNCED(),            // OUTPUT[3:0]
.P_PCS_RX_MCB_STATUS(),         // OUTPUT[3:0]
.P_RCLK2FABRIC(),               // OUTPUT[3:0]
.P_RDATA_0(),                   // OUTPUT[46:0]
.P_RDATA_1(),                   // OUTPUT[46:0]
.P_RDATA_2(),                   // OUTPUT[46:0]
.P_RDATA_3(),                   // OUTPUT[46:0]
.P_RESCAL_I_CODE_O(),           // OUTPUT[5:0]
.P_TCLK2FABRIC(),               // OUTPUT[3:0]
.P_TEST_STATUS_0(),             // OUTPUT[19:0]
.P_TEST_STATUS_1(),             // OUTPUT[19:0]
.P_TEST_STATUS_2(),             // OUTPUT[19:0]
.P_TEST_STATUS_3(),             // OUTPUT[19:0]
.P_CEB_ADETECT_EN(),            // INPUT[3:0]
.P_CFG_ADDR(),                  // INPUT[15:0]
.P_CFG_WDATA(),                 // INPUT[7:0]
.P_CIM_CLK_ALIGNER_RX0(),       // INPUT[7:0]
.P_CIM_CLK_ALIGNER_RX1(),       // INPUT[7:0]
.P_CIM_CLK_ALIGNER_RX2(),       // INPUT[7:0]
.P_CIM_CLK_ALIGNER_RX3(),       // INPUT[7:0]
.P_CIM_CLK_ALIGNER_TX0(),       // INPUT[7:0]
.P_CIM_CLK_ALIGNER_TX1(),       // INPUT[7:0]
.P_CIM_CLK_ALIGNER_TX2(),       // INPUT[7:0]
.P_CIM_CLK_ALIGNER_TX3(),       // INPUT[7:0]
.P_PCS_FAREND_LOOP(),           // INPUT[3:0]
.P_PCS_MCB_EXT_EN(),            // INPUT[3:0]
.P_PCS_NEAREND_LOOP(),          // INPUT[3:0]
.P_PCS_WORD_ALIGN_EN(),         // INPUT[3:0]
.P_PMA_FAREND_PLOOP(),          // INPUT[3:0]
.P_PMA_NEAREND_PLOOP(),         // INPUT[3:0]
.P_PMA_NEAREND_SLOOP(),         // INPUT[3:0]
.P_RESCAL_I_CODE_I(),           // INPUT[5:0]
.P_RX_BUSWIDTH_0(),             // INPUT[2:0]
.P_RX_BUSWIDTH_1(),             // INPUT[2:0]
.P_RX_BUSWIDTH_2(),             // INPUT[2:0]
.P_RX_BUSWIDTH_3(),             // INPUT[2:0]
.P_RX_POLARITY_INVERT(),        // INPUT[3:0]
.P_RX_RATE_0(),                 // INPUT[2:0]
.P_RX_RATE_1(),                 // INPUT[2:0]
.P_RX_RATE_2(),                 // INPUT[2:0]
.P_RX_RATE_3(),                 // INPUT[2:0]
.P_TDATA_0(),                   // INPUT[45:0]
.P_TDATA_1(),                   // INPUT[45:0]
.P_TDATA_2(),                   // INPUT[45:0]
.P_TDATA_3(),                   // INPUT[45:0]
.P_TX_BUSWIDTH_0(),             // INPUT[2:0]
```

```
.P_TX_BUSWIDTH_1(),          // INPUT[2:0]
.P_TX_BUSWIDTH_2(),          // INPUT[2:0]
.P_TX_BUSWIDTH_3(),          // INPUT[2:0]
.P_TX_DEEMP_0(),             // INPUT[1:0]
.P_TX_DEEMP_1(),             // INPUT[1:0]
.P_TX_DEEMP_2(),             // INPUT[1:0]
.P_TX_DEEMP_3(),             // INPUT[1:0]
.P_TX_MARGIN_0(),            // INPUT[2:0]
.P_TX_MARGIN_1(),            // INPUT[2:0]
.P_TX_MARGIN_2(),            // INPUT[2:0]
.P_TX_MARGIN_3(),            // INPUT[2:0]
.P_TX_RATE_0(),              // INPUT[2:0]
.P_TX_RATE_1(),              // INPUT[2:0]
.P_TX_RATE_2(),              // INPUT[2:0]
.P_TX_RATE_3(),              // INPUT[2:0]
.P_CFG_INT(),                // OUTPUT
.P_CFG_READY(),              // OUTPUT
.P_PLL_READY_0(),            // OUTPUT
.P_PLL_READY_1(),            // OUTPUT
.P_PLL_TEST_0(),             // OUTPUT
.P_PLL_TEST_1(),             // OUTPUT
.P_REFCK2CORE_0(),           // OUTPUT
.P_REFCK2CORE_1(),           // OUTPUT
.P_RX_LS_DATA_0(),           // OUTPUT
.P_RX_LS_DATA_1(),           // OUTPUT
.P_RX_LS_DATA_2(),           // OUTPUT
.P_RX_LS_DATA_3(),           // OUTPUT
.P_RX_READY_0(),             // OUTPUT
.P_RX_READY_1(),             // OUTPUT
.P_RX_READY_2(),             // OUTPUT
.P_RX_READY_3(),             // OUTPUT
.P_RX_SATA_COMINIT_0(),      // OUTPUT
.P_RX_SATA_COMINIT_1(),      // OUTPUT
.P_RX_SATA_COMINIT_2(),      // OUTPUT
.P_RX_SATA_COMINIT_3(),      // OUTPUT
.P_RX_SATA_COMWAKE_0(),      // OUTPUT
.P_RX_SATA_COMWAKE_1(),      // OUTPUT
.P_RX_SATA_COMWAKE_2(),      // OUTPUT
.P_RX_SATA_COMWAKE_3(),      // OUTPUT
.P_RX_SIGDET_STATUS_0(),     // OUTPUT
.P_RX_SIGDET_STATUS_1(),     // OUTPUT
.P_RX_SIGDET_STATUS_2(),     // OUTPUT
.P_RX_SIGDET_STATUS_3(),     // OUTPUT
.P_TX_RXDET_STATUS_0(),      // OUTPUT
.P_TX_RXDET_STATUS_1(),      // OUTPUT
.P_TX_RXDET_STATUS_2(),      // OUTPUT
.P_TX_RXDET_STATUS_3(),      // OUTPUT
```

```
.P_TX_SDN0(),                    // OUTPUT
.P_TX_SDN1(),                    // OUTPUT
.P_TX_SDN2(),                    // OUTPUT
.P_TX_SDN3(),                    // OUTPUT
.P_TX_SDP0(),                    // OUTPUT
.P_TX_SDP1(),                    // OUTPUT
.P_TX_SDP2(),                    // OUTPUT
.P_TX_SDP3(),                    // OUTPUT
.P_CFG_CLK(),                    // INPUT
.P_CFG_ENABLE(),                 // INPUT
.P_CFG_PSEL(),                   // INPUT
.P_CFG_RST(),                    // INPUT
.P_CFG_WRITE(),                  // INPUT
.P_CIM_DYN_DLY_SEL_RX0(),        // INPUT
.P_CIM_DYN_DLY_SEL_RX1(),        // INPUT
.P_CIM_DYN_DLY_SEL_RX2(),        // INPUT
.P_CIM_DYN_DLY_SEL_RX3(),        // INPUT
.P_CIM_DYN_DLY_SEL_TX0(),        // INPUT
.P_CIM_DYN_DLY_SEL_TX1(),        // INPUT
.P_CIM_DYN_DLY_SEL_TX2(),        // INPUT
.P_CIM_DYN_DLY_SEL_TX3(),        // INPUT
.P_CIM_START_ALIGN_RX0(),        // INPUT
.P_CIM_START_ALIGN_RX1(),        // INPUT
.P_CIM_START_ALIGN_RX2(),        // INPUT
.P_CIM_START_ALIGN_RX3(),        // INPUT
.P_CIM_START_ALIGN_TX0(),        // INPUT
.P_CIM_START_ALIGN_TX1(),        // INPUT
.P_CIM_START_ALIGN_TX2(),        // INPUT
.P_CIM_START_ALIGN_TX3(),        // INPUT
.P_CTLE_ADP_RST_0(),             // INPUT
.P_CTLE_ADP_RST_1(),             // INPUT
.P_CTLE_ADP_RST_2(),             // INPUT
.P_CTLE_ADP_RST_3(),             // INPUT
.P_HSST_RST(),                   // INPUT
.P_LANE_PD_0(),                  // INPUT
.P_LANE_PD_1(),                  // INPUT
.P_LANE_PD_2(),                  // INPUT
.P_LANE_PD_3(),                  // INPUT
.P_LANE_RST_0(),                 // INPUT
.P_LANE_RST_1(),                 // INPUT
.P_LANE_RST_2(),                 // INPUT
.P_LANE_RST_3(),                 // INPUT
.P_LANE_SYNC_0(),                // INPUT
.P_LANE_SYNC_1(),                // INPUT
.P_LANE_SYNC_EN_0(),             // INPUT
.P_LANE_SYNC_EN_1(),             // INPUT
.P_PCS_CB_RST_0(),               // INPUT
```

```
.P_PCS_CB_RST_1(),              // INPUT
.P_PCS_CB_RST_2(),              // INPUT
.P_PCS_CB_RST_3(),              // INPUT
.P_PCS_RX_RST_0(),              // INPUT
.P_PCS_RX_RST_1(),              // INPUT
.P_PCS_RX_RST_2(),              // INPUT
.P_PCS_RX_RST_3(),              // INPUT
.P_PCS_TX_RST_0(),              // INPUT
.P_PCS_TX_RST_1(),              // INPUT
.P_PCS_TX_RST_2(),              // INPUT
.P_PCS_TX_RST_3(),              // INPUT
.P_PLLPOWERDOWN_0(),            // INPUT
.P_PLLPOWERDOWN_1(),            // INPUT
.P_PLL_REF_CLK_0(),             // INPUT
.P_PLL_REF_CLK_1(),             // INPUT
.P_PLL_RST_0(),                 // INPUT
.P_PLL_RST_1(),                 // INPUT
.P_RATE_CHANGE_TCLK_ON_0(),// INPUT
.P_RATE_CHANGE_TCLK_ON_1(),// INPUT
.P_REFCLKN_0(),                 // INPUT
.P_REFCLKN_1(),                 // INPUT
.P_REFCLKP_0(),                 // INPUT
.P_REFCLKP_1(),                 // INPUT
.P_RESCAL_RST_I(),              // INPUT
.P_RX0_CLK2_FR_CORE(),          // INPUT
.P_RX0_CLK_FR_CORE(),           // INPUT
.P_RX1_CLK2_FR_CORE(),          // INPUT
.P_RX1_CLK_FR_CORE(),           // INPUT
.P_RX2_CLK2_FR_CORE(),          // INPUT
.P_RX2_CLK_FR_CORE(),           // INPUT
.P_RX3_CLK2_FR_CORE(),          // INPUT
.P_RX3_CLK_FR_CORE(),           // INPUT
.P_RXGEAR_SLIP_0(),             // INPUT
.P_RXGEAR_SLIP_1(),             // INPUT
.P_RXGEAR_SLIP_2(),             // INPUT
.P_RXGEAR_SLIP_3(),             // INPUT
.P_RX_HIGHZ_0(),                // INPUT
.P_RX_HIGHZ_1(),                // INPUT
.P_RX_HIGHZ_2(),                // INPUT
.P_RX_HIGHZ_3(),                // INPUT
.P_RX_LANE_PD_0(),              // INPUT
.P_RX_LANE_PD_1(),              // INPUT
.P_RX_LANE_PD_2(),              // INPUT
.P_RX_LANE_PD_3(),              // INPUT
.P_RX_PMA_RST_0(),              // INPUT
.P_RX_PMA_RST_1(),              // INPUT
.P_RX_PMA_RST_2(),              // INPUT
```

```
    .P_RX_PMA_RST_3(),              // INPUT
    .P_RX_SDN0(),                   // INPUT
    .P_RX_SDN1(),                   // INPUT
    .P_RX_SDN2(),                   // INPUT
    .P_RX_SDN3(),                   // INPUT
    .P_RX_SDP0(),                   // INPUT
    .P_RX_SDP1(),                   // INPUT
    .P_RX_SDP2(),                   // INPUT
    .P_RX_SDP3(),                   // INPUT
    .P_TX0_CLK2_FR_CORE(),          // INPUT
    .P_TX0_CLK_FR_CORE(),           // INPUT
    .P_TX1_CLK2_FR_CORE(),          // INPUT
    .P_TX1_CLK_FR_CORE(),           // INPUT
    .P_TX2_CLK2_FR_CORE(),          // INPUT
    .P_TX2_CLK_FR_CORE(),           // INPUT
    .P_TX3_CLK2_FR_CORE(),          // INPUT
    .P_TX3_CLK_FR_CORE(),           // INPUT
    .P_TX_BEACON_EN_0(),            // INPUT
    .P_TX_BEACON_EN_1(),            // INPUT
    .P_TX_BEACON_EN_2(),            // INPUT
    .P_TX_BEACON_EN_3(),            // INPUT
    .P_TX_LANE_PD_0(),              // INPUT
    .P_TX_LANE_PD_1(),              // INPUT
    .P_TX_LANE_PD_2(),              // INPUT
    .P_TX_LANE_PD_3(),              // INPUT
    .P_TX_LS_DATA_0(),              // INPUT
    .P_TX_LS_DATA_1(),              // INPUT
    .P_TX_LS_DATA_2(),              // INPUT
    .P_TX_LS_DATA_3(),              // INPUT
    .P_TX_PMA_RST_0(),              // INPUT
    .P_TX_PMA_RST_1(),              // INPUT
    .P_TX_PMA_RST_2(),              // INPUT
    .P_TX_PMA_RST_3(),              // INPUT
    .P_TX_RXDET_REQ_0(),            // INPUT
    .P_TX_RXDET_REQ_1(),            // INPUT
    .P_TX_RXDET_REQ_2(),            // INPUT
    .P_TX_RXDET_REQ_3(),            // INPUT
    .P_TX_SWING_0(),                // INPUT
    .P_TX_SWING_1(),                // INPUT
    .P_TX_SWING_2(),                // INPUT
    .P_TX_SWING_3()                 // INPUT
);
```

9.6. GTP_ADC_E1 使用说明

9.6.1. 支持器件

表 261 GTP_ADC_E1 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	支持	支持	不支持	不支持	不支持

9.6.2. 功能介绍

GTP_ADC_E1 用于实现 ADC 功能，详细功能请参考《UG020009_Logos 系列 FPGA 模数转换模块（ADC）用户指南》，其结构框图如下图所示。

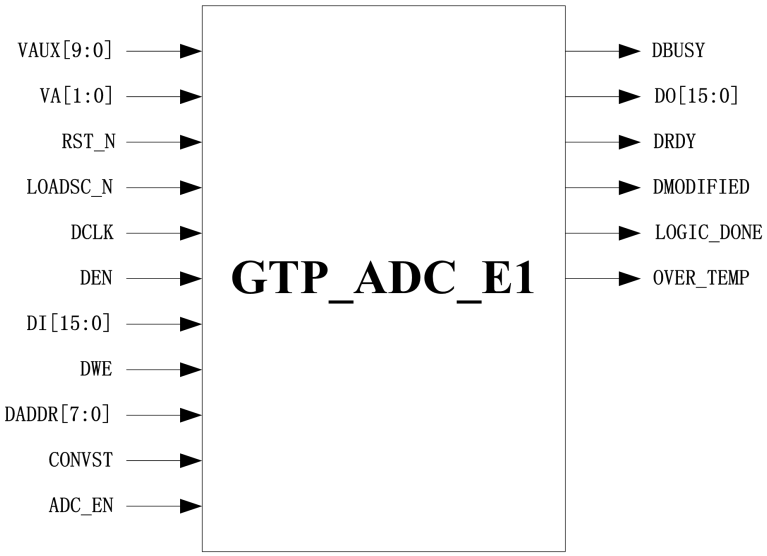


图 122 GTP_ADC_E1 结构框图

9.6.3. 端口描述

表 262 GTP_ADC_E1 端口列表

端口名称	方向	功能描述
VAUX[9:0]	I	输入的模拟信号
VA[1:0]	I	输入的模拟信号
RST_N	I	ADC的系统reset信号，低电平有效
LOADSC_N	I	ADC控制寄存器下载静态配置值信号，低电平有效
DCLK	I	DRP时钟
DEN	I	操作使能，启动一次读/写操作
DI[15:0]	I	数据输入

端口名称	方向	功能描述
DWE	I	写使能，1’b0：读操作；1’b1：写操作；
DADDR[7:0]	I	地址
CONVST	I	主动控制模式的控制信号
ADC_EN	I	ADC使能信号
DBUSY	O	JTAG DRP操作标志，指示JTAG正在通过JDRP指令进行DRP操作
DO[15:0]	O	数据输出
DRDY	O	操作完成标志，DRDY为高
DMODIFIED	O	控制寄存器改动标志，指示控制寄存器被JTAG DRP写过后，用户还未进行DR操作
LOGIC_DONE	O	ADC状态寄存器更新信号
OVER_TEMP	O	ADC温度检测标识信号

9.6.4. 参数描述

表 263 GTP_ADC_E1 参数列表

参数名称	参数类型	有效值	默认值	功能描述
AVERAGE	string	“1”, “16”, “64”, “256”	“1”	ADC 输出 CODE 多次求平均设置
CALIB	string	“NONE”, “OFFSET”, “OFFSET_GAIN”	“NONE”	ADC calibration 设置 “NONE”：offset and gain calculation disable “OFFSET”：offset calculation enable “OFFSET_GAIN”：offset and gain calculation enable
REFERENCE	string	“INTERNAL”, “EXTERNAL”	“INTERNAL”	ADC 参考源设置、 “INTERNAL”：内部参考源 “EXTERNAL”：外部参考源
CALIB_REFERENCE	string	“INTERNAL”, “EXTERNAL”	“INTERNAL”	ADC calibration 的参考源设置 “INTERNAL”：内部参考源 “EXTERNAL”：外部参考源

参数名称	参数类型	有效值	默认值	功能描述
FULL_SWING	string	“0.5V”, “0.6V”, “0.7V”, “0.8V”, “0.9V”, “1.0V”, “1.1V”, “1.2V”	“0.5V”	ADC 量程设置
VCM	string	“0.8V”, “0.9V”, “1.0V”, “1.1V”, “1.2V”,“1.3V”, “1.4V”, “1.5V”	“0.8V”	ADC 内部比较器输入共模电压设置
DIVIDER	string	“2”, “3”, “4”, “5”, “6” ,“7”, “8”, 9” ,“10”, “11” ,“12”, “13”, “14” , “15”, “16”	“2”	输入时钟分频设置
ADC_MODE	string	“DEFAULT”, “SINGLE_PASS”, “CONTINUE_SEQ”, “SINGLE_CHANNEL”	“DEFAULT”	ADC 工作模式选择 “DEFAULT”：默认模式 “SINGLE_PASS”：单次扫描模式 “CONTINUE_SEQ”：连续扫描模式 “SINGLE_CHANNEL”：单通道模式
EVENT_DRIVE	string	“FALSE”, “TRUE”	“FALSE”	采样模式设置 “FALSE”：连续采样模式 “TRUE”：主动控制采样模式
ADC_MODE_1MSPS	string	“FALSE”, “TRUE”	“FALSE”	1MSPS 模式设置 “FALSE”：模式禁止 “TRUE”：模式使能
CLKSWITCH	string	“FALSE”, “TRUE”	“FALSE”	时钟切换设置 “FALSE”：选择 CLK_OSC “TRUE”：选择 DCLK
INTERNAL_VOL_	string	“VDD33”, “VDD11”,	“VDD33”	channel 13 信号选择

参数名称	参数类型	有效值	默认值	功能描述
SEL		“VDDM”		
SINGLE_CH_SEL	string	“0”, “1”, “2”, “3”, “4”, “5”, “6”, “7”, “8”, “9”, “10”, “11”, “12”, “13”, “14”, “15”	“0”	通道选择
SINGLE_CH_IN	string	“SINGLE_END”, “UNIPOLAR”, “BIPOLAR”	“SINGLE_END”	输入模式设置
SEQ_CH11_10_SEL	string	“NONE”, “CH10”, “CH11, “ALL”	“NONE”	通道 11/10 扫描设置 “NONE”：通道 11/10 不扫描 “CH10”：通道 10 扫描 “CH11”：通道 11 扫描 “ALL”：通道 11/10 都扫描
SEQ_CH9_8_SEL	string	“NONE”, “CH8”, “CH9, “ALL”	“NONE”	通道 9/8 扫描设置 “NONE”：通道 9/8 不扫描 “CH8”：通道 8 扫描 “CH9”：通道 9 扫描 “ALL”：通道 9/8 都扫描
SEQ_CH7_6_SEL	string	“NONE”, “CH6”, “CH7, “ALL”	“NONE”	通道 7/6 扫描设置 “NONE”：通道 7/6 不扫描 “CH6”：通道 6 扫描 “CH7”：通道 7 扫描 “ALL”：通道 7/6 都扫描
SEQ_CH5_4_SEL	string	“NONE”, “CH4”, “CH5, “ALL”	“NONE”	通道 5/4 扫描设置 “NONE”：通道 5/4 不扫描 “CH4”：通道 4 扫描 “CH5”：通道 5 扫描 “ALL”：通道 5/4 都扫描
SEQ_CH3_2_SEL	string	“NONE”, “CH2”, “CH3, “ALL”	“NONE”	通道 3/2 扫描设置 “NONE”：通道 3/2 不扫描

参数名称	参数类型	有效值	默认值	功能描述
				“CH2”：通道 2 扫描 “CH3”：通道 3 扫描 “ALL”：通道 3/2 都扫描
SEQ_CH1_0_SEL	string	“NONE”, “CH0”, “CH1, “ALL”	“NONE”	通道 1/0 扫描设置 “NONE”：通道 1/0 不扫描 “CH0”：通道 0 扫描 “CH1”：通道 1 扫描 “ALL”：通道 1/0 都扫描
SEQ_CH11_10_IN	string	“SINGLE_END”, “UNIPOLAR”, “BIPOLAR”	“SINGLE_END”	通道 11/10 输入模式设置
SEQ _CH9_8_IN	string	“SINGLE_END”, “UNIPOLAR”, “BIPOLAR”	“SINGLE_END”	通道 9/8 输入模式设置
SEQ _CH7_6_IN	string	“SINGLE_END”, “UNIPOLAR”, “BIPOLAR”	“SINGLE_END”	通道 7/6 输入模式设置
SEQ _CH5_4_IN	string	“SINGLE_END”, “UNIPOLAR”, “BIPOLAR”	“SINGLE_END”	通道 5/4 输入模式设置
SEQ _CH3_2_IN	string	“SINGLE_END”, “UNIPOLAR”, “BIPOLAR”	“SINGLE_END”	通道 3/2 输入模式设置
SEQ _CH1_0_IN	string	“SINGLE_END”, “UNIPOLAR”, “BIPOLAR”	“SINGLE_END”	通道 1/0 输入模式设置
TEMP_SENSOR_HIGH	integer	0~255	0	温度检测阈值上限
TEMP_SENSOR_LOW	integer	0~255	0	温度检测阈值下限

参数名称	参数类型	有效值	默认值	功能描述
ADC_EN_ENABLE	string	“FALSE”, “TRUE”	“FALSE”	ADC_EN 端口使能设置 “FALSE”：ADC_EN 端口输入无效 “TRUE”: ADC_EN 端口输入有效

9.6.5. 例化模板

```
GTP_ADC_E1 #(
        .AVERAGE("1"),
        .CALIB("NONE"),
        .REFERENCE("INTERNAL"),
        .CALIB_REFERENCE("INTERNAL"),
        .FULL_SWING("0.5V"),
        .VCM("0.8V"),
        .DIVIDER("2"),
        .ADC_MODE("DEFAULT"),
        .EVENT_DRIVE("FALSE"),
        .ADC_MODE_1MSPS("FALSE"),
        .CLKSWITCH("FALSE"),
        .INTERNAL_VOL_SEL("VDD33"),
        .SINGLE_CH_SEL("0"),
        .SINGLE_CH_IN("SINGLE_END"),
        .SEQ_CH11_10_SEL("NONE"),
        .SEQ_CH9_8_SEL("NONE"),
        .SEQ_CH7_6_SEL("NONE"),
        .SEQ_CH5_4_SEL("NONE"),
        .SEQ_CH3_2_SEL("NONE"),
        .SEQ_CH1_0_SEL("NONE"),
        .SEQ_CH11_10_IN("SINGLE_END"),
        .SEQ_CH9_8_IN("SINGLE_END"),
        .SEQ_CH7_6_IN("SINGLE_END"),
        .SEQ_CH5_4_IN("SINGLE_END"),
        .SEQ_CH3_2_IN("SINGLE_END"),
        .SEQ_CH1_0_IN("SINGLE_END"),
        .TEMP_SENSOR_HIGH(0),
        .TEMP_SENSOR_LOW(0),
        .ADC_EN_ENABLE("FALSE")
) GTP_ADC_E1 (
        .DO(),              // OUTPUT[15:0]
        .DADDR(),           // INPUT[7:0]
        .DI(),              // INPUT[15:0]
        .VA(),              // INPUT[1:0]
        .VAUX(),            // INPUT[9:0]
        .DBUSY(),           // OUTPUT
```

```
    .DMODIFIED(), // OUTPUT
    .DRDY(),          // OUTPUT
    .LOGIC_DONE(),// OUTPUT
    .OVER_TEMP(), // OUTPUT
    .ADC_EN(),       // INPUT
    .CONVST(),       // INPUT
    .DCLK(),         // INPUT
    .DEN(),          // INPUT
    .DWE(),          // INPUT
    .LOADSC_N(),    // INPUT
    .RST_N()         // INPUT
);
```

9.7. GTP_PCIEGEN2 使用说明

9.7.1. 支持器件

表 264 GTP_PCIEGEN2 器件支持情况

器件型号	PGL22G	PGL12G	PGL25G	PGL50H	PGL100H
是否支持该GTP	不支持	不支持	不支持	支持	支持

9.7.2. 功能介绍

GTP_PCIEGEN2 (Peripheral Component Interconnect Express)高速串行接口模块兼容 PCIE2.1 协议。主要特性包括：支持 Gen1 和 Gen2 两种速率（2.5Gb/s or 5Gb/s）；支持 EP 和 RC 两种模式并同时支持 x1, x2 和 x4；TLP 包 max_payload 可在 128Bytes，256 Bytes，512 Bytes，1024 Bytes 间灵活配置；BAR 寄存器可配置；同时支持 Legacy, MSI 和 MSIX 三种中断机制；Power Management，Lane Reversal，Lane Polarity Inversion 均可实现。结构简要框图如下所示：

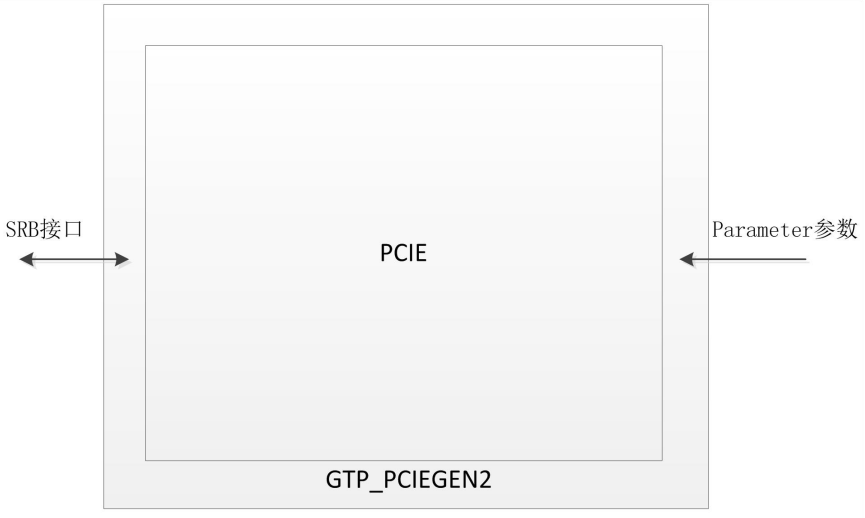


图 123 GTP_ADC_E1 结构框图

9.7.3. 端口描述

表 265 GTP_PCIEGEN2 端口列表

端口名称	方向	宽度	功能描述
PCLK	I	1	PIPE clock from external PHY DFT模式下复用为test_clk Gen1: 125Mhz Gen2: 250Mhz
PCLK_DIV2	I	1	pclk_div2 clock from external PHY, two divided from PCLK DFT模式下复用为test_clk Gen1: 62.5Mhz Gen2: 125Mhz
MEM_CLK	I	1	用于接收memory的时钟，与pclk同源。
BUTTON_RST	I	1	Button reset, Active high
POWER_UP_RST	I	1	Warm reset, Active high
PERST	I	1	Reset from Socket, Active high
CORE_RST_N	O	1	Resets the controller, except for the PMC module, It is recommended that you reset your application logic together with the controller, Active low
TRAINING_RST_N	O	1	Hot reset from upstream component, Actice low
APP_INIT_RST	I	1	Request from your application to send a hot reset to the upstream port; Active high
PHY_RST_N	O	1	For phy rst, Active low
DEVICE_TYPE	I	3	Device/port type. Indicates the specific type of this PCI Express function.

端口名称	方向	宽度	功能描述
			3’b000: PCI Express endpoint 3’b001: Legacy PCI Express endpoint 3’b100: Root port of PCI Express root complex
RX_LANE_FLIP_EN	I	1	Performs manual lane reversal for receive lanes.
TX_LANE_FLIP_EN	I	1	Performs manual lane reversal for transmit lanes
APP_LTSSM_ENABLE	I	1	Driven low by your application after cold, warm or hot reset to hold the LTSSM in the Detect state until your application is ready for the link training to begin.
SMLH_LINK_UP	O	1	PHY Link up/down indicator
RDLH_LINK_UP	O	1	Data link layer up/down indicator
APP_REQ_RETRY_EN	I	1	Provides a capability to defer incoming configuration requests until initialization is complete
SMLH_LTSSM_STATE	O	5	Current state of the LTSSM
AXIS_MASTER_TVALID	O	1	Active high to indicate that the ctrl is driving a valid transfer
AXIS_MASTER_TREADY	I	1	Active high to indicate that the user can accept a transfer in the current cycle
AXIS_MASTER_TDATA	O	128	Primary payload provide to user logic
AXIS_MASTER_TKEEP	O	4	Ativie high to enable related DW in axis_master_tdata； axis_master_tkeep[3] control axis_master_tdata[127:96]; axis_master_tkeep[2] control axis_master_tdata[95:64]; axis_master_tkeep[1] control axis_master_tdata[63:32]; axis_master_tkeep[0] control axis_master_tdata[31:0]; Note：和AXIS标准的区别是这里一个bit对应了4个byte的enable
AXIS_MASTER_TLAST	O	1	Active high to indicate last valid transfer of a packet from ctrl
AXIS_MASTER_TUSER	O	8	Sideband information transmitted alongside axis_master_tdata: 0: radm_trgt1_tlp_abort (Indicates to your application to drop the TLP because of malformed TLP on TRGT1, ECRC error, or completion lookup failures)

端口名称	方向	宽度	功能描述
			1: radm_trgt1_dllp_abort (Indicates to your application to drop the TLP on TRGT1 because of a Data Link Layer error such as LCRC or otherwise) 2: radm_trgt1_ecrc_err (Indicates to your application to drop the TLP because of an ECRC error in the received TLP on TRGT1) 3: radm_trgt1_cpl_last: Indicates the last completion TLP of a split completion transaction. [6:4]: radm_trgt1_in_membar_range[2:0] (Indicates which of the configured BARs contains the target address in the received TLP) 7: radm_trgt1_rom_in_range (Indicates that the target address in the received TLP in range of the expansion ROM)
TRGT1_RADM_PKT_HALT	I	3	Halts the transfer of packets from individual queues. There is one bit of trgt1_radm_pkt_halt for each TLP type for each configured VC: Bit 0: Halt posted TLPs for VC0 Bit 1: Halt non-posted TLPs for VC0 Bit 2: Halt CPL TLPs for VC0
RADM_GRANT_TLP_TYPE	O	6	Indicates that a particular VC and type transaction has been granted to output from the receive queue. There is one bit for each TLP type for each configured VC: Bit [1:0]: Grant posted TLPs for VC0 Bit [3:2]: Grant non-posted TLPs for VC0 Bit [5:4]: Grant CPL TLPs for VC0
AXIS_SLAVE0/1/2_TREADY	O	1	Active high to indicate that the ctrl can accept a transfer in the current cycle
AXIS_SLAVE0/1/2_TVALID	I	1	Active high to indicate that the user is driving a valid transfer

端口名称	方向	宽度	功能描述
AXIS_SLAVE0/1/2_TDATA	I	128	Primary payload provide to ctrl
AXIS_SLAVE0/1/2_TLAST	I	1	Active high to indicate last valid transfer of a packet to ctrl
AXIS_SLAVE0/1/2_TUSER	I	1	Sideband information transmitted alongside axis_slave_tdata: Bit 0: client0/1/2_tlp_bad_eot， Indicates that the current TLP must be nullified.
DBI_ADDR	I	32	Address of the configuration register for the current DBI access [31:2] register address,必须dword-aligned [1] not used [0] DBI access direction,0: internal register 1: external register via ELBI
DBI_DIN	I	32	Write data bus to the selected configuration register
DBI_CS	I	1	Chip select input to access the CDM or ELBI
DBI_CS2	I	1	Additional chip select that enables writing to BAR mask registers
DBI_WR	I	4	Indicates the configuration register access type (read or write).
APP_DBI_RO_WR_DISABLE	I	1	DBI Read-only Write Disable 0: MISC_CONTROL_1_OFF.DBI_RO_WR_EN register field is read-write. 1: MISC_CONTROL_1_OFF.DBI_RO_WR_EN register field is forced to 0 and is read-only.
LBC_DBI_ACK	O	1	Indicates that the requested read or write operation to the selected configuration register is complete.
LBC_DBI_DOUT	O	32	Read data bus from the selected configuration register.
SEDI	I	1	串行信号输入，由slave驱动
SEDI_ACK	I	1	串行信号输入ACK，由slave驱动
SEDO	O	1	串行信号输出，由master驱动
SEDO_EN	O	1	串行信号输出使能，由master驱动
CFG_INT_DISABLE	O	1	When high a functions ability to generate INTx messages is Disabled

端口名称	方向	宽度	功能描述
SYS_INT	I	1	When sys_int goes from low to high, the controller generates an Assert_INTx Message
INTA_GRT_MUX	O	1	EP： 0->1：The signal indicates that the controller sent an Assert_INTA Message to the upstream device. 1->0：The signal indicates that the controller sent an Deassert_INTA Message to the upstream device. RC： 0->1: The signal indicates that the controller received an Assert_INTA Message from the downstream device 1->0: The signal indicates that the controller received an Deassert_INTA Message from the downstream device
INTB_GRT_MUX	O	1	EP:Not used RC： 0->1: The signal indicates that the controller received an Assert_INTB Message from the downstream device 1->0: The signal indicates that the controller received an Deassert_INTB Message from the downstream device
INTC_GRT_MUX	O	1	EP:Not used RC： 0->1: The signal indicates that the controller received an Assert_INTC Message from the downstream device 1->0: The signal indicates that the controller received an Deassert_INTC Message from the downstream device
INTD_GRT_MUX	O	1	EP:Not used RC： 0->1: The signal indicates that the controller received an Assert_INTD Message from the downstream device 1->0: The signal indicates that the controller received an

端口名称	方向	宽度	功能描述
			Deassert_INTD Message from the downstream device
VEN_MSI_REQ	I	1	Request from your application to send an MSI when MSI is enabled
VEN_MSI_TC	I	3	Traffic Class of the MSI request, valid when ven_msi_req is asserted
VEN_MSI_VECTOR	I	5	Used to modulate the lower five bits of the MSI Data register when multiple message mode is enabled
VEN_MSI_GRANT	O	1	One-cycle pulse that indicates that the controller has accepted the request to send an MSI
CFG_MSI_PENDING	I	32	Indication from application about which functions have a pending associated message
CFG_MSI_EN	O	1	Indicates that MSI is enabled (INTx message is not sent)
CFG_MSIX_EN	O	1	The MSI-X Enable bit of the MSI-X Control register in the MSI-X Capability structure
MSIX_ADDR	I	64	The address value for the MSI-X.
MSIX_DATA	I	32	The data value for the MSI-X.
CFG_MSIX_FUNC_MASK	O	1	The function Mask bit of the MSI-X Control register in the MSI-X Capability structure.
CFG_LINK_AUTO_BW_MUX	O	1	cfg_link_auto_bw_int when legacy int is used, cfg_link_auto_bw_msi when MSI or MSI-X is enabled. Only for RC
CFG_BW_MGT_MUX	O	1	cfg_bw_mgt_int when legacy int is used, cfg_bw_mgt_msi when MSI or MSI-X is enabled. Only for RC
CFG_PME_MUX	O	1	cfg_pme_int when legacy int is used, cfg_pme_msi when MSI or MSI-X is enabled.
CFG_AER_RC_ERR_MUX	O	1	cfg_aer_rc_err_int: when MSI/MSI-X is NOT enabled. Other wise used as cfg_aer_rc_err_msi
RADM_PM_TURNOFF	O	1	One-clock-cycle pulse that indicates that the controller received a PME Turnoff message

端口名称	方向	宽度	功能描述
RADM_MSG_UNLOCK	O	1	One-cycle pulse that indicates that the controller received an Unlock message
OUTBAND_PWRUP_CMD	I	1	Wake Up. Used by application logic to wake up the PMC state machine from a D1, D2 or D3 power state.
PM_XTLH_BLOCK_TLP	O	1	Indicates that your application must stop generating new outgoing request TLPs due to the current power management state
PM_STATUS	O	1	PME Status bit from the PMCSR
PM_DSTATE	O	3	The current power management D-state of the function
AUX_PM_EN	O	1	Auxiliary Power Enable bit in the Device Control register
PM_PME_EN	O	1	PME Enable bit in the PMCSR
PM_LINKST_IN_L0S	O	1	Power management is in L0s state.
PM_LINKST_IN_L1	O	1	Power management is in L1 state.
PM_LINKST_IN_L2	O	1	Power management is in L2 state.
PM_LINKST_L2_EXIT	O	1	Power management is exiting L2 state
APP_REQ_ENTR_L1	I	1	Application request to Enter L1 ASPM state
APP_READY_ENTR_L23	I	1	Application Ready to Enter L23.
APP_REQ_EXIT_L1	I	1	Application request to Exit L1
APP_XFER_PENDING	I	1	Indicates that your application has transfers pending and prevents the controller from entering L1.
WAKE	O	1	Wake Up. Wake up from power management unit.
RADM_PM_PME	O	1	One-clock-cycle pulse that indicates that the controller received a PM_PME message
RADM_PM_TO_ACK	O	1	One-clock-cycle pulse that indicates that the controller received a PME_TO_Ack message
APPS_PM_XMT_TURNOFF	I	1	Request from your application to generate a PM_Turn_Off message.
APP_UNLOCK_MSG	I	1	Request from your application to generate an Unlock message
APPS_PM_XMT_PME	I	1	Wake Up. Used by application logic to wake up the PMC state

端口名称	方向	宽度	功能描述
			machine from a D1, D2 or D3 power state
APP_CLK_PM_EN	I	1	Clock PM feature enabled by application.
PM_MASTER_STATE	O	5	Power management master FSM state
PM_SLAVE_STATE	O	5	Power management slave FSM state
SYS_AUX_PWR_DET	I	1	Auxiliary Power Detected. Used to report to the host software that auxiliary power (Vaux) is present
CFG_MAX_RD_REQ_SIZE	O	3	The value of the Max_Read_Request_Size field in the Device Control register.
CFG_BUS_MASTER_EN	O	1	The state of the bus master enable bit in the PCI-compatible Command register.
CFG_MAX_PAYLOAD_SIZE	O	3	The value of the Max_Payload_Size field in the Device Control register
CFG_EXT_TAG_EN	O	1	When enabled, controller supports up to 8-bit tag values.
CFG_RCB	O	1	The value of the RCB bit in the Link Control register.
CFG_MEM_SPACE_EN	O	1	The state of the Memory Space Enable bit in the PCI-compatible Command register.
CFG_PM_NO_SOFT_RST	O	1	This is the value of the No Soft Reset bit in the Power Management Control and Status Register
CFG_CRS_SW_VIS_EN	O	1	Indicates the value of the CRS Software Visibility enable bit in the Root Control register
CFG_NO_SNOOP_EN	O	1	Contents of the “Enable No Snoop” field (PCIE_CAP_EN_NO_SNOOP) in the “Device Control and Status” register (DEVICE_CONTROL_DEVICE_STATUS) register
CFG_RELAX_ORDER_EN	O	1	Contents of the “Enable Relaxed Ordering” field (PCIE_CAP_EN_REL_ORDER) in the “Device Control and Status” register (DEVICE_CONTROL_DEVICE_STATUS) register
CFG_TPH_REQ_EN	O	2	The 2-bit TPH Requester Enabled field of each TPH Requester Control register.

端口名称	方向	宽度	功能描述
CFG_PF_TPH_ST_MODE	O	3	Steering Tag Mode of Operation for Physical Function
CFG_PBUS_NUM	O	8	The primary bus number assigned to the function
CFG_PBUS_DEV_NUM	O	5	The device number assigned to the function
CFG_ATOMIC_REQ_EN	O	1	The AtomicOp Requester Enable field of the Device Control 2 register
CFG_ATOMIC_EGRESS_BLOCK	O	1	The AtomicOp Egress Blocking field of the Device Control 2 register
RBAR_CTRL_UPDATE	O	1	Indicates that a resizable BAR control register has been updated
APP_HDR_VALID	I	1	One-clock-cycle pulse indicating that the data app_hdr_log,
APP_HDR_LOG	I	128	The header of the TLP that contained the error indicated app_err_bus, valid when app_hdr_valid is asserted； Provide app_tlp_prfx_log before app_hdr_valid is asserted if needed
APP_ERR_BUS	I	13	The type of error that your application detected
APP_ERR_ADVISORY	I	1	Indicates that your application error is an advisory error.
CFG_SEND_COR_ERR_MUX	O	1	EP: cfg_send_cor_err Sent Correctable Error. RC: radm_correctable_err, One-clock-cycle pulse that indicates that the controller received an ERR_COR message.
CFG_SEND_NF_ERR_MUX	O	1	EP: radm_nonfatal_err, Sent Non-Fatal Error RC: One-clock-cycle pulse that indicates that the controller received an ERR_NONFATAL message
CFG_SEND_F_ERR_MUX	O	1	EP: radm_fatal_err, Sent Fatal Error RC: One-clock-cycle pulse that indicates that the controller received an ERR_FATAL message.
CFG_SYS_ERR_RC	O	1	System error detected
RADM_CPL_TIMEOUT	O	1	Indicates that the completion TLP for a request has not been received within the expected time window
RADM_TIMEOUT_CPL_TC	O	3	The Traffic Class of the timed out completion

端口名称	方向	宽度	功能描述
RADM_TIMEOUT_CPL_TAG	O	8	The Tag field of the timed out completion
RADM_TIMEOUT_CPL_ATTR	O	2	The Attributes field of the timed out completion.
RADM_TIMEOUT_CPL_LEN	O	11	Length (in bytes) of the timed out completion. For a split completion, it indicates the number of bytes remaining to be delivered when the completion timed out.
DYN_DEBUG_INFO_SEL	I	4	动态debug_info_mux选择
APP_RAS_DES_SD_HOLD_LTSSM	I	1	Hold and release LTSSM. For as long as this signal is '1', thecontroller stays in the current LTSSM.
APP_RAS_DES_TBA_CTRL	I	2	Controls the start/end of time based analysis. You must only set the pins to the required value for the duration of one clock cycle. ■ 2'b00: No action ■ 2'b01: Start ■ 2'b10: End. This setting is only used when the TIME_BASED_DU-RATION_SELECT field of TIME_BASED_ANALYSIS_CON-TROL_REG is set to "manual control". ■ 2'b11: Reserved
RADM_IDLE	O	1	RADM activity status signal
RADM_Q_NOT_EMPTY	O	1	Level indicating that the receive queues contain TLP header/data.
RADM_QOVERFLOW	O	1	Pulse indicating that one or more of the P/NP/CPL receive queues have overflowed
DIAG_CTRL_BUS	I	2	Diagnostic Control Bus, 01: Insert LCRC error by inverting the LSB of LCRC 10: Insert ECRC error by inverting the LSB of ECRC
DEBUG_INFO_MUX	O	133	When DYN_DEBUG_SEL_EN =0,use DEBUG_INFO_SEL

端口名称	方向	宽度	功能描述
			When DYN_DUBUG_SEL_EN =1,use DYN_DEBUG_INFO_SEL *_DEBUG_INFO_SEL=0：MSI [31:0] cfg_msi_mask[31:0] [95:32] cfg_msi_addr[63:0] [127:96] cfg_msi_data[31:0] 128 cfg_msi_64 [131:129] cfg_multi_msi_en[2:0] 132 cfg_msi_ext_data_en *_DEBUG_INFO_SEL=1：MSI-X [10:0] cfg_msix_table_size[10:0] [13:11] cfg_msix_table_bir[2:0] [42:14] cfg_msix_table_offset[28:0] [45:43] cfg_msix_pba_bir[2:0] [74:46] cfg_msix_pba_offset[28:0] *_DEBUG_INFO_SEL=2：TX debug &DL debug 0 xadm_no_fc_credit[NVC-1:0] 1 xadm_notlp_pending 2 xadm_had_enough_credit[NVC-1:0] 3 xdlh_not_expecting_ack 4 xdlh_xmt_pme_ack 5 xdlh_nodllp_pending [14:6] rtfcgen_incr_amt[8:0] 15 rtfcgen_incr_enable [17:16] rtfcgen_fctype[1:0] 18 xdlh_xtlh_halt

端口名称	方向	宽度	功能描述
			19 xtlh_xdlh_badeot 20 xtlh_xdlh_eot 21 xtlh_xdlh_sot [26:22] active_grant[NCL+2-1:0] [31:27] grant_ack[NCL+2-1:0] [36:32] fc_cds_pass[(NCL+2)*NVC-1:0] [41:37] arb_reqs[NCL+2-1:0] 42 xmlh_xdlh_halt [44:43] xdlh_xmlh_sdp[1:0] [46:45] xdlh_xmlh_stp[1:0] [48:47] xdlh_xmlh_eot[1:0] [60:49] rdlh_xdlh_req_acknack_seqnum[11:0] 61 rdlh_xdlh_req2send_nack 62 rdlh_xdlh_req2send_ack_due2dup 63 rdlh_xdlh_req2send_ack [75:64] rdlh_xdlh_rcvd_acknack_seqnum[11:0] 76 rdlh_xdlh_rcvd_ack 77 rdlh_xdlh_rcvd_nack 78 cfg_link_retrain 79 rtlh_req_link_retrain 80 xdlh_smlh_start_link_retrain 81 rdlh_rtlh_tlp_dv [83:82] rdlh_rtlh_tlp_eot[1:0] [85:84] rdlh_rtlh_tlp_sot[1:0] 86 ecrc_err_asserted 87 lcrc_err_asserted *_DEBUG_INFO_SEL=3： 0 unexpected_cpl_err 1 cpl_ca_err

端口名称	方向	宽度	功能描述
			2 cpl_ur_err 3 flt_q_cpl_last 4 flt_q_cpl_abort 5 cpl_mlf_err [8:6]flt_q_header_cpl_status[2:0] [10:9] flt_q_header_destination[1:0] 11 form_filt_ecrc_err 12 form_filt_malform_tlp_err 13 form_filt_dllp_err 14 form_filt_eot [16:15] form_filt_dwen[NW-1:0] 17 form_filt_dv 18 form_filt_hv [20:19] rmlh_rdlh_pkt_err[NW-1:0] 21 rmlh_rdlh_pkt_dv [23:22] rmlh_rdlh_pkt_edb[NW-1:0] [25:24] rmlh_rdlh_pkt_end[NW-1:0] [27:26] rmlh_rdlh_tlp_start[NW-1:0] [29:28] rmlh_rdlh_dllp_start[NW-1:0] [31:30] rmlh_rdlh_nak[NW-1:0] [35:32] smlh_lanes_rcving[NL-1:0] 36 rmlh_rcvd_eidle_set 37 rmlh_rcvd_idle0 38 rmlh_rcvd_idle1 39 smlh_rcvd_lane_rev 40 smlh_ts_link_num_is_k237 41 rmlh_deskew_alignment_err 42 smlh_ts_lane_num_is_k237 43 smlh_ts2_rcvd 44 smlh_ts1_rcvd

端口名称	方向	宽度	功能描述
			45 smlh_ts_rcv_err 46 smlh_inskip_rcv *_DEBUG_INFO_SEL=4： [48:0] cxpl_debug_info[63:0] [64:49] cxpl_debug_info_ei[15:0] [67:65] pm_curnt_state[2:0] 68 pm_sel_aux_clk 69 en_muxd_aux_clk_g 70 en_radm_clk_g 71 link_req_rst_not 72 pm_req_core_rst 73 pm_req_phy_rst 74 pm_req_sticky_rst 75 pm_req_non_sticky_rst *_DEBUG_INFO_SEL=5： [7:0]cfg_int_pin[7:0] [43:8] cfg_rbar_size[35:0] 44 cfg_br_ctrl_serren 45 xdlh_replay_timeout_err 46 xdlh_replay_num_rlover_err 47 rdlh_bad_dllp_err 48 rdlh_bad_tlp_err 49 rdlh_prot_err 50 rtlh_fc_prot_err 51 rmlh_rcvd_err 52 int_xadm_fc_prot_err 53 radm_unexp_cpl_err 54 radm_rcvd_cpl_ur

端口名称	方向	宽度	功能描述
			55 radm_rcvd_cpl_ca 56 radm_rcvd_req_ca 57 radm_rcvd_req_ur 58 radm_ecrc_err 59 radm_mlf_tlp_err 60 radm_rcvd_cpl_poisoned 61 radm_rcvd_wreq_poisoned 62 cfg_sys_err_rc_cor 63 cfg_sys_err_rc_nf 64 cfg_sys_err_rc_f *_DEBUG_INFO_SEL=6： 0 cdm_lbc_ack [4:1]lbc_cdm_wr[3:0] 5 lbc_cdm_cs [37:6] lbc_cdm_data[31:0] [69:38] lbc_cdm_addr[31:0] *_debug_info_sel=7 0 smlh_eidle_inferred_in_l0: Level: Detect EI Infer 1 rmlh_rcvd_err: Pulse: Receiver Error 2 smlh_rx_rcvry_req: Level: Rx Recovery Request 3 smlh_timeout_nfts: Level: FTS Timeout 4 rmlh_framing_err: Pulse: Framing Error 5 rmlh_deskew_alignment_err: Level: Deskew Error 6 rdlh_bad_tlp_err_pertlp : Pulse: BAD TLP 7 rdlh_lcrc_tlp_err_pertlp : Pulse: LCRC Error 8 rdlh_bad_dllp_err_perdllp: Pulse: BAD DLLP 9 xdlh_replay_num_rlover_err: Pulse: Replay_Num Rollover 10 xdlh_replay_timeout_err: Pulse: Replay Timeout

端口名称	方向	宽度	功能描述
			11 rdlh_rcvd_nack_perdllp: Pulse: Rx Nak DLLP 12 xdlh_nak_sent: Pulse: Tx Nak DLLP 13 xdlh_retry_req: Pulse: Retry TLP 14 rtlh_req_link_retrain: Level: FC Timeout [16:15] cfg_poisned_tlp: Pulse: Poisoned TLP [18:17] cfg_ecrc_tlp_err: Pulse: ECRC Error [20:19] cfg_ur_tlp: Pulse: Unsupported Request [22:21] cfg_ca_tlp: Pulse: Completer Abort [24:23] cfg_cpl_timeout[1:0]: Pulse: Completion Timeout 25 smlh_l0_to_recovery: Pulse: L0 to Recovery Entry 26 smlh_l1_to_recovery: Pulse: L1 to Recovery Entry 27 smlh_in_l0s: Level: Tx L0s Entry 28 smlh_in_rl0s: Level: Rx L0s Entry 29 pm_asnak: Level: ASPM L1 reject 30 smlh_in_l1: Level: L1 Entry 31 pm_in_l11: Level: L1.1 Entry 32 pm_in_l12: Level: L1.2 Entry 33 pm_in_l1_short: Level: L1 short duration 34 pm_in_l1_cpm: Level: L1 Clock PM (L1 with REFCLK removal/PLL Off) 35 pm_in_l1_abort: Level: L1.2 abort 36 smlh_in_l23: Level: L2 Entry 37 smlh_spd_change: Pulse: Speed Change 38 smlh_lwd_change: Pulse: Link width Change 39 xdlh_ack_sent: Pulse: Tx Ack DLLP 40 xdlh_update_fc_sent: Pulse: Tx Update FC DLLP 41 rdlh_rcvd_ack_perdllp: Pulse: Rx Ack DLLP 42 rtlh_rcvd_ufc_perdllp: Pulse: Rx Update FC DLLP 43 rdlh_nulified_tlp_err_pertlp : Pulse: Rx Nullified TLP 44 xtlh_xadm_restore_enable: Pulse: Tx Nullified TLP

端口名称	方向	宽度	功能描述
			45 rdlh_duplicate_tlp_err_pertlp: Pulse: Rx Duplicate TLP 46 xtlh_tx_memwr_evt: Pulse: Tx Memory Write 47 xtlh_tx_memrd_evt: Pulse: Tx Memory Read 48 xtlh_tx_cfgwr_evt: Pulse: Tx Config Write 49 xtlh_tx_cfgrd_evt: Pulse: Tx Config Read 50 xtlh_tx_iowr_evt: Pulse: Tx IO Write 51 xtlh_tx_iord_evt: Pulse: Tx IO Read 52 xtlh_tx_cplwod_evt: Pulse: Tx Completion wo data 53 xtlh_tx_cplwd_evt: Pulse: Tx Completion w data 54 xtlh_tx_msg_evt: Pulse: Tx Message 55 xtlh_tx_atmcop_evt: Pulse: Tx AtomicOp 56 xtlh_tx_tlpwprefix_evt: Pulse: Tx TLP with Prefix 57 rtlh_rx_memwr_evt: Pulse: Rx Memory Write 58 rtlh_rx_memrd_evt: Pulse: Rx Memory Read 59 rtlh_rx_cfgwr_evt: Pulse: Rx Config Write 60 rtlh_rx_cfgrd_evt: Pulse: Rx Config Read 61 rtlh_rx_iowr_evt: Pulse: Rx IO Write 62 rtlh_rx_iord_evt: Pulse: Rx IO Read 63 rtlh_rx_cplwod_evt: Pulse: Rx Completion wo data 64 rtlh_rx_cplwd_evt: Pulse: Rx Completion w data 65 rtlh_rx_msg_evt: Pulse: Rx Message TLP 66 rtlh_rx_atmcop_evt: Pulse: Rx Atomic 67 rtlh_rx_tlpwprefix_evt: Pulse: Rx TLP with Prefix 68 xtlh_tx_ccix_tlp_evt: Pulse: Tx CCIX TLP 69 rtlh_rx_ccix_tlp_evt: Pulse: Rx CCIX TLP 82:70 cdm_ras_des_ec_info_l0[12:0] 95:83 cdm_ras_des_ec_info_l1[12:0] 108:96 cdm_ras_des_ec_info_l2[12:0] 121:109 cdm_ras_des_ec_info_l3[12:0] 128:122 cdm_ras_des_tba_info_common[6:0]

端口名称	方向	宽度	功能描述
			*_debug_info_sel=8 [4:0]pm_master_state[4:0]: Level: PM Internal State (Master) [8:5]pm_slave_state[3:0]: Level: PM Internal State (Slave) [15:9] rmlh_framing_err_ptr[6:0]: Pulse: 1st Framing Error Pointer [16] smlh_lane_reversed: Level: Lane Reversal Operation [17] pm_pme_resend_flag: Pulse: PME Re-Send flag [33:18] smlh_ltssm_variable [15:0]: Level: LTSSM Variable [36:34] ltssm_powerdown[1:0]: Level: PIPE: Power Down [44:37] latched_ts_nfts[7:0]: Level: Latched NFTS [46:45] rdlh_dlcntrl_state [1:0]: Level: DLCM [47] rdlh_vc0_initfc1_status: Level: Init-FC Flag1 VC0 [48] rdlh_vc0_initfc2_status: Level: Init-FC Flag2 VC0 [60:49] rdlh_curnt_rx_ack_seqnum[11:0]: Level: Rx ACK SEQ# [72:61] xdlh_curnt_seqnum [11:0]: Level: Tx TLP SEQ# 85:73 cdm_ras_des_sd_info_l0[12:0] 98:86 cdm_ras_des_sd_info_l1[12:0] 111:99 cdm_ras_des_sd_info_l2[12:0] 124:112 cdm_ras_des_sd_info_l3[12:0] *_debug_info_sel=9 119:0 cdm_ras_des_sd_info_v0[239:120] *_debug_info_sel=10 119:0 cdm_ras_des_sd_info_v0[119:0]
CFG_IDO_REQ_EN	O	1	ID-Based Ordering Requests Enabled

端口名称	方向	宽度	功能描述
CFG_IDO_CPL_EN	O	1	ID-Based Ordering Completions Enabled
XADM_PH_CDTS[7:0]	O	8	The amount of posted header buffer space currently available at the receiver at the other end of the link
XADM_PD_CDTS[11:0]	O	12	The amount of posted data buffer space currently available at the receiver at the other end of the link
XADM_NPH_CDTS[7:0]	O	8	The amount of non-posted header buffer space currently available at the receiver at the other end of the link
XADM_NPD_CDTS[11:0]	O	12	The amount of non-posted data buffer space currently available at the receiver at the other end of the link
XADM_CPLH_CDTS[7:0]	O	8	The amount of completion header buffer space currently available at the receiver at the other end of the link
XADM_CPLD_CDTS[11:0]	O	12	The amount of completion data buffer space currently available at the receiver at the other end of the link
MAC_PHY_POWERDOWN	O	2	Power control bits to the PHY 00: P0 (L0): normal 01: P0s (L0s): low recovery time, power saving. 10: P1 (L1): longer recovery time, additional power saving. 11: P2 (L2): lowest power state.
PHY_MAC_RXELECIDLE	I	4	Indicates receiver detection of an Electrical Idle for each lane
PHY_MAC_PHYSTATUS	I	4	Communicates completion of PHY functions, including power management transitions, receiver detection, speed change.
PHY_MAC_RXDATA	I	128	Parallel received data, 32 bits per lane
PHY_MAC_RXDATAK	I	16	Control (K character) indicator bits for received data
PHY_MAC_RXVALID	I	4	Indicates symbol lock and valid data for each lane
PHY_MAC_RXSTATUS	I	12	Receive status and error codes for each lane(3bits per lane)
MAC_PHY_TXDATA	O	128	Parallel data for transmission
MAC_PHY_TXDATAK	O	16	Control (K character) indicator bits for transmitted data:
MAC_PHY_TXDETECTRX_LOOPBACK	O	4	Combined loopback and transmit detect control, as per PIPE specification.

端口名称	方向	宽度	功能描述
MAC_PHY_TXELECIDLE_H	O	4	Forces transmit output to Electrical Idle for each lane which it is asserted.
MAC_PHY_TXELECIDLE_L	O	4	详见pcie2_hard_pipe_adapter章节
MAC_PHY_TXCOMPLIANCE	O	4	Sets the running disparity to negative
MAC_PHY_RXPOLARITY	O	4	Directs the PHY to perform a polarity inversion the received data the specified lanes
MAC_PHY_RATE	O	1	Controls the link signaling rate
MAC_PHY_TXDEEMPH	O	2	For Gen2 configurations, this two bits wide output selects transmitter de-emphasis as follows
MAC_PHY_TXMARGIN	O	3	Selects transmitter voltage levels
MAC_PHY_TXSWING	O	1	Controls the PHY transmitter voltage swing level
CFG_HW_AUTO_SP_DIS	O	1	Autonomous speed disable.
RAM_TEST_EN	I	1	external_ram 测试使能，高有效
RAM_TEST_ADDRH	I	1	测试模式： 1：test addr在数据高位 0：test addr 在数据低位
RETRY_TEST_DATA_EN	I	1	测试模式(只用于在单口ram)： 1：测试数据线 0：测试控制线
P_DATAQ_DATAOUT	I	66	Data buffer数据输出
P_DATAQ_ADDRA	O	10	Data buffer端口a地址
P_DATAQ_ADDRB	O	10	Data buffer端口b地址
P_DATAQ_DATAIN	O	66	Data buffer数据输入
P_DATAQ_ENA	O	1	Data buffer端口a使能
P_DATAQ_ENB	O	1	Data buffer端口b使能
P_DATAQ_WEA	O	1	Data buffer端口a写使能
RETRYRAM_XDLH_DATA	I	68	retry buffer数据输出

端口名称	方向	宽度	功能描述
XDLH_RETRYRAM_ADDR	O	10	retry buffer地址
XDLH_RETRYRAM_DATA	O	68	retry buffer数据输入
XDLH_RETRYRAM_WE	O	1	retry buffer写使能
XDLH_RETRYRAM_EN	O	1	retry buffer使能
P_HDRQ_DATAOUT	I	138	Hdr buffer数据输出
P_HDRQ_ADDRA	O	9	Hdr buffer端口a地址
P_HDRQ_ADDRB	O	9	Hdr buffer端口b地址
P_HDRQ_DATAIN	O	138	Hdr buffer数据输入
P_HDRQ_ENA	O	1	Hdr buffer端口a使能
P_HDRQ_ENB	O	1	Hdr buffer端口b使能
P_HDRQ_WEA	O	1	Hdr buffer端口a写使能
RAM_TEST_MODE_N	I	1	ram_test_mode_n用于打开ram测试模式， Active low

注：以上端口为function工作模式下所有端口名及其功能描述，若为DFT测试模式下，将增加以下三个端口

TEST_SE_N	I	1	Test shift enable, active low
TEST_RST_N	I	1	Test 模式复位，active low
TEST_MODE_N	I	1	Active low to enable test mode

9.7.4. 参数描述

表 266 GTP_PCIEGEN2 参数列表

参数名称	参数类型	有效值	默认值	功能描述
GRS_EN	<string>	"FALSE","TRUE"	TRUE	Bypass global rst
PIN_MUX_INT_FORCE_EN	<string>	"FALSE","TRUE"	FALSE	When setting “TRUE”, it uses “PIN_MUX_INT_DISABLE” to judge whether “legacy int” should be disable .
PIN_MUX_INT_DISABLE	<string>	"FALSE","TRUE"	FALSE	Replace internal signal: “cfg_int_disable” when setting

				this signal and “PIN_MUX_INT_FORCE_EN” are “TRUE”.
DIAG_CTRL_BUS_B2	<string>	"NORMAL", "FAST_LINK_MODE"	NORMAL	Select fast Link Mode for simulation when it is“FAST_LINK_MODE ”
DYN_DEBUG_SEL_EN	<string>	"FALSE","TRUE"	FALSE	FALSE: “DEBUG_INFO_SEL” controls “debug_info_mux” TRUE: “DYN_DEBUG_INFO_SEL” controls “debug_info_mux”
DEBUG_INFO_SEL	<integer>	0 to 15	0	Control “DEBUG_DEBUG_INFO_MUX”
BAR_RESIZABLE	<integer>	“ 0 to 56” Note: Please do not set more than 3 resizable bars at the same time	21 Note: 6’b010101 in binary	高有效表示对应的bar设为resizable，注意：该BAR必须已使能；有对应的BAR_INDEX_0/1/2指示，且同时有效的位数最大为3个。
NUM_OF_RBARS	<integer>	"0", "1", "2", "3"	3	Number of Resizable Bars in Resizable Bar Control Register
BAR_INDEX_0	<integer>	“0 to 5”	0	对应于Resizable BAR Control Register(at offset 008h)中的 bar index
BAR_INDEX_1	<integer>	“0 to 5”	2	对应于Resizable BAR Control Register(at offset 010h)中的 bar index
BAR_INDEX_2	<integer>	“0 to 5”	4	对应于Resizable BAR Control Register(at offset 018h)中的 bar index

TPH_DISABLE	<string>	"FALSE","TRUE"	FALSE	Bypass “TLP Processing Hints Supported”
MSIX_CAP_DISABLE	<string>	"FALSE","TRUE"	FALSE	Bypass “MSIX Capability”
MSI_CAP_DISABLE	<string>	"FALSE","TRUE"	FALSE	Bypass “MSI Capability”
MSI_PVM_DISABLE	<string>	"FALSE","TRUE"	FALSE	Bypass “Per vertor Masking Capable”
BAR_MASK_WRITABLE	<integer>	“0 to 63”	32	高有效表示对应的bar_mask值可写
APP_DEV_NUM	<integer>	“0 to 31”	0	Device number. Your application must drive this signal to set the device number in the Requester ID for RC mode.
APP_BUS_NUM	<integer>	“0 to 255”	0	Bus number. Your application must drive this signal to set the bus number in the Requester ID for RC mode.
RAM_MUX_EN	<string>	"FALSE","TRUE"	FALSE	rcv hdr ram和rcv data ram输出合并使能
ATOMIC_DISABLE	<string>	"FALSE","TRUE"	FALSE	用来配置是否关闭ATOMIC功能，高有效

9.7.5. 例化模板

```
GTP_PCIEGEN2 #(
    .GRS_EN("TRUE"),
    .PIN_MUX_INT_FORCE_EN("FALSE"),
    .PIN_MUX_INT_DISABLE("FALSE"),
    .DIAG_CTRL_BUS_B2("NORMAL"),
    .DYN_DEBUG_SEL_EN("FALSE"),
    .DEBUG_INFO_SEL(0),
    .BAR_RESIZABLE(21),
    .NUM_OF_RBARS(3),
    .BAR_INDEX_0(0),
    .BAR_INDEX_1(2),
    .BAR_INDEX_2(4),
    .TPH_DISABLE("FALSE"),
    .MSIX_CAP_DISABLE("FALSE"),
```

```
        .MSI_CAP_DISABLE("FALSE"),
        .MSI_PVM_DISABLE("FALSE"),
        .BAR_MASK_WRITABLE(32),
        .APP_DEV_NUM(0),
        .APP_BUS_NUM(0),
        .RAM_MUX_EN("FALSE"),
        .ATOMIC_DISABLE("FALSE")
    ) GTP_PCIEGEN2_inst (
        .AXIS_MASTER_TDATA(),              // OUTPUT[127:0]
        .AXIS_MASTER_TKEEP(),              // OUTPUT[3:0]
        .AXIS_MASTER_TUSER(),              // OUTPUT[7:0]
        .CFG_MAX_PAYLOAD_SIZE(),           // OUTPUT[2:0]
        .CFG_MAX_RD_REQ_SIZE(),            // OUTPUT[2:0]
        .CFG_PBUS_DEV_NUM(),               // OUTPUT[4:0]
        .CFG_PBUS_NUM(),                   // OUTPUT[7:0]
        .CFG_PF_TPH_ST_MODE(),             // OUTPUT[2:0]
        .CFG_TPH_REQ_EN(),                 // OUTPUT[1:0]
        .DEBUG_INFO_MUX(),                 // OUTPUT[132:0]
        .LBC_DBI_DOUT(),                   // OUTPUT[31:0]
        .MAC_PHY_POWERDOWN(),              // OUTPUT[1:0]
        .MAC_PHY_RXPOLARITY(),             // OUTPUT[3:0]
        .MAC_PHY_TXCOMPLIANCE(),           // OUTPUT[3:0]
        .MAC_PHY_TXDATA(),                 // OUTPUT[127:0]
        .MAC_PHY_TXDATAK(),                // OUTPUT[15:0]
        .MAC_PHY_TXDEEMPH(),               // OUTPUT[1:0]
        .MAC_PHY_TXDETECTRX_LOOPBACK(),// OUTPUT[3:0]
        .MAC_PHY_TXELECIDLE_H(),           // OUTPUT[3:0]
        .MAC_PHY_TXELECIDLE_L(),           // OUTPUT[3:0]
        .MAC_PHY_TXMARGIN(),               // OUTPUT[2:0]
        .PM_DSTATE(),                      // OUTPUT[2:0]
        .PM_MASTER_STATE(),                // OUTPUT[4:0]
        .PM_SLAVE_STATE(),                 // OUTPUT[4:0]
        .P_DATAQ_ADDRA(),                  // OUTPUT[9:0]
        .P_DATAQ_ADDRB(),                  // OUTPUT[9:0]
        .P_DATAQ_DATAIN(),                 // OUTPUT[65:0]
        .P_HDRQ_ADDRA(),                   // OUTPUT[8:0]
        .P_HDRQ_ADDRB(),                   // OUTPUT[8:0]
        .P_HDRQ_DATAIN(),                  // OUTPUT[137:0]
        .RADM_GRANT_TLP_TYPE(),            // OUTPUT[5:0]
        .RADM_TIMEOUT_CPL_ATTR(),          // OUTPUT[1:0]
        .RADM_TIMEOUT_CPL_LEN(),           // OUTPUT[10:0]
        .RADM_TIMEOUT_CPL_TAG(),           // OUTPUT[7:0]
        .RADM_TIMEOUT_CPL_TC(),            // OUTPUT[2:0]
        .SMLH_LTSSM_STATE(),               // OUTPUT[4:0]
        .XADM_CPLD_CDTS(),                 // OUTPUT[11:0]
        .XADM_CPLH_CDTS(),                 // OUTPUT[7:0]
        .XADM_NPD_CDTS(),                  // OUTPUT[11:0]
```

```
.XADM_NPH_CDTS(),                // OUTPUT[7:0]
.XADM_PD_CDTS(),                 // OUTPUT[11:0]
.XADM_PH_CDTS(),                 // OUTPUT[7:0]
.XDLH_RETRYRAM_ADDR(),           // OUTPUT[10:0]
.XDLH_RETRYRAM_DATA(),           // OUTPUT[67:0]
.APP_ERR_BUS(),                  // INPUT[12:0]
.APP_HDR_LOG(),                  // INPUT[127:0]
.APP_RAS_DES_TBA_CTRL(),         // INPUT[1:0]
.AXIS_SLAVE0_TDATA(),            // INPUT[127:0]
.AXIS_SLAVE1_TDATA(),            // INPUT[127:0]
.AXIS_SLAVE2_TDATA(),            // INPUT[127:0]
.CFG_MSI_PENDING(),              // INPUT[31:0]
.DBI_ADDR(),                     // INPUT[31:0]
.DBI_DIN(),                      // INPUT[31:0]
.DBI_WR(),                       // INPUT[3:0]
.DEVICE_TYPE(),                  // INPUT[2:0]
.DIAG_CTRL_BUS(),                // INPUT[1:0]
.DYN_DEBUG_INFO_SEL(),           // INPUT[3:0]
.MSIX_ADDR(),                    // INPUT[63:0]
.MSIX_DATA(),                    // INPUT[31:0]
.PHY_MAC_PHYSTATUS(),            // INPUT[3:0]
.PHY_MAC_RXDATA(),               // INPUT[127:0]
.PHY_MAC_RXDATAK(),              // INPUT[15:0]
.PHY_MAC_RXELECIDLE(),           // INPUT[3:0]
.PHY_MAC_RXSTATUS(),             // INPUT[11:0]
.PHY_MAC_RXVALID(),              // INPUT[3:0]
.P_DATAQ_DATAOUT(),              // INPUT[65:0]
.P_HDRQ_DATAOUT(),               // INPUT[137:0]
.RETRYRAM_XDLH_DATA(),           // INPUT[67:0]
.TRGT1_RADM_PKT_HALT(),          // INPUT[2:0]
.VEN_MSI_TC(),                   // INPUT[2:0]
.VEN_MSI_VECTOR(),               // INPUT[4:0]
.AUX_PM_EN(),                    // OUTPUT
.AXIS_MASTER_TLAST(),            // OUTPUT
.AXIS_MASTER_TVALID(),           // OUTPUT
.AXIS_SLAVE0_TREADY(),           // OUTPUT
.AXIS_SLAVE1_TREADY(),           // OUTPUT
.AXIS_SLAVE2_TREADY(),           // OUTPUT
.CFG_AER_RC_ERR_MUX(),           // OUTPUT
.CFG_ATOMIC_EGRESS_BLOCK(),      // OUTPUT
.CFG_ATOMIC_REQ_EN(),            // OUTPUT
.CFG_BUS_MASTER_EN(),            // OUTPUT
.CFG_BW_MGT_MUX(),               // OUTPUT
.CFG_CRS_SW_VIS_EN(),            // OUTPUT
.CFG_EXT_TAG_EN(),               // OUTPUT
.CFG_HW_AUTO_SP_DIS(),           // OUTPUT
.CFG_IDO_CPL_EN(),               // OUTPUT
```

```
.CFG_IDO_REQ_EN(),                // OUTPUT
.CFG_INT_DISABLE(),               // OUTPUT
.CFG_LINK_AUTO_BW_MUX(),          // OUTPUT
.CFG_MEM_SPACE_EN(),              // OUTPUT
.CFG_MSIX_EN(),                   // OUTPUT
.CFG_MSIX_FUNC_MASK(),            // OUTPUT
.CFG_MSI_EN(),                    // OUTPUT
.CFG_NO_SNOOP_EN(),               // OUTPUT
.CFG_PME_MUX(),                   // OUTPUT
.CFG_PM_NO_SOFT_RST(),            // OUTPUT
.CFG_RCB(),                       // OUTPUT
.CFG_RELAX_ORDER_EN(),            // OUTPUT
.CFG_SEND_COR_ERR_MUX(),          // OUTPUT
.CFG_SEND_F_ERR_MUX(),            // OUTPUT
.CFG_SEND_NF_ERR_MUX(),           // OUTPUT
.CFG_SYS_ERR_RC(),                // OUTPUT
.CORE_RST_N(),                    // OUTPUT
.INTA_GRT_MUX(),                  // OUTPUT
.INTB_GRT_MUX(),                  // OUTPUT
.INTC_GRT_MUX(),                  // OUTPUT
.INTD_GRT_MUX(),                  // OUTPUT
.LBC_DBI_ACK(),                   // OUTPUT
.MAC_PHY_RATE(),                  // OUTPUT
.MAC_PHY_TXSWING(),               // OUTPUT
.PHY_RST_N(),                     // OUTPUT
.PM_LINKST_IN_L0S(),              // OUTPUT
.PM_LINKST_IN_L1(),               // OUTPUT
.PM_LINKST_IN_L2(),               // OUTPUT
.PM_LINKST_L2_EXIT(),             // OUTPUT
.PM_PME_EN(),                     // OUTPUT
.PM_STATUS(),                     // OUTPUT
.PM_XTLH_BLOCK_TLP(),             // OUTPUT
.P_DATAQ_ENA(),                   // OUTPUT
.P_DATAQ_ENB(),                   // OUTPUT
.P_DATAQ_WEA(),                   // OUTPUT
.P_HDRQ_ENA(),                    // OUTPUT
.P_HDRQ_ENB(),                    // OUTPUT
.P_HDRQ_WEA(),                    // OUTPUT
.RADM_CPL_TIMEOUT(),              // OUTPUT
.RADM_IDLE(),                     // OUTPUT
.RADM_MSG_UNLOCK(),               // OUTPUT
.RADM_PM_PME(),                   // OUTPUT
.RADM_PM_TO_ACK(),                // OUTPUT
.RADM_PM_TURNOFF(),               // OUTPUT
.RADM_QOVERFLOW(),                // OUTPUT
.RADM_Q_NOT_EMPTY(),              // OUTPUT
.RBAR_CTRL_UPDATE(),              // OUTPUT
```

```
.RDLH_LINK_UP(),                    // OUTPUT
.SEDO(),                            // OUTPUT
.SEDO_EN(),                         // OUTPUT
.SMLH_LINK_UP(),                    // OUTPUT
.TRAINING_RST_N(),                  // OUTPUT
.VEN_MSI_GRANT(),                   // OUTPUT
.WAKE(),                            // OUTPUT
.XDLH_RETRYRAM_EN(),                // OUTPUT
.XDLH_RETRYRAM_WE(),                // OUTPUT
.APPS_PM_XMT_PME(),                 // INPUT
.APPS_PM_XMT_TURNOFF(),             // INPUT
.APP_CLK_PM_EN(),                   // INPUT
.APP_DBI_RO_WR_DISABLE(),           // INPUT
.APP_ERR_ADVISORY(),                // INPUT
.APP_HDR_VALID(),                   // INPUT
.APP_INIT_RST(),                    // INPUT
.APP_LTSSM_EN(),                    // INPUT
.APP_RAS_DES_SD_HOLD_LTSSM(),       // INPUT
.APP_READY_ENTR_L23(),              // INPUT
.APP_REQ_ENTR_L1(),                 // INPUT
.APP_REQ_EXIT_L1(),                 // INPUT
.APP_REQ_RETRY_EN(),                // INPUT
.APP_UNLOCK_MSG(),                  // INPUT
.APP_XFER_PENDING(),                // INPUT
.AXIS_MASTER_TREADY(),              // INPUT
.AXIS_SLAVE0_TLAST(),               // INPUT
.AXIS_SLAVE0_TUSER(),               // INPUT
.AXIS_SLAVE0_TVALID(),              // INPUT
.AXIS_SLAVE1_TLAST(),               // INPUT
.AXIS_SLAVE1_TUSER(),               // INPUT
.AXIS_SLAVE1_TVALID(),              // INPUT
.AXIS_SLAVE2_TLAST(),               // INPUT
.AXIS_SLAVE2_TUSER(),               // INPUT
.AXIS_SLAVE2_TVALID(),              // INPUT
.BUTTON_RST(),                      // INPUT
.DBI_CS(),                          // INPUT
.DBI_CS2(),                         // INPUT
.MEM_CLK(),                         // INPUT
.OUTBAND_PWRUP_CMD(),               // INPUT
.PCLK(),                            // INPUT
.PCLK_DIV2(),                       // INPUT
.PERST(),                           // INPUT
.POWER_UP_RST(),                    // INPUT
.RAM_TEST_ADDRH(),                  // INPUT
.RAM_TEST_EN(),                     // INPUT
.RAM_TEST_MODE_N(),                 // INPUT
.RETRY_TEST_DATA_EN(),              // INPUT
```

```
    .RX_LANE_FLIP_EN(),                 // INPUT
    .SEDI(),                           // INPUT
    .SEDI_ACK(),                        // INPUT
    .SYS_AUX_PWR_DET(),                   // INPUT
    .SYS_INT(),                        // INPUT
    .TX_LANE_FLIP_EN(),                 // INPUT
    .VEN_MSI_REQ()                      // INPUT
);
```

免责声明

版权声明

本文档版权归深圳市紫光同创电子有限公司所有，并保留一切权利。未经书面许可，任何公司和个人不得将此文档中的任何部分公开、转载或以其他方式披露、散发给第三方。否则，公司必将追究其法律责任。

免责声明

1、本文档仅提供阶段性信息，所含内容可根据产品的实际情况随时更新，恕不另行通知。如因本文档使用不当造成的直接或间接损失，本公司不承担任何法律责任。

2、本文档按现状提供，不负任何担保责任，包括对适销性、适用于特定用途或非侵权性的任何担保，和任何提案、规格或样品在他处提到的任何担保。本文档在此未以禁止反言或其他方式授予任何知识产权使用许可，不管是明示许可还是暗示许可。

3、公司保留任何时候在不事先声明的情况下对公司系列产品相关文档的修改权利。